

INDIAN INSTITUTE OF TECHNOLOGY DELHI

Department of Electrical Engineering

ELL305: Computer Architecture

Modern Processor Design Beyond Monolithic Scaling

Chiplet Architectures, Cache Hierarchies, and 3D-Stacked Cache — A Case Study of AMD Zen Processors

Term Paper

Submitted by

Ishan Krishna Agrawal	2024MT60860	mt6240860@maths.iitd.ac.in
Vaidik Goyal	2024EE11030	ee1241030@ee.iitd.ac.in
Moon Saraswat	2024EE11163	ee1241163@ee.iitd.ac.in

github.com/TheInvincible2111/ELL305-Term-Paper

8 November 2026

Copyright and Intellectual Property Rights Assignment

All intellectual property rights in the original content of this report belong to the Indian Institute of Technology Delhi, including its Department of Electrical Engineering.

Intellectual property rights in material cited or otherwise attributed to external sources remain with the respective authors, publishers, or other rights holders.

Ishan Krishna Agrawal
Vaidik Goyal
Moon Saraswat

8 November 2026

Acknowledgements

The authors acknowledge Prof. Sumantra Dutta Roy as the faculty advisor for this term-paper project.

The authors thank Prof. Subrat Kar for introducing the term-paper component of the course, which gave the authors the opportunity to undertake this work.

The authors also acknowledge the Department of Electrical Engineering and the Indian Institute of Technology Delhi for the academic environment, facilities, and institutional resources supporting this work.

This paper is prepared collectively by Ishan Krishna Agrawal, Vaidik Goyal, and Moon Saraswat.

Use of AI Tools

Because of the scale of this term paper, AI tools were used to help draft and organise the text and to help draw up its schematics and diagrams. The research questions, ideas, analysis and conclusions are the authors' own, and the authors directed, checked and take full responsibility for all of the content.

Abstract

For several decades, processor performance improved largely by integrating more transistors on a single, monolithic die. That approach is now constrained by die size limits, manufacturing yield, the uneven scaling of logic, SRAM and analog circuits, and the power and thermal budget of the package. This paper studies two linked architectural responses to these constraints: chiplet-based processor organisation, which partitions a processor into separately manufactured dies, and three-dimensional stacking of cache, which adds last-level-cache capacity without enlarging the compute die. AMD Zen processors are the principal case study, tracing the progression from the compute-die and I/O-die partitioning of Zen 2, through the unified eight-core L3 organisation of Zen 3, to the hybrid-bonded 3D V-Cache products. For each technique the paper asks which bottleneck it addresses, what benefit it provides and what new cost or trade-off it introduces, and it grades the published evidence behind each answer. Chiplets answer the cost and size limits of large dies: smaller dies are cheaper per good die, the I/O moves to an older node, and core counts beyond the reticle limit become possible, at the price of a die-to-die hop on every L3 miss and of non-uniform memory access (NUMA) across the socket. Stacking a 64 MB SRAM die on a Zen 3 compute die expands its L3 from 32 to 96 MB for about four extra cycles of latency, a cost measured independently, by one outlet, across three generations. The benefit depends strongly on the workload: it is large for working sets between the base and stacked capacities and small or slightly negative otherwise, and in the first two generations (Zen 3 and Zen 4) it is partly offset by lower clocks. A simple model calibrated on published measurements shows that the 7% clock gap of one Zen 4 part raises the number of misses the larger cache must remove to break even from about 0.1 to about 0.35 per thousand instructions. A trace-driven cache model, calibrated on the same measurements, illustrates these mechanisms with every other factor held fixed and shows which assumptions the conclusions depend on most. Measurements on the group's own Ryzen 5 5600X, a single-CCD chiplet part, show that the published L1 and L2 latencies the model takes as inputs agree with that machine's, and that its L3 and memory latencies are within about 20%. Close to each cache's capacity, however, the model's latency is too low, by up to 2.4 times (at 32 MB, measured in a virtual machine). The analysis is grounded in the memory-hierarchy and multiprocessor foundations of the course textbook and uses published research and benchmark evidence, complemented by these measurements on one processor without stacked cache.

Keywords: processor architecture, chiplets, memory hierarchy, last-level cache, 3D-stacked cache, hybrid bonding, AMD Zen, ELL305.

Contents

List of Tables **10**

List of Figures **12**

Glossary **15**

I Introduction and Research Framing 17

1 Introduction 17

1.1 Motivation: why monolithic scaling is becoming difficult 17
1.2 A thought experiment: one die or eight? 18
1.3 Two linked scaling problems 18
1.4 Research questions and objectives 20
1.5 Scope 21
1.6 Related work 22
1.7 Methodology 23
1.8 Contributions 23
1.9 Main findings 24
1.10 Organisation of the paper 25

II Architectural Foundations 26

2 Processor Organisation and Performance Metrics 26

2.1 The performance equation 27
2.2 Baseline processor organisation 27

3 Pipelining, Power and Thermal Limits 27

3.1 Pipelining and its limits 27
3.2 Dynamic and leakage power 28
3.3 Temperature 29

4 The Memory Hierarchy 29

4.1 Why a hierarchy is needed . . . 29
4.2 SRAM and DRAM arrays . . . 30
4.3 Average memory access time . . . 31
4.4 Classifying cache misses . . . 32
4.5 Cache organisation and policies . . . 32
4.6 Writes and inclusion . . . 33
4.7 Virtual memory and address translation . . . 35
4.8 Overlapping misses and multithreading . . . 36
4.9 Beyond SRAM: alternative LLC technologies . . . 37

5 Multicore Processors, Coherence and Interconnects 37
5.1 From one core to many . . . 37
5.2 How far parallelism goes: Amdahl's law . . . 37
5.3 Cache coherence . . . 38
5.4 On-chip and on-package interconnection networks . . . 40

III Chiplet-Based Processor Architecture 42

6 Why Large Monolithic Dies Become Costly 42
6.1 The reticle limit and die size . . . 42
6.2 Manufacturing yield . . . 43
6.3 Uneven scaling of logic, SRAM and analog circuits . . . 44

7 Compute Dies, I/O Dies and Core Complexes 45
7.1 Inside the I/O die: memory, I/O and NUMA settings . . . 46

8 Infinity Fabric and Die-to-Die Communication 48
8.1 Latency, bandwidth and interconnect power . . . 48
8.2 Sharing data across dies . . . 50
8.3 Scalability . . . 52
8.4 How the dies are connected: organic substrate, 2.5D and 3D . . . 53

9 Yield, Process-Node Specialisation and Cost 55

10 Power, Clocking and Product Flexibility **55**

10.1 Power and clocking 56

10.2 One compute die, many products 57

11 The Same Core With and Without Chiplets **58**

12 Summary of Part III **60**

IV 3D-Stacked Cache and AMD 3D V-Cache **62**

13 Why Cache Capacity Matters **62**

14 Why Expanding Cache in Two Dimensions Became Difficult **63**

14.1 Area and yield 64

14.2 Wire length and access latency 64

14.3 Power 65

14.4 Summary of the planar limits 65

15 Side-by-Side Versus Vertically Stacked Cache Dies **65**

15.1 Placing the cache die beside the compute die 65

15.2 Stacking the cache die on the compute die 67

16 Stacked-Die Integration and Hybrid Bonding **67**

16.1 The Zen 3 3D V-Cache stack 67

16.2 Hybrid bonding versus micro-bumps 68

16.3 What a finer pitch buys 69

16.4 How a hybrid bond is made 70

17 Cache Access, Latency, Bandwidth and Hierarchy **71**

17.1 Applying the trade-off to a measured workload 71

17.2 Inside the cache interface 73

18 Thermal Management, Power Delivery and Manufacturing Complexity **74**

18.1 Power delivery through the stack 75

18.2 Evidence of the thermal cost and AMD's response 75

18.3 Manufacturing: thinning, assembly, yield and test 76

19 Workloads That Benefit From a Larger LLC 79

20 Summary of Part IV 81

V AMD Case Study and Quantitative Analysis 82

21 Method for Analysing Published Evidence 82

22 From Zen 2 Chiplets to Zen 3 and 3D V-Cache 83

22.1 Zen 2 to Zen 3: a same-core-count comparison 84

23 Comparing Architectural Configurations 86

24 Published Benchmarks and Performance Measurements 87

24.1 The evidence ledger 88

24.2 Latency through the whole hierarchy 89

24.3 Lam's six workloads in detail 89

24.4 A working-set sweep: MiniFE 92

24.5 OpenFOAM in detail 92

24.6 The next generation: Genoa-X 93

24.7 Do the independent sources agree? 94

24.8 Breadth: 350 desktop results 95

25 Cache-Sensitive Versus Cache-Insensitive Workloads 96

26 Separating Cache Effects From Other Factors 96

26.1 Correcting for clock speed 98

27 Cache Against Clock 99

27.1 Cache against clock on one chip 99

27.2 Refining the rule: memory time does not shrink with clock 100

28 Simulation 101

28.1 The model and how the experiments are run 101

28.2 S0: What the calibration fixes, and how the model compares with hardware . . 102
28.3 S1: The working-set window 105
28.4 S2: Stacked, planar or beside? 106
28.5 S3: Cache against clock 107
28.6 S4: The cost of the chiplet split 110
28.7 S5: Zen 2 to Zen 3, one factor at a time 111
28.8 S6: When bandwidth binds 112
28.9 Sensitivity to the assumed inputs 113
28.10 What the model adds, and what it cannot show 113

29 Summary of Part V **115**

VI Comparative Synthesis, Limitations and Future Directions **116**

30 Monolithic, Chiplet-Based and Vertically Stacked Designs Compared **116**

31 Other Ways to Enlarge the Last-Level Cache **117**

32 Interactions Between Chiplet Organisation and Cache Design **119**

33 Workload-Dependent Trade-Offs and Remaining Bottlenecks **119**

34 The Memory-Bandwidth Wall Behind Both Techniques **120**

35 Future Directions in Heterogeneous Integration and Advanced Packaging **122**

36 How Strong Is the Evidence? **123**

37 The Case Against, and How It Fares **125**

38 Threats to Validity **125**

39 Implications for Practice **126**

40 Conclusions **128**

41 Open Research Questions **129**

References **131**

A Plagiarism Report **134**

B Reading Indices **135**

C Yield and Cost-per-Good-Die Model **136**

C.1 From Poisson to Murphy yield 136

C.2 Cost per good die 136

C.3 Splitting a design into chiplets 137

C.4 Silicon across a product line 137

C.5 Good dies per wafer 138

D Organisation of the Stacked L3 Arrays **139**

E Simulation Set-up and Reproduction **140**

F Further Calculations **143**

F.1 A directory protocol, step by step 143

F.2 TLB reach against cache capacity 143

F.3 Calibration of the cache–clock model 144

G Measurements on the Group's Ryzen 5 5600X **146**

H Evidence Ledger **149**

Index **150**

List of Tables

1 One die or eight: the 64-core thought experiment, answered from later sections 19
2 Where the research questions are answered 21
3 Selected memory-system optimisations and their costs, after Sarangi 34
4 Distinct on-chip capacity of one Zen 3 CCX under the two inclusion policies . . 35
5 Private versus shared caches, after Sarangi 38
6 Block states and bus traffic for two cores sharing one block 39
7 Diameter and bisection bandwidth of common topologies 41
8 Illustrative Murphy-model yield 44
9 NUMA settings of an EPYC 7003 processor 47
10 Memory latency by NUMA distance, Naples and Rome, as reported by AMD . 49
11 Core-to-core message-passing latency (Reguly) 51
12 Total CCD–IOD link bandwidth against peak memory bandwidth 52
13 Die-to-die connection methods 54
14 One compute die, many products 58
15 Zen 3 with and without the chiplet split: Cezanne against Vermeer and Milan . 59
16 Qualitative comparison of side-by-side and vertically stacked cache dies 67
17 Silicon consumed per good part for a planar 96 MB CCD and the stacked design 79
18 L3 organisation per compute die across the generations studied 83
19 3D V-Cache across Zen generations 86
20 Processor pairs that differ mainly in L3 capacity 87
21 AMD-reported performance figures relevant to this study 87
22 Independent measurements of the effect of 3D V-Cache 88
23 Lam's per-CCD measurements on the Ryzen 9 7950X3D, both CCDs at 4.2 GHz 90
24 Cycles saved per removed L3 miss 91
25 Cross-check of independent results across generations 95
26 Which confounding factors each available comparison controls 98
27 Speedups bounded by a clock correction 98
28 IPC gain of the V-Cache CCD against the clock gap 100
29 The simulation experiments 102
30 S0: overlap factors fitted to Lam's three workloads 105

31 Overlap of independent misses on the group's Ryzen 5 5600X 105
32 S2: the slowest planar 96 MB L3 that still beats the baseline 107
33 S2: a side die against the stacked L3 108
34 S3: break-even misses removed for each clock gap 109
35 Sensitivity of S3: the break-even when one input is changed 109
36 S4: two CCDs against a hypothetical single die 111
37 Sensitivity of S1 to the assumed inputs 114
38 Replacement policy and the gain from a larger L3 114
39 Comparison of the three design approaches 117
40 Ways to enlarge the last-level cache 118
41 Memory bandwidth and L3 capacity per core on EPYC processors with all cores busy 121
42 Strength of the evidence for the paper's main claims 124
43 Turnitin similarity report 134
44 Reading indices of the main text 135
45 Average silicon per good part across a product line 138
46 Dies and good dies per 300 mm wafer 138
47 Default model parameters and their sources 141
48 The five accesses of the MSI trace under a directory protocol 143
49 Data-TLB reach of Zen 3 against its cache capacities 144
50 Net speed-up of the V-Cache CCD over the plain CCD for the libx264 profile . 145
51 Conditions of the 5600X measurements 146
52 Latency at each level on the 5600X against the model's default inputs 147
53 Evidence ledger of the independent sources on the stacked cache 149

List of Figures

1 The steps of AMD's Zen design studied in this paper . . . 18
2 Research framework . . . 20
3 Overview of the research questions of section 1.4 . . . 22
4 Overview of the paper . . . 25
5 Overview of Part II . . . 26
6 The heat path from a processor die to the air, redrawn after Sarangi . . . 29
7 A multi-level cache hierarchy and the AMAT quantities . . . 30
8 SRAM cell and array . . . 31
9 DRAM cell and array . . . 31
10 Lookup in a 4-way set-associative cache . . . 33
11 Overview of table 3 . . . 34
12 The write-invalidate (MSI) snooping protocol . . . 39
13 Three topologies . . . 41
14 Overview of Part III . . . 42
15 The 64-core Rome against the reticle limit . . . 43
16 Murphy yield model for three illustrative defect densities . . . 44
17 Area scaling by circuit type and cost per yielded mm^2 (AMD) . . . 45
18 A monolithic die against a chiplet package . . . 45
19 Schematic of an AMD EPYC 7002/7003-style package . . . 47
20 The path of a load that misses in the L3 of a chiplet processor . . . 49
21 Memory latency at each NUMA distance in Naples and Rome, as reported by AMD . . . 50
22 Which side limits a Rome socket's memory bandwidth as compute dies are added 52
23 The lane trade in a dual-socket EPYC 7003 system . . . 53
24 AMD's packaging steps as presented by AMD . . . 54
25 Cost of chiplet and hypothetical monolithic Rome-generation products (AMD) . 56
26 Overview of the chiplet trade-off as Part III found it . . . 61
27 Overview of Part IV . . . 62
28 Two of the charts with which AMD's ISSCC 2022 presentation opens, redrawn 63
29 L3 miss rate against working-set size (conceptual) . . . 64

30 Extra latency to each L3 slice of a Zen 3 core (Ryzen 5 5600G) 65
31 Two ways to add cache capacity to a compute die 66
32 Cross-section of the Zen 3 3D V-Cache stack 68
33 Connections per mm^2 at each interconnect pitch 70
34 One hybrid-bond connection in the Zen 3 stack 70
35 The libx264 calculation as a sequence 72
36 Top view of how the L3D connects to the CCD, redrawn from AMD's descriptions 74
37 Overview of the costs of stacking and AMD's responses 75
38 The two stack arrangements of 3D V-Cache 77
39 The steps that a V-Cache CCD needs and a planar one does not, grouped by purpose 78
40 IPC gain from 3D V-Cache with clock, core design and memory held equal . . 81
41 Overview of Part V 82
42 Evidence-analysis procedure for Part V 83
43 Compute die organisation from Zen 2 to Zen 3 and Zen 3 with 3D V-Cache . . 84
44 AIDA64 latency of a Zen 2 and a Zen 3 eight-core processor 86
45 Range of results for the stacked cache: AMD against independent sources . . . 88
46 Cache and memory latency of Milan and Milan-X against data size 90
47 MiniFE speed-up of Milan-X over Milan as the problem size grows 93
48 OpenFOAM speed-up of Milan-X over Milan by mesh size 93
49 Distribution of Phoronix's 350 significant differences, 5800X3D against 5800X 96
50 Overview of the confounding factors of section 26 97
51 Comparisons before and after a linear clock correction 99
52 Cache against clock for the libx264 profile 101
53 S0: simulated latency sweep against Milan and Milan-X 103
54 The group's Ryzen 5 5600X against the simulation 104
55 S1: the effect of tripling the L3 against working-set size 106
56 S2: ways to provide about 96 MB of last-level cache 107
57 S3: cache against clock in the simulation 108
58 S4: cross-CCD to same-CCD ping-pong ratio against hop cost 110
59 S5: Zen 2 to Zen 3 L3 organisation and latency, separated 112
60 S6: up to eight cores streaming through private data 113

61 Overview of Part VI 116
62 When does a stacked L3 pay off? 120
63 Cache-to-memory bandwidth ratio and the gain from tiling (Reguly) 122
64 Overview of the future directions of section 35 123
65 A possible thread-placement policy for mixed CCDs 127
66 Overview of the guidance for practice 128

Glossary

Definitions follow Sarangi [1] unless stated otherwise.

3D V-Cache AMD's name for a design that uses three-dimensional die integration to add stacked cache capacity above a processor die.

AMAT (average memory access time) A weighted average of memory-access latencies that accounts for cache hits, cache misses, and the penalties of accessing lower levels of the memory hierarchy.

cache coherence The mechanisms and rules used to maintain a consistent view of shared data when multiple caches can hold copies of the same memory location.

cache hierarchy A set of cache levels organised to exploit locality by balancing capacity, access latency, and implementation cost.

CCD (Core Complex Die) AMD terminology for a compute die containing processor cores and associated cache resources in selected Zen-based processors.

CCX (Core Complex) AMD terminology for a group of cores that share one L3 cache; in Zen 2 a CCD contains two four-core CCXs, and in Zen 3 a CCD contains one eight-core CCX.

chiplet A separately fabricated die designed to operate with one or more other dies as part of a packaged system.

CPI (cycles per instruction) The average number of clock cycles taken per dynamic instruction; its reciprocal is IPC.

defect density The average number of yield-killing manufacturing defects per unit area of silicon, written D_0 in simple yield models.

die-to-die hop The extra latency of a transfer between two dies in a package, such as between a CCD and the I/O die. In the simulations of Part V a hop is one CCD–IOD round trip, by default AMD's 9 ns.

hybrid bonding A fine-pitch die-stacking technique that forms direct mechanical and electrical connections between aligned die surfaces.

IFOP (Infinity Fabric On-Package) The on-package serial link used by AMD to connect compute dies to the I/O die over short organic-package routes.

Infinity Fabric AMD's interconnect architecture for communication among processor components and, in relevant products, among separate dies.

IOD (I/O Die) AMD terminology for a die that centralises input/output, memory-interface, and supporting interconnect functions in selected chiplet-based Zen processors.

IPC (instructions per cycle) The average number of dynamic instructions completed per clock cycle; the reciprocal of CPI.

L3D (L3 die) AMD's name for the stacked SRAM die of 3D V-Cache, called the cache die in this paper. A CCD carrying one is a V-Cache CCD; a CCD without one is a plain CCD. A product built from V-Cache CCDs is a V-Cache part.

LLC (last-level cache) The final cache level checked before a request proceeds to main memory in the cache hierarchy under discussion.

memory hierarchy An organisation of storage levels with different capacities, costs, and access times, arranged to exploit temporal and spatial locality.

memory-level parallelism The overlap of several outstanding cache misses, which lets an out-of-order core wait roughly once for several misses.

micro-bump A small solder-based bump used to connect stacked or adjacent dies; its pitch limits the achievable interconnect density compared with hybrid bonding.

monolithic die A single semiconductor die on which the principal processor resources under discussion are integrated rather than partitioned across multiple dies.

reticle limit The maximum die area that a lithography system can pattern in a single exposure field, which bounds the size of a monolithic die.

SRAM (static random-access memory) A memory built from bistable cells that retain data while powered without refresh; the usual technology for on-chip caches.

TLB (translation lookaside buffer) A small cache of recent virtual-to-physical address translations, consulted on every memory access.

TSV (through-silicon via) A vertical electrical connection passing through a silicon die to support communication in a stacked-die assembly.

yield The fraction of manufactured dies that function correctly; larger dies generally yield worse for a given defect density.

Part I

Introduction and Research Framing

1 Introduction

1.1 Motivation: why monolithic scaling is becoming difficult

For most of the history of the microprocessor, a new product generation could be built by placing more transistors on a single piece of silicon. The course textbook summarises this trend as Moore's law: the number of transistors on a chip has roughly doubled every one to two years since the 1960s [1, sec. 12.1.1].

How processors used to scale

The textbook describes three phases in how designers spent those transistors [1, sec. 12.1.2]. First, a smaller transistor was faster and more power-efficient, so each generation built a bigger processor: more complex units, larger caches, wider issue and more functional units. Second, until about 2002, pipelines also grew deeper and clock frequencies rose with them. Third, after about 2002, power and temperature became the limit: processors began to exceed 100 W and on-chip temperatures 100 °C, which effectively ended the growth in core complexity and clock frequency. Designers instead began to place more cores of an unchanged design on each chip, so that the number of cores could double with each process generation [1, sec. 12.1.2].

Over the same period the smallest feature that could be manufactured shrank by a factor of about $\sqrt{2}$ every two years, from 130 nm in 2001 to 22 nm in 2011, doubling the number of transistors per unit area each time [1, sec. 12.1.1, Table 12.1]. Writing around 2012, the textbook expected processors with 32 or 64 cores within five to ten years [1, sec. 12.1.2]. The prediction came true almost exactly: AMD's second-generation EPYC offered 64 cores in 2019. But it did not arrive in the way the multicore era had assumed. Those 64 cores were not on one larger die but on eight small ones, joined in one package to a separate I/O die [2]. Explaining why is the starting point of this paper.

Three pressures now make it harder to keep growing a single monolithic die:

- **Die size and yield.** A lithography system can only pattern a die up to the reticle limit, and the probability that a die contains a fatal defect rises with its area. AMD states that the 64-core server configuration of its second-generation EPYC processors would lie well beyond the maximum reticle size if built as one chip [2]. Section 6.2 quantifies the yield effect with a simple model.

- **Uneven scaling of circuit types.** Logic, SRAM (static random-access memory) and analog/I-O circuits do not shrink equally when moving to a new process node. Naffziger *et al.* note that analog circuitry shrinks far less than digital logic and SRAM, so memory interfaces and serialiser–deserialiser (SerDes) links gain little from an advanced node [2]. AMD's 2022 charts show SRAM, too, falling behind logic from 7 nm, while the cost of each yielded square millimetre rose to almost five times its 45 nm value at 5 nm (figure 17) [3, slide 6].
- **Power and temperature.** Dynamic power grows with switched capacitance, the square of the supply voltage and clock frequency, while leakage power is consumed even when circuits are idle and itself rises with temperature [1, sec. 10.10]. Every additional square millimetre of active silicon must be powered and cooled within a fixed package budget.

These pressures have changed how processors grow (figure 1). Instead of one ever-larger die, designers increasingly assemble a processor from several dies inside one package, and increasingly use the vertical dimension as well as the horizontal one. This paper studies that shift through two concrete techniques and one principal case study.

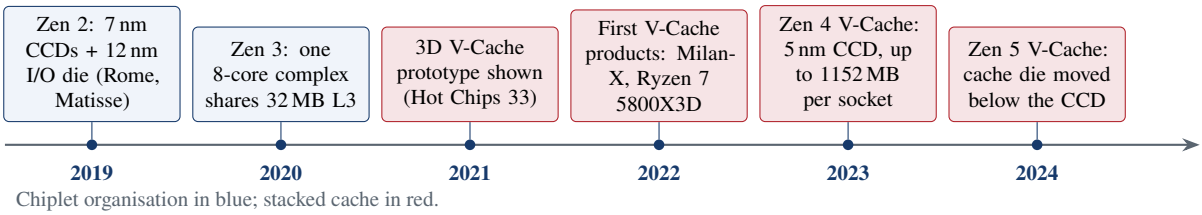


Figure 1: The steps of AMD's Zen design studied in this paper, with the year each was disclosed or launched [2], [3], [4], [5], [6], [7], [8], [9], [10].

1.2 A thought experiment: one die or eight?

Consider two ways to build a 64-core processor. Processor A places all 64 cores on one large die. Processor B places them on eight smaller dies of eight cores each, connected inside one package to a separate I/O die. Both offer 64 cores. Are they equally expensive to make? Do they communicate equally quickly? Can they use the same manufacturing process?

Using AMD's Zen 2 dimensions, with the compute dies at 74 mm^2 each and a monolithic equivalent of $8 \times 74 = 592\ mm^2$ for the compute area alone, the later parts of this paper answer these questions (table 1).

The answers are not all in favour of B. By AMD's account the full 64-core design could not be built as one die at all, and the models of Part III make B far cheaper, but it pays for that with a die-to-die hop on every memory access and with caches that are private to each die. The rest of the paper examines both sides of this trade, and how 3D stacking changes the cache side of it.

1.3 Two linked scaling problems

The paper is organised around two related problems (figure 2).

Table 1: One die or eight: the 64-core thought experiment, answered from later sections (model results are illustrative).

Question	A: one 64-core die	B: eight 8-core dies + I/O die	Where
Can it be built?	For 64 cores with I/O, beyond the maximum reticle size [2]	Yes; each die is small	section 6
Modelled yield ($D_0 = 0.2\,\mathrm{cm}^{-2}$)	38%	87% per compute die	section 6.2
64-core sets per 300 mm wafer	about 35	about 95	section 9
AMD's cost estimate	roughly 2× at 32 and 48 cores	1× (reference)	figure 25
Manufacturing process	everything on the newest node	cores on 7 nm, I/O on cheaper 12 nm	section 7
Communication	on-die network only	every memory access crosses a die-to-die link (AMD: under 9 ns round trip [2])	section 8
Memory latency	not available for comparison	AMD reports Rome's average at about 0.81× that of its multi-die predecessor [2]	table 10
Cache per core	one shared design choice	each die has its own L3; can be enlarged per die by stacking	Part IV

Problem 1: scaling processor resources beyond a single die. If a product needs more cores, memory channels and I/O than one economical die can hold, the processor can be partitioned into chiplets. AMD's Zen 2-based products separate eight-core compute dies (CCDs) built on a 7 nm process from an I/O die (IOD) built on a mature 12 nm process [2], [4].

Problem 2: scaling cache capacity without spending planar die area. A larger LLC (last-level cache) can reduce costly trips to main memory, but on a planar die every extra megabyte of SRAM enlarges the die, lowers yield and lengthens wires. AMD's 3D V-Cache instead stacks an additional SRAM die on top of the compute die, expanding the Zen 3 L3 from 32 MB to 96 MB per CCD [3], [11].

The problems are linked: the chiplet organisation makes the CCD a small, self-contained compute die with its own L3, and that die is the natural host for a stacked cache. The paper therefore follows the argument in four steps:

1. Monolithic CPUs face die-size, yield and scaling limits; chiplets address compute scaling [2].

2. Chiplets alone leave the cost of large on-die caches unsolved: AMD's ISSCC 2022 presentation names the slowing scaling and rising cost of SRAM as barriers to large on-die caches, in cost, product flexibility and latency, and argues that 2.5D chiplets address only part of the problem [3, slides 6–7].

3. Vertical stacking addresses those cache-specific limits [3], [6].

4. Evaluation evidence shows where the stacked cache improves performance, and which thermal, packaging and cost trade-offs it introduces (Part V).

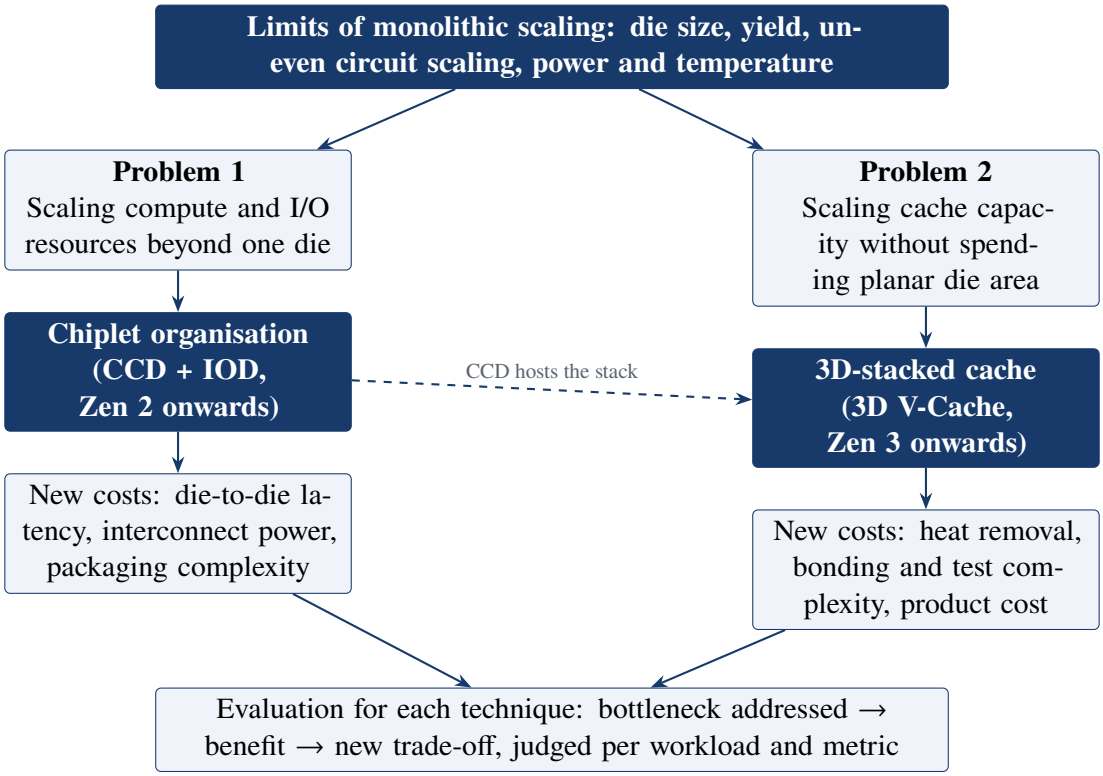


Figure 2: Research framework: two linked scaling problems, the AMD responses studied in this paper, and the evaluation question applied to each.

1.4 Research questions and objectives

The central research question is:

> *How do chiplet-based organisation and 3D-stacked cache help modern processors address limits in performance, scalability, memory latency, power and die-level integration, and what new trade-offs do they introduce?*

It is divided into four sub-questions, the second with a supporting question:

RQ1 Which architectural and manufacturing limits make a single large die an increasingly costly way to scale a processor?

RQ2 How does partitioning a processor into compute and I/O dies change memory latency, bandwidth, power and product flexibility?

RQ2a How does AMD divide processor functionality between the CCDs and the IOD, and why does it connect them through the organic package substrate rather than a 2.5D or 3D interconnect?

RQ3 Why does last-level-cache capacity matter, and what does vertical stacking change compared with expanding the cache in the plane of the die?

RQ4 For which workloads does a larger stacked L3 produce measurable benefit, and how can that benefit be separated from differences in frequency, core count and memory configuration?

For every technique the paper applies the same three-part test: *which bottleneck does it address, what benefit does it provide, and what new bottleneck, cost or trade-off does it create?* Table 2 shows where each question is answered.

Table 2: Where the research questions are answered.

Question	Main evidence	Sections
RQ1	Reticle limit, yield and dies-per-wafer models, uneven scaling of circuit types	Part III (sections 6 and 6.2); appendix C.5
RQ2, RQ2a	AMD's chiplet disclosures, die-to-die link figures, non-uniform memory access (NUMA) latencies, packaging options; the chiplet simulation	Part III (sections 7, 8 and 8.4); section 28.6
RQ3	Planar limits, hybrid bonding, the stacked-cache interface, thermal and manufacturing evidence; the simulated design comparison	Part IV; section 28.4
RQ4	AMD-reported and independent measurements, graded for confounding factors; the simulations	Part V (section 28)
All	Comparison, interactions and conclusions	Part VI

Figure 3 shows the same questions at a glance.

1.5 Scope

AMD Zen processors are the principal case study because AMD has published unusually detailed technical descriptions of both its chiplet organisation and its stacked cache [2], [4], [6], [11]. They are, however, a case study and not the whole subject: the architectural principles apply to chiplet and die-stacked designs in general.

The following are deliberately outside scope:

- fabrication chemistry and process details beyond what is needed to explain an architectural decision;
- reverse engineering of undisclosed Zen internals;
- a catalogue of product specifications;
- vendor contests. Comparisons are always tied to a stated workload, metric and time period, and the paper makes no general claim that one vendor "beats" another.

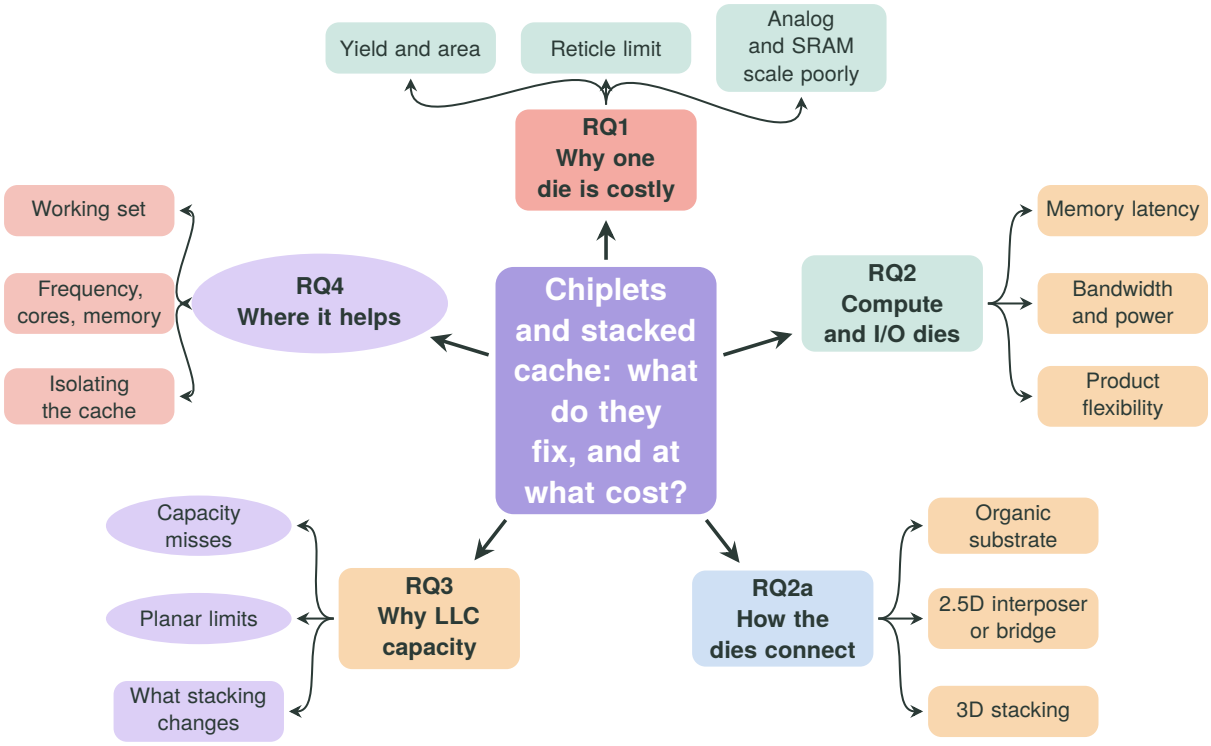


Figure 3: Overview of the research questions of section 1.4. Every branch is answered with the same test: which bottleneck the technique addresses, what it gains, and what new trade-off it creates.

1.6 Related work

Earlier surveys cover each technique on its own. Liu, Li and Yin review chiplet-based design as a whole, dividing it into system architecture and interconnection. They identify as the main challenges the mapping of functions onto chiplets, the co-optimisation of package and architecture, the added latency of communication between dies, and keeping independently designed chiplets deadlock-free when they are combined [12]. Their review is broad and spans many vendors and accelerators, rather than tracing one product family through its generations. Mittal and Vetter survey techniques for building large last-level caches from DRAM, motivated by the low density and high leakage of SRAM [13]. Their survey predates hybrid-bonded SRAM stacking in products, so it treats a different technology from the one AMD chose (section 4.2). At the level of individual designs, Black *et al.* simulated stacking SRAM and DRAM caches on a processor in 2006, including their thermal cost [14], and Dong and Xie built a system-level cost model for 3D integrated circuits, which finds that stacking pays only above a certain design size [15]. Domke *et al.* come closest to this paper's subject. They measured Milan-X against Milan on high-performance computing (HPC) proxy applications, and simulated a hypothetical future processor with very large stacked SRAM, reporting an average 9.56× per-chip gain for cache-sensitive HPC applications [16]. Their focus is HPC and future designs, not the trade-offs of the products already shipped.

This paper differs from these works in three ways. It treats chiplets and stacked cache as two answers to related limits of one design, the AMD Zen family. It explains both with the models of an undergraduate textbook, so that each claim can be checked by hand. And it grades the published evidence by what each comparison controls, rather than quoting headline figures.

1.7 Methodology

The study combines five kinds of evidence:

1. **Textbook foundations.** Chapters 7 and 9–12 of Sarangi's *Basic Computer Architecture* supply the models used throughout. The topic maps mainly to Chapters 11 (the memory system) and 12 (multiprocessors), with foundations from Chapters 7, 9 and 10; all lie within Chapters 1–12, so the faculty advisor is Prof. Sumantra Dutta Roy. The models used are the performance equation, pipelining limits, power and temperature, the memory hierarchy and average memory access time, and multicore coherence and interconnects [1].

2. **Primary technical disclosures.** Peer-reviewed conference papers and vendor technical presentations describe what was built and why. Vendor presentations are attributed to the vendor and treated as engineering claims rather than independent measurements.

3. **Published research and benchmarks.** Part V analyses published measurements of chiplet and stacked-cache processors. Apart from the one set of measurements on a single-CCD part described below, the paper performs no new measurements of these processors; its quantitative contribution lies in organising, normalising and critically comparing existing results (section 21).

4. **A simulation.** A trace-driven model of the cache hierarchy, built by the group and calibrated against published measurements, changes one factor at a time where no published comparison does (section 28). Its results are labelled as model outputs throughout.

5. **The group's own measurements.** Latency, overlap of misses, bandwidth and core-to-core tests on the group's Ryzen 5 5600X, a single-CCD Zen 3 chiplet processor without stacked cache, compare the model with a machine it was not built from (appendix G).

Two practices run through the analysis. First, *calculations* apply the textbook's models to published numbers: yield and cost per wafer, connection density, average memory access time, the stacked-cache trade-off and clock correction. Each states its inputs, and any assumed value is labelled as such. Second, sources are graded. Vendor presentations are used for what was built; independent measurements, preferably with fixed clocks or performance counters, are used for what it achieves; and where a value had to be read from a published chart rather than a table, the text says so and gives the reading accuracy.

1.8 Contributions

This paper makes no new measurements of stacked-cache processors or of traffic between CCDs, and one set on a Ryzen 5 5600X, a single-CCD chiplet part without stacked cache; its contributions are otherwise analytical and, in one part, computational:

- a single argument linking chiplet partitioning and 3D-stacked cache as responses to two related scaling limits, die cost and last-level cache capacity;
- an explanation of both techniques in terms of the textbook's performance, power, thermal and memory-hierarchy models;

- a structured comparison of planar, side-by-side and vertically stacked cache expansion;
- a synthesis of AMD-reported and independent performance evidence for stacked cache across three Zen generations, graded by which confounding factors each comparison controls (table 26);
- an account of how AMD's move to an inverted stack in Zen 5 bears on the thermal cost of the first design;
- seven simulations, most changing one factor at a time, with the model, scripts and raw results on GitHub for repetition (`github.com/TheInvincible2111/ELL305-Term-Paper`; section 28 and appendix E);
- measurements on the group's own Ryzen 5 5600X, compared with the model on a machine it was not built from (section 28.2 and appendix G).

1.9 Main findings

The main findings, each developed in the part cited, are as follows.

- **Chiplets trade a small latency cost on every L3 miss for cost and flexibility.** Smaller dies yield far better, I/O moves to a cheaper node (section 6.2), and within each generation one compute die serves a wide range of products, from desktop parts to 64-core (Zen 3) or 96-core (Zen 4) servers (section 10.2). Every L3 miss pays a die-to-die round trip that AMD gives as under 9 ns, yet AMD reports that Rome's average memory latency was 19% lower than its predecessor's (section 8). The largest cost is in sharing: in the one measurement available, passing data between cores on different CCDs took about 4.5 times as long as within one (section 8.2). The power cost can only be estimated, at about 2% of a Genoa processor's power budget for memory traffic alone (section 10.1).
- **Stacking triples the L3 for about four cycles.** AMD's first cache die expands a Zen 3 CCD's L3 from 32 to 96 MB for about four extra cycles of latency, a cost measured in three generations by one independent outlet (Part IV, section 24.2).
- **The benefit depends on the working set.** At equal clock, the best-controlled study finds gains in instructions per cycle (IPC) of 4.9–19.7% in five workloads, a geometric mean of 11.4% by the authors' calculation, and a small loss in a sixth (section 24.3). Memory-bound computational fluid dynamics (OpenFOAM) gains up to 1.5–1.6× and one HPC proxy up to 3.4× at favourable problem sizes, while working sets that already fit gain nothing (Part V).
- **A removed miss is worth less than a memory access.** In the measured workloads each counted miss removed was worth an apparent 50 to 130 cycles, well below the memory latency. Overlap of misses by out-of-order cores and the counting of prefetches as misses are two candidate explanations; the sources do not separate them (sections 17.1 and 24.3).
- **Lower clocks offset part of the gain.** In the first two generations the V-Cache parts clocked lower. A simple model calibrated on published measurements shows that a 7% clock gap raises the number of misses the larger cache must remove to break even from about 0.1 to about 0.35 per thousand instructions. The simulation, built on the same calibration and so not an independent check, gives 0.38 at that gap (sections 27.2 and 28.5).
- **A simulation illustrates these mechanisms.** A trace-driven cache model, calibrated on the same measurements, reproduces the working-set window, the small latency cost and the

weight of the clock gap with every other factor fixed. It also shows which inputs matter: across the overlap factors fitted to Lam's workloads the break-even at a realistic clock gap changes about threefold, and the two link round trips of one transfer between CCDs explain only part of the measured cost of sharing (section 28). Its results are model outputs, not measurements.

- **The group's own measurements check the model's inputs and expose its limits.** On a Ryzen 5 5600X, the published Zen 3 L1 and L2 latencies the model uses agree with the measured ones, and its L3 and memory latencies are within about 20%. The core keeps at least sixteen misses in flight, enough for the overlap the calibration implies. Between the cache plateaus, where the model's own behaviour shows, its latency close to each cache's capacity is too low, by up to 2.4 times (at 32 MB, measured under the Windows Subsystem for Linux, WSL2; section 28.2).

1.10 Organisation of the paper

Part II reviews the architectural foundations. Part III analyses chiplet-based organisation. Part IV examines 3D-stacked cache and AMD 3D V-Cache. Part V presents the AMD case study and quantitative analysis, and Part VI compares the approaches, identifies limitations and outlines future directions. Self-contained derivations longer than about half a page appear as appendices. Figure 4 maps the whole paper.

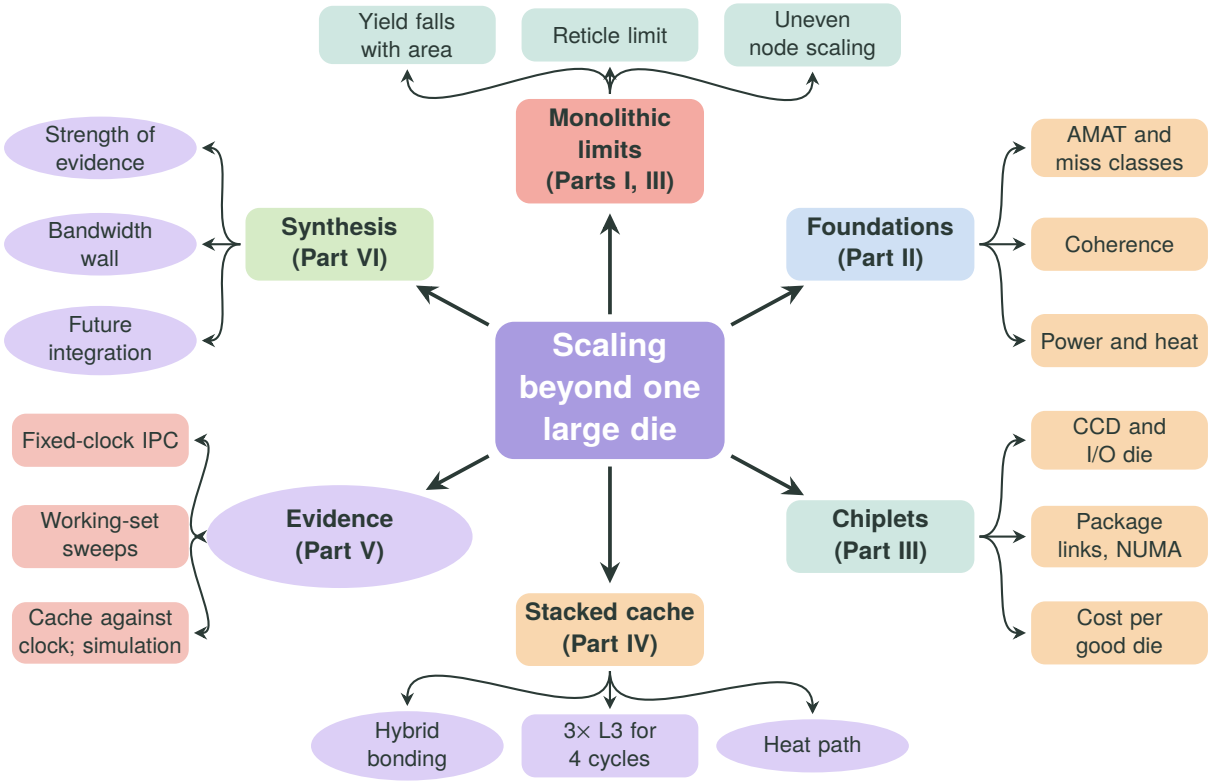


Figure 4: Overview of the paper: the problem at the centre, one branch per part, and the main idea each part contributes.

Part II

Architectural Foundations

Figure 5 maps the topics of this part.

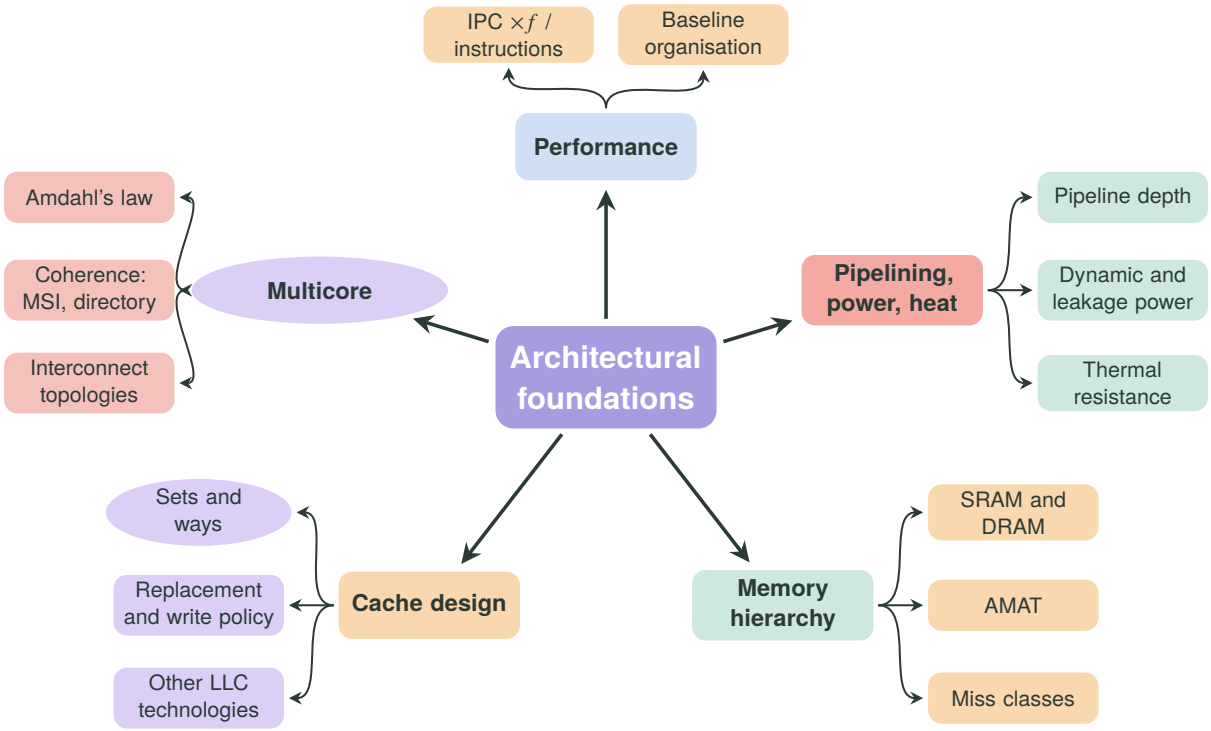


Figure 5: Overview of Part II: the textbook concepts that the later parts use to judge chiplets and stacked cache.

This part sets out the models that the rest of the paper uses to judge chiplets and stacked cache. It follows Chapters 7 and 9–12 of the course textbook [1]. The textbook supplies the theory; it does not describe modern compute-die/I/O-die partitioning or stacked cache, which later parts support with external sources.

2 Processor Organisation and Performance Metrics

2.1 The performance equation

The textbook defines the performance of a processor only with respect to a given program, as the reciprocal of its execution time [1, sec. 10.9.1]. Execution time τ can be split into three factors:

$$\tau = \frac{\#\text{instructions} \times \text{CPI}}{f}, \qquad P \propto \frac{\text{IPC} \times f}{\#\text{instructions}}, \tag{1}$$

where f is the clock frequency, CPI (cycles per instruction) is the average number of cycles per dynamic instruction, and IPC (instructions per cycle) is its reciprocal [1, sec. 10.9.1].

Equation (1) is the lens for the whole paper. A larger cache does not change the instruction count or, directly, the clock frequency; it acts through IPC by reducing the cycles spent waiting for memory. A chiplet organisation, by contrast, can change all three: it can enable more cores (more throughput), alter memory latency (IPC), and change the frequency and power each die can sustain. When published results compare two processors, they must therefore be checked for differences in f as well as in cache size (section 26).

2.2 Baseline processor organisation

The textbook builds its baseline processor, SimpleRisc, from five stages: instruction fetch (IF), operand fetch (OF), execute (EX), memory access (MA) and register writeback (RW) [1, sec. 9.9.1]. The *data path* holds and processes information (registers, memory elements, the ALU), while the *control path* generates the signals that move instructions and data through it; the control path can be a hardwired control unit or a microprogrammed one, which trades speed for flexibility [1, secs. 9.2–9.8].

Two features of this baseline matter for the rest of the paper. First, the MA stage is the processor's only contact with memory: every load and store passes through it, so the speed of the memory system directly sets how long that stage can take. Second, the organisation assumes a single processor on a single piece of silicon. Everything that follows, from pipelining to multicore processors and finally to chiplets, can be read as a series of departures from this baseline, each made to use a larger transistor budget without lengthening the clock cycle or exceeding the power budget.

3 Pipelining, Power and Thermal Limits

3.1 Pipelining and its limits

In the basic processor, most of the hardware is idle at any moment: when an instruction is in the MA stage, the IF stage has nothing to do. Pipelining overlaps instructions so that each of the five stages works on a different instruction in the same cycle, with pipeline registers between stages carrying each instruction's state forward [1, sec. 10.12.1]. Ideally this completes one instruction per cycle while allowing the clock frequency to rise roughly in proportion to the number of stages.

The ideal is broken by *hazards*. A data (read-after-write) hazard occurs when an instruction

needs a value that an earlier instruction has not yet produced; a control hazard occurs because the outcome of a branch is not known when the following instructions are fetched; a structural hazard occurs when two instructions need the same hardware unit [1, sec. 10.4]. Interlocks resolve hazards by stalling, and forwarding passes results directly between stages, which removes every data hazard except the load-use hazard [1, sec. 10.12.1]. The effect on performance is captured by

$$\text{CPI} = \text{CPI}_{\text{ideal}} + \text{stall rate} \times \text{stall penalty}, \tag{2}$$

so the CPI grows with every stall cycle. Deeper pipelines raise the frequency but also the stall penalty, which is why the textbook derives an optimal pipeline depth rather than an unlimited one [1, sec. 10.9.3].

The textbook's model of pipeline depth [1, sec. 10.9.3] finds an optimum of about 13 stages for typical values and draws three conclusions: programs with many dependencies favour shallower pipelines, better forwarding favours deeper ones, and the optimal depth has changed little over time because logic and latches have been getting faster at about the same rate. Deepening pipelines therefore has limits of its own, besides the power limit that ended the growth in clock frequency (section 1). In our reading, a design that reduces the stall rate, such as a larger cache that removes memory stalls, is a useful alternative to a higher clock.

Equation (2) is the link between pipelining and the memory hierarchy. A load that misses in the cache is a stall whose penalty is not one or two cycles but the miss penalty of equation (5), which can be tens or hundreds of cycles. Out-of-order processors soften this: they fetch and decode in order, rename registers to remove false dependencies, and issue any instruction whose operands are ready from an instruction window that typically holds 64–128 entries [1, sec. 10.11.4]. Independent work can therefore proceed while a load waits. But the window is finite, so a long miss to main memory eventually fills it and stalls the pipeline. Reducing the number of such misses, which is what a larger last-level cache does, therefore still matters even for aggressive out-of-order cores.

3.2 Dynamic and leakage power

The textbook models dynamic power, the energy dissipated when circuit nodes switch, as

$$P_{\text{dyn}} \propto \sum_i \alpha_i \, C_i \, V^2 f, \tag{3}$$

where α_i is the activity factor and C_i the capacitance of node i, V the supply voltage and f the frequency [1, sec. 10.10.2]. Leakage (static) power arises because transistors do not switch off perfectly; it is consumed all the time regardless of activity, and the textbook notes that it can account for roughly 20–40% of a high-performance processor's power budget [1, sec. 10.10.3].

Both terms matter for this paper:

- **Wires are capacitance.** Long on-die or die-to-die wires increase C in equation (3), so moving data farther costs energy as well as time. This is one reason the distance between cores and cache matters (section 14).
- **Large SRAM arrays leak.** A large cache adds leakage even when most of it is idle, which is part of the cost of simply growing the LLC on a planar die.

3.3 Temperature

Power that is dissipated becomes heat, which must flow from the transistors through the die, a thermal interface material and a heat spreader to the cooler [1, sec. 10.10.1] (figure 6). The textbook gives a sense of the scale: a 200–400 mm^2 die dissipating 60–100 W would heat up to around 200 °C without these measures, and even with them processors run at 60–100 °C, reaching 120 °C under heavy load [1, sec. 10.10.1].

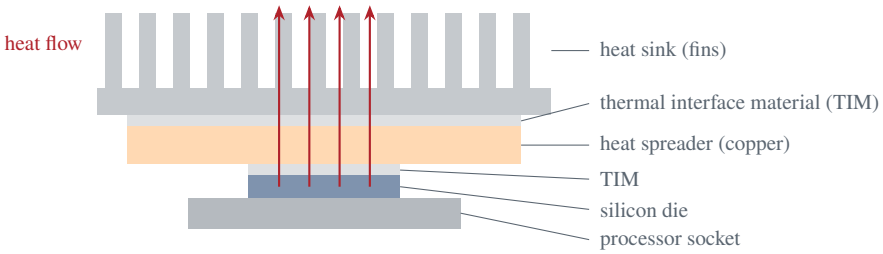


Figure 6: The heat path from a processor die to the air, redrawn after Sarangi [1, sec. 10.10.1, Fig. 10.36]. Each layer adds thermal resistance; stacking a second die on the processor adds another layer between the transistors and the spreader (section 18).

Higher temperature raises leakage, which raises power further, and accelerates ageing [1, sec. 10.10.1]. Stacking a die on top of a compute die changes this heat path, which is the main thermal question raised in section 18.

Heat is not produced evenly. The textbook notes that the ALUs dissipate a lot of heat while memory elements stay relatively cool, and that the heat spreader exists to even out the resulting hot spots [1, sec. 10.10.1]. It models the steady-state temperature by dividing the die into a grid of m points:

$$T - T_{\text{amb}} \;=\; \Delta T \;=\; A \times P_{\text{tot}}, \tag{4}$$

where P_{tot} is the vector of power dissipated at each grid point, T_{amb} the ambient temperature and A the $m \times m$ *thermal resistance matrix* [1, sec. 10.10.4]. Because $P_{\text{tot}} = P_{\text{dyn}} + P_{\text{leak}}$ and leakage itself grows with temperature, equation (4) and the leakage model form a feedback loop that must be iterated until it converges [1, sec. 10.10.4]. Two consequences follow for stacking. Placing an extra layer over a hot region increases the entries of A for that region, so the same power produces a higher temperature. A cool structure such as a cache array is the cheaper thing to put in the heat path. 3D V-Cache follows both rules: its cache die sits over the CCD's L3, and only structural silicon covers the cores (section 18).

4 The Memory Hierarchy

4.1 Why a hierarchy is needed

The textbook frames memory design as a conflict between capacity and speed. Per unit of silicon, DRAM stores far more data than SRAM but is much slower; a memory built entirely of DRAM would make every load and store wait for main memory, while a memory built entirely of fast on-chip SRAM cannot hold a program's data [1, sec. 11.1.1].

The way out is *locality*: programs tend to reuse recently used data (temporal locality) and to access data near recently used data (spatial locality) [1, sec. 11.1.2–11.1.6]. A memory hierarchy keeps recently used blocks in small, fast caches close to the core and backs them with progressively larger, slower levels (figure 7).

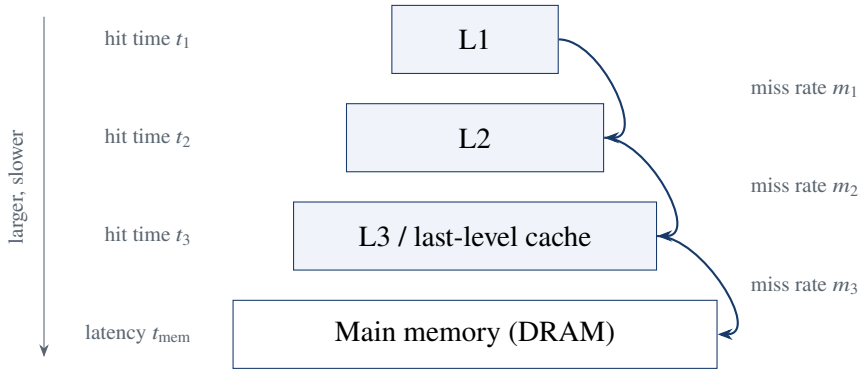


Figure 7: A multi-level cache hierarchy with the quantities used in the recursive AMAT expression (equation (5)). Capacity grows and speed falls moving down the hierarchy.

4.2 SRAM and DRAM arrays

Caches are built from SRAM. A standard SRAM cell uses six transistors: two cross-coupled inverters hold the bit, and two access transistors, switched by a shared *word line*, connect it to a pair of complementary *bit lines* [1, sec. 7.4.1]. Cells are laid out as a two-dimensional array: a decoder raises one word line, every cell in that row drives its bit lines, and a column multiplexer selects the bits of interest [1, sec. 7.4.1]. Because hundreds of cells share a bit line, the bit line has a high capacitance and would be slow to charge fully. Reads therefore pre-charge both bit lines to half the supply voltage and let *sense amplifiers* detect a small voltage difference [1, sec. 7.4.1]. The textbook also notes that as an array grows it must be kept roughly square, otherwise the capacitive load on its word lines or bit lines becomes prohibitive [1, sec. 7.4.1]. Large caches are therefore built as many smaller arrays and banks (section 5), connected by wires whose length grows with total capacity. Figure 8 shows the cell and the array.

Cell size sets the density gap between SRAM and DRAM. Using 2012-era figures, the textbook gives an SRAM cell an area of about $0.08\,\mu m^2$ and an array access time of 1–5 cycles, against about $0.005\,\mu m^2$ and 50–200 cycles for a DRAM cell [1, sec. 11.1.1, Table 11.1]. SRAM is thus roughly sixteen times larger per bit but far faster, which is why it is used for on-chip caches and why a cache of tens of megabytes takes a large share of a die (section 14).

Main memory is built from DRAM instead (figure 9). A DRAM cell stores a bit as charge on a capacitor, reached through a single access transistor that the word line switches on, so it needs one transistor where SRAM needs six [1, sec. 7.4.3]. The charge leaks away slowly, and reading the cell also drains some of it onto the bit line, so every cell must be rewritten after each read and refreshed periodically. A DRAM array therefore has a refresh circuit on its bit lines, a single bit line per column, and sense amplifiers placed before the column multiplexer, where they keep the whole open row (a DRAM *page*) so that further accesses to that row are served quickly [1, sec. 7.4.3]. The density is what makes DRAM suitable for large off-chip memories, and the extra steps are part of why it is so much slower.

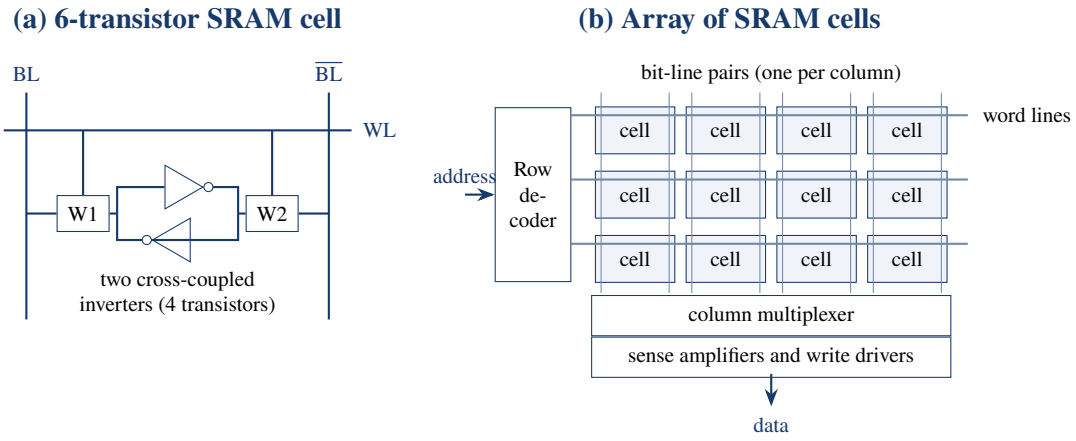


Figure 8: (a) A six-transistor SRAM cell: the word line (WL) turns on the access transistors W1 and W2, connecting the cross-coupled inverters to the complementary bit lines. (b) Cells are arranged in a two-dimensional array; a decoder raises one word line, every cell in that row drives its bit lines, and a column multiplexer and sense amplifiers select and read the bits required. Redrawn after Sarangi [1, sec. 7.4.1, Figs. 7.26–7.27].

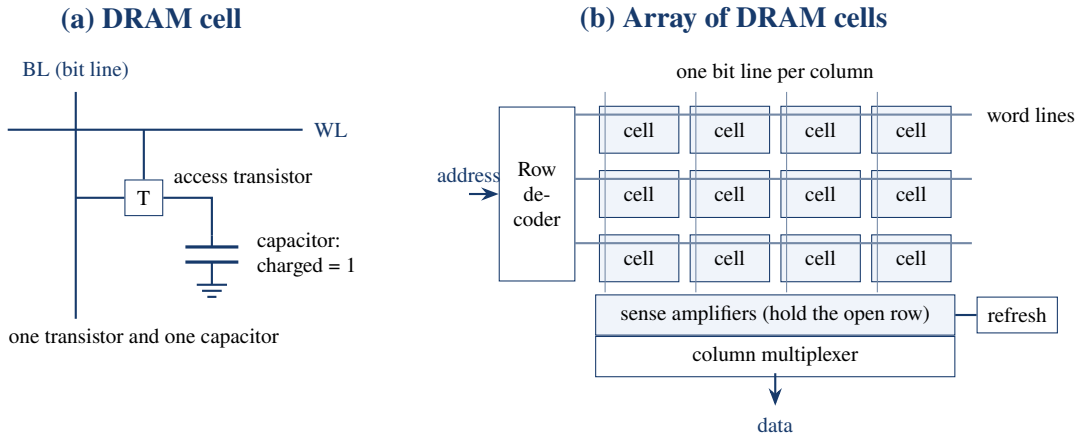


Figure 9: (a) A DRAM cell: one access transistor, switched by the word line, connects a capacitor to a single bit line; a charged capacitor stores a 1. (b) Compared with the SRAM array of figure 8, a DRAM array has one bit line per column instead of two, a refresh circuit that writes the data back after every read and periodically, and sense amplifiers placed before the column multiplexer, where they hold the whole open row so that further accesses to it are fast. Redrawn after Sarangi [1, sec. 7.4.3, Figs. 7.30–7.31].

4.3 Average memory access time

The average memory access time (AMAT (average memory access time)) of a hierarchy is defined recursively [1, sec. 11.3.1]:

$$\begin{aligned} \text{AMAT} &= t_1 + m_1 \times \text{penalty}_1, \\ \text{penalty}_k &= t_{k+1} + m_{k+1} \times \text{penalty}_{k+1}, \end{aligned} \tag{5}$$

where t_k is the hit time and m_k the local miss rate of level k, and the penalty of the last cache level is the main-memory latency. Because the lower levels are weighted by the miss rates above them, a long memory latency adds little to the average. With the latencies measured on a Zen 2 and a Zen 3 desktop part [17] and illustrative local miss rates of 5, 30 and 40% for L1, L2 and L3,

the L3 and a 60–70 ns memory together add only about half a nanosecond to an average access of about 1.5 ns. Each load that does miss still stalls for the full latency, which an out-of-order core can only partly overlap.

Equation (5) shows precisely what a larger last-level cache can and cannot do:

- it can lower m_{LLC}, so fewer accesses pay the main-memory latency;
- but if making the cache larger also lengthens its hit time t_{LLC}, part of the gain is lost on every LLC hit.

The design problem for a large LLC is therefore to add capacity while keeping the added hit latency small. That is the problem 3D stacking attacks (Part IV).

4.4 Classifying cache misses

The textbook divides misses into three classes [1, sec. 11.3.2]:

- **Compulsory (cold) misses** occur the first time a block is accessed.
- **Capacity misses** occur when the program's working set—the blocks it uses over a typical interval—is larger than the cache.
- **Conflict misses** occur in direct-mapped and set-associative caches when more blocks map to one set than the set can hold.

A larger LLC primarily removes capacity misses. It therefore helps workloads whose working set exceeds the original cache but fits in the larger one, and helps little when the working set either already fits or is far larger than either size. This simple argument predicts the strongly workload-dependent results examined in Part V.

4.5 Cache organisation and policies

A cache stores fixed-size *blocks* (typically 32–128 bytes) and records which memory addresses it holds in a *tag array* beside the *data array*. An address is divided into a tag, an index and an offset within the block. In a *fully associative* cache a block may be placed anywhere, so every tag must be compared on each access; this is practical only for small structures because of its area and power. A *direct-mapped* cache allows each block exactly one location, selected by the index bits, which is simple and fast but suffers conflict misses. A *set-associative* cache is the compromise used almost universally: the index selects a set of K locations (the associativity, typically 2, 4 or 8 in the textbook's examples), and the tags of that set are compared in parallel [1, sec. 11.2.2]. AMD's Zen 3 L3, for comparison, is 16-way set associative both with and without V-Cache [3, slides 19–20]. Figure 10 shows the lookup.

Tags in a large cache. The index and offset bits are fixed by the number of sets and the block size, so the tag takes the remaining address bits [1, sec. 11.2.2]. With Zen 3's 64-byte lines and a 48-bit physical address [18, p. 10], the 16-way 32 MB L3 [3, slide 19] still stores a 27-bit tag per line, so tag storage grows almost in proportion to capacity (section 17.2). The 8-way 32 KB L1 uses only the low 12 address bits for its index and offset, the offset within a 4 KB page, which lets it be indexed while the address is being translated (section 4.7).

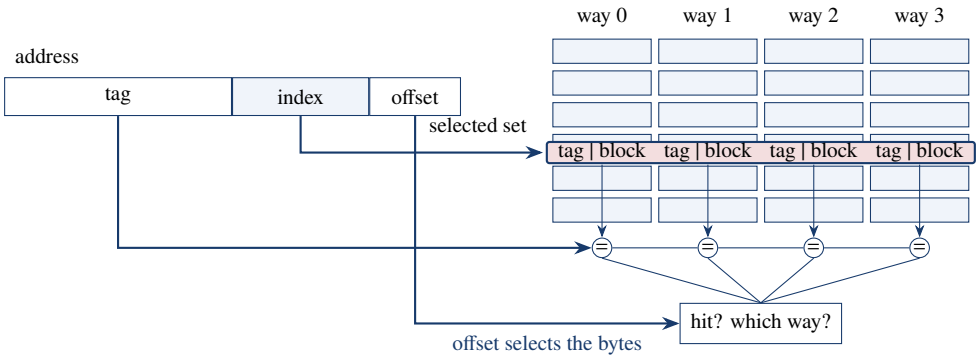


Figure 10: Lookup in a 4-way set-associative cache. The index bits select one set; the tags stored in all four ways of that set are compared with the address tag in parallel; a match identifies the way holding the block, and the offset selects the bytes within it. After Sarangi [1, sec. 11.2.2].

Organisation can also be matched to how a program walks through its data. Wang, for example, proposes an L1 cache for two-dimensional data, such as matrices and images, that stores data in a hierarchical Z-order layout across multiple banks so that either a square tile or a row can be read in one access. In matrix-multiplication simulations this reduced both L1 cache misses and misses in the translation lookaside buffer (TLB, section 4.7) compared with a conventional row-by-row layout [19]. The example is far from AMD's last-level cache, but it illustrates the same principle from the other direction: instead of adding capacity to hold a working set, it rearranges data so that the existing capacity is used better.

When a set is full, a *replacement policy* chooses a victim: random replacement is simplest, FIFO evicts the oldest block, and least-recently-used (LRU), which follows directly from the stack-distance view of temporal locality, is among the most effective [1, sec. 11.2.5]. Writes are handled either *write-through*, sending every write to the next level, or *write-back*, marking a block as modified and writing it back only on eviction, which reduces traffic at the cost of tracking state [1, sec. 11.2.3]. The stacked V-Cache keeps this organisation intact: each die's portion of the L3 has its own data, tag and LRU arrays [3, slide 20].

The textbook summarises the main cache optimisations and their side effects; table 3 reproduces the entries most relevant to this paper. The table makes the paper's central tension explicit: the standard remedy for capacity misses is a larger cache, and its listed costs are higher latency, higher power and more area.

Figure 11 arranges the same techniques around the terms of equation (5) that they target.

3D V-Cache can be read as an attempt to take the third row of table 3 while avoiding most of its disadvantages: the area cost moves to a separate die, and the latency cost is held to four cycles (section 17).

4.6 Writes and inclusion

Before a store can write, the whole block must be in the cache, so even a one-byte store first brings in the entire block. A write-back cache then updates only its own copy and writes the block down when it is evicted, so writes are cheap and evictions carry the traffic [1, sec. 11.2.3].

Write-back is the usual choice at the larger levels, because the more often a block is rewritten

Table 3: Selected memory-system optimisations and their costs, after Sarangi [1, Table 11.2].

Technique	Targets	Disadvantages
Larger block size	compulsory misses	fewer blocks in the cache
Prefetching	compulsory and capacity misses	complexity; may displace useful data
Larger cache	**capacity misses**	**higher latency, higher power, more area**
Higher associativity	conflict misses	higher latency, higher power
Victim cache	conflict misses	extra complexity
Small, simple cache	hit time	higher miss rate
Write buffer, critical word first	miss penalty	extra complexity

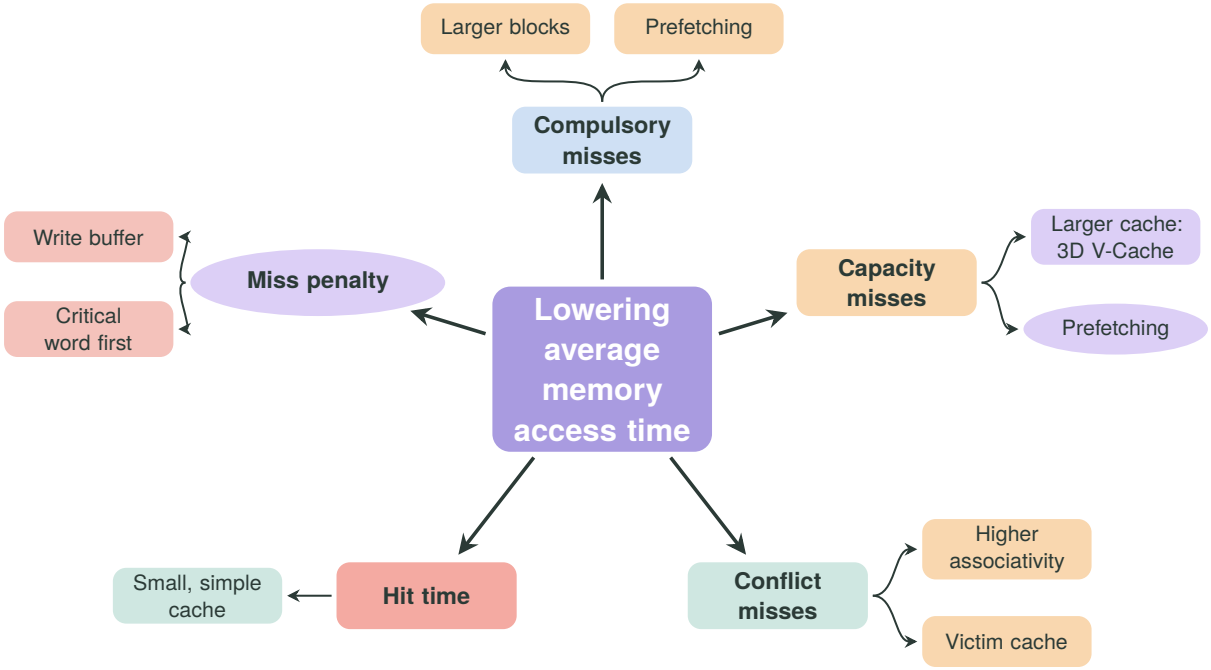


Figure 11: Overview of table 3: the three miss classes and the AMAT terms, and the techniques that target each [1, Table 11.2]. The stacked cache of Part IV is a way of making the cache larger, so it attacks capacity misses, and it pays the price the table lists for that technique: higher latency, power and area.

before eviction, the more traffic it saves. The same reasoning applies at the bottom of the hierarchy. In a chiplet processor, the memory controllers sit on the I/O die (section 8), so modified blocks written back from a compute die's L3 must cross the die-to-die link to reach memory. A larger L3 that keeps blocks longer before eviction can therefore reduce write traffic on that link as well as read misses.

Write buffers and critical-word-first fetching shorten the miss penalty rather than the miss rate, and complement a larger cache rather than replace it [1, sec. 11.3.3].

Inclusion

A hierarchy is *inclusive* if every level holds a subset of the blocks in the level below it, and *exclusive* if it does not follow strict inclusion [1, sec. 11.1.7, Definition 100]. The textbook calls inclusive caches the convention, because they are simpler to design and avoid some correctness problems in multiprocessors, but notes that some advanced processors (as of 2025) use exclusive caches where they benefit from more flexible data placement [1, sec. 11.1.7]. Inclusion has a direct consequence for capacity. In an inclusive hierarchy the caches add no storage, since each holds only copies of data in the level below; the total size of the memory system is that of main memory [1, sec. 11.3]. The same holds between caches: an inclusive L3 keeps a copy of every block in the L2s above it, so the number of distinct blocks a die holds on chip is set by the L3 alone. An exclusive L3 adds its capacity to that of the L2s. A second consequence follows from the definition. In an inclusive hierarchy, a block that is not in the shared level cannot be in any private cache above it, so one lookup in the shared level answers for the whole group of cores.

Table 4 applies this to a Zen 3 core complex (CCX), the group of cores that shares one L3, whose eight cores have up to 4 MB of L2 between them and share a 32 MB L3, or 96 MB with 3D V-Cache [11, sec. 2.2]. The sources used in this paper do not state which inclusion policy Zen 3's L3 follows, so the table gives both cases.

Table 4: Distinct on-chip capacity of one Zen 3 CCX under the two inclusion policies (8 cores × 512 KB L2 [11, sec. 2.2]).

L3	Inclusive L3	Exclusive L3	Difference
32 MB (planar)	32 MB	36 MB	12.5%
96 MB (V-Cache)	96 MB	100 MB	4.2%
Ratio, V-Cache to planar	3.0×	2.8×	

The policy matters less as the L3 grows. With the cache die, the L2s are a small fraction of the total, so V-Cache roughly triples the distinct capacity of a CCX under either policy. Comparisons that hold the policy fixed, as all the comparisons in Part V do because the plain and V-Cache parts share one core design, are not affected by it.

4.7 Virtual memory and address translation

Programs use *virtual* addresses, which are translated page by page to *physical* addresses through a per-process *page table* [1, sec. 11.4.3]. With 4 KB pages, the low 12 bits of an address are the offset within the page and pass through translation unchanged; only the page number is replaced. Recent translations are cached in a small TLB, and a TLB miss stalls the pipeline for the tens of cycles needed to read the page table [1, sec. 11.4.3].

TLB reach. The memory a TLB can translate without a miss, its *reach*, is its number of entries times the page size. With 4 KB pages, Zen 3's 64-entry first-level and 2048-entry second-level data TLBs [20] reach 256 KB and 8 MB, a quarter of a 32 MB L3 and a twelfth of a 96 MB stacked L3; with 2 MB pages the first level alone covers 128 MB, more than either L3 (appendix F.2). A program that touches a large working set at random with 4 KB pages can therefore hit in the L3 and still miss in both TLBs. The group's measurement on a Ryzen 5 5600X agrees (appendix G):

with 4 KB pages latency starts to rise just above 256 KB and again from about 7 MB, and at 1 GB address translation adds about 17 ns to every access, a figure that the nested translation of the virtual machine (the Windows Subsystem for Linux, WSL2) probably inflates.

Three points follow for the rest of the paper. First, latency sweeps with 4 KB pages measure cache and TLB effects together, which Part V has to allow for when it reads the steps in a latency curve (section 24.2). Second, for a program using 4 KB pages, the benefit of a larger L3 is the benefit to the whole memory system, including any translation cost the L3 does not remove. Third, the observation in section 4.5 that the L1 index and offset use only the low 12 bits matters here: those bits are the page offset, which translation does not change, so the L1 lookup can start before the TLB has finished.

4.8 Overlapping misses and multithreading

The textbook's memory-stall model, CPI = $\mathrm{CPI}_{\mathrm{ideal}} + f_{\mathrm{mem}} \times (\mathrm{AMAT} - 1)$, where f_{mem} is the fraction of instructions that access memory, assumes that the pipeline stalls for every cycle of every access beyond the first [1, sec. 11.3.1]. That is exact for the textbook's five-stage in-order pipeline, but it overstates the stall of an out-of-order core. Such a core keeps issuing independent instructions from its window while a load waits (section 3), and those instructions can include other loads, so several misses can be outstanding at once. If n independent misses are in flight together, the core waits about one memory latency for all of them, so each costs roughly t_{mem}/n. This overlap is usually called *memory-level parallelism*. A simple way to express it is a hidden fraction h (the model of section 28 uses its complement, the exposed fraction $\omega = 1 - h$, which it calls the overlap factor):

$$\text{stall cycles per thousand instructions} \;\approx\; \mathrm{MPKI} \times (1 - h) \times t_{\mathrm{mem}}, \tag{6}$$

where MPKI is the number of misses to memory per thousand instructions and t_{mem} the memory latency in cycles. With $h = 0$ this is the textbook's model; with independent misses, h rises towards $1 - 1/n$.

For the textbook's memory latency of 100–300 cycles [1, sec. 12.4.8], a core that keeps two independent misses in flight ($h = 0.5$) waits 50–150 cycles per miss, and one that keeps four ($h = 0.75$) waits 25–75. Removing a miss saves only what it would have cost, so these values bound what a larger cache can gain per removed miss; Part IV compares them with what a measurement implies (section 17.1).

A Zen 3 measurement shows the same effect directly for TLB misses. 7-cpu.com reports a first-level TLB miss penalty of 7 cycles when misses occur one at a time, but only 1.7 cycles per access when independent misses overlap; for the second-level TLB the figures are 24 and 5 cycles [20]. The hardware thus overlaps the equivalent of about four to five TLB misses.

The overlap explains a pattern that recurs in Parts IV and V. A latency sweep walks a chain of pointers in which each load's address depends on the previous one, so no two misses can overlap and the sweep sees the full latency of every level: a nearly five-fold difference between Milan and Milan-X at 64 MB (section 24.2). Real programs usually have some independent misses, so removing a miss saves less, and whole-program gains are measured in per cent rather than multiples. Programs that chase pointers, such as graph traversals, are closer to the latency sweep and should gain more per removed miss; programs that stream through memory overlap their

misses well and gain less.

A second way to hide memory latency is to give the pipeline other threads to run. The textbook describes three forms of *multithreading* [1, sec. 12.4.8]. A *coarse-grained* multithreaded processor keeps the context of several threads in the pipeline and switches among them every n cycles or on a long-latency event such as an L2 miss, so while one memory-intensive thread waits 100–300 cycles for memory, another can do useful work. *Fine-grained* multithreading switches every one or two cycles. *Simultaneous multithreading* (SMT), the form used in modern multiple-issue cores, shares the issue slots of each cycle dynamically among the threads, so a stall in one thread is filled with instructions from the others. The textbook lists its costs: a more complex fetch stage, harder coherence and consistency, and a higher memory-bandwidth requirement than an equivalent single-threaded core [1, sec. 12.4.8]. The threads also split the core's resources between them [1, sec. 12.4.8, Important Point 20], including its caches.

Zen 3 supports SMT, so one CCX runs up to 16 hardware threads: each core's two threads share its 512 KB L2, and all sixteen share the 32 MB L3, or 96 MB with V-Cache [11, sec. 2.2]. With every thread busy, that is $32/16 = 2$ MB of L3 per thread on a planar CCD and $96/16 = 6$ MB on a stacked one. SMT and a larger cache therefore interact. SMT hides latency but divides capacity and raises the demand for bandwidth; a larger L3 restores capacity per thread and removes some of the extra misses. Part VI returns to the bandwidth side of this interaction (section 34).

4.9 Beyond SRAM: alternative LLC technologies

Stacking SRAM is not the only way to obtain a larger last-level cache. Denser technologies such as stacked DRAM [13] and STT-RAM [21] change the *technology* of the cache, whereas AMD's 3D V-Cache keeps SRAM and changes its *placement*; section 31 compares them with the figures the sources report.

5 Multicore Processors, Coherence and Interconnects

5.1 From one core to many

The textbook traces the move to multicore processors from the mid-2000s as transistor budgets continued to grow while single-core power and complexity limits tightened [1, sec. 12.1]. Multicore processors raise two questions that matter directly for chiplets: how cores keep a consistent view of shared data, and how requests travel between cores, caches and memory.

5.2 How far parallelism goes: Amdahl's law

More cores help only to the extent that a program can use them. If a fraction f_{seq} of a program's running time is sequential and the rest divides perfectly among P processors, the speedup is

$$S = \frac{1}{f_{\text{seq}} + (1 - f_{\text{seq}})/P}, \tag{7}$$

which approaches $1/f_{\text{seq}}$ however many processors are added [1, sec. 12.2.3]. With $f_{\text{seq}} = 5\%$ the limit is 20, and the textbook notes that there is then no appreciable difference between 35 and 200 processors [1, sec. 12.2.3].

Amdahl's law has two consequences for this paper. First, a 64-core chiplet processor does not mainly serve one parallel program: its typical use is to run many unrelated programs or requests at once, which the textbook calls loosely coupled multiprocessing and which scales almost perfectly because the programs barely interact [1, sec. 12.2.1]. That is the market the chiplet organisation serves. Second, for any single program the sequential part still runs on one core, so the performance of each core, set by its clock, its IPC and the cache it can reach, still decides the limit. Stacked cache improves exactly that per-core performance, which is why it complements adding cores.

5.3 Cache coherence

When several caches can hold copies of the same address, cache coherence rules ensure that all threads observe writes to that address in a consistent order [1, sec. 12.4.2]. In a chiplet processor, a coherence transaction between cores on different compute dies must cross the die-to-die interconnect, so the physical partitioning becomes visible as latency (section 8).

A multicore memory system can use *shared* caches, which every core accesses, or *private* caches, each serving one core or a small group of cores. The textbook compares them directly (table 5): private caches are small, fast and close to their cores but must be kept coherent and replicate shared data, while shared caches avoid replication and coherence traffic but are large, slow and far from most cores [1, sec. 12.4.4]. Its conclusion is that the first levels should be private and the lower, larger levels shared.

Table 5: Private versus shared caches, after Sarangi [1, Table 12.3].

Attribute	Private cache	Shared cache
Area	low	high
Speed	fast	slow
Proximity to the processor	near	far
Scalability in size	low	high
Data replication	yes	no
Complexity	high (needs coherence)	low

A large shared cache is not built as one monolithic array. To serve many cores at once, it is split into *banks*, each a smaller cache in its own right selected by some of the address bits; banking reduces contention and makes each bank faster and more power-efficient [1, sec. 12.4.5].

Private caches are made to look like one shared cache by a coherence protocol. The textbook states the requirement as two axioms: *completion*, every write must eventually complete, and *order*, all threads must see the writes to one address in the same order [1, sec. 12.4.2].

In a *snooping* protocol the caches share a bus and broadcast read misses, write misses and writes; each block is tracked in a small state machine with the states Modified, Shared and Invalid [1, sec. 12.4.6]. Because the bus carries one message at a time, every cache sees the same order

of messages, which satisfies the order axiom. The textbook develops two such protocols. In *write-update*, every write is broadcast so that all copies are updated. In *write-invalidate*, a cache that wants to write first invalidates all other copies; a block may then be shared by many readers or owned by a single writer, but never both, and writes to an owned block need no messages at all [1, sec. 12.4.6]. Figure 12 shows the write-invalidate state machine.

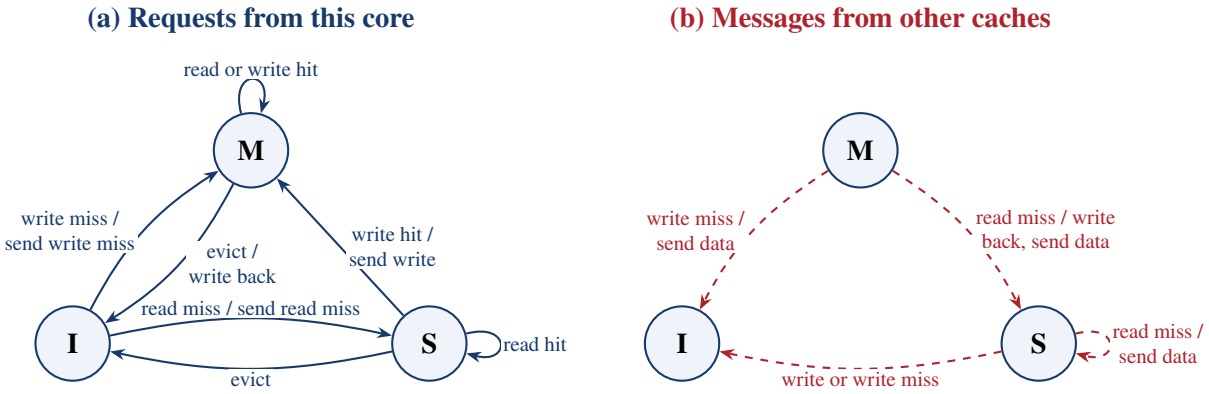


Figure 12: The write-invalidate (MSI) snooping protocol for one cache block, redrawn after Sarangi [1, sec. 12.4.6, Figs. 12.13–12.14]. (a) Transitions caused by this cache's own processor; (b) transitions caused by messages that other caches place on the bus. A block may be held by many caches in S, but by only one in M.

Table 6 traces a write-invalidate protocol through five accesses by two cores, A and B, to the same block, using the transitions of figure 12. Every step generates a bus message, and steps 2 and 4 also move the block's data between the caches.

Table 6: Block states and bus traffic for two cores sharing one block (write-invalidate MSI).

	Access	A	B	Bus message and data movement
1	A reads x	I → S	I	read miss; data from the lower level
2	B reads x	S	I → S	read miss; A or the lower level supplies the data
3	A writes x	S → M	S → I	write; B invalidates its copy
4	B reads x	M → S	I → S	read miss; A writes the block back and sends it to B
5	B writes x	S → I	S → M	write; A invalidates its copy

On a chiplet processor, every message and every data transfer in the table must travel between the two caches. If A and B are on the same Zen CCD, the transfer stays on the die; if they are on different CCDs, each one crosses the die-to-die links through the I/O die (section 8). A block that is written alternately by two cores also ping-pongs between them, so the cost of the hop is paid repeatedly. Software that places communicating threads on the same CCD avoids this, which is one reason EPYC processors expose their CCD structure to the operating system (section 8). Broadcasting does not scale, so larger systems use a *directory* that records, for each block, which caches hold a copy, and sends messages only to those sharers [1, sec. 12.4.6].

The textbook's directory keeps the sharers as a bit vector with one bit per cache [1, sec. 12.4.6], which adds $N/512$ to every tracked 64-byte block for N caches (12.5% for 64); coherence tracking therefore grows harder as more caches, and more dies, share one memory. AMD's implementation is not described in the sources used here.

What a directory buys. Appendix F.1 repeats the five accesses of table 6 with a directory instead of a bus, under counting assumptions of the authors' own. The directory sends 18 messages whatever the number of caches, because only two caches ever hold the block, whereas a bus delivers each of the five steps to every other cache: $5(N-1)$ deliveries, 35 for eight caches and 315 for 64. The price is indirection: each step after the first passes through the directory and takes two sequential round trips instead of one bus transaction. A directory therefore trades latency for bandwidth, which is the right trade when there are many caches and few of them share any one block.

In a chiplet processor the indirection has a physical cost. If the requester, the directory and the owner sat on different dies, each arrow in table 48 (appendix F.1) would cross a die-to-die link, and a four-message step would take two link round trips. The sources used here do not say where AMD keeps its coherence directory, so this is a statement about the textbook scheme. It is still a useful bound: section 8 gives the cost of one such round trip on Rome, and Part III returns to what coherence across dies costs in practice.

AMD's Zen designs combine both ideas at different scales. Within a Zen 3 CCD, eight cores share one L3; across CCDs, each L3 is private to its die, and the dies communicate through the I/O die over Infinity Fabric [11]. EPYC 7003 can even expose each CCD's L3 as a separate non-uniform memory access (NUMA) domain to software [11]. The textbook's trade-off therefore reappears one level up: the per-CCD L3 is "private" with respect to the whole socket, which keeps it close and fast but means that data shared across CCDs must be kept coherent over a die-to-die link (section 8).

5.4 On-chip and on-package interconnection networks

On a single die, cores, cache banks and memory controllers are usually arranged as tiles, each with a *router*; the routers and the links between them form a network-on-chip (NoC) that carries coherence messages, LLC requests and responses, and memory traffic [1, sec. 12.6.1]. Two figures of merit describe a topology: the *bisection bandwidth*, the minimum number of links that must be cut to split the network into two equal halves, which measures both reliability and aggregate bandwidth; and the *diameter*, the longest of all shortest paths between nodes, which bounds latency [1, sec. 12.6.2]. Table 7 lists the textbook's values.

At eight nodes the ring is perfectly adequate: no node is more than four hops away. At 64 nodes the same ring would put cores up to 32 hops apart with a bisection of only 2, which is why large monolithic processors use meshes. AMD's chiplet designs avoid the problem by never building a 64-node network on one die. Within each CCD, a ring connects the eight cores to the L3 slices [22]; across the package, the CCDs form a star around the I/O die, in which any two CCDs are two links apart (figure 13). Chips and Cheese attributes AMD's L3 latency, 24–28 cycles lower on average than that of Intel's mesh-based server processors in its tests, to this choice: a ring has lower latency and higher bandwidth, while a mesh scales better for a large monolithic die [23]. The cost of the star is that its hub carries all traffic between dies, which is the bandwidth limit examined in section 8.3.

What changes when a link leaves the die is the subject of Part III. On a die, a link is a set of short on-chip wires; between chiplets, it is a serialised link routed through the package, with its own transmitters, receivers and clocking. AMD's Rome processors connect every CCD to a single central I/O die, a star-like arrangement, which AMD credits with reducing the number of

Table 7: Diameter and bisection bandwidth of common topologies with N nodes, from Sarangi [1, sec. 12.6.3], with values for $N = 8$ (the CCDs of a Rome or Milan package, or the cores of a Zen 3 CCD) and $N = 64$ (the cores of the full processor).

	Formula		$N = 8$		$N = 64$	
Topology	**Diameter**	**Bisection**	**Diam.**	**Bisec.**	**Diam.**	**Bisec.**
Chain	$N - 1$	1	7	1	63	1
Ring	$N/2$	2	4	2	32	2
Fat tree	$2 \log N$	$N/2$	6	4	12	32
Mesh	$2\sqrt{N} - 2$	$\sqrt{N}$	—	—	14	8
Torus	$\sqrt{N}$	$2\sqrt{N}$	—	—	8	16
Hypercube	$\log N$	$N/2$	3	4	6	32
Butterfly	$\log N + 1$	$N/2$	4	4	7	32

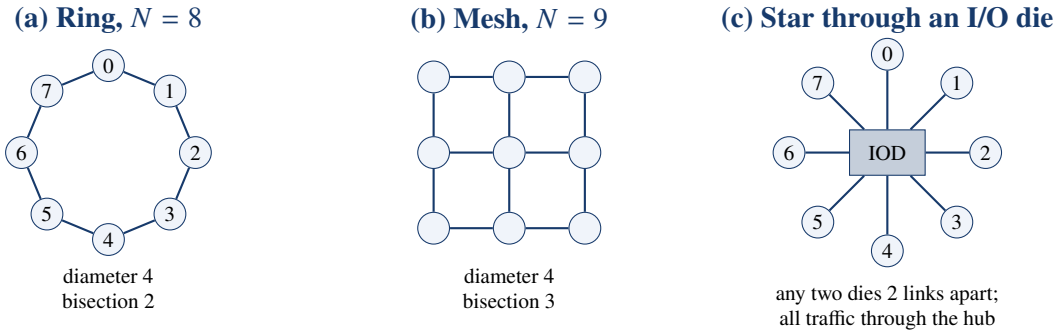


Figure 13: Three topologies. The ring and mesh follow Sarangi's definitions [1, sec. 12.6.3]; the star is the arrangement of Rome's eight CCDs around one I/O die (section 7). Within a CCD, Zen 3 and Zen 4 connect cores to L3 slices by a ring [22], [23].

NUMA domains and the distances to memory compared with the previous generation [2].

Part III

Chiplet-Based Processor Architecture

Figure 14 maps the topics of this part.

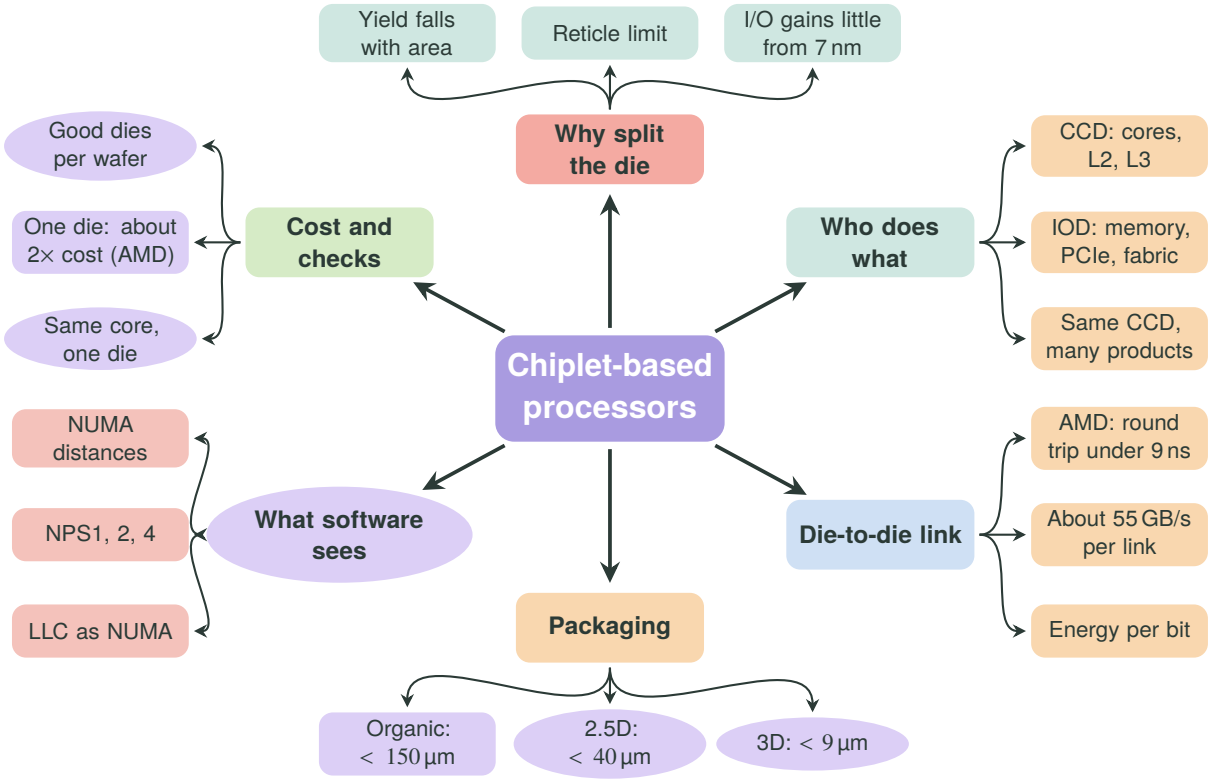


Figure 14: Overview of Part III. The figures are those discussed in the text: link latency and bandwidth for Rome (section 8), interconnect pitches from AMD's packaging talk (table 13), AMD's cost estimate (figure 25) and the Cezanne comparison (section 11).

6 Why Large Monolithic Dies Become Costly

6.1 The reticle limit and die size

A single die cannot be larger than the area a lithography tool can expose in one field. AMD's Hot Chips 33 presentation plots server CPU and GPU die sizes growing towards this reticle limit over roughly the last fifteen years [6]. For its second-generation EPYC ("Rome") processor, AMD states that the 64-core top configuration would not be practical as a monolithic chip and would lie well beyond the maximum reticle size [2]. The numbers show by how much (figure 15).

AMD's chart of die sizes draws the reticle limit at about 800 mm^2 (read from the chart) [6, slide 3]. Rome's eight 74 mm^2 CCDs and its 416 mm^2 I/O die add up to 1008 mm^2 [2], the "about 1000 mm^2" AMD quotes [4, slide 19]. Built as one die with the same areas, the 64-core part would therefore be about a quarter larger than the reticle allows, and even its compute alone, $8 \times 74 = 592$ mm^2, would be about three quarters of the limit.

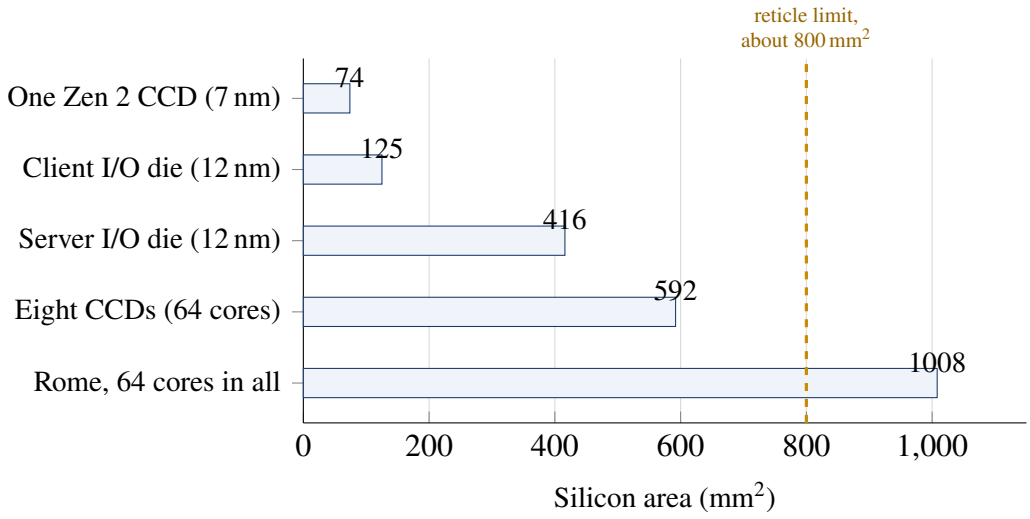


Figure 15: The 64-core Rome against the reticle limit. Die areas are AMD's [2]; together they make the "about 1000 mm^2" AMD gives for Rome [4, slide 19]. The reticle limit is the line drawn in AMD's chart of server CPU and GPU die sizes [6, slide 3], read from its logarithmic axis as about 800 mm^2 (approximate). A single die holding all of Rome would exceed it by about a quarter, and even the eight CCDs' 592 mm^2 of compute alone would be a very large die.

6.2 Manufacturing yield

Manufacturing defects are scattered across a wafer, so a larger die is more likely to contain one. The simplest model assumes that defects fall independently with a fixed average density D_0, which gives a Poisson yield e^{-AD_0} for a die of area A. Real defect densities vary across a wafer and from wafer to wafer, and the Poisson model ignores this. This paper therefore uses the yield model adopted in the 3D-IC cost analysis of Dong and Xie [15, Eq. 15]:

$$Y_{\text{die}} = Y_{\text{wafer}} \cdot \frac{1 - e^{-2AD_0}}{2AD_0}, \tag{8}$$

where A is the die area, D_0 the average defect density and Y_{wafer} the wafer yield. This is one of the yield functions derived by Murphy, who averaged the Poisson yield over a defect density that varies from chip to chip; this form assumes the density is spread uniformly between 0 and $2D_0$ [24, Eq. 11]. Appendix C derives it and compares it with Murphy's preferred intermediate model.

Table 8 gives an illustrative calculation with $D_0 = 0.2$ defects/cm^2 and $Y_{\text{wafer}} = 1$. It is a model result, not a measured processor result.

Doubling the die area lowers the modelled yield from about 82.4% to 68.8% (figure 16). Cost per good die rises faster than area for two reasons: fewer dies fit on each wafer, and a smaller fraction

Table 8: Illustrative Murphy-model yield for $D_0 = 0.2$ defects/cm^2 and $Y_{\text{wafer}} = 1$.

Die area	$2AD_0$	**Modelled yield Y_{die}**
100 mm^2 (1 cm^2)	0.4	82.4%
200 mm^2 (2 cm^2)	0.8	68.8%

of them work. The exact increase depends on wafer cost, process node, defect distribution and test cost; appendix C derives the cost-per-good-die expression and shows why partitioning a design into smaller dies can lower the silicon cost even after paying for extra interface area.

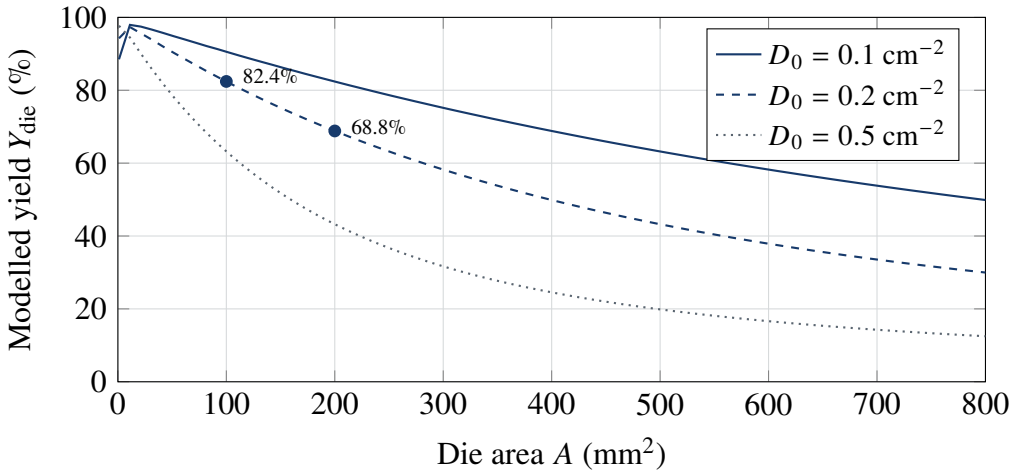


Figure 16: Murphy yield model (equation (8), $Y_{\text{wafer}} = 1$) for three illustrative defect densities. The marked points reproduce the calculation in section 6.2. Illustrative calculation, not measured processor data.

AMD's own material makes the same qualitative point. Its Zen 2 presentation lists "small die improve yield" among the motivations for the chiplet strategy [4], and its Hot Chips 33 presentation shows normalised cost per yielded square millimetre rising at each newer process node [6].

6.3 Uneven scaling of logic, SRAM and analog circuits

Moving a design to a newer node does not shrink every circuit equally. Naffziger *et al.* state that shrink factors at advanced nodes are much lower for analog circuitry than for digital logic and SRAM, and that memory interfaces and SerDes therefore gain very little performance, power or area from moving from 12 nm to 7 nm [2]. Building those circuits on the expensive leading-edge node wastes its cost. This observation is the basis of process-node specialisation (section 9).

AMD's own charts show the trend (figure 17). Logic and SRAM shrink together down to 10 nm, after which SRAM falls behind, while analog and I/O circuits hardly shrink at all after 16 nm [3, slide 6]. At the same time the cost of each yielded square millimetre rises: read from AMD's chart, it is about 2.2 times the 45 nm cost at 14 nm, 3.7 times at 7 nm and 4.9 times at 5 nm. A circuit that does not shrink on the new node therefore costs several times as much per function there as on an older one.

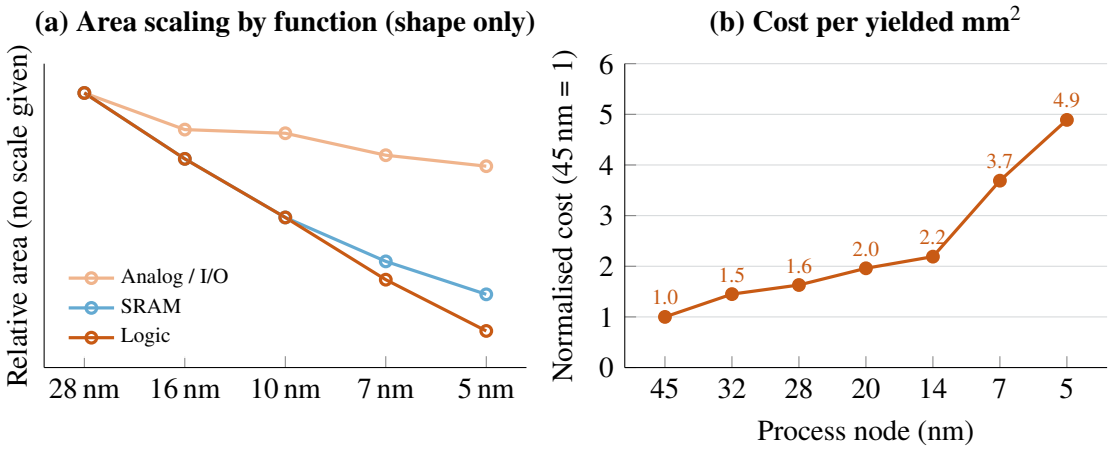


Figure 17: Why SRAM and analog circuits no longer shrink with logic, and why each newer node costs more. Both panels are redrawn from AMD's ISSCC 2022 presentation [3, slide 6]; AMD's 2025 packaging talk shows the same curves [25, slide 3]. (a) AMD's chart has no values on its area axis, so only the shape is reproduced, normalised to 28 nm: logic and SRAM shrink together to 10 nm, SRAM then falls behind, and analog and I/O circuits hardly shrink after 16 nm. (b) Cost per yielded mm^2 for a 250 mm^2 die, read from AMD's chart (about ±0.1): about 2.2 times the 45 nm cost at 14 nm and almost 5 times at 5 nm.

Where the Zen 2 gains came from. AMD's Zen 2 presentation splits the doubling of instructions per unit of energy over the previous generation into three factors: 1.15× from higher IPC, 1.17× from design improvements and 1.47× from the 7 nm process [4, slide 4]. Multiplied together, $1.15 \times 1.17 \times 1.47 \approx 1.98$, which is AMD's "2×". The process contributed the largest single share, but only to the circuits that shrink with it. Logic and SRAM in the cores benefit; memory interfaces and SerDes, by Naffziger *et al.*'s account, barely do [2]. The chiplet split therefore places the 7 nm process exactly where its 1.47× applies and spends the cheaper process everywhere else. The related, more recent observation that SRAM is falling behind logic is taken up in section 13.

7 Compute Dies, I/O Dies and Core Complexes

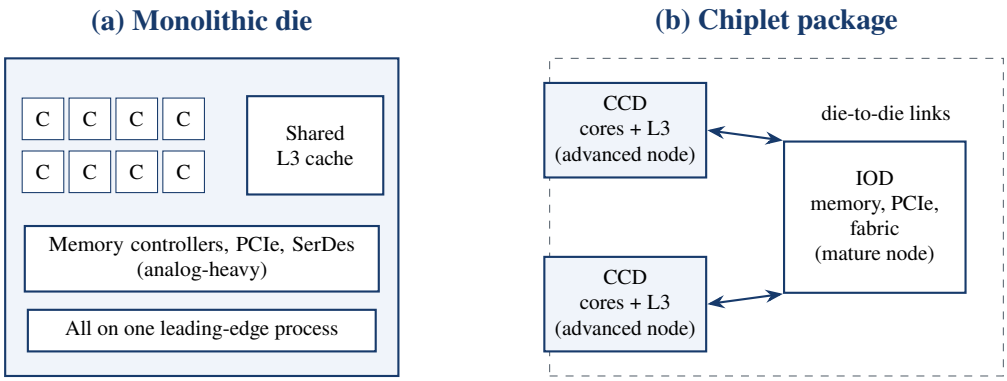


Figure 18: Schematic contrast between a monolithic processor die and a chiplet package with compute dies (CCDs) and an I/O die (IOD). Illustrative only; not to scale and not an actual AMD floorplan.

AMD's Zen 2 products ("Rome" for servers and "Matisse" for desktops) are built from three kinds of die [2], [4]:

- an eight-core CCD (Core Complex Die) of about 74 mm^2 on a 7 nm process, containing the cores and their L3 cache;
- a server IOD (I/O Die) of about 416 mm^2 on a 12 nm process;
- a smaller client IOD of about 125 mm^2, which, packaged on its own, is also the X570 chipset.

The same CCD is reused across server and desktop products; only the IOD and the number of populated CCDs change [2]. Within each Zen 2 CCD, the cores are grouped into two four-core CCX (Core Complex)s, each with up to 16 MB of L3 [4]. Inside a CCX, each core has a private 512 KB L2, and the 16 MB L3 is built from four 4 MB slices, one beside each core, with 32-byte-per-cycle paths throughout; the CCX doubled both the L3 capacity and the L1 load/store bandwidth of the first Zen generation [4, slide 12].

AMD stated the problem and the response in two lists [4, slide 13]. The challenges were a higher cost per mm^2 at the leading-edge node, analog and I/O circuits that "do not scale well" and are hard to port to a new process, and a wide range of markets to serve. The chiplet strategy answered each in turn: small dies to improve yield, analog and I/O on 12 nm to reduce cost and complexity, and a configurable set of parts.

Of Rome's 1008 mm^2 of silicon (figure 15), 592 mm^2 (59%) is on the expensive 7 nm process and 416 mm^2 (41%) on the cheaper 12 nm [2].

The partition places latency-critical, logic-dense circuits—cores and caches—on the newest node, and places memory controllers, PCI Express (PCIe) links and other I/O, which scale poorly, on a cheaper mature node (figure 18).

Naffziger *et al.* describe how this partition was turned into products [2]. The server package places up to eight CCDs around a central IOD, in two-deep columns that were only possible because the organic-package link reaches farther than a silicon interposer would. Each CCD-to-IOD link is routed to the centre of the CCD to shorten the path to its L3. Not every server model populates all eight CCDs, so one package design yields a range of products. The desktop part reuses the same CCDs with a smaller IOD in the existing AM4 socket, and that IOD, packaged on its own, doubles as the platform's chipset. The previous server generation ("Naples") tied its interface clocks together, whereas Rome decouples the fabric, memory and I/O clocks so that each can be set independently, with a latency-optimised mode when fabric and memory clocks are equal [2]. Figure 19 sketches the resulting server package.

7.1 Inside the I/O die: memory, I/O and NUMA settings

AMD's EPYC 7003 overview describes what the I/O die of the Zen 3 server generation contains and how software sees it [11]. Each CCD connects to the IOD through its own Global Memory Interconnect (GMI) link. The IOD holds eight Unified Memory Controllers, one per DDR4 channel, each supporting up to two memory modules (DIMMs), for up to 4 TB of memory per socket; eight x16 I/O links that provide up to 128 lanes of PCIe 4.0; and the Infinity Fabric links that join the dies. The fabric clock can run at up to 1600 MHz, which lets it run in step with DDR4-3200 memory, whose memory clock is also 1600 MHz, a pairing AMD says improves memory latency [11]. The IOD can be used with four, six or eight populated memory channels;

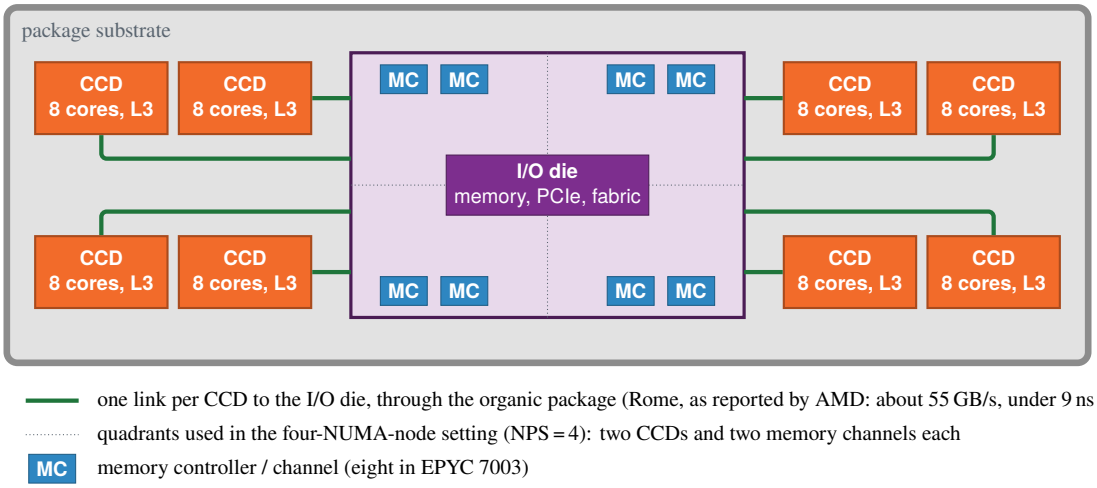


Figure 19: Schematic of an AMD EPYC 7002/7003-style package: eight CCDs in two-deep columns around a central I/O die, each with its own link to the I/O die [2], [11]. Positions, link routes and memory-controller placement are illustrative, not an actual AMD floorplan.

eight is the most common and generally fastest configuration, and the six-channel option, with its own interleave mode, is new in this generation [11]. AMD's figure labels this IOD as 14 nm [11, Fig. 2-4], while Naffziger *et al.* give 12 nm for the Rome IOD [2]; the sources used here do not resolve the difference.

NUMA settings. Because a core's distance to each memory controller depends on where both sit on the IOD, EPYC is a non-uniform memory access (NUMA) design, and a BIOS setting called NUMA Nodes Per Socket (NPS) lets the user choose how that structure is presented to software [11]. The setting also decides how memory addresses are spread, or *interleaved*, across channels: consecutive blocks of memory go to different channels in turn, so that a stream of accesses uses several channels at once. Table 9 summarises the options.

Table 9: NUMA settings of an EPYC 7003 processor with eight CCDs and eight memory channels, from AMD's description [11], with peak interleaved bandwidth worked out for DDR4-3200 (25.6 GB/s per channel).

Setting	Each NUMA domain contains	Channels inter-leaved	Peak bandwidth per domain
NPS1	the whole socket: all 8 CCDs, all channels, all PCIe devices	8	204.8 GB/s
NPS2	half the socket: 4 CCDs and 4 channels	4	102.4 GB/s
NPS4	one quadrant: 2 CCDs, 2 channels and 1 I/O hub	2	51.2 GB/s
LLC as NUMA	one CCD and its L3 (8 domains per socket)	—	—

What the setting trades. In every NPS mode the socket's total memory bandwidth is the same, $8 \times 25.6 = 204.8$ GB/s, and so is the share per core when all 64 cores are busy, $204.8/64 = 3.2$ GB/s (table 12). What changes is locality and reach. Under NPS4, a thread whose memory is allocated in its own quadrant reaches the nearest two controllers, the shortest core-to-memory distance on the die, but its interleaved accesses can draw on at most 51.2 GB/s. Under NPS1, the same thread's accesses are spread over all eight channels and can draw on

the full 204.8 GB/s, but on average three quarters of them go to controllers outside its own quadrant, including the diagonal quadrant that AMD identifies as the farthest [11]. NPS4 therefore suits many independent, well-placed workloads, and NPS1 suits a single program that needs bandwidth more than latency. The "LLC as NUMA" option goes one step further and tells the operating system which cores share an L3, so that threads that share data can be kept on one CCD and avoid the cross-die coherence traffic of section 5 [11]. The chiplet structure is thus exposed to software as a tuning choice: none of these settings would exist on a single die with uniform access to its memory controllers.

8 Infinity Fabric and Die-to-Die Communication

Splitting a processor into dies turns some on-chip wires into package-level links. AMD connects each CCD to the IOD with an IFOP (Infinity Fabric On-Package) serial link routed over 10–20 mm of organic package [2]. AMD reports, for Rome [2]:

- about 55 GB/s of effective bandwidth per IFOP link at a 1.46 GHz fabric clock;
- a round-trip latency from CCD to IOD and back of 13 fabric clocks, under 9 ns;
- longer reach than a 2.5D silicon interposer, which allowed two-deep columns of chiplets and hence eight CCDs in one package;
- a 19% average reduction in memory latency for the socket compared with the previous generation, attributed to the central IOD, decoupled clocks and the latency-optimised link.

The predecessor "Zeppelin" link achieved about 2 pJ/bit [2]. These are AMD-reported figures for specific configurations and are not independent measurements.

The link is engineered for low latency and power rather than maximum reach. Each IFOP lane runs at 14.6 GT/s with a 1.46 GHz fabric clock. It uses single-ended signalling to save power and package routing layers, forwards a clock alongside each bundle of data lanes to track skew, recovers each lane's timing with a digitally controlled clock-and-data-recovery circuit, and avoids active equalisation in favour of passive T-coils, again to save power [2]. The package itself had to be designed with the dies: the Rome package uses a 9-2-9 layer build-up (four signal-routing layers above the core) against 5-2-5 for Matisse. The dense arrays of package vias must lie outside the footprints of the CCDs and IOD because of the high density of power vias beneath them [2]. These are the costs of a serial, package-level link that the 3D interface of section 17.2 avoids.

8.1 Latency, bandwidth and interconnect power

The textbook models of Part II show what these figures mean.

Latency. Every L3 miss in a CCD must now travel to the IOD, where the memory controllers sit, and back. In equation (5) this adds the die-to-die round trip (under 9 ns in Rome) to $\text{penalty}_{\text{LLC}}$ for every access that misses in the L3 [2]. The same hop appears whenever data must move between CCDs, for example when two threads on different CCDs share a cache block and the coherence mechanism of section 5 has to transfer it. This is why EPYC 7003 lets software treat each CCD's L3 as its own NUMA domain, and why its NPS settings group cores, memory

channels and I/O into quadrants: the closest core-to-memory distance is within a quadrant and the farthest is between diagonal quadrants [11]. On a monolithic die the same traffic would cross on-chip wires instead. Figure 20 traces this path.

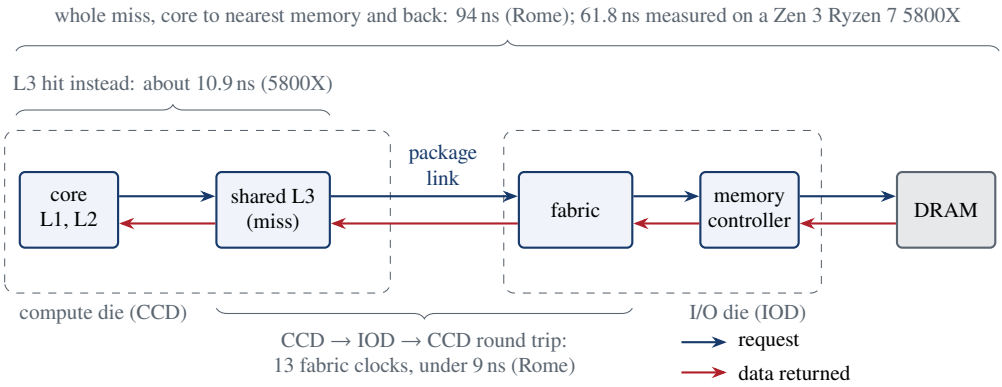


Figure 20: The path of a load that misses in the L3 of a chiplet processor. The request leaves the compute die, crosses the package link to the I/O die, where the memory controllers sit, and the data returns the same way. Times from AMD for Rome [2] and from AIDA64 measurements of the Ryzen 7 5800X [17]; the figures come from different processors and are shown for scale only.

The central IOD also changed how uneven memory latency is across the socket. In the previous generation, Naples, each of four dies had its own memory controllers, so a core saw three different memory distances; Rome's single IOD reduced this to two (table 10).

Table 10: Memory latency by NUMA distance, Naples (EPYC 7001) and Rome (EPYC 7002), as reported by AMD [2, Fig. 2.2.6].

Naples: 8 NUMA domains, 3 distances		**Rome: 2 NUMA domains, 2 distances**	
Domain	Latency (ns)	Domain	Latency (ns)
NUMA1	90	Near 2 channels	94
NUMA2	141	NUMA1	104
NUMA3	234	NUMA2	201
Average local	128	Local average vs Naples	0.81×

Figure 21 plots these numbers so that what the table lists can be seen at once: Rome's nearest access is slightly slower than Naples's, but its latencies span a much narrower range.

The fastest access became slightly slower (94 against 90 ns), probably because every memory access now crosses the IFOP link to the IOD. AMD nevertheless reports that the average fell by 19%, which it attributes to the central I/O die together with decoupled clocks and the latency-optimised link; no access now has to cross to another compute die's memory controller inside the socket. This is the trade the chiplet organisation makes: a small penalty paid by every memory access in place of large, uneven ones.

Bandwidth. A single link limits how much memory bandwidth one CCD can draw. As an illustration, eight 64-bit DDR4-3200 channels, the configuration of both Rome and the most common EPYC 7003 systems [4, slide 19], [11], provide $8 \times 3200 \times 10^6 \times 8\,\mathrm{B} = 204.8\,\mathrm{GB/s}$ at peak, the figure AMD gives for Rome [4, slide 19]. The desktop Matisse part has two such

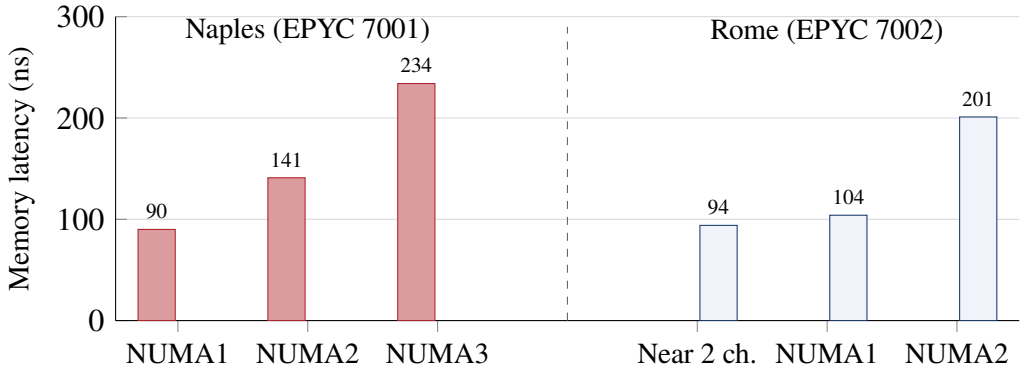


Figure 21: Memory latency at each NUMA distance in Naples and Rome, as reported by AMD [2, Fig. 2.2.6] (labels as in table 10). Rome's nearest access is slightly slower, probably because every memory access crosses the package link to the I/O die, but the spread of latencies within the socket is much narrower, and the average local latency fell to 0.81 of Naples's.

channels and 51.2 GB/s [4, slide 15]; with two CCDs and 16 cores that is $51.2/16 = 3.2$ GB/s per core, exactly the share of a fully loaded 64-core Rome ($204.8/64$). The memory system was scaled with the number of compute dies, not the other way round. Naffziger *et al.* give about 55 GB/s of effective bandwidth per Rome CCD link [2], roughly a quarter of that figure. Both numbers now describe Rome, but one is a peak rate and the other an effective rate, so the ratio is only indicative. It is, however, consistent with AMD's statement that at least four CCDs are populated in most Rome configurations to balance overall bandwidth [2].

Power. Moving a bit across the package costs more energy than moving it across a die, because the link drives longer, more capacitive wires (equation (3)) and needs serialisers and receivers at each end. AMD reports about 2 pJ/bit for the predecessor link and states that the Rome link was designed to minimise power, avoiding active equalisation for that reason [2]. At 55 GB/s, 2 pJ/bit would correspond to roughly $55 \times 10^9 \times 8 \times 2 \times 10^{-12} \approx 0.9$ W per fully loaded link; this is an order-of-magnitude illustration using the older generation's efficiency, not a measured Rome figure.

8.2 Sharing data across dies

The hop on an L3 miss is paid once per miss. Sharing between cores on different CCDs is costlier, because a block that two threads write in turn moves between their caches each time (table 6), and in a chiplet processor every such move between CCDs passes through the I/O die. Part II showed that a directory protocol adds steps that go through a third party (table 48); if the directory sat on another die, each of those steps would also cross the package, although the sources do not say where AMD keeps it (section 5).

Reguly measured what this costs on a Milan-X processor, an EPYC 7V73X with 3D V-Cache, running as an Azure virtual machine with SMT disabled [26]. The test passes messages between two cores through shared cache lines, one core writing and the other reading, which is the access pattern of table 6. Table 11 gives the results, together with the same test on two Intel Xeon processors.

Sharing *within* a CCD is fast: at 58.6 ns, two cores that share the CCD's L3 exchange data faster than adjacent cores on either Intel processor. Sharing *across* CCDs takes about 4.5 times as

Table 11: Message-passing latency between cores through shared cache lines (one writer, one reader), read from Reguly's Fig. 2 [26]. The EPYC system ran as a cloud virtual machine with SMT disabled; the Intel columns' first row is between two hardware threads of one core.

Pair of cores	EPYC 7V73X	Xeon Platinum 8360Y	Xeon Max 9480
Two threads of one core	(SMT off)	0.1 ns	0.7 ns
Adjacent cores (EPYC: same CCD)	58.6 ns	106.1–109.5 ns	83.6–84.4 ns
EPYC: different CCD, same socket	265.2–266.2 ns	—	—
Other socket	396.9–415.3 ns	228.3–233.9 ns	241.6–246.6 ns

long as within one, and on this system a core on another CCD in the same socket is farther away (265 ns) than a core on the other socket of either Intel machine (228–247 ns). Crossing to the other EPYC socket costs about 1.5 times the cross-CCD figure; Reguly reports the cross-socket figure as 1.6 times worse than on the Intel systems and warns that the virtual machine may have affected the results [26]. The processors also differ in clock, core count and generation, so the comparison between vendors is indicative only. The comparison between rows of the EPYC column is cleaner, because all three were measured on the same system.

The group's own Ryzen 5 5600X gives a comparable same-CCD figure. Its six cores share one CCD, and in the group's ping-pong test (appendix G) a cache line passed from one core to another in about 49 ns, half of a 98.5 ns round trip, the same order as Reguly's 58.6 ns. Like Reguly's, the measurement ran in a virtual machine, and the two tests time slightly different things, so the agreement is one of magnitude. The 5600X has a single CCD, so it cannot measure the cost of crossing between CCDs.

The extra cost of crossing CCDs, about $265 - 59 \approx 206$ ns, is about eleven times the 18 ns that the link itself accounts for. AMD gives under 9 ns for one round trip between a CCD and the I/O die on Rome (section 8), and a request from one CCD to another and its reply cross that link four times, two such round trips. The link itself is therefore not most of the cost. A transfer between CCDs also has to pass through the fabric on the I/O die and through the steps of the coherence protocol at each end, and the sources used here do not break the total down. The figure is also not the same as the latency of a single miss: the test measures how long one message takes to travel from writer to reader, which includes the invalidation and the transfer.

For two threads that hand one shared block, such as a counter or a lock, back and forth two million times, the latencies of table 11 give about 0.12 s on one CCD, 0.53 s across CCDs and 0.80 s across sockets. The work is the same in all three cases; only the placement differs. A program that shares data heavily between threads can therefore run several times slower if the operating system spreads those threads across CCDs. This is the practical reason behind the NUMA options of section 7: when EPYC 7003 exposes each CCD's L3 as a separate NUMA domain [11], the operating system and the programmer can see the boundary and keep threads that communicate on the same die. Programs that share little, such as many independent requests or processes, are hardly affected, which is the loosely coupled use that Amdahl's law favours (section 5).

A stacked cache does not change this cost. The V-Cache enlarges each CCD's L3 but leaves the path between CCDs as it was, and the EPYC 7V73X in table 11 is itself a V-Cache part. What a larger L3 can change is how often a thread needs data held elsewhere: if more of its working set fits on its own CCD, fewer of its accesses leave the die at all.

8.3 Scalability

The chiplet organisation scales by adding compute dies, but the IOD is shared. Adding CCDs increases core count and total L3 capacity, while memory channels and I/O stay fixed by the IOD and the socket. As more cores share the same memory bandwidth, the bandwidth per core falls, which raises the value of a large L3 that keeps traffic off the memory channels. AMD's own Rome rule of thumb, at least four populated CCDs for bandwidth balance [2], shows the opposite limit: with too few CCDs, the per-CCD links rather than the memory channels become the bottleneck. A star of point-to-point links to one IOD also has a fixed number of link ports, so the number of CCDs is bounded by the IOD design as well as by package area.

Balancing link and memory bandwidth

Naffziger *et al.* give about 55 GB/s of effective bandwidth per Rome IFOP link and state that at least four CCDs are populated in most configurations to balance overall bandwidth [2]. Table 12 shows why, comparing the total link bandwidth with the 205 GB/s peak of eight DDR4-3200 channels computed above. As before, the link figure is effective and the memory figure is peak, so the comparison is indicative only.

Table 12: Total CCD–IOD link bandwidth against peak memory bandwidth (55 GB/s per link [2]; 8 channels of DDR4-3200).

CCDs	Cores	Total link bandwidth	Limiting side	Memory bandwidth per core
2	16	110 GB/s	links	$110/16 \approx 6.9$ GB/s
4	32	220 GB/s	roughly balanced	$205/32 \approx 6.4$ GB/s
8	64	440 GB/s	memory channels	$205/64 \approx 3.2$ GB/s

Figure 22 extends the table to every CCD count from one to eight. It shows where the two limits cross, at about 3.7 CCDs, which the table's three rows only bracket.

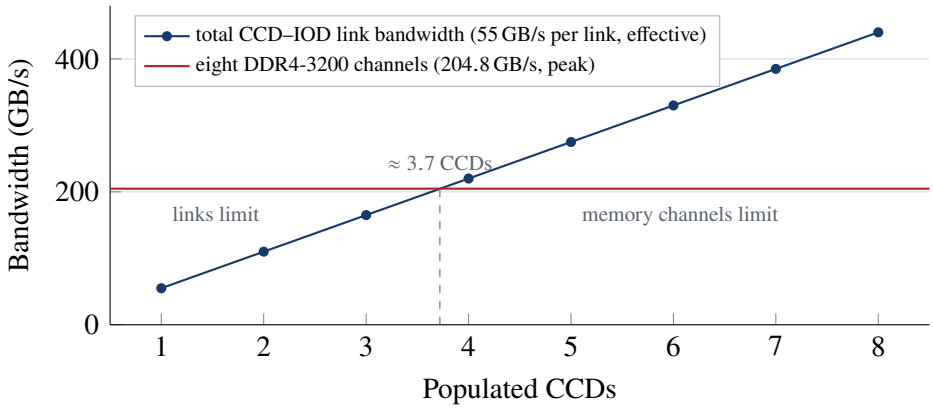


Figure 22: Which side limits a Rome socket's memory bandwidth as compute dies are added: below about four CCDs the die-to-die links are the bottleneck, above it the memory channels. This is consistent with AMD's statement that at least four CCDs are populated in most configurations [2]. The link figure is effective and the memory figure peak, so the crossing point is indicative (table 12).

With two CCDs the links, not the memory, are the bottleneck; at four the two sides are roughly

matched, which is AMD's rule of thumb; at eight, the memory channels limit, and each core's share of memory bandwidth halves. The last column is the point that connects Part III to Part IV: the more cores share a socket's memory channels, the more valuable it is to satisfy requests from a large on-package cache instead.

Scaling beyond one socket

The same links extend to two sockets. Two identical EPYC 7003 processors connect through their external Global Memory Interconnect (xGMI) Infinity Fabric links, which use the same physical connections as the PCIe lanes [11]. Each link uses 16 lanes, so a socket that devotes four links to the other socket gives up 64 of its 128 lanes, leaving $2 \times 64 = 128$ lanes for the whole system. A designer who uses three links instead frees $128 - 48 = 80$ lanes per socket, 160 in total, at the cost of inter-socket bandwidth [11]. A dual-socket system has sixteen memory channels and can be configured with between 1 and 16 NUMA domains. The trade between I/O lanes and inter-processor bandwidth is the package-level version of the trade between link width and die area seen within the package (section 8.4). Figure 23 shows the two configurations.

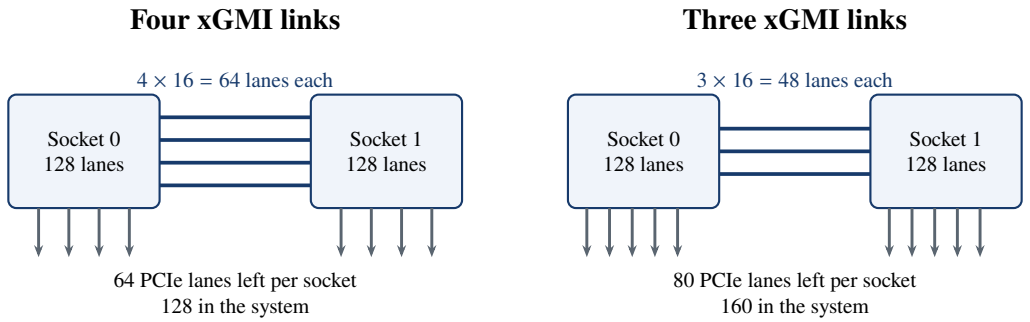


Figure 23: The lane trade in a dual-socket EPYC 7003 system: the links between sockets (xGMI, blue) use the same 16-lane connections as PCIe, so each link given to the other socket removes 16 lanes of I/O (grey arrows, schematic) [11]. Fewer links mean more I/O but less bandwidth between the sockets.

8.4 How the dies are connected: organic substrate, 2.5D and 3D

Chiplets can be joined in three broad ways, which differ mainly in how finely their connections can be spaced (table 13).

- **On the organic package substrate** (a multi-chip module, sometimes called 2D packaging). Each die is mounted on the substrate with C4 solder bumps, and the substrate's copper traces connect the dies. This is the cheapest option and allows long links, but the connections are coarse.
- **2.5D integration.** The dies still sit side by side, but they are joined through a finer layer of wiring placed under or between them: a silicon interposer, a small embedded silicon bridge, or a fan-out redistribution layer. Connections are much denser, so a link of the same width carries more bandwidth with less energy per bit, but the extra silicon adds cost and limits how far apart the dies can be.
- **3D integration.** One die is stacked directly on another and connected through through-silicon

vias (TSVs) and micro-bumps or hybrid bonds (Part IV). This gives the densest and shortest connections.

Table 13: Die-to-die connection methods, with AMD's interconnect pitch targets [25, slide 13] and AMD examples.

Method	Pitch	AMD examples	Main trade-off
Organic substrate (C4 bumps)	< 150 µm	EPYC 7002 and Ryzen 3000 CCD–IOD links [2]	long reach and low cost; coarse links
2.5D (interposer, bridge or fan-out)	< 40 µm	Fiji GPU with high-bandwidth memory (HBM) on a silicon interposer (2015) [6, slide 9]; Ryzen AI Max PRO 300 [25, slide 8]	denser, lower-energy links; added silicon and shorter reach
3D (hybrid bonding)	< 9 µm	3D V-Cache [3]	densest, shortest links; heat and test complexity

AMD's choice for its chiplet CPUs was deliberate. Naffziger *et al.* contrast the organic-package SerDes link with a 2.5D interposer and with Intel's embedded bridge (EMIB), and state that its key benefit is longer reach, which allowed chiplets to be arranged in columns two deep; without that, eight CCDs would not have fitted around one IOD [2]. Rome and Matisse therefore connect their CCDs to the IOD through the organic substrate, and the later Zen CPUs studied in this paper use 3D stacking for the extra cache; the sources used here do not describe a change to the CCD–IOD connection in those later generations (section 8, Part IV).

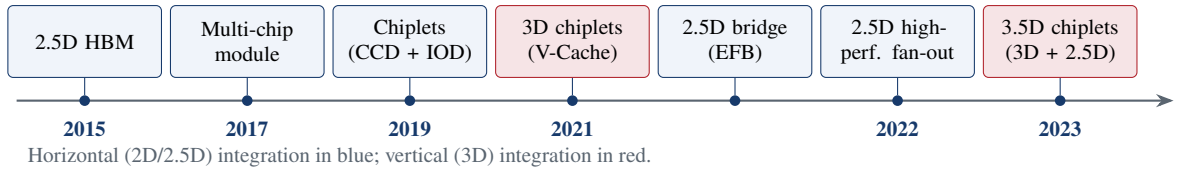


Figure 24: AMD's packaging steps as presented by AMD [25, slide 5]. The slide gives no separate year for the 2.5D bridge (EFB). The CPUs studied in this paper use the 2019 chiplet organisation and, from 2021, 3D-stacked cache.

2.5D integration is nevertheless part of AMD's wider packaging history (figure 24). Its own timeline runs from 2.5D HBM (2015) through the multi-chip module (2017), chiplets (2019) and 3D chiplets (2021) to a 2.5D bridge (labelled EFB on AMD's slide), 2.5D high-performance fan-out (2022) and "3.5D" chiplets (2023) [25, slide 5]. The Instinct MI300 accelerator combines both: dies hybrid-bonded on top of I/O dies, which sit side by side on a 2.5D silicon interposer [25, slide 10]. Among Zen-based processors, the Ryzen AI Max PRO 300 series uses 2.5D fan-out packaging, which AMD describes as a "continued strategic use of 2.5D advanced packaging to reduce die-to-die power and latency" [25, slide 8]. Swaminathan names the parameters that decide between these options: cost and volume, *bandwidth density* (the area spent on die-to-die communication) and *energy per bit* [6, slide 7]. The organic substrate wins on cost and reach, 2.5D on bandwidth density and energy, and 3D on both of the latter at the price of the thermal and manufacturing issues of section 18.

9 Yield, Process-Node Specialisation and Cost

Chiplets combine two cost advantages. First, small dies yield better (section 6.2). Second, each die can use the process that suits it: AMD keeps memory interfaces and SerDes on 12 nm because they would see little benefit at 7 nm [2]. AMD also tests every CCD before assembly (section 10.1), which limits the cost of combining several dies in one package.

These benefits are not free. AMD describes the design as requiring package–silicon co-design, new interconnect circuits, a mixed-process package (with copper pillars on the 12 nm IOD to match die heights), and new test and power-management infrastructure [2].

The model of appendix C gives a rough sense of the silicon saving. Consider eight Rome-sized CCDs of 74 mm^2 each, 592 mm^2 in total, against a hypothetical monolithic die with the same compute area (ignoring the I/O, which in the chiplet design moves to a cheaper node). With $D_0 = 0.2\,cm^{-2}$ and the yield model of equation (8), a 74 mm^2 CCD yields about 87% and the 592 mm^2 die about 38%, so eight tested CCDs cost about 44% of the silicon of one good monolithic die; with $D_0 = 0.1\,cm^{-2}$ the figure is about 63%. These numbers are illustrative only: they ignore interface area, packaging, assembly yield and test cost, and Dong and Xie find an analogous effect for 3D stacking, where the extra bonding and test costs outweigh the yield gain for small designs, so that stacking pays only above a size threshold [15]. They also ignore the separate saving from building the I/O on a mature node [2]. The qualitative conclusion is robust, however: for a design of server size, the yield advantage of small dies is large.

Yield is only half of the cost story; the other half is how many dies fit on a wafer, which falls faster than area because partial dies are lost around the circular edge. Counting both (appendix C.5), a 300 mm wafer of 74 mm^2 CCDs supplies the compute silicon for about 95 64-core processors, against 35 for a 592 mm^2 monolithic die, a factor of 2.7 (1.9 with $D_0 = 0.1$, 3.5 with $D_0 = 0.3$). The calculation ignores the I/O die, packaging and test, and assumes that such a large die could be made at all.

AMD's own estimate points the same way (figure 25). Naffziger *et al.* compare the cost of Rome chiplet products with hypothetical monolithic 7 nm designs of the same core count [2, Fig. 2.2.2]. Read from the chart, the monolithic designs cost roughly twice as much at 48 and 32 cores and about 1.8× at 16 cores, and the chiplet design scales its cost down with core count by leaving CCD positions empty ("depopulation") and using partly defective dies ("harvesting"). Two details of the chart should be noted: its legend describes the chiplet design as "7 nm + 14 nm", whereas the paper's text gives 12 nm for the I/O die, and the chart's axis is a normalised cost without units. The chart is AMD's model, not a disclosed cost, but its factor of about two agrees with the order of magnitude of table 46.

10 Power, Clocking and Product Flexibility

Research question RQ2 asks what the split into compute and I/O dies changes for power and for product flexibility as well as for latency and bandwidth. The evidence on the two is very different. On flexibility the AMD sources are detailed and the product lists can be checked; on power they describe design choices but give no measurement of the I/O die or of the package at idle.

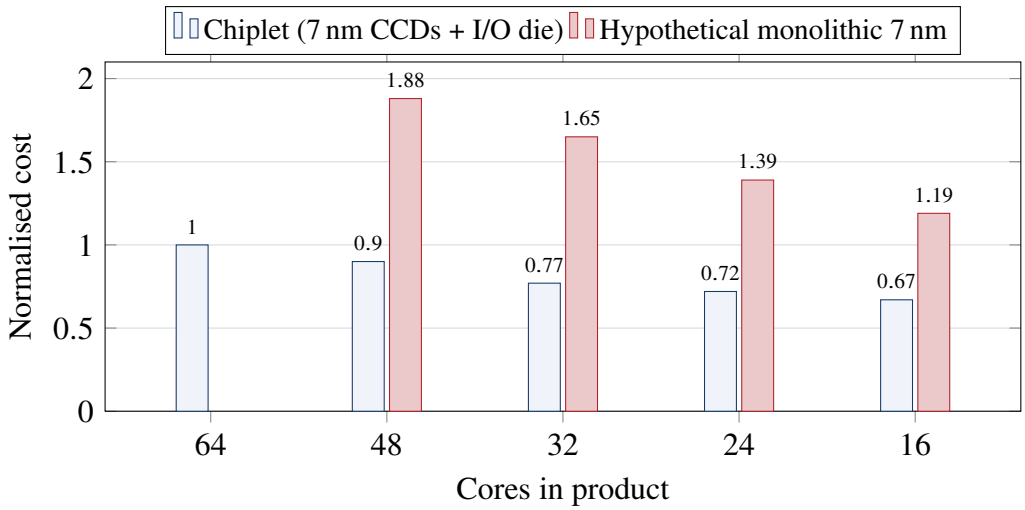


Figure 25: Cost against core count for Rome-generation chiplet products and hypothetical monolithic 7 nm equivalents, normalised to the 64-core chiplet part. Values were read by eye from AMD's chart [2, Fig. 2.2.2] and are accurate only to about ±0.05. AMD shows no monolithic 64-core bar; the text states that such a die would exceed the maximum reticle size.

10.1 Power and clocking

Two kinds of power cost come with the split. The first is the energy of the links, discussed in section 8: about 2 pJ/bit for the predecessor link, with the Rome link designed to reduce it further by using single-ended signalling, a termination scheme that needs little power to maintain, no active equalisation and a bandwidth-limited clock distribution [2]. The second is that the I/O die is a separate chip with its own clocks and voltage. Here the chiplet design turned a cost into a control. In Naples, the clocks of the fabric, the memory and the inter-socket links were tied together; in Rome they can be set independently. AMD states that this clocking flexibility, together with dynamic voltage and frequency scaling (DVFS) of the uncore and a variable memory clock, enables power management that shifts power between the CPU cores and the uncore within a fixed thermal design power (TDP) [2]. The same source adds a qualification: Rome and Matisse have clocking modes that improve latency when the fabric clock equals the memory clock [2], so decoupling the clocks saves power at some cost in latency. The Cezanne comparison of section 11, where the 5700G ran a 1600 MHz fabric clock against the 5800X's 1800 MHz, shows how strongly such settings enter measured latencies.

Two further mechanisms let a chiplet product match its power to its configuration. AMD tests every CCD before assembly, and if a CCD is damaged during assembly, firmware "down-bins" it, putting the CCD and its interconnect logic into a known low-power state [2]. And in the fourth-generation EPYC processors, a CCD can connect to the I/O die through one link or two. Models with four or fewer CCDs use two links per CCD to double the bandwidth, while models with more CCDs use one. The I/O die's twelve links allow 4, 8 or 12 CCDs "depending on the performance and power requirements" of the product [9].

How much power the links could take

No source used here gives the power of the Rome or Genoa links, so this example estimates it with the one figure available, 2 pJ/bit for the predecessor link [2]. Two observations make

the estimate simple. A 64-byte line is 512 bits, so moving one line across a link costs about $512 \times 2\,\text{pJ} \approx 1\,\text{nJ}$. And memory traffic can cross the links no faster than the memory channels supply it.

- **Rome**, eight channels of DDR4-3200 at 204.8 GB/s (section 8): $204.8 \times 10^9 \times 8 \times 2 \times 10^{-12} \approx 3.3\,\text{W}$ with the memory fully loaded.
- **Genoa**, twelve channels of DDR5-4800 at 460.8 GB/s [27]: about 7.4 W on the same assumption, against a default TDP of 360 W for the 96-core EPYC 9654 [27], or about 2%.

Traffic between CCDs, I/O and the link controllers' own idle power come on top, and newer links are probably more efficient per bit than the predecessor's, so these numbers are an order of magnitude, not a measurement. They suggest that the energy of moving memory data across the package is a small share of the processor's power budget. They do not show that the I/O die as a whole is cheap in power, because its memory controllers, fabric and I/O are not counted. The power side of RQ2 can therefore be answered only for link energy, and otherwise qualitatively, from the sources used in this paper.

10.2 One compute die, many products

The flexibility gain is easier to document. AMD describes three chiplets "leveraged to many products and markets" and states that the chiplet architecture brings leading-edge CPUs to several markets with high reuse for faster time-to-market [2, Fig. 2.2.1]. The same CCD serves Rome and Matisse; different Rome models leave some CCD positions empty "to optimize cost and performance"; and the client I/O die, packaged on its own, doubles as the X570 chipset [2]. The fourth-generation white paper calls the CPU die "a unit of innovation" that AMD can "swap in and out" to match products to workloads [9].

Table 14 tests the claim against AMD's own product lists. Each row is a product built from its generation's compute die. The number of CCDs and of active cores per CCD is not stated in the datasheets; the table infers it from the L3 size and the 32 MB of L3 that each CCD carries in all three generations, two 16 MB halves on Zen 2 and one 32 MB L3 on Zen 3 and Zen 4, or 96 MB with V-Cache [3], [4], [9]. The inference is unreliable when part of a CCD's L3 is disabled: the same datasheet lists an EPYC 9224 with 24 cores and 64 MB, which by this rule would be two CCDs holding at most 16 cores [27]. The configurations in the table are therefore plausible readings, not facts; only the EPYC 9174F's is stated by AMD.

The table shows how wide a range one die design covers. The Zen 4 server products in the table alone span 16 to 96 cores, 64 MB to 1152 MB of L3 and 4 to 48 MB of L3 per core, with only two variants of the compute die (with and without the stacked cache) and one server I/O die. The datasheet's EPYC 9224 is lower, at 2.7 MB per core [27], and the 112- and 128-core parts use a different, Zen 4c, compute die [9]. A monolithic product line would need a separate die for each core count or would have to disable cores on one large die, paying for silicon it does not sell. Products with fewer active cores per CCD also give a use for dies on which not every core is needed, and possibly for dies on which not every core works, although the sources used here do not say how AMD sorts its dies.

Two rows make a point that only chiplets can. The EPYC 9124 and 9174F both have 16 cores. AMD states that the 9174F spreads them over eight CCDs with two active cores each, and that this "spreads the thermal load across the processor package and enables us to increase the clock

Table 14: One compute die, many products: Zen 2 and Zen 3 desktop processors [28] and Zen 4 server processors [27]. CCDs × active cores inferred from the L3 size (32 MB per CCD, 96 MB with V-Cache), except where stated; the rule fails when L3 is partly disabled, as on the EPYC 9224 (24 cores, 64 MB).

Product	Cores	L3 (MB)	CCDs × cores	L3 per core	Default TDP
Ryzen 9 3950X / 5950X	16	64	2 × 8	4 MB	105 W
Ryzen 9 3900X / 5900X	12	64	2 × 6	5.3 MB	105 W
Ryzen 7 3800X / 5800X	8	32	1 × 8	4 MB	105 W
Ryzen 5 3600 / 5600X	6	32	1 × 6	5.3 MB	65 W
EPYC 9654	96	384	12 × 8	4 MB	360 W
EPYC 9554	64	256	8 × 8	4 MB	360 W
EPYC 9354	32	256	8 × 4	8 MB	280 W
EPYC 9334	32	128	4 × 8	4 MB	210 W
EPYC 9124	16	64	2 × 8	4 MB	200 W
EPYC 9174F	16	256	8 × 2 (stated)	16 MB	320 W
EPYC 9684X (V-Cache)	96	1152	12 × 8	12 MB	400 W
EPYC 9184X (V-Cache)	16	768	8 × 2	48 MB	320 W

frequency", to a base of 4.10 GHz against 3.00 GHz [9]. The datasheet shows the cost: a default TDP of 320 against 200 W [27]. The spread also brings eight CCDs' worth of L3, 256 MB against the 9124's 64 MB [27], and eight links to the I/O die. On a single die, sixteen cores cannot be spread over eight dies' worth of area and cache; in a chiplet package, the number of dies becomes a design variable that trades power and price for clock, cache per core and bandwidth.

The same logic explains how 3D V-Cache entered the product lines. In EPYC 7003, the X-series models triple the L3 of the standard models using the same Zen 3 core [11, sec. 1.1], and in EPYC 9004 three V-Cache models sit beside the standard ones [27]. Because the stacked cache is a property of the CCD, AMD can offer it on a few products without designing a new processor, which is part of the answer to RQ3 as well as to RQ2 (Part IV).

11 The Same Core With and Without Chiplets

Zen 3 offers something close to a natural experiment. The same core appears in two desktop designs. Vermeer, the Ryzen 5000 series used throughout this paper, follows the chiplet organisation of this part: an eight-core CCD with 32 MB of L3, joined through the package to a separate I/O die. Cezanne, the Ryzen 5000G series, places eight Zen 3 cores, a 16 MB L3, the memory controllers and integrated graphics on one monolithic die, the same die AMD uses in its mobile processors [29]. If the die-to-die hop of section 8 were a large cost, Cezanne should show it by its absence. Table 15 collects what the sources measured.

L3 latency does not depend on the split. The monolithic Cezanne and the chiplet-based Milan reach their L3 in about the same number of cycles, 47 against 47.5 by pointer chasing, although Cezanne's L3 is half the size [20], [31]. AIDA64 gives a similar match, about 50 cycles on the 5700G at 4 GHz against about 53 on the 5800X [17], [30]. This is what the organisation predicts:

Table 15: Zen 3 with and without the chiplet split: Cezanne (monolithic) against Vermeer and Milan (chiplet). Values come from different tools, memory settings and processors, as stated in each cell; they are not a controlled comparison.

	Cezanne (one die)	**Vermeer / Milan (CCD + I/O die)**
L3 per eight cores	16 MB	32 MB
Boost clock; TDP	4.6 GHz; 65 W (5700G) [29]	4.7 GHz; 105 W (5800X) [29]
PCIe	3.0 [29]	4.0 [29]
L3 hit latency	47 cycles (5600G, pointer chase) [20]; 12.6 ns, about 50 cycles (5700G at 4 GHz, AIDA64) [30]	47.49 cycles (Milan, same CCD design) [31]; about 53 cycles from 10.9 ns (5800X, AIDA64) [17]
Memory latency	76.1 ns (5700G at 4 GHz, AIDA64, DDR4-3200 CL16, 1600 MHz fabric clock) [30]; 47 cycles + 74 ns, about 85 ns (5600G, pointer chase, DDR4-3000 CL15, 2 MB pages) [20]	61.8 ns (5800X, AIDA64, DDR4-3600 CL16, 1800 MHz fabric clock) [17]
Memory latency, same memory and clock	5700G against a 3950X with one CCD active, both at 4 GHz with the same DDR4-3200 CL16 kit: "very similar" (only the 5700G's value, 76.1 ns, is shown) [30]	
Performance, 5700G against 5800X	Cinebench R20 multi-core about 10% lower; 7-Zip compression 9% lower; games at 1080p with an RTX 3090 11% lower on average (4–20% by title); total system power about 15% lower [29]	

in both designs the L3 sits beside the cores on the same die, so the chiplet boundary lies beyond it. The 7-cpu measurements also show where the cycles go on Cezanne: about 40 cycles to the nearest of the eight L3 slices, plus up to 20 more for the farthest [20], the slice-and-ring structure of figure 30. Halving the capacity of each slice did not shorten the trip around the ring.

Memory latency: no advantage for the single die. Measured with the same tool, AIDA64, the monolithic 5700G took 76.1 ns to reach memory and the chiplet-based 5800X 61.8 ns [17], [30]. That is the opposite of what removing the die-to-die hop would suggest, but the 5700G ran with slower memory (DDR4-3200 against DDR4-3600), a slower fabric clock (1600 against 1800 MHz) and a lower core clock (4.0 against about 4.85 GHz). CapFrameX notes that the core clock affects AIDA64's latency result and that AIDA64's absolute values have been criticised as too low [30]. The difference therefore cannot be credited to the organisation. The one comparison with matched memory and clocks, by CapFrameX, found the 5700G's memory latency "very similar" to that of a chiplet-based Zen 2 part with one CCD active [30]. CapFrameX offers an explanation from an annotated die photograph: even on Cezanne's single die, the memory controller is reached from the L3's ring of slices through the Infinity Fabric, and the transition from ring to fabric costs extra cycles [30]. If that is right, putting everything on one die removes the package link but not the fabric, and the package link is the smaller part: AMD gives a round trip of 13 fabric clocks, under 9 ns (figure 20), roughly a tenth of a memory access of 60 to 90 ns. The sources thus show no memory-latency advantage for the monolithic die, though none measures the two designs under identical conditions.

Performance favours the chiplet part. In TechSpot's tests the 5700G trailed the 5800X by about 10% in multi-threaded applications and by 11% on average in games, with the gap reaching

20% in one title [29]. TechSpot does not report the clocks reached during its runs. The rated boost clocks differ by only about 2% (4.6 against 4.7 GHz), but measured clocks may differ more: CapFrameX saw the 5700G at 4.5–4.55 GHz in games, and the 5800X ran at about 4.85 GHz in The FPS Review's AIDA64 run, a gap of up to about 8% [17], [30]. Clock speed is nevertheless unlikely to explain most of the gaming gap: CapFrameX found that raising the 5700G's clock by more than 10%, from a fixed 4 GHz to its stock boost, gained only 3.22% in games on average [30]. Other differences remain: the 5700G has half the L3, a 65 W rather than 105 W TDP, and PCIe 3.0 rather than 4.0, and TechSpot measured about 15% less power at the wall for the whole system [29]. The power limit can explain much of the multi-threaded gap, but games load fewer cores and are less limited by power. The reviewer's tentative reading is that, in CPU-limited gaming, "it appears" that cache capacity matters more than having all cores on one die; the reviewer also notes that at 1440p or higher, with a less extreme graphics card, there would be basically no difference between the two [29]. That reading matches the working-set argument of Part IV and the size of the V-Cache gains in games (figure 40).

What the comparison shows. It is not a controlled experiment: L3 capacity, power limits, I/O and memory settings all differ between the two parts, and no source measures them at equal L3 size. Nor can it bound the cost of the die-to-die hop. The one matched latency comparison sets Cezanne against a Zen 2 part, and CapFrameX itself asks whether Cezanne's memory interface and controller were designed with compromises for energy [30], which could hide whatever the single die gains. CapFrameX also expected the monolithic die to recover the latency that chiplets lose, and counts the result as a point against Cezanne's design [30]. The sources therefore show no visible hop, without showing how large it is. In TechSpot's CPU-limited tests at 1080p the chiplet part was the faster of the two, and the most likely reason, though not an isolated one, is the 16 MB of extra L3 it carries. For this pair, at least, choosing one die did not buy speed. The monolithic design has other reasons to exist, among them integrated graphics and the mobile market for which AMD designed the die [29], but the sources used here do not quantify them.

12 Summary of Part III

Figure 26 summarises what the chiplet organisation gains and what it pays.

Applying the paper's three-part test to chiplets:

- **Bottleneck addressed.** Die size beyond the reticle limit, falling yield with area, and the poor scaling of analog and I/O circuits on advanced nodes (section 6).
- **Benefit.** Core counts that would be impractical on one die, much better silicon yield, I/O on a cheaper node, and, within each generation, one compute die reused across a wide range of products. On Zen 3 it spans 6-core desktop parts to 64-core servers. In the Zen 4 server products of table 14 it spans 16 to 96 cores with 4 to 48 MB of L3 per core (section 10.2) [2], [4].
- **New trade-offs.** An extra die-to-die hop on every L3 miss. Sharing between CCDs takes about 4.5 times as long as within one (section 8.2). Further costs are per-link bandwidth limits, interconnect power (estimated at about 2% of a Genoa processor's power budget for memory traffic alone, section 10.1), NUMA effects that software may need to manage, and more complex package design and test.

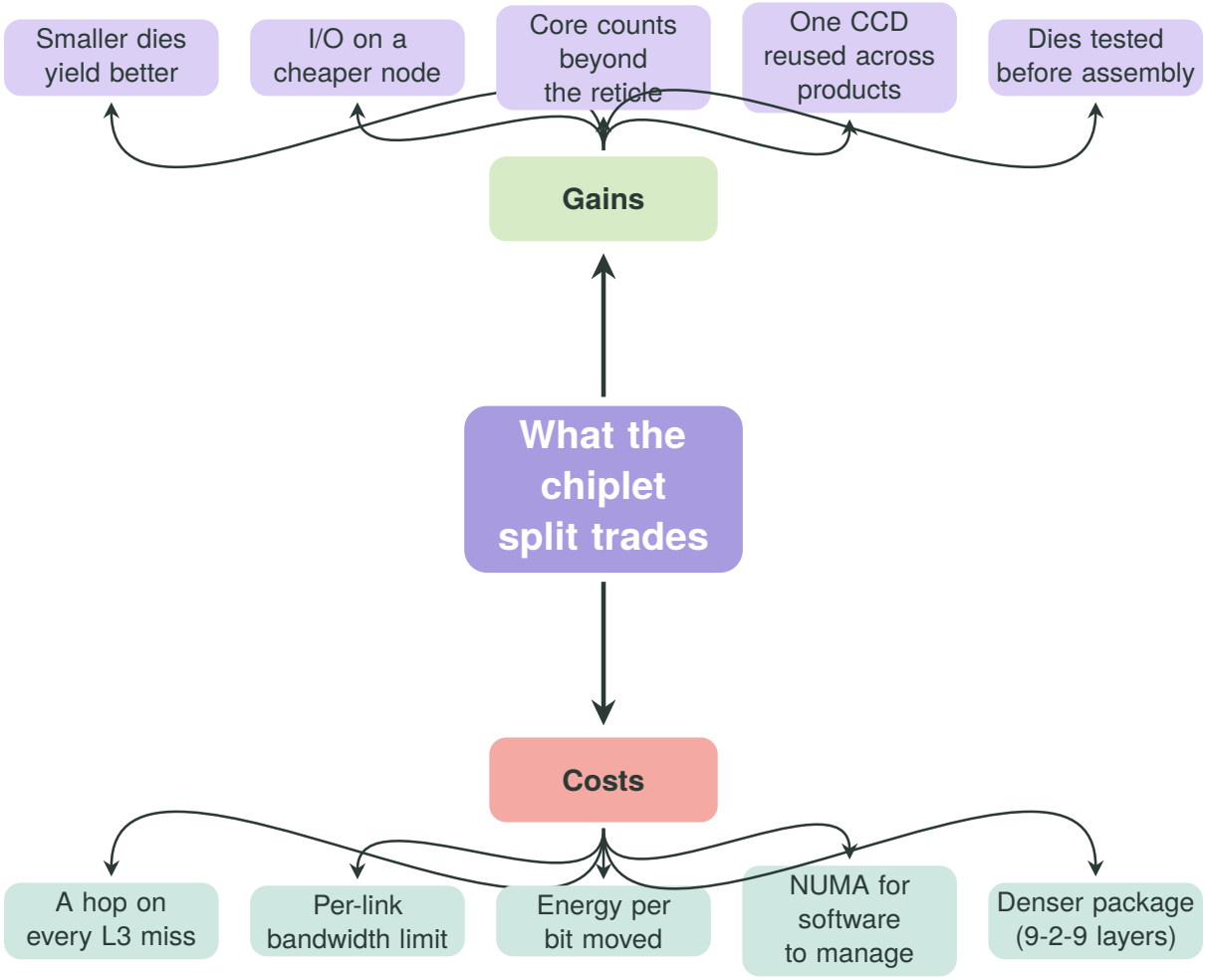


Figure 26: Overview of the chiplet trade-off as Part III found it: the gains above (sections 6.2 and 7) against the costs below (section 8). The Cezanne comparison (section 11) shows no visible latency cost, but cannot say how large it is.

One limit is untouched: each CCD still carries its own planar L3. Growing that cache runs into the same area, yield and wire-length costs at the scale of the compute die, which is the problem Part IV takes up.

Part IV

3D-Stacked Cache and AMD 3D V-Cache

Figure 27 maps the topics of this part.

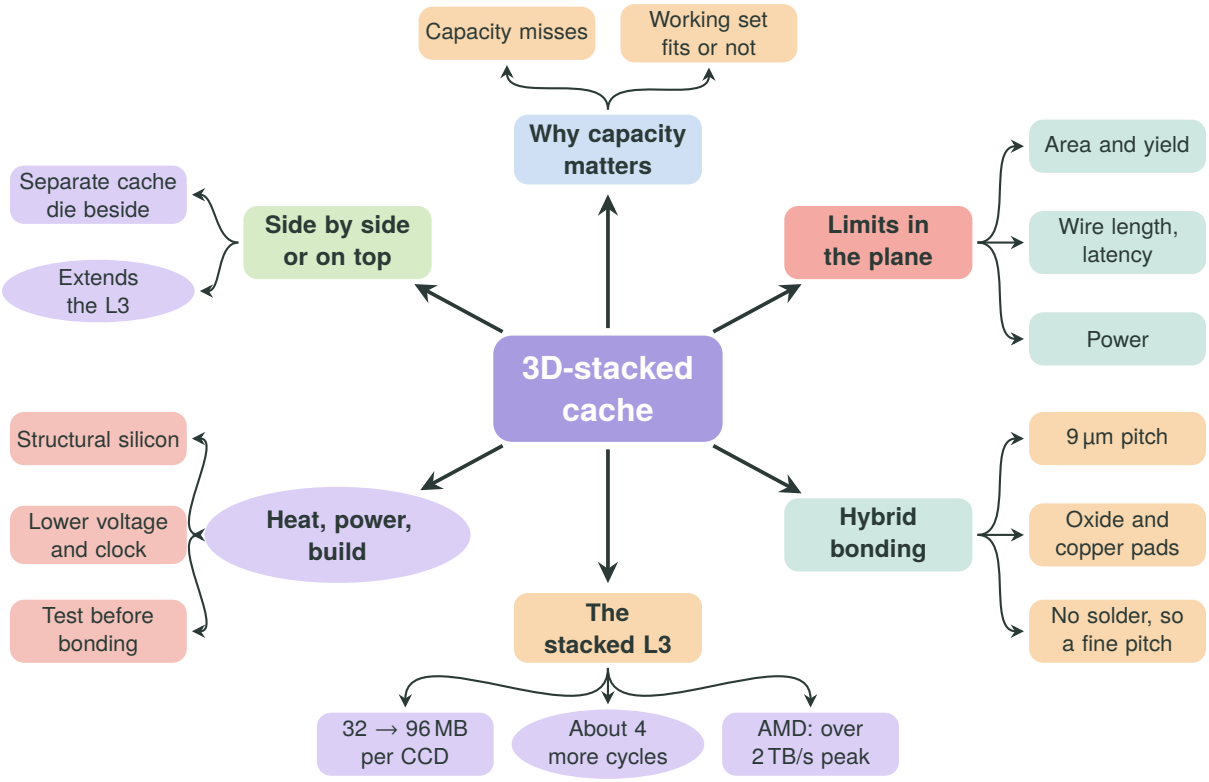


Figure 27: Overview of Part IV, with the values discussed in the text (sections 16 to 18).

13 Why Cache Capacity Matters

Sections 4.3 and 4.4 showed that a larger LLC lowers the miss rate by removing capacity misses, provided its hit time does not grow too much. Whether that matters depends on the workload: a program whose working set already fits in the cache gains little, while one whose working set lies just above the cache size can gain a great deal.

AMD's ISSCC 2022 presentation of 3D V-Cache opens with two charts that frame this part [3, slides 5–6]. The first, titled "Large L3 Caches Provide IPC Uplift", plots IPC uplift on a linear scale rising steadily as L3 size grows from 1× to 32×; the slide gives no values on the uplift axis and does not name the workloads. The second, "Silicon Area Scaling by Function", shows logic

area continuing to shrink from 28 nm to 5 nm while SRAM scaling falls behind from about 7 nm and analog circuits shrink least; a companion chart on the same slide shows normalised cost per yielded mm^2 (for a 250 mm^2 die) rising from 45 nm to 5 nm. The slide summarises these as "SRAM scaling & cost = barriers to large on-die caches". This extends Naffziger *et al.*'s 2020 observation that analog circuits scale worst [2]: by the 5 nm generation SRAM, too, has become comparatively expensive. Figure 28 redraws the IPC chart and the slide's third chart, the slowing pace of new nodes; the area and cost charts are in figure 17.

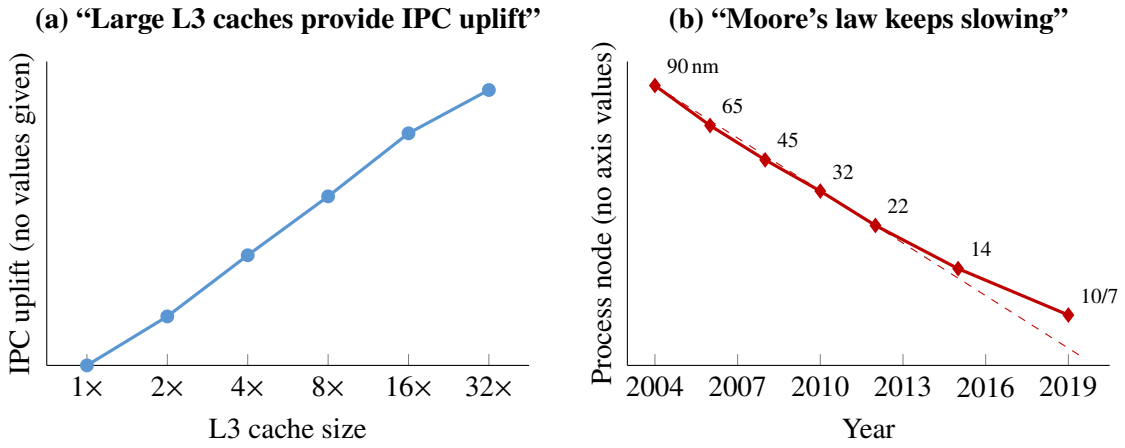


Figure 28: Two of the charts with which AMD's ISSCC 2022 presentation opens, redrawn [3, slides 5–6]. (a) IPC uplift against L3 size on a linear uplift axis. The slide gives no values and names no workloads, so only the shape is shown: each doubling of the L3 up to 16× adds a similar step, and the last doubling a smaller one. (b) Process node against year of introduction, with the dashed line continuing the pace of 2004–2012. The slide gives no axis values, so only the positions are reproduced: after 22 nm each node lies above the earlier trend, that is, it arrived later than that pace would predict. Positions are read from the slide and are approximate.

Both charts are vendor material, and the first has no stated workloads or values; neither is evidence that every program benefits. Part V tests the claim against published measurements.

The textbook's miss classes explain why the benefit of a larger cache is uneven. Compulsory misses occur on the first access to a block and no cache size removes them; conflict misses depend on associativity; capacity misses occur because the cache cannot hold everything the program is actively using [1, sec. 11.3.2]. Only capacity misses fall as the cache grows, and they fall sharply only when the cache becomes large enough to hold a program's *working set*, the data it reuses over a period of time. Figure 29 sketches the consequence. A program whose working set fits in 32 MB sees almost no change from a 96 MB cache; one whose working set is far larger than 96 MB also sees little change, because most of its data still does not fit; only programs whose working sets lie between the two sizes move from frequent misses to frequent hits. The useful range is set by the ratio of the two capacities, three times, rather than by the extra 64 MB alone.

14 Why Expanding Cache in Two Dimensions Became Difficult

Growing the L3 in the plane of the compute die meets four related obstacles.

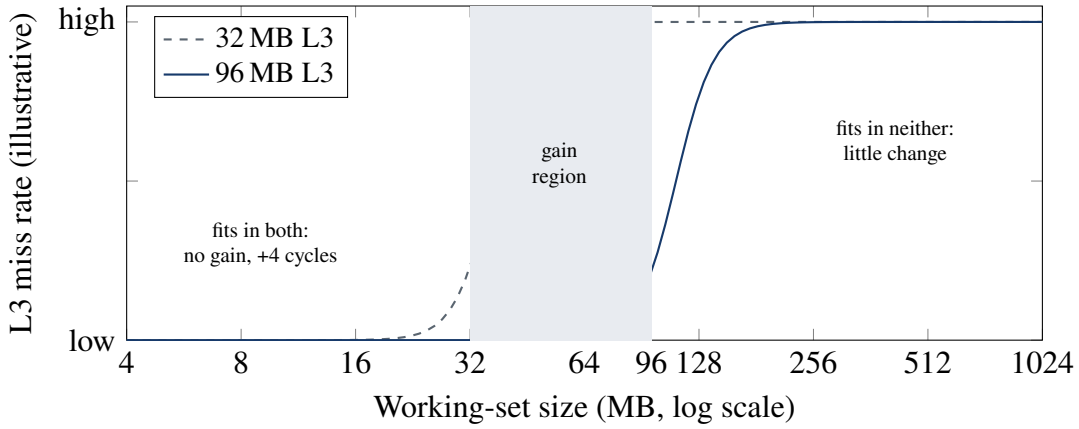


Figure 29: Conceptual shape of the L3 miss rate against working-set size for a workload with reuse, for a 32 MB and a 96 MB L3. The curves are illustrative, not measured; real transitions are smoother and depend on access pattern, associativity and sharing. They show the three workload classes of section 19.

14.1 Area and yield

SRAM cells no longer shrink at the rate of logic (section 13), so a cache of a given capacity occupies a growing share of the die at each new node. AMD lists cost, product flexibility and latency as the resulting barriers to large on-die caches [3, slide 6]. Mittal and Vetter add that, if the large caches needed by future workloads were built from SRAM, they could occupy up to 90% of the chip area [13, sec. 1]. A larger die yields fewer good dies per wafer (section 6.2), so doubling the L3 on a planar CCD would raise the cost of every CCD, including those sold for workloads that do not need the extra cache. It would also make the CCD less flexible as a building block reused across products.

14.2 Wire length and access latency

A larger planar cache spreads over a larger area, so the average distance between a core and the cache bank holding its data increases. Longer wires have more resistance and capacitance; their RC delay increases access latency, and the energy needed to charge their capacitance increases dynamic power (equation (3)). Inside each array the same effect appears at smaller scale: more cells per bit line mean more capacitance to drive, which is why large caches are divided into many small arrays and banks (section 4.2).

The effect is measurable even within today's L3. Zen 3 divides its L3 into eight slices, and measurements on a Ryzen 5 5600G show that a core reaches its nearest slice in 40 cycles but needs up to 20 cycles more, 60 in total, for the farthest one (figure 30) [20]. Because consecutive lines are spread across the slices, the average over all eight is what a program sees: $(0 + 6 + 1 + 10 + 6 + 13 + 10 + 20)/8 \approx 8$ extra cycles for core 0, or about 48 cycles in total, which matches the 47-cycle L3 latency measured on the same processor [20]. Distance on the die is therefore already a visible part of the L3's hit time; a planar L3 three times larger would spread its slices over a larger area and lengthen the farthest paths further.

In terms of equation (5), part of the miss-rate gain from a bigger cache is paid back as a longer hit time t_{LLC}. AMD makes the same argument for stacking: compared with spreading a large cache across the plane, 3D integration "improves effective memory latency" and "reduces long



Figure 30: Measured extra latency, in cycles beyond the 40-cycle minimum, for a Zen 3 core to reach each of the eight slices of its L3, on a Ryzen 5 5600G [20]. The arrangement of slices follows the source's own table; it is a logical layout, not a floorplan.

datapath and I/O's dynamic powers" [3, slide 7].

14.3 Power

Large SRAM caches consume dynamic power on every read and write and leakage power even when idle (section 3.2). Mittal and Vetter's survey of DRAM-cache architectures argues that the low density and high leakage power of SRAM make very large SRAM caches infeasible, and that such caches may need expensive cooling, such as liquid cooling, which adds substantial cost and complexity [13, sec. 1]. This is one reason researchers have explored denser technologies, such as stacked DRAM, for last-level caches.

14.4 Summary of the planar limits

Planar expansion therefore trades miss rate against die cost, hit latency and power all at once. The question for the next section is whether a separate cache die can break that coupling.

15 Side-by-Side Versus Vertically Stacked Cache Dies

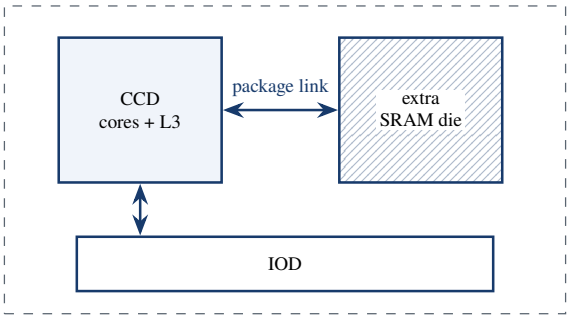
If extra cache is to be made on its own die, it can be placed either *beside* the compute die in the package or *on top of* it (figure 31). Both are valid chiplet designs; the choice between them is a trade-off.

15.1 Placing the cache die beside the compute die

A side-by-side cache die is a perfectly workable chiplet design. It could be connected through the organic substrate or, more densely, through 2.5D packaging (section 8.4), but either way it has three costs:

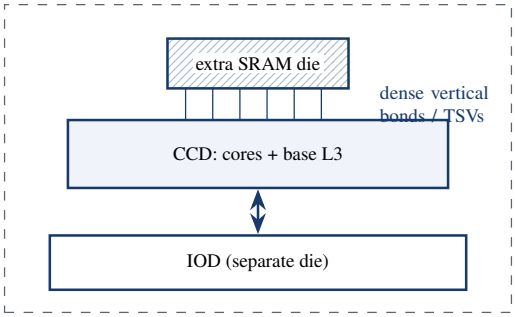
- **Package footprint.** The extra die occupies additional area next to the CCD. This enlarges the *package*, even though it does not enlarge the compute die itself. The distinction between

(a) Cache die placed beside the CCD (top view)



Larger package footprint; cache traffic crosses a side-by-side die-to-die interface.

(b) Cache die stacked on the CCD (side view)



Little extra footprint; many short vertical connections; heat path from cores is altered.

Figure 31: Two ways to add cache capacity to a compute die. Illustrative arrangements only; not actual AMD package layouts and not to scale.

die area (one piece of silicon) and *package footprint* (the area of the assembled processor) is important throughout this part.

- **Interconnect.** Every access to the extra cache must cross a die-to-die interface through the package or an interposer. Package-level connections are far coarser than on-die wiring, which limits how many parallel connections—and hence how much bandwidth—the link can offer.
- **Latency and power.** Data must travel between the dies, adding interface circuitry and wire delay to every access to the extra cache, and energy to every bit moved.

A side-by-side die does not automatically push the CCD farther from the IOD: the package designer chooses where each die goes, and the cache die could be placed on the far side of the CCD. However, package area is limited, the CCD–IOD links are already routed with care to minimise memory latency [2], and every additional die competes for the same routing resources. At high signalling rates, longer and more numerous package routes bring RC delay, power, signal-integrity and routing-congestion problems. These can be mitigated with better interconnect design, buffering and packaging technology, so they are engineering costs.

The more important comparison is therefore not the CCD-to-IOD distance but the latency and bandwidth of the path *between the cores and the additional cache*.

AMD's own framing of the choice matches this analysis. Its ISSCC 2022 presentation illustrates

a hypothetical processor whose large cache is split off onto a side-by-side chiplet. It states that "2.5D chiplets can provide product flexibility and reduce cost" but that "3D can be even better", because it improves effective memory latency, reduces long-datapath and I/O dynamic power, and "fits more transistors within a given package cavity size" [3, slide 7].

15.2 Stacking the cache die on the compute die

Vertical stacking changes the trade-off:

- **Footprint.** The cache die overlaps the CCD's existing footprint, so capacity grows with little additional package area.
- **Interconnect density.** The two dies can be joined by a very large number of short vertical connections, which supports the bandwidth a large cache needs and keeps the added distance small.
- **Thermal design.** The stack can obstruct heat flow from the compute die, depending on which die is on top and what lies above the cores. This is the central new engineering problem (section 18).

Table 16: Qualitative comparison of side-by-side and vertically stacked cache dies.

Factor	Cache die beside the CCD	Cache die stacked on the CCD
Package footprint	Larger	More compact
Connections	Side-by-side package or interposer link	Dense vertical bonds and through-silicon vias
Cache access latency	Depends on the interface design	Can benefit from short, dense connections
Cooling	Easier to arrange	More difficult; depends on stack order
Manufacturing	Extra die plus package assembly	Extra die plus bonding, thinning and stacking

Table 16 summarises the comparison. The case for vertical stacking is not that side-by-side chiplets cannot work. It is that stacking can add substantial cache capacity without enlarging the package by the same amount, with potentially better interconnect characteristics. The question worth investigating is *when the extra complexity of 3D integration is worth the smaller footprint and denser interconnect*.

16 Stacked-Die Integration and Hybrid Bonding

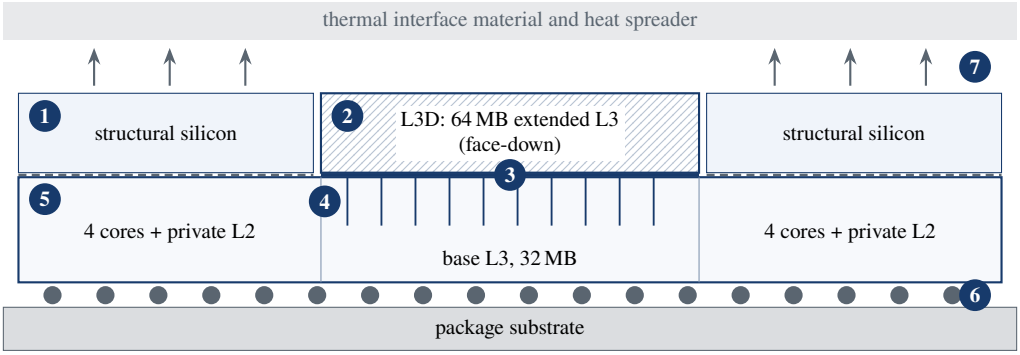
16.1 The Zen 3 3D V-Cache stack

This paper calls the arrangement of the Zen 3 and Zen 4 V-Cache parts, with the cache die on top of the CCD, the *cache-on-top stack*; section 18.2 explains why AMD's generation labels are avoided. AMD's Hot Chips 33 presentation describes the Zen 3 stack as a 64 MB L3 cache

die attached to an up-to-eight-core Zen 3 CCD by a direct copper-to-copper bond, with TSV (through-silicon via)s carrying signals and power through the CCD [6]. AMD's EPYC 7003 documentation states that the L3 cache tiles are stacked on the central L3 portion of the base CCX [11]. The ISSCC 2022 presentation describes the three components of the stack and how they are assembled [3, slides 10–12, 14, 16–17]:

- the Zen 3 CCD, built on TSMC's 7 nm process with eight cores sharing 32 MB of L3 in an area of 81 mm^2, with V-Cache support designed in "from Day 1";
- the extended L3 die (L3D in AMD's slides, which the following paragraphs describe; the cache die elsewhere in this paper), also 7 nm, carrying 64 MB of L3 in 41 mm^2, roughly half the CCD area;
- structural silicon dies, which provide mechanical support for the thinned CCD and a path for heat from the CPU cores.

The CCD is mounted face-down on the package substrate through conventional C4 bumps and connects to the L3D through TSVs. The L3D is mounted face-down on the CCD by hybrid bonding, using TSMC's System on Integrated Chips (SoIC) process with a minimum TSV pitch of 9 µm, while the structural dies are oxide-bonded beside it. Support silicon brings the stack to the same height as a plain CCD, so server parts with and without V-Cache fit the same package. The cache die sits over the CCD's central L3 region [11] and the structural silicon over the cores, so the hottest circuits are covered only by passive silicon. Figure 32 shows the arrangement in cross-section.



1 Structural silicon dies, oxide-bonded to the CCD: support the thinned CCD and conduct heat from the cores.

2 L3D: 7 nm, 41 mm^2, 64 MB of L3 over the CCD's central L3 region.

3 Hybrid bond: direct copper-to-copper connection (TSMC SoIC) between CCD and L3D.

4 Through-silicon vias in the thinned CCD; minimum 9 µm pitch.

5 CCD: Zen 3, 7 nm, 81 mm^2, eight cores; the shared L3 grows from 32 to 96 MB.

6 C4 bumps connect the face-down CCD to the substrate.

7 Heat from the cores rises through the structural silicon to the heat spreader.

Figure 32: Schematic cross-section of the Zen 3 3D V-Cache stack, drawn from AMD's descriptions [3, slides 10–12, 14, 16–17] and [11]. Not to scale; layer thicknesses and the number of TSVs are illustrative.

16.2 Hybrid bonding versus micro-bumps

Earlier 3D stacking typically joined dies with solder micro-bumps. Hybrid bonding instead bonds aligned copper pads and surrounding dielectric directly. AMD reports, from its internal engineering analysis [6]:

- a hybrid-bond pitch of 9 µm, compared with 50 to 36 µm for micro-bump 3D;
- more than 15× the interconnect density of micro-bump 3D;
- more than 3× better interconnect power efficiency than micro-bump 3D;
- lower TSV capacitance and inductance, which benefits signal and power delivery.

AMD's EPYC 7003 documentation similarly claims over 200× the interconnect density of 2D packaging and over 15× that of solder-bump 3D [11]. These are AMD's own comparisons and should be read as engineering claims.

Interconnect density matters architecturally for three reasons. First, *bandwidth*: the number of parallel connections between two dies limits how much data can cross between them per cycle, and a large L3 serving eight cores needs very wide paths; AMD's stack delivers, by AMD's figure, more than 2 TB/s [3, slide 20]. Second, *latency*: vertical connections a few micrometres long add very little wire delay, which helps explain why the extra 64 MB costs only four cycles. Third, *energy*: short, low-capacitance connections cost less energy per bit (equation (3)), which AMD quantifies as more than 3× better than micro-bump stacking [3, slide 15]. The low parasitics of hybrid bonding also let AMD use a simple digital signal interface between the dies rather than the serialising transceivers needed by package-level chiplet links [3, slide 26]. These are exactly the costs that section 15 identified for a side-by-side cache die.

16.3 What a finer pitch buys

Pitch is the distance between the centres of neighbouring connections. If pads are laid out on a square grid of pitch p, a square millimetre holds

$$n = \left(\frac{1000\,\mu\text{m}}{p}\right)^2 \tag{9}$$

connections. At the 36 µm micro-bump pitch quoted by AMD this gives $(1000/36)^2 \approx 772$ connections per mm^2; at the 9 µm hybrid-bond pitch it gives $(1000/9)^2 \approx 12\,346$, sixteen times as many, which is consistent with AMD's claim of more than 15× the density [6]. Against C4 bumps at AMD's 150 µm limit [25, slide 13], about 44 per mm^2, the hybrid bond offers roughly 280 times as many, the same order as AMD's "over 200×" comparison with 2D packaging [11]. Because density grows with the square of the pitch reduction, each halving of the pitch gives four times as many connections (figure 33).

The architectural meaning becomes clear from the area involved. Suppose, for illustration only, that a 1 mm^2 strip of the CCD were given over entirely to signal connections. At the hybrid-bond pitch it could carry about twelve thousand signals; a 32-byte (256-bit) interface for each of eight cores in both directions needs $8 \times 256 \times 2 = 4096$ data wires, a third of that budget before clocks, control and spares. At the micro-bump pitch the same interface would need more than 5 mm^2, and at the C4 pitch more than 90 mm^2, larger than the whole CCD. This is a model calculation, not AMD's layout, but it shows that such an interface is impractical at C4 pitch and would cost several mm^2 at micro-bump or 2.5D pitch, whereas hybrid bonding fits it in a fraction of a square millimetre. Package-level chiplet links, which use C4-class bumps on an organic substrate, must instead serialise data over a few fast lanes (section 8).

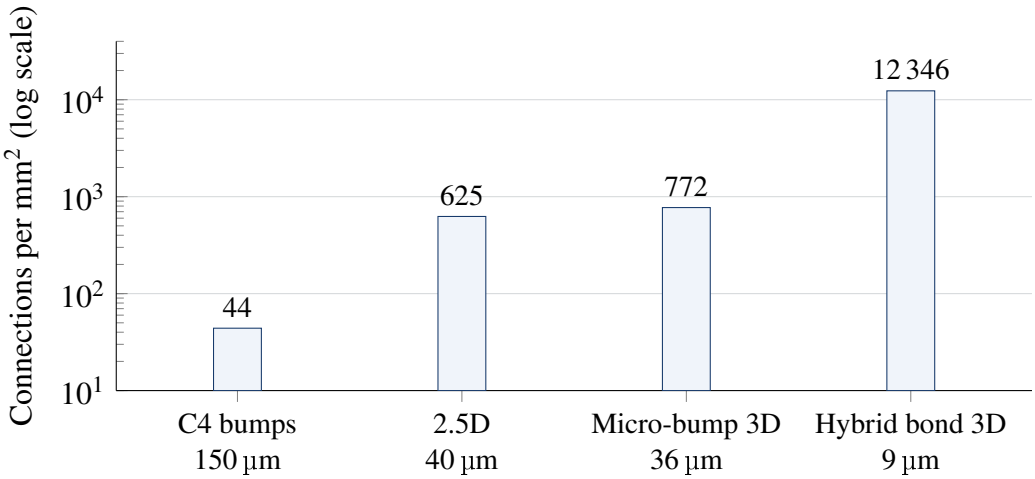


Figure 33: Connections per square millimetre for a square grid of pads at each pitch, $(1000/p)^2$ for a pitch p in micrometres. Pitches are AMD's stated limits for C4 bumps and 2.5D [25, slide 13], the finest micro-bump pitch AMD quotes, and the 9 µm hybrid-bond pitch [6]. Real layouts are not full square grids, so the values are upper bounds for comparison only.

16.4 How a hybrid bond is made

Hybrid bonding joins two surfaces that each contain both oxide and copper pads. The oxide surfaces bond first; the copper pads start slightly recessed, and during the anneal the copper diffuses across the interface and joins them [8, slide 9]. No solder is involved, and solder is what limits the pitch of micro-bumps (section 16), which is why the pitch can be so much finer.

Figure 34 shows one connection. In the Zen 3 stack, the copper pads on each die are called bond-pad metal (BPM). On the CCD, the BPM sits on the end of a TSV; on the L3D, a bond-pad via (BPV) connects the BPM to the die's top copper layer, M13 [3, slide 16]. The L3D has thirteen copper layers and one aluminium layer [3, slide 11]. For Zen 4, AMD changed this detail: the stack became explicitly face-to-back, with a 7 nm cache die on a 5 nm CCD, and the BPV lands on the top aluminium pad instead of the top copper layer [8, slides 11–12]. The process is TSMC's SoIC [3, slide 16].

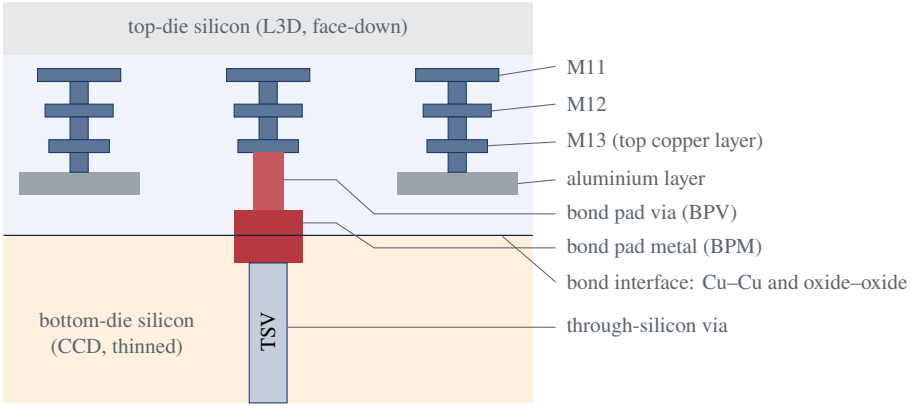


Figure 34: Schematic of one hybrid-bond connection in the Zen 3 stack, redrawn from AMD's description [3, slide 16]. Copper bond-pad metal (BPM) on the two dies is bonded directly; on the top die a bond-pad via (BPV) connects the pad to the top copper layer (M13), and on the bottom die the pad lands on a TSV. In the Zen 4 stack the BPV lands on the top aluminium pad instead [8, slides 11–12]. Not to scale.

17 Cache Access, Latency, Bandwidth and Hierarchy

With the cache die, the L3 of a Zen 3 CCD grows from 32 MB to 96 MB, shared by up to eight cores [3], [11]; an EPYC 7003 processor with eight such CCDs carries up to 768 MB of L3 [11]. AMD reports more than 2 TB/s of bandwidth for the V-Cache [3]. The extra capacity remains part of the same L3 level, so the cache hierarchy above it (private L1 and L2 per core) is unchanged [11].

AMD's presentation gives the cost of the extra capacity in latency: the 96 MB L3 remains 16-way set associative with a 32 B/cycle interface to each core, delivers more than 2 TB/s of L3 bandwidth, and adds *four cycles* of latency [3, slide 20]. Independent measurements agree: Chips and Cheese measured roughly three to four extra cycles of L3 latency on Milan-X compared with Milan [23], about four extra cycles on the V-Cache CCD of the Zen 4 Ryzen 9 7950X3D [22], and the same four cycles on the Zen 5 Ryzen 7 9800X3D, whose cache die sits below the CCD [32]. AMD's bandwidth figure is a peak capability: on Milan-X, single-thread L3 bandwidth measured about 25% more bytes per cycle than Milan rather than the doubling AMD had announced [23]. Each die's portion of the L3 has its own data, tag and LRU arrays, so a tag lookup does not have to cross between dies before the data array is accessed [3, slide 20].

Equation (5) shows how to judge this trade-off. For an access that has already missed in L2, the extra capacity changes the average cost by approximately

$$\Delta T \;\approx\; \Delta t_{\mathrm{L3}} \;-\; \Delta m_{\mathrm{L3}} \cdot t_{\mathrm{mem}}, \tag{10}$$

where $\Delta t_{\mathrm{L3}} = 4$ cycles is the added hit latency, Δm_{L3} the reduction in the L3 miss rate and t_{mem} the main-memory latency in cycles. The textbook puts the stall for a miss that goes to main memory at 100–300 cycles [1, sec. 12.4.8]. Over that range, the larger L3 pays off once it removes more than about $4/300 \approx 1.3$ to $4/100 = 4$ percentage points of L3 misses. A workload whose working set already fits in 32 MB gains nothing and pays the four cycles on every L3 hit; a workload whose working set lies between 32 MB and 96 MB can gain substantially.

17.1 Applying the trade-off to a measured workload

Lam's fixed-frequency measurements of the Ryzen 9 7950X3D give enough data to apply equation (10) to a real program [22]. For libx264 video encoding, the plain CCD had 1.48 L3 misses per thousand instructions (MPKI). Lam's charts give an L3 hit rate of 61.55% and an IPC of 1.36 on the plain CCD, against 71.86% and 1.43 on the V-Cache CCD. (The article's text gives the second hit rate as 72.8%, but its chart and its stated 16.75% improvement both correspond to 71.86%, which is used here.)

1. **L3 accesses.** If 38.45% of L3 accesses miss and this gives 1.48 misses per thousand instructions, the program makes $1.48/0.3845 \approx 3.85$ L3 accesses per thousand instructions.

2. **Misses removed.** With V-Cache, $3.85 \times 0.2814 \approx 1.08$ of those accesses still miss, so about $1.48 - 1.08 = 0.40$ misses per thousand instructions are removed.

3. **Cycles saved.** At the textbook's 100–300 cycles per memory access [1, sec. 12.4.8], that saves $0.40 \times (100 \text{ to } 300) \approx 40$ to 120 cycles per thousand instructions.

4. **Cycles added.** Every L3 access pays four more cycles: $3.85 \times 4 \approx 15$ cycles per thousand instructions.

5. **Net effect.** About 25 to 105 cycles saved per thousand instructions.

At an IPC of 1.36, a thousand instructions take $1000/1.36 \approx 735$ cycles, so the model predicts 3–14% fewer cycles. The measurement lies inside that range: at an IPC of 1.43 the same work takes about 699 cycles, 4.9% fewer (an IPC gain of 5.1% from the rounded chart values; Lam states 4.9%). Figure 35 shows how this change divides between the two effects.

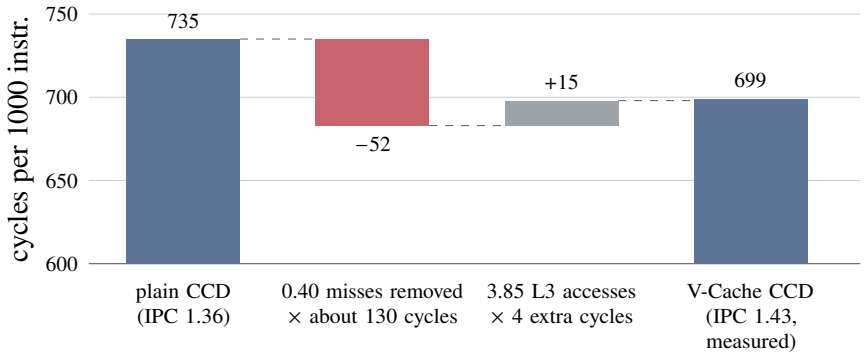


Figure 35: The libx264 calculation of section 17.1 as a sequence: starting from the plain CCD's 735 cycles per thousand instructions, the misses removed by the larger L3 save about 52 cycles and the four extra cycles on every L3 access cost about 15, leaving about 698 against the 699 implied by the measured IPC. The stall of about 130 cycles per removed miss is itself inferred from the measurement, so the chart shows how the measured change divides between the two effects rather than predicting it independently. Both CCDs fixed at 4.2 GHz; data from Lam [22]. The vertical axis starts at 600.

The measurement can also be turned around. The observed saving of about 36 cycles per thousand instructions, plus the 15 cycles of added L3 latency, implies that each removed miss saved about $(36 + 15)/0.40 \approx 130$ cycles. That is at the low end of the textbook's 100–300-cycle range, which is expected if an out-of-order core overlaps some memory misses with other work (section 4.8): removing a miss then saves less than its full latency.

The 130 cycles should be read as an *apparent* saving per counted miss, for two reasons. First, Lam counts L3 accesses from the L3's side, including speculative accesses and without separating prefetches from demand requests [22]. A removed prefetch miss need not have stalled the core at all, so prefetching is a second explanation, besides overlap, for a saving below the full latency. Second, the calculation rests on four assumptions besides the textbook's latency range. They are that all of the IPC change comes from L3 misses and L3 latency; that every L3 access pays all four extra cycles; that both CCDs make the same number of L3 accesses per thousand instructions; and that the counted misses are the ones the core waits for. The third can be checked on the one workload for which Lam states both MPKIs, the flight simulator DCS discussed next. Its 0.35 misses at a 10.9% miss rate imply 3.2 L3 accesses per thousand instructions on the plain CCD, but 0.28 at 7.6% imply 3.7 on the V-Cache CCD, about 15% more; the method would have predicted 0.24 MPKI with V-Cache rather than the 0.28 measured. The 130 cycles is therefore an estimate with an uncertainty of this order, not a measured stall.

The same arithmetic explains the small loss Lam observed in a flight simulator with only 0.35 MPKI, falling to 0.28 with V-Cache [22]. Removing 0.07 misses per thousand instructions saves at most 7–21 cycles, which can be outweighed by four extra cycles on each of its L3 hits.

The model's range brackets the measurement, and both agree with the miss-class argument of section 13.

17.2 Inside the cache interface

AMD's ISSCC presentation describes how the stacked half of the L3 is organised and connected, and the details explain why the latency cost is so small (figure 36).

Where the connections are. On the CCD, two columns of signal TSVs run along either side of the central 32 MB L3, between the L3 and the two rows of four cores [3, slide 10]. The L3D has matching "CCD signal interface" columns directly above them [3, slide 11]. The L3 already sits at the centre of the CCD, with every core's 32 B/cycle interface entering it from the side [3, slides 19, 21], so the stacked cache is reached through the same central region rather than through a new path across the die. AMD states that support for V-Cache was integrated in the CCD "from Day 1" [3, slide 10]: the TSV columns occupy CCD area in every Zen 3 CCD, including those sold without a cache die.

How the stacked arrays are organised. The L3D holds 512 data macros and 1088 tag and LRU macros, so each portion of the L3 keeps its own tags and the lookup does not cross between dies. Appendix D works through the macro counts, the cost of the tags (about 10% of the data capacity) and the set count of the combined 96 MB cache [3, slides 20, 25].

Checking the bandwidth claim. Each core has a 32 B/cycle interface to the L3, and AMD describes the stacked connection as a 32 B/cycle bidirectional bus [3, slides 20–21]. If all eight cores transfer 32 bytes per cycle in each direction at an assumed L3 clock of 4 GHz, the total is

$$8 \times 32\,\mathrm{B} \times 2 \times 4 \times 10^{9}\,\mathrm{s}^{-1} \;\approx\; 2.0\,\mathrm{TB/s},$$

consistent with AMD's "more than 2 TB/s" [3, slide 20]. The figure is therefore an aggregate peak across all cores and both directions, and no independent test has reached it. Chips and Cheese's single-thread measurement on Milan-X found about 25% more bytes per cycle than on Milan, with an estimated clock, but its test of one whole CCD found Milan-X about 12.5% *lower* in bytes per cycle [23]. Lam's all-core test on the 7950X3D, at a fixed 4.2 GHz, found the V-Cache CCD averaging 18.45 bytes per cycle per core, slightly below the plain CCD [22]. For eight cores that is about $8 \times 18.45 \times 4.2 \times 10^{9} \approx 0.62$ TB/s at the 4.2 GHz of that test, or about 0.71 TB/s at the 4.8 GHz the V-Cache CCD sustained at stock clocks, consistent with the 702 GB/s of Lam's multithreaded test. The 2 TB/s is therefore AMD's peak for the interface, not a bandwidth that programs have been shown to obtain.

A simple signal interface. Because hybrid-bond connections have low parasitics, each signal crosses between the dies through an ordinary digital path: a flip-flop and driver on one side, the TSV, electrostatic-discharge protection and an isolation stage on the other, and a receiving flip-flop [3, slide 26]. No serialiser, equaliser or separate link clock is needed. By contrast, the CCD-to-IOD link of section 8 is a serial interface with a round trip of 13 fabric clocks. This difference probably explains much of why a stacked cache can add only four cycles while a die-to-die hop through the package costs several nanoseconds (under 9 ns for the round trip in Rome [2]), against about 1 ns for four cycles at 4 GHz.

The L3 remains private to each CCD, so a thread running on one CCD does not benefit from another CCD's stacked cache.

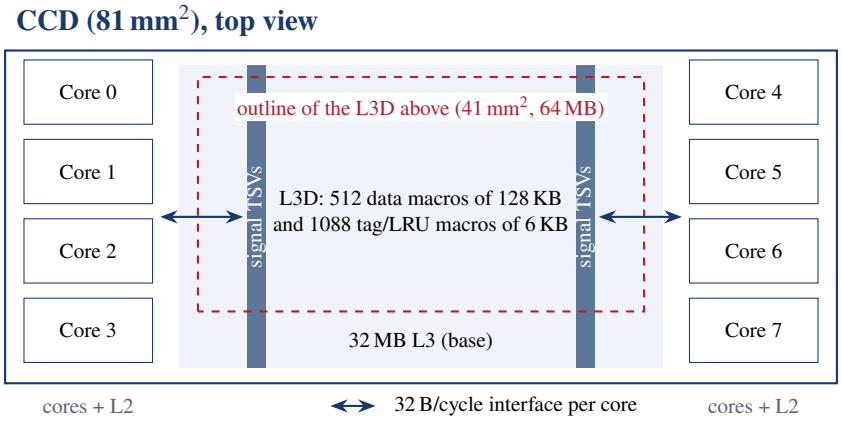


Figure 36: Top view of how the L3D connects to the CCD, redrawn from AMD's descriptions [3, slides 10–11, 19–21, 25]. Two columns of signal TSVs run along the CCD's central L3, and matching interface columns on the L3D lie directly above them. Each core reaches the L3 over a 32 B/cycle interface. Positions and sizes are schematic; the L3D outline is drawn only approximately over the L3 region.

18 Thermal Management, Power Delivery and Manufacturing Complexity

Stacking changes the heat path described in section 3.2: heat from the cores must now pass through additional silicon and bonding layers before it reaches the heat spreader. The cache-on-top design limits the penalty by placing the cache die over the CCD's central L3 region [11] and covering the cores only with structural silicon, whose stated purposes include "thermal dissipation for CPU cores" [3, slide 12]. AMD also credits hybrid bonding with "superior thermal conductance" compared with micro-bump stacking [3, slide 15], although this is an AMD claim based on internal simulation. Earlier academic work gives a sense of scale. Black *et al.* simulated an extra 8 MB of SRAM stacked on a Core 2 Duo-class processor: the peak temperature rose from 88.35 °C to 92.85 °C, whereas a 32 MB stacked DRAM cache raised it only to 88.43 °C, because SRAM has the higher power density [14]. These are 2006 simulations of an older stacking technology and are indicative only.

Equation (4) gives a way to reason about these numbers. Treat a core's hot spot as one grid point with power P and an effective thermal resistance R to ambient, so that its temperature rise is $\Delta T = RP$. Any layer added above the core adds its own resistance in series, raising R and therefore ΔT for the same power. As an illustration with assumed values only: a hot spot that runs 60 K above ambient through its original heat path runs 66 K above ambient if the added layer raises R by 10%. Because leakage rises with temperature, the extra 6 K also raises P slightly, and the feedback loop of section 3.2 increases the final temperature further. The designer then has two choices: accept the higher temperature, or lower voltage and frequency until P falls enough to compensate. The measured clock and voltage limits of the cache-on-top parts (section 18.2) are consistent with the second choice, although the sources attribute them at least partly to the stacked cache's lower voltage tolerance rather than to temperature alone [22]. Black *et al.*'s simulations show the same effect in reverse: the stacked SRAM die adds both its own power and a layer, so its 4.5 K rise is much larger than that of a lower-power stacked DRAM [14].

Figure 37 collects the costs of stacking discussed in this section and AMD's responses to them.

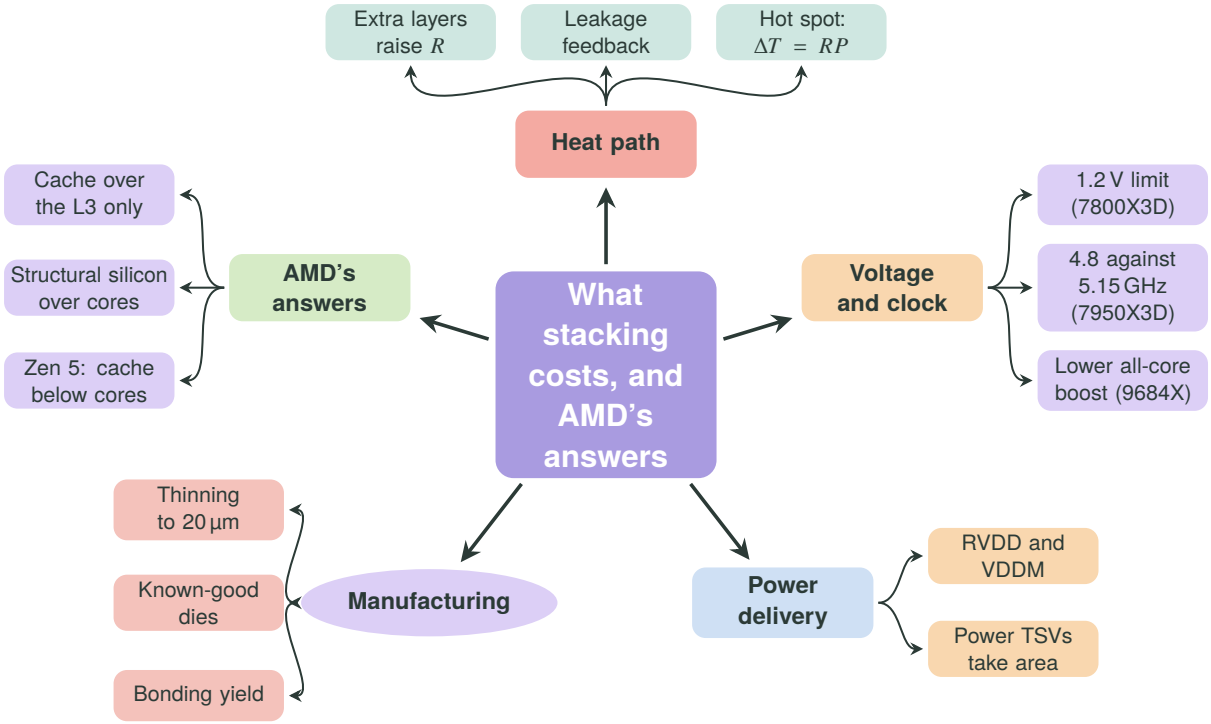


Figure 37: Overview of the costs of stacking discussed in section 18, with the measurements cited there, and of AMD's design responses. The sources attribute the voltage and clock limits at least partly to the stacked cache's voltage tolerance as well as to heat (section 18.2).

18.1 Power delivery through the stack

Power must also be delivered upwards, through the CCD, to the cache die. The Zen 3 CCD has three primary supplies: an ungated supply (RVDD), a per-core gated supply (VDD), and a gated supply for the L2 and L3 SRAM arrays (VDDM) [3, slide 23]. The L3D needs two of these, RVDD and VDDM, and receives them through dedicated power TSVs placed in channels between the CCD's array macros [3, slide 24]. Inside the L3D the same split continues: VDDM powers the bit cells and RVDD the peripheral circuits [3, slide 25]. Supplying the bit cells separately lets them run at the voltage that keeps SRAM data stable, while the periphery can follow the rest of the logic. Each of these power paths is a set of TSVs that occupies CCD area and must carry current with little voltage drop. Hybrid bonding's lower TSV capacitance and inductance help here as well [6].

18.2 Evidence of the thermal cost and AMD's response

AMD's own naming is inconsistent: one 2024 AMD presentation calls the Zen 4 stack the "2nd generation" [8], while AMD's 2024 press release and a 2025 presentation use the same label for the inverted Zen 5 stack [10], [25]. This paper therefore names designs by product where it can, and calls the arrangement with the cache die on top, used in the Zen 3 and Zen 4 parts, the cache-on-top stack.

Measurements of shipping parts show that the cache-on-top arrangement did carry a cost. In the cache-on-top parts the CCD is thinned to about 20 µm, and the V-Cache parts accepted a lower voltage limit: the Zen 4 Ryzen 7 7800X3D has a maximum of 1.2 V [32]. Clock speeds fall

accordingly. On the Ryzen 9 7950X3D, which pairs one V-Cache CCD with one plain CCD, the plain CCD clocks about 7% higher on average, and under an all-core load on an L3-sized data set the V-Cache CCD sustained 4.8 GHz against 5.15 GHz [22]. On servers, the 96-core EPYC 9684X with V-Cache has a lower all-core boost than the otherwise similar EPYC 9654 (3.42 against 3.55 GHz) and a higher default power (400 against 360 W), although its base clock is higher (2.55 against 2.40 GHz) and both reach the same 3.70 GHz maximum boost [27].

The Zen 4 generation kept the cache-on-top arrangement: a 5 nm CCD and a 7 nm cache die, both face-down, joined by hybrid bonding at a 9 µm minimum pitch, with structural dies alongside [8]. For the Zen 5 Ryzen 9000X3D series AMD inverted the stack. AMD states that the 64 MB cache "has been relocated below the processor, which puts the core complex die (CCD) closer to the cooling solution" [10], and its packaging engineers describe the new arrangement as placing the CPU die on top of the stack with face-to-back hybrid bonding at below 9 µm pitch [25, slides 6–7]. The two structural dies that sat beside the cache die, directly over the cores, are gone (figure 38); AMD's schematic still shows a structural silicon layer above the whole stack [25, slide 7]. Independent testing found that the Ryzen 7 9800X3D held its 4.7 GHz base clock while running at 95 °C, and characterised the change as removing the biggest weakness of the cache-on-top design, the extra thermal resistance of the support silicon over the cores; a boost-clock deficit of about 300 MHz against the equivalent non-V-Cache part remains [32]. The redesign is consistent with heat removal having been a major constraint of the first design. It was not the only one: both Cozma and Lam describe the stacked cache of the cache-on-top parts as less tolerant of high voltage, which also limited boost clocks [22], [32]. How the clock penalty divides between the two causes is left open (section 41).

What the remaining deficit suggests. The Zen 5 part allows a rough separation of the two causes, and one result is firm: the voltage limit cannot explain the clock deficit that remains. Cozma reports that the inverted stack also removed the voltage handicap: the 9800X3D and its plain counterpart, the Ryzen 7 9700X, both have a maximum of 1.4 V and a nominal 1.28–1.31 V, against 1.2 V maximum and 1.1 V nominal for the Zen 4 7800X3D, which uses the cache-on-top arrangement [32]. The 9800X3D also clocked uniformly, within about 10 MHz across its cores, against about 70 MHz on the 7950X3D's V-Cache CCD [32]. Yet the 300 MHz boost deficit against the 9700X remains, about 5.5% of the 9700X's implied 5.5 GHz given the 9800X3D's 5.2 GHz maximum boost [32], [33]. The gaps on the cache-on-top parts were 7% between the two CCDs of a 7950X3D and about 10% between the 5800X3D and 5950X [22]. With the voltage limit equalised and the support silicon over the cores gone, a deficit of similar size therefore remained. It is either a remaining thermal cost, since AMD's schematic still shows a silicon layer above the whole stack, or a cause that the sources do not name: power delivery up through the cache die, which Cozma notes now carries additional power distribution under the cores [32], or the binning and positioning of the product. The comparison is indicative only: it mixes rated maximum clocks with Lam's measured average gap and spans three generations, and Cozma's 9800X3D was a sample supplied by AMD [32].

18.3 Manufacturing: thinning, assembly, yield and test

Stacking adds steps that a planar die does not need. The CCD is thinned, to about 20 µm in the cache-on-top parts [32]; a die that thin cannot support itself, which is one reason for the structural silicon [3, slide 12]. Support silicon then restores the stack to the height of an ordinary CCD, so that server packages with and without V-Cache are the same [3, slide 17]. Each of

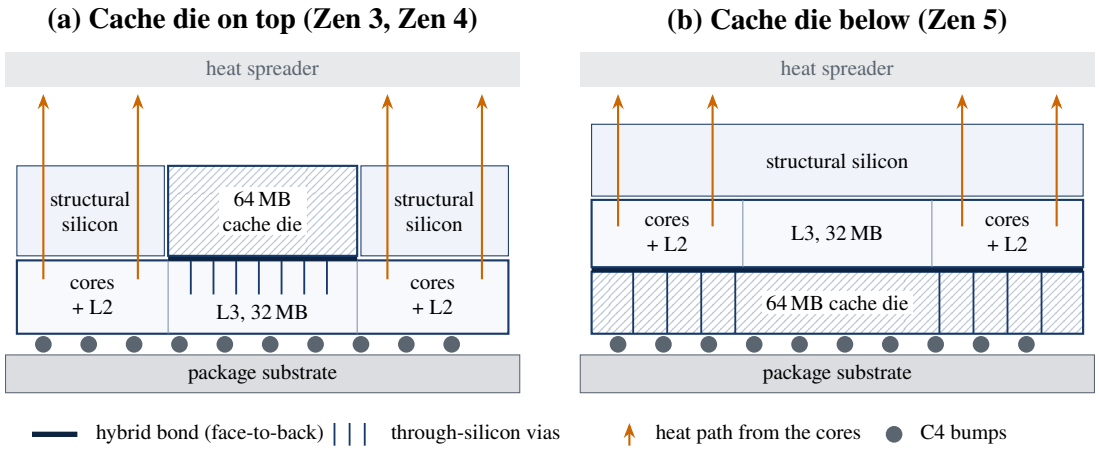


Figure 38: The two stack arrangements of 3D V-Cache, drawn from AMD's comparison [25, slide 7] and descriptions [3], [8], [32]. In (a) the cache die sits over the CCD's central L3 and the cores lie under structural silicon dies; in (b) the cache die, now the full width of the CCD, carries the through-silicon vias and the cores sit directly beneath a single structural layer. Schematic; not to scale.

these steps costs money and adds a new way for a part to fail. Figure 39 collects the extra steps described in this part.

Die-to-wafer or wafer-to-wafer? Dies can be bonded one at a time onto a wafer of other dies (die-to-wafer, D2W), or two whole wafers can be bonded together (wafer-to-wafer, W2W). Dong and Xie summarise the trade-off: D2W can achieve a higher yield because each die can be tested first and only *known-good dies* (KGD) stacked, whereas W2W needs no test before bonding and is easier to align, with higher throughput, at the cost of yield [15, sec. IV.C]. AMD's 2024 comparison adds that in D2W the top and bottom dies need not be the same size and that D2W allows more heterogeneous integration, while W2W requires dies of the same or similar size [8, slide 10]. V-Cache stacks a 41 mm^2 cache die on an 81 mm^2 CCD, dies of different sizes from different wafers, so it falls on the D2W side of this comparison.

Dong and Xie's cost model shows why testing before bonding matters. For an N-layer stack bonded die-to-wafer, the cost of the bare stack is

$$C_{\text{D2W}} = \frac{\sum_{i=1}^{N} \left(C_{\text{die},i} + C_{\text{KGD test}}\right) / Y_{\text{die},i} \;+\; (N-1)\, C_{\text{bonding}}}{Y_{\text{bonding}}^{N-1}}, \tag{11}$$

while for wafer-to-wafer bonding the die yields multiply:

$$C_{\text{W2W}} = \frac{\sum_{i=1}^{N} C_{\text{die},i} \;+\; (N-1)\, C_{\text{bonding}}}{\prod_{i=1}^{N} Y_{\text{die},i}\; Y_{\text{bonding}}^{N-1}} \tag{12}$$

[15, Eqs. 16–17]. In W2W, a defect in either die scraps the whole stack, so the stack's yield is as low as that of a single large die; Dong and Xie find W2W stacking always more expensive than 2D in their study, and D2W cheaper than W2W from the yield perspective [15, sec. IV.C].

Does stacking save silicon? Compare a hypothetical planar CCD with all 96 MB of L3 on one die against the actual two-die stack, using the yield model of equation (8) with $D_0 = 0.2\,\text{cm}^{-2}$ and $Y_{\text{wafer}} = 1$. The planar die is taken as the sum of the two real dies, $81 + 41 = 122\,\text{mm}^2$; this ignores the area of the TSVs and the possibility of a denser layout, so it is only an approximation.

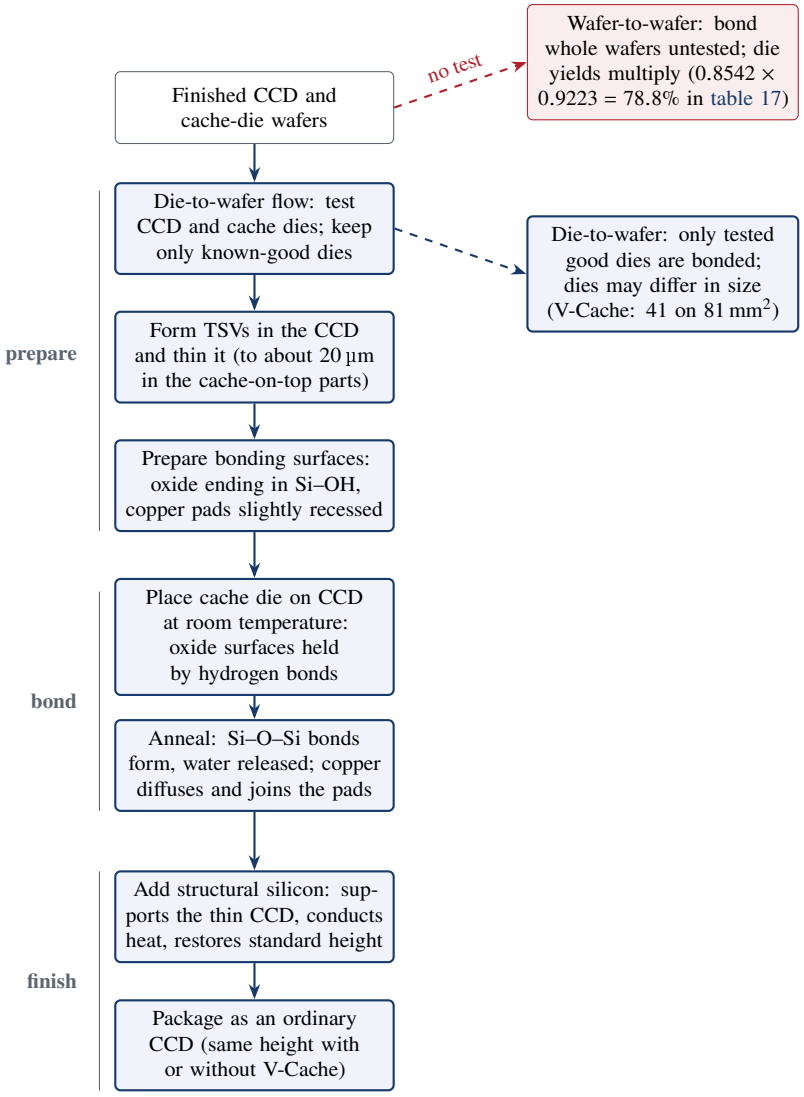


Figure 39: The steps that a V-Cache CCD needs and a planar one does not, grouped by purpose. The sources describe the steps but not their exact order in AMD's production flow, so the order shown is indicative. Sources: thinning [32]; bonding chemistry [8, slide 9]; structural silicon and package height [3, slides 12, 17]; die-to-wafer against wafer-to-wafer [15], [8, slide 10].

Table 17 gives the result.

With known-good-die testing, the stack consumes about 10% less silicon than the planar die, as long as bonding loses fewer than about 10% of stacks ($139/154 \approx 0.90$). With $D_0 = 0.1$ the saving shrinks to about 5%, and with $D_0 = 0.3$ it grows to about 14%. Without testing, the stack has almost exactly the planar die's yield, as Dong and Xie's analysis predicts, and gains nothing. The silicon saving is modest; thinning, bonding and testing all cost extra, so on this measure alone stacking is not obviously cheaper. The stronger economic argument is the one AMD gives: *product flexibility* [3, slide 6]. The same 81 mm^2 CCD is sold with or without a cache die, so only the products that need 96 MB pay for it, while a planar 122 mm^2 CCD would charge every product for the larger die.

Silicon across a product line. The flexibility argument can be put in the same units as table 17. Appendix C.4 compares, in silicon per good part, three ways of building a product line in which some parts need 96 MB of L3: one planar 96 MB die for everything, two separate planar dies,

Table 17: Silicon consumed per good part for a planar 96 MB CCD and the stacked design (Murphy model, $D_0 = 0.2\,\mathrm{cm}^{-2}$; illustrative).

Design	Die area(s)	Modelled yield	Silicon per good part
Planar CCD with 96 MB L3	122 mm^2	79.1%	154 mm^2
Stack, dies tested first (D2W)	81 + 41 mm^2	85.4% and 92.2%	94.8 + 44.5 = 139 mm^2 $\div Y_{\text{bonding}}$
Stack, untested (W2W)	81 + 41 mm^2	$0.8542 \times 0.9223 =$ 78.8%	122/0.788 = 155 mm^2 $\div Y_{\text{bonding}}$

and one CCD with an optional cache die. Against the single large die, stacking saves about a third of the silicon when few parts need the large cache, because the parts that do not need it no longer pay for it. Against two planar dies, the saving in silicon is only a few per cent, so stacking's real advantage there is outside the calculation: one CCD design instead of two, and a product mix decided at assembly rather than when wafers are started. This is how AMD describes its products: the Zen 3 V-Cache parts use a thinned version of the standard CCD and fit the same server package [3, slide 17], and in the fourth-generation EPYC processors the additional cache is stacked "on top of each CPU die" of the standard design [9].

The node of the cache die adds a second, separate saving. In the Zen 4 generation the CCD moved to 5 nm while the cache die stayed on 7 nm [8]. AMD's own chart of normalised cost per yielded mm^2 rises by roughly a third from 7 nm to 5 nm (read from the chart; approximate), while the same slide shows SRAM area scaling falling behind logic at those nodes [3, slide 6]. A planar Zen 4 CCD with 96 MB would have placed its extra 64 MB of SRAM on the more expensive node, where it shrinks less than logic does. Stacking lets the SRAM stay on the older, cheaper node while the cores move on, the same reasoning that keeps the I/O die on an older node in the chiplet split (section 6.2).

Reliability. A bonded interface must survive years of heating and cooling. AMD reports that packaged Zen 4 V-Cache parts passed 96 hours of unbiased highly accelerated stress testing, 2500 temperature cycles and 1000 hours of high-temperature storage, with no apparent damage to the bond interface [8, slide 14]. These are AMD's qualification results, not independent tests.

Co-design. AMD's own summary of the project is that "product definition and design [were] co-optimized up front with technology development" [3, slide 33]. The TSV columns, power channels and isolation circuits had to be in the CCD before the first V-Cache product existed, and the CCD's floorplan, with the L3 at the centre and the cores at the edges, made a cool, central place for the cache die. Stacked cache was therefore a design decision taken years before it shipped, not a packaging option added at the end.

19 Workloads That Benefit From a Larger LLC

AMD's figures for V-Cache in games come from two different comparisons. Its Hot Chips 33 comparison of a 12-core 3D-chiplet prototype with the 12-core Ryzen 9 5900X, both fixed at 4.0 GHz, reports gains from 1.04× to 1.25× across five games, 15% on average [6, slide 19]. Holding frequency fixed is valuable because it isolates the cache effect, but the result covers only the disclosed games and configuration and should not be generalised.

The ISSCC 2022 presentation adds product results, again from AMD's own testing [3, slides 29, 31, 35]. The eight-core Ryzen 7 5800X3D is reported as about 15% faster in games than the twelve-core Ryzen 9 5900X at 1080p with high settings, ranging from up to 1.36× in one title to a tie in another. On servers, a 16-core EPYC with V-Cache completed about 66% more electronic-design jobs (register-transfer-level, or RTL, simulation of chip designs) per hour than a 16-core EPYC 73F3 without it. The spread from a tie to 1.36× already illustrates the workload dependence predicted in section 4.4; section 26 discusses what else differs between the compared parts.

The miss classification of section 4.4 predicts which workloads should benefit. A larger L3 removes capacity misses, so the gain depends on where a workload's working set lies relative to the 32 MB and 96 MB capacities:

- **Working set already within 32 MB.** Few capacity misses remain to remove, and every L3 hit pays the extra four cycles. Little or no gain, possibly a small loss.
- **Working set between 32 MB and 96 MB, with reuse.** Many accesses that went to memory now hit in the L3. This is where large gains are expected.
- **Working set far larger than 96 MB, or streaming with little reuse.** The data still does not fit or is not reused, so the miss rate barely changes.

Mittal and Vetter note that modern scale-out workloads such as business analytics and web search can demand hundreds of megabytes of last-level cache [13, sec. 1]. AMD's gaming and RTL-simulation results above fit this picture, but they are vendor results and Part V examines them more critically.

Independent measurements support this classification. Lam measured each CCD of a Ryzen 9 7950X3D separately, with boost disabled and both CCDs held at 4.2 GHz, and used performance counters to compare IPC and L3 hit rate [22]. Because clock, core design, memory and software are identical, the differences isolate the cache. Workloads with many L3 misses gained most: IPC rose by over 19% in *Call of Duty: Black Ops Cold War*, whose hit rate rose by 47%, and by 13% in *Cyberpunk 2077* (from 1.26 to 1.43). Workloads with few misses to remove gained little: libx264 video encoding, with 1.48 L3 misses per thousand instructions, gained 4.9%, less than the plain CCD's 7% clock advantage, and a flight simulator with only 0.35 misses per thousand instructions lost a little IPC, which Lam suggests is likely due to the added latency. These are the first two classes above, measured directly (figure 40); section 17.1 works through the libx264 case with equation (10).

HPC studies show the third class. On a memory-bound proxy application, Domke *et al.* found that Milan-X was up to 3.4× faster than Milan, but only for a subset of problem sizes [16]. Lawenda *et al.* found the same pattern in OpenFOAM: Milan-X was up to 1.54× faster than Milan for the mesh size that best fits its larger cache, the advantage fell to 1.24× for the largest mesh, and on some smaller meshes Milan-X was up to 19% slower. The larger L3 cut the L3 miss rate by up to 30 percentage points [34]. On desktop software, Phoronix's 420-test comparison of the eight-core Ryzen 7 5800X3D with the eight-core 5800X found large V-Cache leads in a minority of tests, while the 5800X won, mostly by a couple of percent, where the cache could not be used and its slightly higher clocks paid off [35].

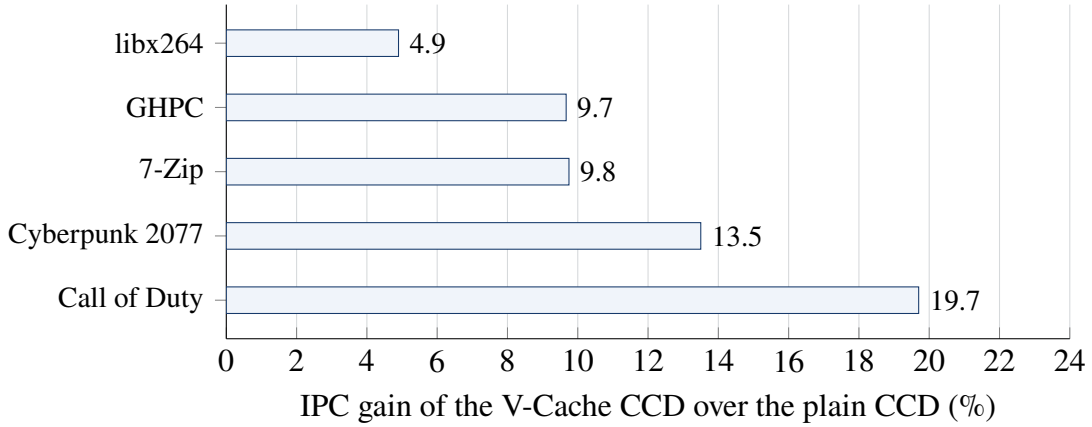


Figure 40: IPC gain from 3D V-Cache with clock, core design and memory held equal: the two CCDs of one Ryzen 9 7950X3D, both fixed at 4.2 GHz, measured with performance counters by Lam [22]. Cyberpunk 2077 is computed from the reported IPC of 1.26 and 1.43; the Call of Duty value is computed from the IPC of 0.71 and 0.85 read from Lam's chart (the text says "over 19%"). A sixth workload, a flight simulator with very few L3 misses, lost a small, unquantified amount of IPC and is not plotted.

20 Summary of Part IV

Applying the three-part test to 3D-stacked cache:

- **Bottleneck addressed.** The area, yield, wire-length and power costs of growing an SRAM cache in the plane of the compute die, made worse by SRAM's slowing area scaling (section 14).
- **Benefit.** Three times the L3 per CCD (32 to 96 MB) for four extra cycles of latency, with AMD's quoted peak of more than 2 TB/s (not reproduced by independent CCD-wide tests, section 17) and no change to the package footprint or the cache hierarchy above the L3 [3], [11].
- **New trade-offs.** An altered heat path from the cores, power delivery through TSVs, die thinning, bonding and test complexity, a second die to pay for, and a benefit that depends strongly on the workload.

The calculations of this part make these points quantitative. The 9 µm bond pitch gives about sixteen times the connection density of micro-bumps (section 16.3), which allows a wide, parallel interface in little area and helps keep the latency cost to four cycles. Applied to Lam's libx264 measurements, the simple trade-off of equation (10) gives a range of 3–14% that brackets the measured gain (section 17.1). The yield model suggests that stacking with tested dies saves only about a tenth of the silicon of an equivalent planar die (table 17). The economic case therefore rests mainly on product flexibility: one CCD with an optional cache die uses about as little silicon as two separate planar designs while needing only one design, and lets the SRAM stay on an older node (table 45). The heat path, which the cache-on-top parts managed with structural silicon, is the reason AMD gives for inverting the stack in Zen 5 (section 18.2); the lower clocks of those parts are attributed partly to the stacked cache's voltage tolerance as well. The inverted part still clocks about 5.5% below its plain counterpart with an equal voltage limit, which suggests that factors besides the two named in the sources remain. Part V now turns from the mechanism to the evidence across products.

Part V

AMD Case Study and Quantitative Analysis

Figure 41 maps the topics of this part.

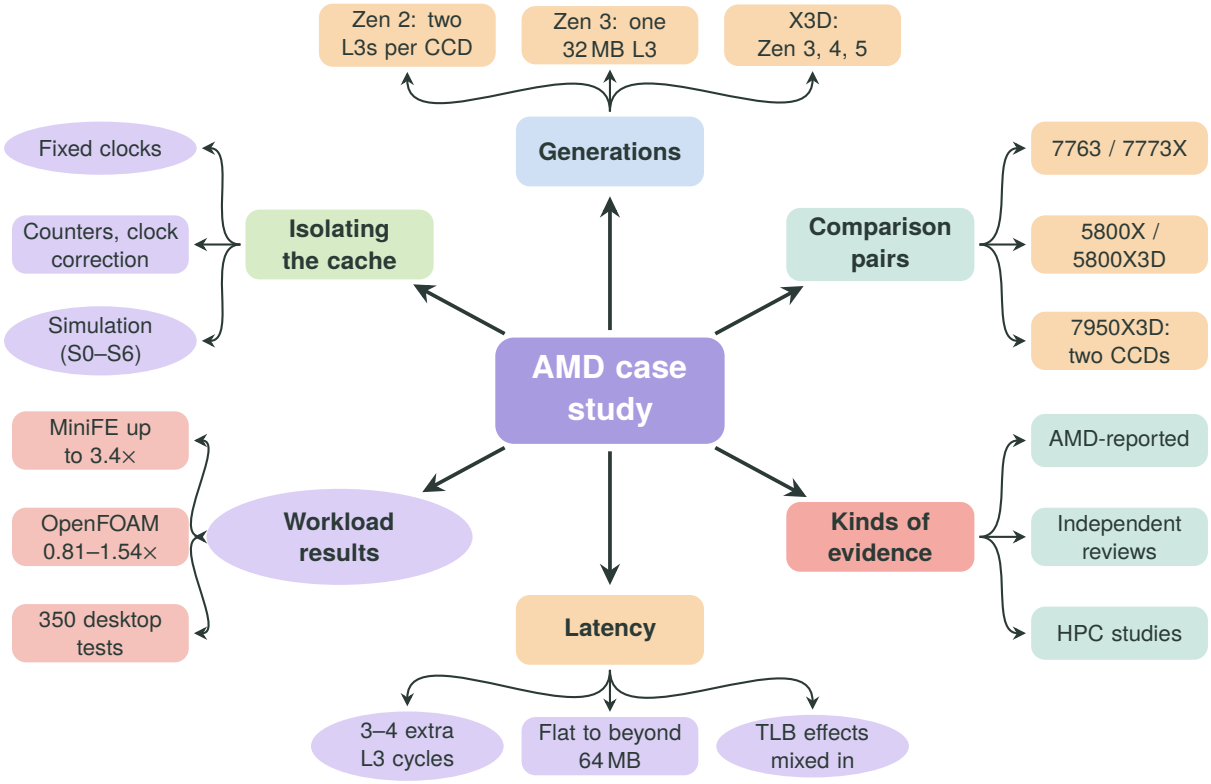


Figure 41: Overview of Part V: the products, the comparisons, the kinds of evidence and the main results, with values as reported in sections 24 and 26; the simulation is section 28.

21 Method for Analysing Published Evidence

The quantitative analysis uses published research and benchmarks (figure 42), complemented by a simulation that changes one factor at a time (section 28), which the group compared with its own Ryzen 5 5600X (appendix G). Published evidence is the most manageable option given that the group has no processor with stacked cache. Its limitation is dependence on the experimental conditions and measurements that others chose to publish; the analysis can only be as controlled as its sources.

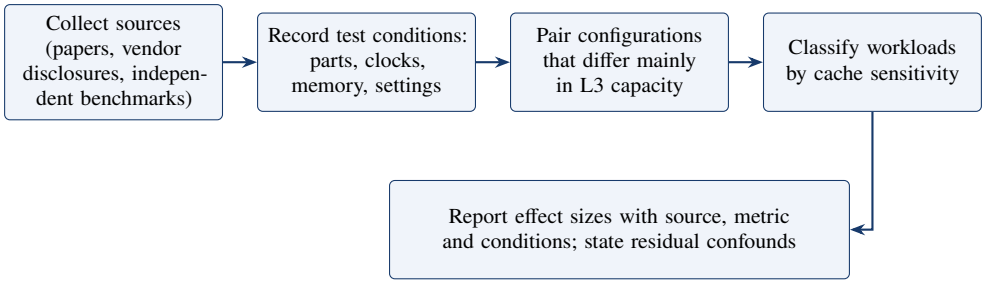


Figure 42: Evidence-analysis procedure for Part V (published research and benchmarks; the group's own measurements on a Ryzen 5 5600X are in appendix G).

Each source is recorded with: the processors compared; clock frequencies (fixed or boost); memory type, speed and timings; power limits; software and settings; the metric (for example, frames per second, IPC, runtime or throughput); and whether the source is the vendor or an independent party. Results are reported as relative changes with the source, metric and conditions stated, never as a single headline number.

Source selection. Sources are admitted in three tiers: peer-reviewed papers; independent benchmark publications that state their hardware, settings and method; and vendor material, which is used only when labelled as such. Every source used must have its PDF in the project's source folder. Where sources conflict, the one with the better-controlled comparison (section 26) is preferred and the disagreement is reported. Results across workloads are reported as ranges, and where a source gives them, as medians, rather than as averages that would hide their spread. Where this paper computes a summary itself, such as the geometric mean of section 24.3, it says so and gives the range beside it.

22 From Zen 2 Chiplets to Zen 3 and 3D V-Cache

Table 18 and figure 43 summarise the changes in cache organisation that matter for this paper, using only AMD's published descriptions.

Table 18: L3 organisation per compute die across the generations studied (AMD-published figures).

	Zen 2 CCD	**Zen 3 CCD**	**Zen 3 CCD with 3D V-Cache**
Cores per CCD	8	8	8
CCX organisation	Two 4-core CCXs	One 8-core CCX	One 8-core CCX
L3 shared by one core	Up to 16 MB (its CCX)	32 MB	96 MB
L3 per CCD	2×16 MB	32 MB	32 MB base + 64 MB stacked
Private L2 per core	512 KB	512 KB	512 KB
CCD area	74 mm^2	81 mm^2	81 mm^2 + 41 mm^2 cache die
Added L3 latency	—	—	+4 cycles
Sources	[2], [4]	[3], [11]	[3], [11]

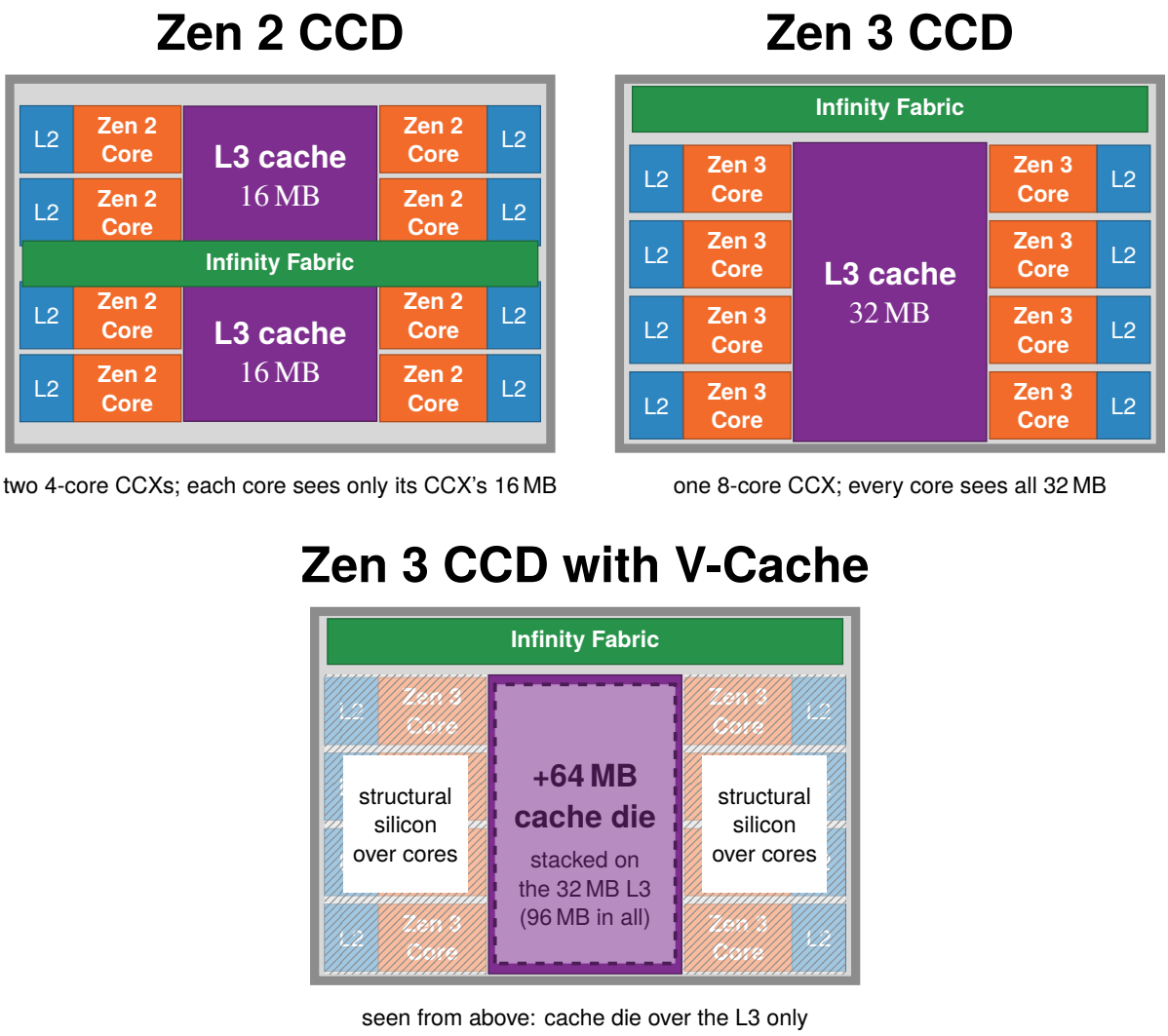


Figure 43: Compute die (CCD) organisation from Zen 2 to Zen 3 and Zen 3 with 3D V-Cache: cores (orange), private 512 KB L2 caches (blue), L3 (purple) and the Infinity Fabric interface (green). Zen 2 splits the CCD into two four-core complexes with 16 MB of L3 each; Zen 3 gives all eight cores one 32 MB L3; V-Cache stacks a 64 MB cache die over the central L3 and covers the cores only with structural silicon. Drawn from AMD's descriptions [3], [4], [11]; a schematic, not a floorplan, and not to scale.

Zen 3 kept the L3 capacity per CCD and changed its *sharing*: each core can now use the whole 32 MB rather than only its own CCX's 16 MB [4], [11]. 3D V-Cache then tripled the capacity seen by each core without changing the CCD's footprint or the sharing structure [3], [11]. These two steps give a natural experiment for the effect of L3 capacity, as long as other differences are controlled (section 26).

22.1 Zen 2 to Zen 3: a same-core-count comparison

The Zen 2 to Zen 3 step can be examined with two eight-core, single-CCD desktop parts. The Ryzen 7 3700X (Zen 2) and Ryzen 7 5800X (Zen 3) both have eight cores, sixteen threads and 32 MB of L3, but the 3700X splits its L3 into two 16 MB halves, one per four-core CCX, while the 5800X shares all 32 MB among its eight cores [4], [11]. They differ in other ways too: base and maximum boost clocks of 3.6 and 4.4 GHz against 3.8 and 4.7 GHz, and a TDP of 65 W

against 105 W [28].

AMD attributes a 19% IPC increase to Zen 3, citing the unified eight-core complex with direct access to 32 MB of L3 among the core improvements. The figure comes from 25 workloads run at a locked 4 GHz on the eight-core Ryzen 7 3800XT and 5800X, both with DDR4-3600 memory [5, note 4]. Because clock, core count and memory are held equal, this is a cleaner comparison than most vendor figures. It still cannot separate the shared L3 from the many other changes to the Zen 3 core.

Independent testing of the two retail parts shows the combined effect. With the same DDR4-3200 CL14 memory, TechSpot measured the 5800X ahead of the 3700X by 22% in Cinebench R20 multi-core and by 24% and 19% in 7-Zip compression and decompression. In rendering it led by 16% in Blender, 30% in V-Ray and 26% in Corona, and elsewhere by 12% in code compilation, 6% in DaVinci Resolve, 14% in Premiere Pro, 27% in Photoshop and 23% in After Effects [36]. During the multi-core Cinebench run the 5800X held about 4.55 GHz, and it drew about 20 W more than the 3700X [36]. These are whole-processor differences. The 5800X runs at higher clocks under a higher power limit and has a redesigned core, so the gains cannot be credited to the unified L3 alone. The gains are broad rather than concentrated in cache-sensitive work: the largest is in V-Ray, a multi-threaded renderer (30%), and large gains also appear in Photoshop (27%), which the reviewer describes as relying heavily on single-core performance [36]. In our reading, this points to the core and clock as much as to the cache.

Cache measurements give a more direct view. The FPS Review ran AIDA64's cache and memory benchmark on both processors with identical DDR4-3600 CL16 memory and the same fabric clock (1800 MHz) [17]. Memory read bandwidth was the same, about 50.7 GB/s on both, and memory latency fell from 68.8 ns on the 3700X to 61.8 ns on the 5800X. L3 read bandwidth was also nearly identical (633 against 631 GB/s). L3 latency, however, *rose* from 9.9 ns to 10.9 ns [17]. At the core clocks the tool reported during the runs (4.40 and 4.85 GHz), this is roughly 44 cycles against 53 cycles. The figures come from screenshots of a single run on each processor, so the cycle counts are only approximate.

Chips and Cheese's latency measurements across Zen generations agree: unifying the two 16 MB caches of a Zen 2 CCD into one 32 MB cache added 7 to 8 cycles of L3 latency. By contrast, doubling the size of each L3 slice from Zen to Zen 2 added about 5 cycles and tripling it with V-Cache only 3 to 4, which suggests that enlarging slices is cheaper in latency than merging separate caches into one [23]. Chips and Cheese adds that how much of the 7 to 8 cycles came from AMD preparing the design for V-Cache cannot be known [23]. Figure 44 plots the AIDA64 latencies.

This result fits the trade-off described in section 14. Doubling the cache a core can reach, from 16 to 32 MB, removes misses for working sets between the two sizes and the traffic between CCXs, but a larger shared array serving more cores takes longer to reach. The 5800X's faster single-threaded applications, in which the 32 MB working-set limit rarely matters, therefore most likely owe their gains to the core and clock, not to faster cache access. Isolating the benefit of the unified L3 itself would require fixed-clock measurements of workloads with working sets between 16 and 32 MB, or of communication between cores in different CCXs, which none of the sources in hand provides.

Two later generations extend the same pattern (table 19). With Zen 4, AMD kept the stack unchanged in principle but moved the CCD to 5 nm while the cache die stayed at 7 nm [8]; on servers, up to twelve CCDs per package each reach 96 MB of L3 with V-Cache, for up to

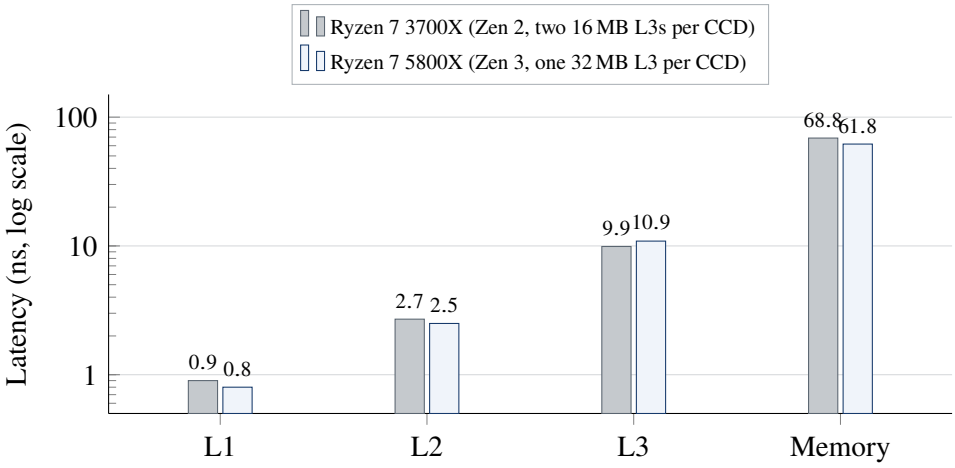


Figure 44: AIDA64 cache and memory latency of a Zen 2 and a Zen 3 eight-core processor with identical DDR4-3600 CL16 memory and the same 1800 MHz fabric clock, from The FPS Review [17]. Unifying the L3 made it slower by about a nanosecond, while memory latency fell by 7 ns; the 5800X also runs at a higher clock, so the nanosecond values mix the cache change with the clock change.

1152 MB per socket [9]. With Zen 5, AMD inverted the stack so that the cache die sits below the CCD (section 18.2).

Table 19: 3D V-Cache across Zen generations (AMD-published figures unless noted).

	Zen 3	**Zen 4**	**Zen 5**
Example parts	Ryzen 7 5800X3D; EPYC 7773X	Ryzen 7 7800X3D; EPYC 9684X	Ryzen 7 9800X3D
CCD / cache die	7 nm / 7 nm	5 nm / 7 nm	4 nm (TSMC) / not stated
Stack order	cache die on top of CCD	cache die on top of CCD	CCD on top of cache die
L3 per V-Cache CCD	96 MB	96 MB	96 MB (32 + 64)
Largest socket L3	768 MB	1152 MB	—
Added L3 latency	3–4 cycles (measured)	about 4 cycles (measured)	4 cycles (measured)
Sources	[3], [11], [23]	[8], [9], [22]	[10], [25], [32], [33]

AMD lists the Ryzen 7 9800X3D with 96 MB of L3, cores made in TSMC's 4 nm process and an I/O die in 6 nm [33]; it does not state the process used for the cache die.

23 Comparing Architectural Configurations

The comparisons in this part rely on pairs of processors that differ mainly in L3 capacity. Table 20 lists the pairs used and what else differs between them.

Table 20: Processor pairs that differ mainly in L3 capacity.

Pair	Cores	L3	Other differences	Source
EPYC 7763 vs 7773X (Zen 3)	64	256 vs 768 MB	base clock 2.45 vs 2.20 GHz; same 3.5 GHz boost	[7]
Ryzen 7 5800X vs 5800X3D (Zen 3)	8	32 vs 96 MB	5800X clocks slightly higher	[35]
7950X3D: plain CCD vs V-Cache CCD (Zen 4)	8 per CCD	32 vs 96 MB	plain CCD about 7% faster at stock; identical when both are fixed at 4.2 GHz	[22]
EPYC 9654 vs 9684X (Zen 4)	96	384 vs 1152 MB	all-core boost 3.55 vs 3.42 GHz; base 2.40 vs 2.55 GHz; same 3.70 GHz maximum; default power 360 vs 400 W	[27]

24 Published Benchmarks and Performance Measurements

The evidence falls into two groups: AMD-reported results (table 21) and independent measurements (table 22).

Table 21: AMD-reported performance figures relevant to this study (vendor claims, not independent measurements).

Claim	Conditions stated	Source
About 15% faster gaming, 8-core Ryzen 7 5800X3D vs 12-core Ryzen 9 5900X; per-title results from a tie to 1.36×	Six games at 1080p, high preset; both with DDR4-3600 and RTX 3080; AMD testing, Dec. 2021	[3, slides 29, 35]
About 66% more RTL-simulation jobs per hour (24.4 to 40.6), 16-core EPYC with V-Cache vs 16-core EPYC 73F3	Synopsys VCS; same reference platform; AMD testing, Sept. 2021	[3, slides 31, 35]
15% average gaming uplift, 12-core 3D-chiplet prototype vs Ryzen 9 5900X	Both at the same fixed frequency; disclosed games only	[6]
15% IPC improvement from Zen to Zen 2	Many simultaneous changes, including doubled L3	[4]
19% IPC improvement from Zen 2 to Zen 3 (one AMD test, reported in both sources)	25 workloads at a fixed 4 GHz, 3800XT vs 5800X, equal memory; but a redesigned core as well as the unified L3	[5], [3, slides 10, 35]
19% average memory-latency reduction, Rome vs Naples	Central IOD, decoupled clocks, optimised IFOP	[2]

The independent results agree with AMD's on direction but not on size: they show a wider spread, including small losses, and the best-controlled of them, Lam's fixed-frequency per-CCD measurements, shows single-digit to high-teen IPC gains rather than uniform double-digit

Table 22: Independent measurements of the effect of 3D V-Cache.

Result	Conditions	Source
IPC +4.9% to +19.7% across five workloads (geometric mean +11.4% by the authors' calculation, section 24.3); one workload slightly negative	7950X3D, each CCD at a fixed 4.2 GHz; performance counters	[22]
Up to 3.4× on MiniFE for a subset of problem sizes	EPYC 7773X vs 7763, 64 cores each	[16]
OpenFOAM from 0.81× to 1.54× (motorBike) and up to 1.47× (urban air pollution); L3 miss rate down by up to 30 points	EPYC 7773X vs 7763, dual socket, clocks not fixed	[34]
gem5 +7.6%, libx264 and 7-Zip about +5% on one CCD; Milan-X clocked about 5% lower	Milan-X vs Milan cloud instances, clocks estimated	[23]
Of 350 significant differences among 420 tests, 106 favour the 5800X3D (12 by 50% or more) and 244 the 5800X (all but 3 by under 10%)	Ryzen 7 5800X3D vs 5800X, Linux	[35]
L3 latency +3 to +4 cycles	Milan-X, 7950X3D, 9800X3D	[22], [23], [32]

speedups. Figure 45 shows the difference in spread.

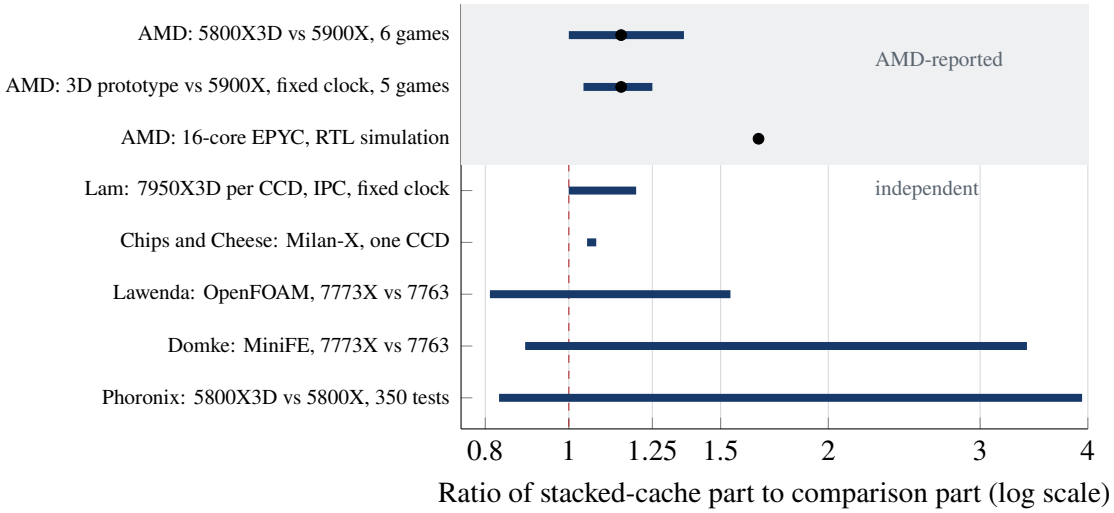


Figure 45: Range of reported results for the stacked cache, AMD's own (shaded) against independent sources (bars: lowest to highest result; dots: averages or single results). The metrics differ (frame rate, jobs per hour, IPC, run time) and so do the comparison parts, so the chart compares spreads, not values. Lam's lower end is a small, unquantified loss and is drawn at 1; Phoronix's lower end is the largest 5800X lead (just under 20%) and its upper end the largest single result, 294% (3.94×). Sources: [3], [6], [16], [22], [23], [34], [35].

24.1 The evidence ledger

The method of section 21 asks that every source be recorded with its systems, clocks, memory, metric and independence. Table 53 in appendix H is that record for the independent sources

used in this part. It complements table 26, which grades only which confounding factors each comparison controls.

Only one source, Lam's, holds clock, core design, memory and the rest of the system equal, and it covers only six workloads. The sources with many workloads (Larabel) or with systematic problem-size sweeps (Domke, Lawenda) let the clock float, so their results combine the cache with a clock difference of 5–10%. The detailed reading below therefore starts with Lam, then uses the sweeps to show how the effect depends on working-set size.

24.2 Latency through the whole hierarchy

The clearest picture of what the stacked cache changes comes from a latency sweep, in which a test program walks through a region of memory of increasing size and measures the time per load. Chips and Cheese ran such a sweep on Milan and Milan-X cloud instances (figure 46) [31]. The two curves coincide through L1 (4 cycles) and L2 (about 12 cycles). In the L3 range Milan-X is slightly slower, 51.14 against 47.49 cycles, the three to four extra cycles discussed in section 17. The curves separate at 32 MB: Milan's latency climbs towards memory, while Milan-X's stays flat until beyond about 64 MB and then rises as its 96 MB L3 is exceeded. Between those sizes a load costs about 51 cycles on Milan-X, while on Milan it rises to about 240 cycles at 64 MB, nearly five times as long, and further at 96 MB. Beyond 96 MB both go to memory, at 342 cycles for Milan and 313 for Milan-X; in nanoseconds Milan-X's memory latency was also lower (89 against 106 ns), and the article notes that Milan-X was boosting better than Milan in this test [31].

A third source agrees. On a Zen 3 Ryzen 5 5600G, which has a 16 MB L3 on a single die, 7-cpu.com measured 4 cycles for L1, 12 for L2, 47 for L3 and 47 cycles plus 74 ns for memory, about 375 cycles at the chip's 4.45 GHz [20]. The same source shows a second effect that latency sweeps can mix up with cache misses: with ordinary 4 KB pages, latency at 512 KB was 21 cycles, against 12 with 2 MB pages, although the data still fits in L2. The source's table attributes part of the rise to misses in the 64-entry first-level data TLB, which can no longer hold the address translations, and leaves the exact 4 KB penalty unstated [20]. Sweeps such as Chips and Cheese's therefore measure cache and TLB effects together, and the steps in figure 46 are not purely cache capacities.

These measurements also check the textbook's numbers. The measured memory latency of 313–342 cycles lies just above the textbook's 100–300-cycle range. Latency is not the same as the stall a removed miss saves, however: section 17.1 found that a removed miss saved only an apparent 130 cycles or so on the 7950X3D, probably because an out-of-order core overlaps misses, and possibly because the counters include prefetches. With equation (10), the break-even for Milan-X therefore lies between $3.65/313 \approx 1.2$ and $3.65/130 \approx 2.8$ percentage points of L3 misses removed, depending on how much of each miss the core hides.

24.3 Lam's six workloads in detail

Lam's measurements are the best-controlled evidence available (section 26), so they are worth reading workload by workload. Lam tested four games, GHPC (which simulates the fire-control systems of armoured vehicles), Cyberpunk 2077, the flight simulator DCS and Call of Duty, together with libx264 video encoding and 7-Zip compression [22]. Table 23 collects the L3 hit

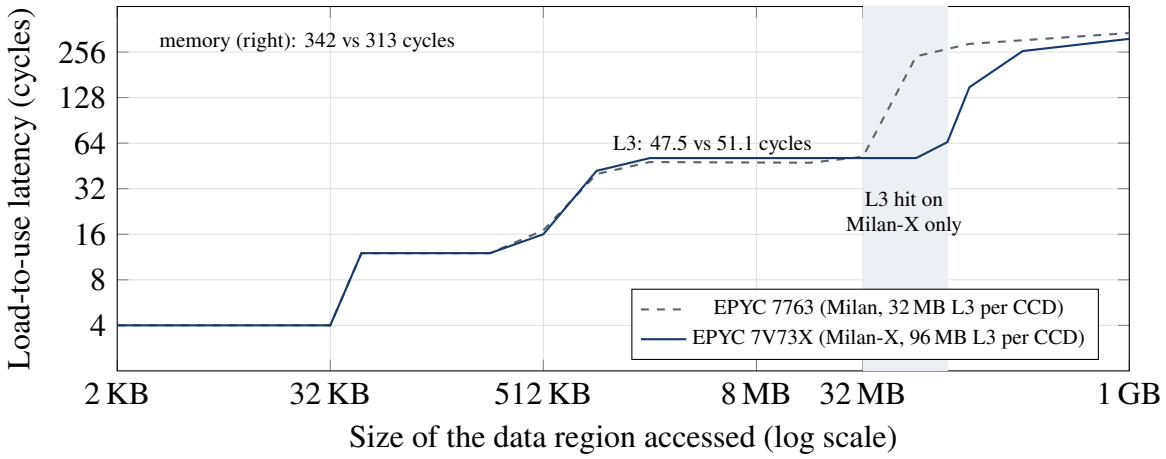


Figure 46: Cache and memory latency against data size for Milan and Milan-X, redrawn from Chips and Cheese's measurements [31]. The labelled values (L1 4 cycles, L2 about 12 cycles, L3 47.49 and 51.14 cycles, memory 342 and 313 cycles) are taken from the chart's own labels; the transitions between them were read by eye and are approximate. In the shaded range, from 32 MB to beyond about 64 MB, Milan-X still hits in its L3 while Milan already goes to memory.

rates and IPC values from Lam's charts and the misses per thousand instructions (MPKI) from the text [22].

Table 23: Lam's per-CCD measurements on the Ryzen 9 7950X3D, both CCDs at 4.2 GHz [22]. Hit rates and IPC read from the article's charts (IPC to two decimals); IPC gains as stated in the text where given (Call of Duty: computed from the chart; the text says "over 19%"); MPKI on the plain CCD as stated in the text. Lam counts L3 accesses from the L3's side, including speculative accesses and prefetches. The Cyberpunk run was limited by the graphics card, which is why Lam compares IPC rather than frame rates.

Workload	**L3 hit rate** plain → V-Cache	**L3 miss rate** plain → V-Cache	**IPC** plain → V-Cache	**IPC gain**	**MPKI** plain
Call of Duty	46.5 → 68.7%	53.5 → 31.3%	0.71 → 0.85	19.7%	not legible
Cyberpunk 2077	46.0 → 63.7%	54.0 → 36.3%	1.26 → 1.43	13.5%	5
7-Zip	45.8 → 59.2%	54.2 → 40.8%	1.33 → 1.46	9.75%	not stated
GHPC	58.7 → 78.0%	41.4 → 22.0%	1.15 → 1.27	9.67%	5.5
libx264	61.6 → 71.9%	38.5 → 28.1%	1.36 → 1.43	4.9%	1.48
DCS (flight simulator)	89.1 → 92.4%	10.9 → 7.6%	about 1.7 (both)	small loss	0.35

A summary figure. The method of this part reports ranges rather than a single average, but a summary helps when comparing with AMD's figures. Over the five workloads with a quantified gain, the geometric mean of the IPC ratios, which is the authors' calculation and not Lam's, is $(1.049 \times 1.0967 \times 1.0975 \times 1.1349 \times 1.197)^{1/5} \approx 1.114$, an 11.4% gain at equal clock, and the median is 9.75%. DCS lost a little, by an amount the article does not state. Counting it as no change gives $1.114^{5/6} \approx 1.094$, so the geometric mean over all six workloads is at most 9.4%. These six workloads were chosen by the author, four of them games, so the mean describes this sample and not software in general; the 350 Phoronix results of section 24.8 are the better guide to how common such gains are.

What the hit rates say. In every workload the V-Cache raised the L3 hit rate, by 3 to 22 percentage points, and removed between a quarter and a half of the L3 misses (from 54.2 to

40.8% for 7-Zip, a quarter; from 41.4 to 22.0% for GHPC, almost half). The capacity argument of section 13 predicts exactly this: tripling the L3 turns some capacity misses into hits but leaves the misses of data that does not fit even in 96 MB, or that is touched for the first time.

What the IPC gains say. The ranking of gains follows the ranking of misses only loosely. DCS, with 0.35 MPKI, had almost nothing to gain and lost a little, which Lam attributes to the added L3 latency; libx264, with 1.48 MPKI, gained 4.9%; the four miss-heavy workloads (Call of Duty, Cyberpunk, 7-Zip and GHPC, which miss on 41–54% of their L3 accesses) gained 9.67–19.7%. Among them, however, GHPC had the highest stated MPKI (5.5) and the largest relative fall in miss rate, yet gained less than Cyberpunk. The data allow a rough check of why, using the method of section 17.1. For each workload with a stated MPKI, the number of L3 accesses per thousand instructions is MPKI divided by the miss rate (for GHPC, $5.5/0.414 \approx 13.3$). The misses removed follow from the two miss rates ($5.5 - 13.3 \times 0.220 \approx 2.57$), and the cycles saved follow from the two IPC values, to which the four extra cycles on every L3 access are added back (table 24).

Table 24: Cycles saved per removed L3 miss, derived from table 23 with four extra cycles per L3 access (approximate; see text).

	per thousand instructions				Cycles per
Workload	**L3 accesses**	**Misses removed**	**Cycles saved**	**Cycles added**	**removed miss**
GHPC	13.3	2.57	77	53	about 50
Cyberpunk 2077	9.3	1.64	94	37	about 80
libx264	3.85	0.40	36	15	about 130

The pattern in table 24 is that the more misses a workload has, the less each removed miss appears to be worth. The overlap of misses in section 4.8 would produce this: a workload with many outstanding misses hides more of each one, so removing a miss saves less of its latency. Prefetches counted as misses would have the same effect (section 17.1). The values are therefore *apparent cycles saved per counted miss*, and they inherit the caveats of section 17.1, including the 15% difference in L3 accesses per instruction between the two CCDs found for DCS. The figures are approximate. The IPC values are read to two decimals, so for GHPC the chart's 1.15 and 1.27 give a 10.4% gain against the 9.67% in the text, which would raise its figure to about 53 cycles. The calculation also assumes that all the IPC change comes from L3 misses and latency; and each workload was measured in one scenario. The ordering, however, is not sensitive to these errors, because the three values differ by more than a factor of two.

What else limits the gain. Call of Duty shows that the cache is not the only bottleneck even in a miss-heavy game. Lam's breakdown of pipeline slots shows the V-Cache cutting the share of slots lost to the back end, which waits for data, from 29.7% to 24.2%, while the share lost to the front end, which supplies instructions, stayed at about 42–43% [22]. Lam concludes that most of the hit-rate gain came from data-side misses. The game's IPC of 0.85 with V-Cache is still the lowest of the six, so the 19.7% gain is large in relative terms but leaves the core mostly waiting on instruction supply.

At stock clocks. In normal operation, Lam's plain CCD clocked about 7% higher than the V-Cache CCD [22]. If performance scaled linearly with clock, dividing each IPC ratio by 1.07 would give the net effect of choosing the V-Cache CCD: 0.98× for libx264, 1.03× for GHPC and 7-Zip, 1.06× for Cyberpunk and 1.12× for Call of Duty. Lam notes that performance does

not scale linearly with clock, because memory latency does not improve with core frequency, so these values are lower bounds for the V-Cache CCD; libx264, with few misses, is the case closest to linear and the only one likely to favour the faster plain CCD. Section 27.2 develops this trade-off into a model.

24.4 A working-set sweep: MiniFE

Domke *et al.* ran the MiniFE finite-element proxy application on dual-socket Milan and Milan-X systems with the same 64 cores per socket, 280 W TDP and 16-channel DDR4 memory, while scaling the problem from 100^3 to 400^3 grid points [16]. Figure 47 shows the result: a sweep through working-set sizes of exactly the kind sketched in figure 29.

- At the smallest sizes ($n = 100$ and 110) Milan-X is about 10% *slower*. Both systems hold the data in L3, so the difference is probably Milan's higher base clock (2.45 against 2.20 GHz); section 26.1 tests this.
- From $n = 120$ the advantage rises steeply, to 3.4× at $n = 160$. In this range Milan's 512 MiB of L3 (two sockets) no longer holds the data, while Milan-X's 1536 MiB still does.
- Beyond $n = 170$ the advantage falls quickly and settles at about 1.04× from $n \approx 330$ onwards, once the data exceeds both caches.

Inferring the working set. The curve can be used to estimate how much data each grid point needs, with no information about MiniFE's data structures. Milan's own performance falls sharply between $n = 120$ and 130 [16, Fig. 1], that is, between 1.73 and 2.20 million points; if this is where the data outgrows 512 MiB, each point needs about $512 \times 2^{20} / (1.73 \text{ to } 2.20 \times 10^6) \approx 245$–310 bytes. Milan-X's performance falls between $n = 170$ and 180, between 4.91 and 5.83 million points; with 1536 MiB of L3 that gives about 275–330 bytes per point. The two independent estimates agree, which supports the reading that the speedup curve is a cache-capacity effect: at about 300 bytes per point, Milan-X's L3 holds three times as many grid points as Milan's. The estimate is approximate, because not all of the L3 holds useful data and the two sockets' caches are not perfectly shared.

24.5 OpenFOAM in detail

Lawenda *et al.*'s OpenFOAM study adds measured miss rates and clock speeds [34]. For the dual-socket 64-core Rome, Milan and Genoa systems, all with 256 MB of L3 per socket, the L3 miss rate rose from about 10–15% on the smallest motorBike meshes to about 80% on the largest. Milan-X, with 768 MB per socket, cut the miss rate by up to 30 percentage points on the mid-sized to large meshes. Its performance relative to Milan ranged from 0.81× to 1.54× across the eight mesh sizes: slower on some small meshes, fastest at the "mhigh" mesh of 6.7 million cells, and falling back to 1.24× on the largest (xhigh) mesh of 39.4 million cells, where the data exceeds even Milan-X's cache (1.22× in their Fig. 4). Expressed per core, Lawenda *et al.* place Milan-X's advantage between about 15 000 and 100 000 cells per core. Its best utilisation came at about 14 800 cells per core, against about 4700 for Rome and Milan. The newer generation shows the same pattern (section 24.6). Figure 48 plots the ratios Lawenda *et al.* label in their figure for the five larger meshes.

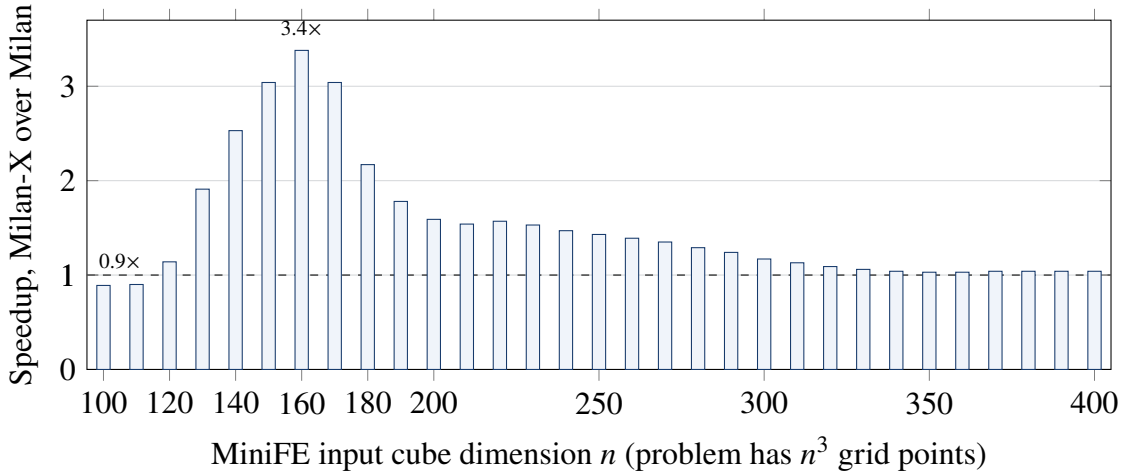


Figure 47: MiniFE on a dual-socket EPYC 7773X (Milan-X, 2×768 MiB L3) against a dual-socket EPYC 7763 (Milan, 2×256 MiB L3) as the problem size grows. Values were read by eye from Domke *et al.*'s figure [16, Fig. 1] and are accurate to about ±0.05. Milan-X runs at a lower base clock (2.20 against 2.45 GHz).

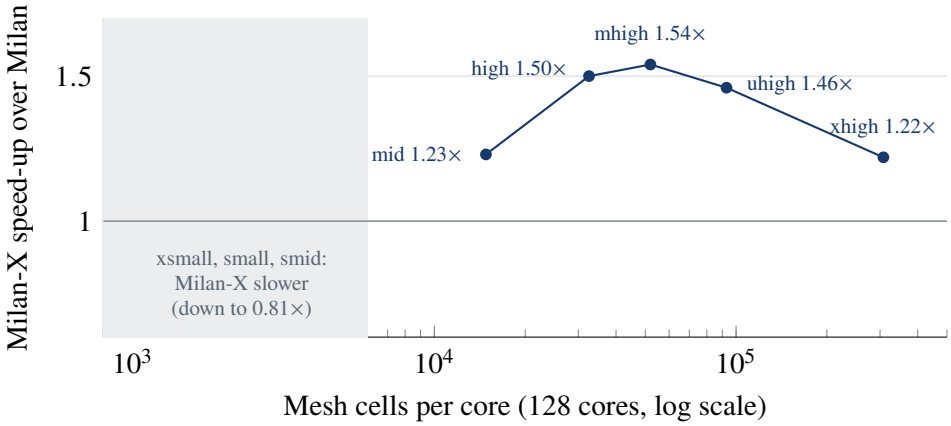


Figure 48: OpenFOAM motorBike: speed-up of a dual-socket EPYC 7773X (Milan-X) system over a dual-socket EPYC 7763 (Milan) system by mesh size, read from Lawenda *et al.*'s Fig. 4, with cells per core from their Table 3 [34]. The gain peaks where the working set fits the larger L3 but not the smaller one, and falls on the largest mesh, which exceeds both. The figure labels the xhigh gain 1.22× while the article's text gives 1.24×. For the three smallest meshes the figure gives no ratio; the article's Table 6 gives a lowest value of 0.81×. Clocks were not fixed (table 27).

The study's measured clocks show the confound discussed in section 26: during the runs, Milan ran at 2.9–3.2 GHz and Milan-X at 2.8–3.0 GHz [34, Table 5]. Milan-X's losses on the small meshes, where both systems' caches suffice, are therefore at least partly a clock effect, as in MiniFE.

24.6 The next generation: Genoa-X

Lawenda *et al.* also measured the fourth generation: dual-socket systems with the 64-core EPYC 9554 and the 96-core EPYC 9654 (Genoa) and the 96-core EPYC 9684X (Genoa-X), which adds a cache die to each of its twelve CCDs [34]. Their results allow three comparisons

that the third-generation data cannot.

The same pattern, a wider window. Against the 96-core Genoa, Genoa-X ranged from 0.98× to 1.60× across the motorBike meshes, against 0.81× to 1.54× for Milan-X against Milan. As with Milan-X, it was not faster on the smaller meshes, and its largest gains came on the mid-sized to large ones. Lawenda *et al.* place Genoa-X's advantage between about 10 000 and 200 000 cells per core, against 15 000 to 100 000 for Milan-X [34]. The two stacked products have the same L3 per core: 768/64 = 12 MB per core for the EPYC 7773X [34] and 1152/96 = 12 MB for the 9684X [27]. The wider window therefore cannot come from more L3 per core, although the L2 does double, from 512 KB to 1 MB per core [34]. One candidate is memory bandwidth: Genoa has twelve channels of DDR5-4800, 460.8 GB/s per socket or 4.8 GB/s per core on the 96-core parts [27], against 204.8 GB/s or 3.2 GB/s per core for a 64-core Milan (section 8). The source does not test this explanation, so it remains a hypothesis.

A smaller loss on small meshes. Milan-X's worst case was 0.81×, and section 26.1 found that the measured clocks explain only part of it. Genoa-X's worst case was 0.98×, a loss of 2%. The clock table of Lawenda *et al.* gives 3.4–3.5 GHz and 3.2–3.4 GHz for two of the fourth-generation systems, but its row labels for the Genoa family appear to be misprinted (two rows are labelled "64-core Genoa" and none "Genoa X") [34, Table 5]. If those two rows are the 96-core Genoa and Genoa-X, in the order the paper uses elsewhere, Genoa-X ran about 4% slower, and the 2% loss is no more than its clock deficit. The reading is plausible but rests on that assumption.

A new generation outweighs a stacked cache. Without any stacked cache, the 64-core Genoa was 1.24–1.70× as fast as Milan-X on every mesh [34], and Genoa-X was up to 2.67× as fast as Milan-X. The gain from one generation of core, memory (DDR5) and clock was larger than the gain from tripling the L3 within a generation. This is a caution against reading too much into any single factor: the stacked cache is one way to buy performance among several, and it is most valuable where the other ways run out, as section 34 argues.

24.7 Do the independent sources agree?

Three articles from one independent outlet, Chips and Cheese, measured the same programs or the same quantity on different generations, which allows a direct check of consistency (table 25). Cozma compared one CCD of Milan-X with one CCD of Milan in cloud instances, estimated that Milan-X ran about 5% slower (3.15 against 3.3 GHz), and corrected those results for that difference [23]. Lam measured the Zen 4 7950X3D at a fixed clock [22].

The latency cost agrees across all three generations, as section 17 noted. The 7-Zip results also agree: about 10% at equal clock on both Zen 3 and Zen 4. The libx264 results do not: Cozma's corrected figure is about twice Lam's. Several differences could explain this, and the sources do not allow a choice among them. Cozma's clocks are estimated and the correction assumes performance proportional to clock, which overstates the per-clock gain of a program that waits on memory (section 26.1). The two used different videos and possibly different settings. And the Zen 4 core has a 1 MB L2 [9], twice Zen 3's 512 KB [3, slide 19], so more of libx264's reuse may be caught before the L3 on Zen 4, leaving less for the V-Cache to remove. The last point is a hypothesis; neither source measured Zen 3 and Zen 4 under the same conditions.

A later measurement shows the same pattern on Zen 5. In SPEC CPU 2017, the eight-core

Table 25: Cross-check of independent results for the same program or quantity on different generations.

Quantity	Zen 3: Milan-X vs Milan, one CCD [23]	Zen 4: 7950X3D CCDs at 4.2 GHz [22]	Agreement
7-Zip	+5% measured; about +10% at equal clock (source's correction)	+9.75% IPC	close
libx264	+5% measured; about +10% at equal clock	+4.9% IPC	differs by half
gem5	+7.6% measured; about +12.5% at equal clock	not tested	—
Added L3 latency	3–4 cycles (47.49 to 51.14 on Milan-X [31])	about 4 cycles	close; also 4 on Zen 5 [32]

9800X3D ran ahead of the sixteen-core 9950X in the integer suite and almost made up its 500 MHz clock deficit in the floating-point suite. In the floating-point suite, the two most memory-bound tests gained most, 7.5% for 549.fotonik3d and 15% for 554.roms [32]. With the clock deficit of about 9% (5.2 against the 9950X's rated 5.7 GHz), those gains imply per-clock improvements of very roughly 18–26% for the two memory-bound tests, on the same simple correction and with the same caveat.

The cross-check shows that the mechanism is stable across generations: the latency cost is the same, and the programs that gain most are usually among those that miss most, though not always: Cozma notes that 523.xalancbmk, the most memory-bound SPEC test on Zen 5, sees little benefit from V-Cache [32], and Lam's GHPC gained less than Cyberpunk despite more misses. The size of the gain for a given program is less stable, because it depends on the rest of the hierarchy, on the input and on how the clock is controlled. This is why section 21 reports each result with its conditions.

24.8 Breadth: 350 desktop results

Phoronix compared the eight-core Ryzen 7 5800X3D and 5800X across 420 benchmarks on Linux and charted the 350 with a statistically significant difference [35]. Classifying each result by the colour of its bar in that chart gives the distribution in figure 49. The 5800X was faster in 244 tests, but all except three of those margins were under 10% and the median was about 4%, consistent with its higher clocks. The 5800X3D was faster in 106 tests, with a median margin of about 11%; 58 of its wins were 10% or more and 12 exceeded 50%, led by ASKAP's tConvolve gridding kernel (294%), Zstd compression, oneDNN, CloverLeaf and OpenFOAM (68%). Phoronix's own summary describes the 5800X winning "a small set of tests"; the chart shows that the 5800X won more tests but by much smaller margins. The distribution is the clearest available picture of the workload dependence: for most desktop software the stacked cache costs a few percent, and for a minority it is worth tens of percent or more.

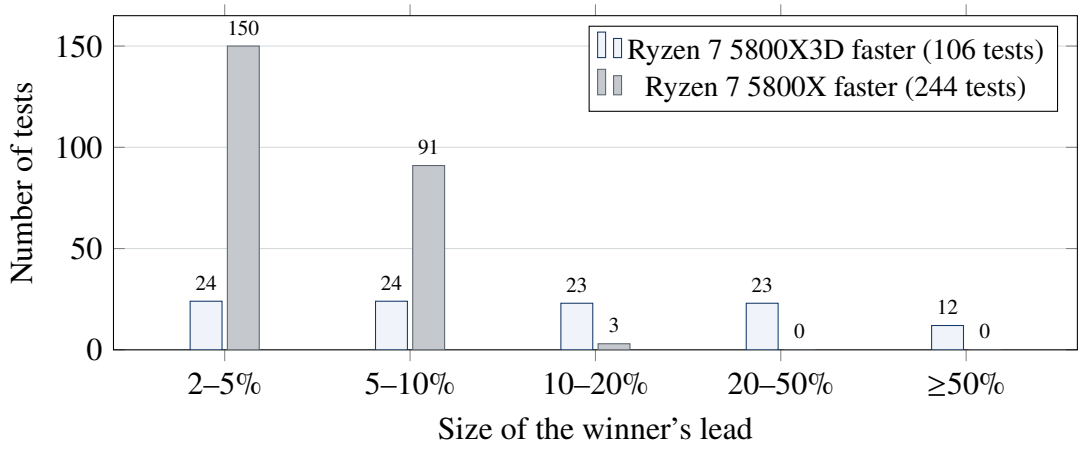


Figure 49: Distribution of the 350 statistically significant differences among Phoronix's 420 benchmarks of the Ryzen 7 5800X3D and 5800X [35]. The winner of each test was identified from the colour of its bar in Phoronix's comparison chart and matched to the chart's printed percentages; this classification is ours. Each bin includes its lower bound (for example, 5–10% means at least 5% and less than 10%).

25 Cache-Sensitive Versus Cache-Insensitive Workloads

The evidence sorts workloads into three groups, matching the working-set argument of section 19:

- **Strongly cache-sensitive:** memory-bound HPC and engineering codes whose working sets fall between the base and stacked capacities (MiniFE, OpenFOAM at mid-sized meshes, RTL simulation) and games with high L3 miss rates. Gains range from about 10% to more than 3× [16], [22], [34].
- **Mildly sensitive:** workloads with moderate miss rates, such as compression and video encoding, gain around 5–10% at equal clock [22], [23], which a V-Cache part's lower clock can cancel.
- **Insensitive or negative:** workloads whose working sets already fit (small OpenFOAM meshes, the flight simulator in Lam's tests) or that are compute-bound gain nothing and can lose slightly from the added latency or lower clocks [22], [34], [35].

A useful first screen is therefore not the application type but its L3 miss rate on the base configuration: in Lam's results, the workloads with the most L3 misses gained most and those with the fewest gained least or lost [22]. The order is not strict, because each removed miss is worth less in a workload that overlaps many misses (section 24.3).

26 Separating Cache Effects From Other Factors

A difference in measured performance between two processors can rarely be attributed to cache size alone. Using equation (1) and equation (5), the main confounding factors are:

- **Clock frequency.** If the larger-cache part runs at a different frequency, f in equation (1) changes as well as IPC. Fixed-frequency comparisons, like the one AMD reports for its prototype [6], remove this factor.

- **Core count and topology.** More cores or a different number of CCDs changes throughput and the cache available per core. AMD's headline desktop comparison sets an eight-core, single-CCD Ryzen 7 5800X3D against a twelve-core, two-CCD Ryzen 9 5900X [3, slide 29], so it mixes cache capacity with core count, CCD topology and clock speed. Its server comparison holds the core count at 16 but uses the EPYC 73F3 as the baseline, an F-series model that AMD positions for high frequency [3], [11].
- **Memory bandwidth and latency.** Memory speed, timings and the fabric clock change the miss penalty in equation (5) and can make a larger cache appear more or less valuable.
- **Concurrent microarchitectural changes.** Generational comparisons mix many changes. AMD's 15% Zen-to-Zen 2 IPC figure, for example, combines a new branch predictor, a larger op cache, wider floating-point paths and a doubled L3, among others [4]; it cannot be credited to the cache.
- **Power limits and software.** Package power limits, operating-system scheduling across CCDs, and application versions all affect results.

Figure 50 arranges these factors in one figure.

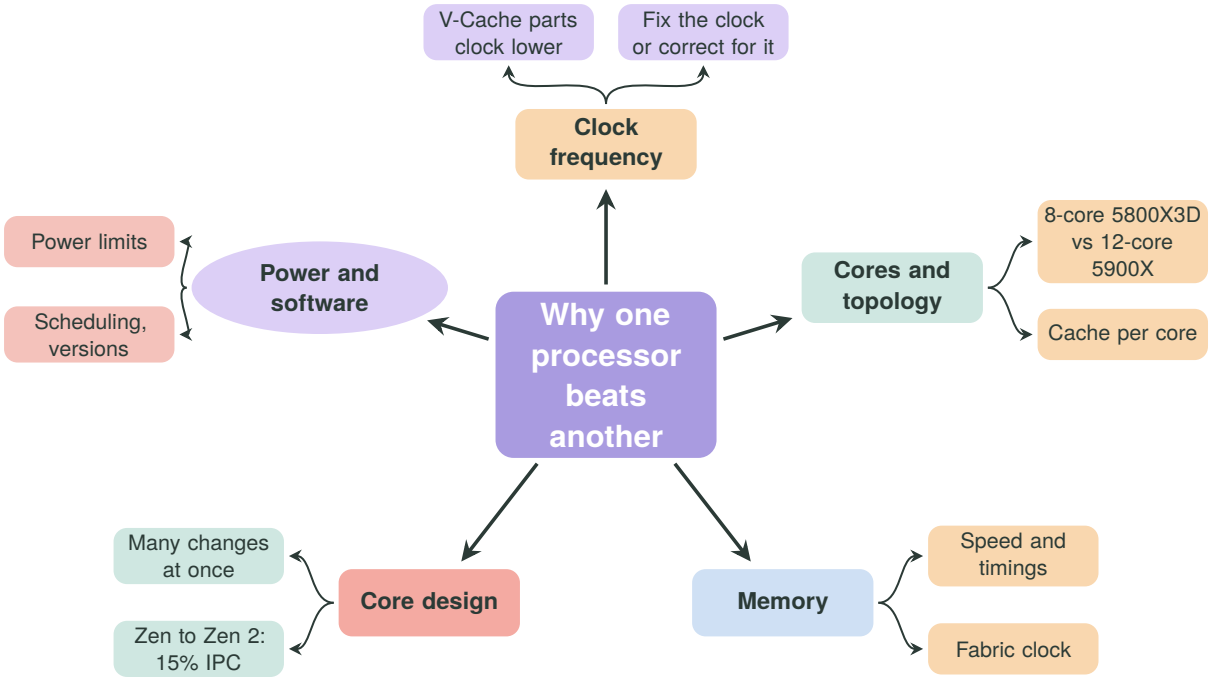


Figure 50: Overview of the confounding factors of section 26. A difference in measured performance can be credited to the cache only when the comparison controls each branch (table 26).

Table 26 applies this checklist to the comparisons available so far. None of AMD's published product comparisons controls every factor. The fixed-frequency prototype comparison controls clock speed and core count but does not state the memory configuration, and the server comparison controls core count but not frequency. The independent comparisons are graded the same way below the line.

The prototype comparison holds both clock (4.0 GHz) and core count (12) constant but does not state the memory configuration [6, slide 19]. Among the independent results, Lam's per-CCD measurements control every listed factor, which makes them the strongest available evidence for the effect of the stacked cache itself [22].

Table 26: Which confounding factors each available comparison controls (✓ = controlled, × = not controlled, ~ = estimated and normalised, ? = not stated; AMD-reported rows above the line, independent below).

Comparison	Clock	Cores	Memory	Same core design
12-core 3D prototype vs Ryzen 9 5900X, fixed frequency [6]	✓	✓	?	✓
Ryzen 7 5800X3D vs Ryzen 9 5900X [3, slides 29, 35]	×	×	✓	✓
16-core EPYC with V-Cache vs EPYC 73F3 [3, slides 31, 35]	×	✓	?	✓
Zen to Zen 2 IPC [4]	?	?	?	×
Zen 2 to Zen 3 IPC, 3800XT vs 5800X at 4 GHz [5]	✓	✓	✓	×
7950X3D V-Cache CCD vs plain CCD, both at 4.2 GHz [22]	✓	✓	✓	✓
EPYC 7773X vs 7763 [16], [34]	×	✓	✓	✓
Milan-X vs Milan, one CCD [23]	~	✓	?	✓
Ryzen 7 5800X3D vs 5800X [35]	×	✓	?	✓
Ryzen 7 5800X vs 3700X [36]	×	✓	✓	×

26.1 Correcting for clock speed

Where clocks were not fixed, a rough correction is possible if the clocks during the test are known. If performance scaled linearly with frequency, dividing a measured speedup by the ratio of clock speeds would give the per-clock effect. Memory-bound code does not scale linearly, because memory latency in nanoseconds does not improve with core clock, so the true per-clock effect lies between the raw ratio and the corrected one. Table 27 applies this to the comparisons whose clocks were measured or stated.

Table 27: Speedups bounded by a clock correction (corrected = measured ÷ clock ratio).

Comparison	Clock ratio (V-Cache / plain)	Measured	Clock-corrected
OpenFOAM, best mesh [34]	2.9/3.05 ≈ 0.95 (midpoints of measured ranges)	1.54×	1.62×
OpenFOAM, worst mesh [34]	≈ 0.95	0.81×	0.85×
gem5, one CCD [23]	3.15/3.3 ≈ 0.95 (estimated)	1.076×	1.13×
MiniFE, smallest input [16]	2.20/2.45 ≈ 0.90 (base clocks)	0.89×	0.99×

Figure 51 plots each comparison before and after the correction, which shows which comparisons reach parity once clock is accounted for (MiniFE's smallest input) and which stay below it (OpenFOAM's worst mesh).

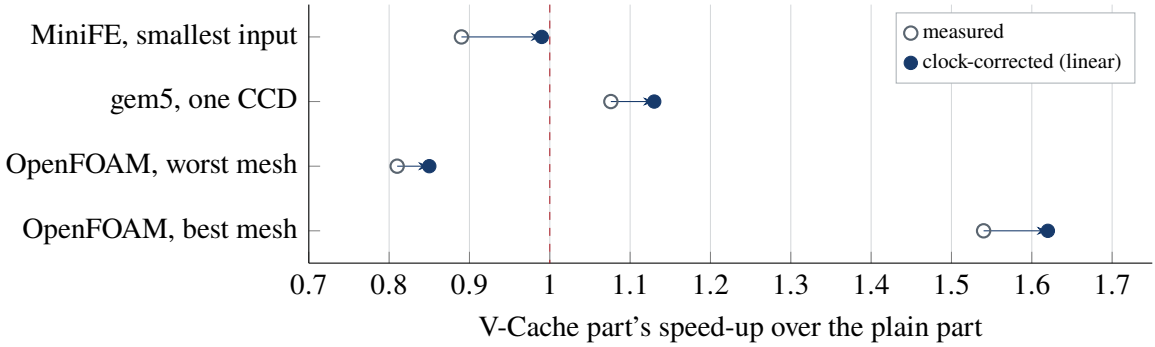


Figure 51: The comparisons of table 27 before and after a linear clock correction. The correction moves every point to the right, because the V-Cache part ran at a lower clock in each test; MiniFE's smallest input moves to parity, while OpenFOAM's worst mesh stays clearly below it. The true per-clock effect lies between the two markers, since memory-bound code does not speed up in proportion to clock. Sources: [16], [23], [34].

The correction changes the picture in two ways. For MiniFE's smallest input the whole 11% loss disappears, which supports the reading that Milan-X loses there only because of its clock. For OpenFOAM's worst mesh a loss of about 15% remains even after correction, so the clock alone does not explain it; Lawenda *et al.* do not analyse that case, and the four extra cycles of L3 latency cannot account for so large a difference on their own. This is a question the published evidence leaves open. Because base clocks and measured ranges are crude stand-ins for the actual frequency during each run, the corrected values are indicative only.

27 Cache Against Clock

In the first two generations the V-Cache parts clock lower than their plain counterparts (section 26), so whether the stacked cache pays off depends on how its gain at equal clock compares with the clock it gives up. This section works it through for the Ryzen 9 7950X3D with Lam's per-CCD measurements (section 24.3), first with a linear rule and then with a model in which memory time does not shrink with clock.

27.1 Cache against clock on one chip

The Ryzen 9 7950X3D makes the trade-off concrete, because its two CCDs differ only in the stacked cache and in clock speed: the plain CCD runs about 7% faster [22]. If performance scaled linearly with clock, the V-Cache CCD would win whenever its IPC gain at equal clock exceeded 7%. Table 28 applies this rule to Lam's fixed-clock measurements.

The linear rule overstates the plain CCD's advantage, because memory-bound programs do not speed up in proportion to clock; Lam makes this point for libx264, expecting the two CCDs to come close [22]. The table's net values are therefore conservative for the V-Cache CCD. Even so, the result shows why AMD placed ordinary applications on the higher-clocked CCD by default, and why the choice still depends on the program [22].

Table 28: IPC gain of the V-Cache CCD at equal clock [22] against the plain CCD's 7% clock advantage (net = $(1 + \text{gain})/1.07 - 1$).

Workload	IPC gain at 4.2 GHz	Net, with plain CCD 7% faster	Better CCD
libx264	+4.9%	−2.0%	plain
GHPC	+9.7%	+2.5%	V-Cache
7-Zip	+9.8%	+2.6%	V-Cache
Cyberpunk 2077[a]	+13.5%	+6.1%	V-Cache
Call of Duty	> +19%	> +11%	V-Cache

[a] Lam's Cyberpunk 2077 run was limited by the graphics card, so the IPC gain did not appear as frame rate (0.5 FPS) [22].

27.2 Refining the rule: memory time does not shrink with clock

The linear rule treats every cycle as if it shrank in proportion to clock frequency. A memory access does not: its latency is set mainly by the DRAM and by the path through the I/O die, and is roughly fixed in nanoseconds (section 26.1). Splitting the time per thousand instructions into a part that scales with clock and a part that does not gives

$$T(f) = \frac{C + A\,\Delta t_{\mathrm{L3}}}{f} + (M - \Delta M)\,s, \tag{13}$$

where C is the number of cycles per thousand instructions not spent waiting for memory, and A and M are the L3 accesses and L3 misses per thousand instructions on the plain CCD. Δt_{L3} is the added L3 latency (zero on the plain CCD, four cycles on the stacked one), ΔM is the number of misses the larger cache removes, and s is the effective stall per miss in nanoseconds. Equation (13) combines equations (1) and (10); the only new assumption is that s does not depend on the core clock.

Appendix F.3 calibrates the model on Lam's libx264 run ($C \approx 543$ cycles, $s \approx 31$ ns, clocks of 4.8 and 5.15 GHz) and tabulates the result in table 50; at equal clocks it gives a 5.2% gain, against 5.1% from Lam's chart IPCs and the 4.9% Lam states.

First, table 50 and figure 52 show that the clock gap roughly triples the number of misses the stacked cache must remove to break even, from about 0.12 per thousand instructions at equal clocks of 4.2 GHz to about 0.35, so the clock penalty matters about twice as much as the four cycles of added latency. The factor depends on the clock at which the equal-clock case is taken: at equal clocks of 4.8 GHz the break-even would be about 0.10, and at 5.15 GHz also about 0.10, so the factor lies between about three and three and a half. Second, the linear rule exaggerates the penalty: for libx264 it predicts a 1.9% loss, with the model's 5.2% and the measured 1.073 (table 28 gives 2.0% from Lam's 4.9% and a rounded 1.07). The refined model instead predicts a 1.0% gain, because about 30% of the plain CCD's time at 5.15 GHz (26% at the 4.2 GHz calibration point) goes to memory stalls that a faster clock does not shorten. Lam's expectation that the two CCDs would come close on this program [22] is what the refined model gives. Third, with no misses removed the V-Cache CCD loses 6.7%, which is the cost a cache-insensitive program pays for running on it. The model is calibrated on one program and assumes that the clocks and the stall per miss carry over, so its numbers are illustrative; its value is in showing which quantities decide the outcome: the misses removed, the share of time spent in memory,

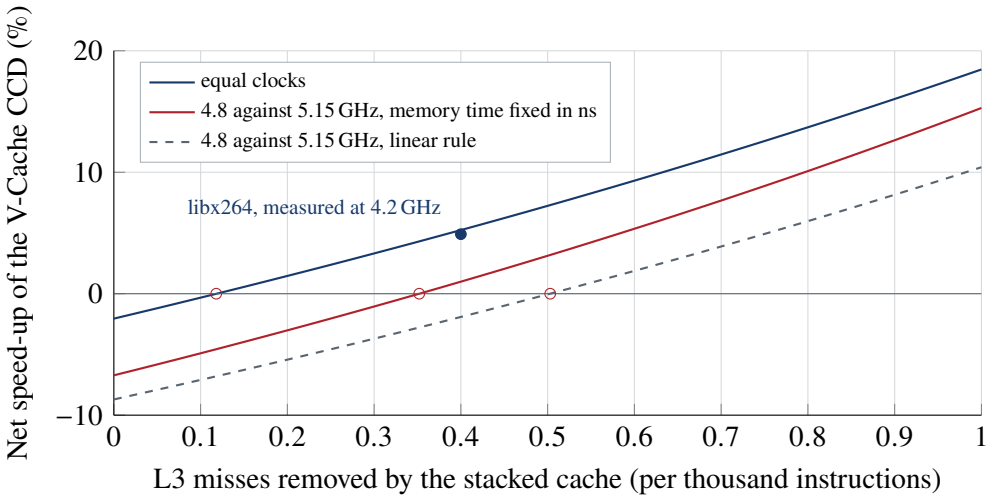


Figure 52: Cache against clock for the libx264 profile of section 17.1, from equation (13). Open circles mark the break-even points: about 0.12 misses removed per thousand instructions at equal clocks, 0.35 with the measured clock gap if memory time is fixed in nanoseconds, and 0.50 under the linear rule. Model calibrated on Lam's fixed-clock measurements [22]; clocks are those Lam measured under an all-core L3-sized load, not during libx264 itself.

and the clock gap.

28 Simulation

The published comparisons of this part never change only one thing. Even Lam's per-CCD measurements, the best controlled of them, cover six workloads on one processor (section 24.3), and every other comparison also changes the clock, the core count or the generation (table 26). A simulation can hold everything fixed and change one factor at a time. This section uses the group's cache-hierarchy model for that purpose. Its results are *model outputs*, not measurements. Several of its inputs are fitted to the very measurements it is set beside, and several of its results follow from its synthetic workloads by construction, so the model *illustrates* the mechanisms of the earlier sections in isolation; it does not confirm them independently. Each subsection says which of its agreements are built in and which are not. One comparison, with the group's own Ryzen 5 5600X, uses data the model was not built from; S0 sets out what it can and cannot show (section 28.2).

28.1 The model and how the experiments are run

The model is trace-driven and functional. It follows every memory access through private L1 and L2 caches, one or more shared L3s, an optional L4 and DRAM, with a directory for coherence, and charges each access the latency of the level that served it (appendix E). It has no pipeline, so it does not predict IPC. Instead, an *overlap factor* multiplies the part of every latency beyond an L2 hit, standing for the share of a miss that an out-of-order core cannot hide (section 4.8). The model therefore reports two averages: the *raw* latency per access, which is what a latency sweep measures, and the *exposed* latency per access after the overlap factor,

which the experiments compare unless a caption says otherwise. The default parameters are the measured Zen 3 latencies and AMD's published figures listed in table 47; the inputs that are assumptions are labelled as such there and where they are used. Every point is the mean of five runs with different random seeds, each with a warm-up and 10^6 measured accesses (2.5×10^5 per thread in S6). For every plotted quantity the spread between seeds was below 1% of the mean, too small to show in the figures; for the miss counts stored alongside, it was at most 1.4%. The CSV files on GitHub (appendix E) give the minimum and maximum for each point. Table 29 lists the seven experiments.

Table 29: The simulation experiments, the factor each one changes, and what each is compared with. S0, S3 and S4 are compared with measurements their inputs are fitted to, except for the 5600X check in S0; S1 is compared in shape only.

	Question	Factor changed (all else equal)	Compared with
S0	What the calibration fixes, and how close the model is to hardware	L3 size, 32 against 96 MB; overlap factor	Milan and Milan-X latency sweep; Lam's three workloads; the group's Ryzen 5 5600X (not fitted)
S1	Where does a larger L3 help? (RQ4)	L3 size, across working sets	MiniFE sweep
S2	Stacked, planar or beside? (RQ3)	Arrangement and latency of the extra capacity	— (no such products exist)
S3	Cache against clock	Clock gap, 0 to 10%	Lam's 7950X3D; section 27.2
S4	What does the split cost? (RQ2)	Two dies or one; cost of the hop	Reguly's core-to-core test
S5	Zen 2 to Zen 3	L3 organisation, then L3 latency	—
S6	When does bandwidth bind?	Number of streaming threads	section 34

28.2 S0: What the calibration fixes, and how the model compares with hardware

Figure 53 runs the model as a latency sweep, with random accesses and no overlap, and sets it beside Chips and Cheese's Milan and Milan-X curves. The model's L3 plateaus, 46.4 and 50.8 cycles, match the measured 47.49 and 51.14, and its latency at 1 GB, 334 and 319 cycles, lies near the measured 342 and 313 [31]. Neither match is a test. The plateaus are inputs, and the 1 GB values follow from the inputs (47 or 51 cycles plus 74 ns at 4 GHz, the 74 ns measured on a different Zen 3 chip). The two pairs also differ for different reasons: the model's 1 GB values differ because the larger L3 still holds a small fraction of the data, whereas the measured ones were taken on two instances whose clocks differed, Cozma noting that the Milan-X instance boosted higher.

A comparison with the group's Ryzen 5 5600X. The comparison with Milan cannot test the model, because its inputs come from the same kind of measurement. The group's own Ryzen 5

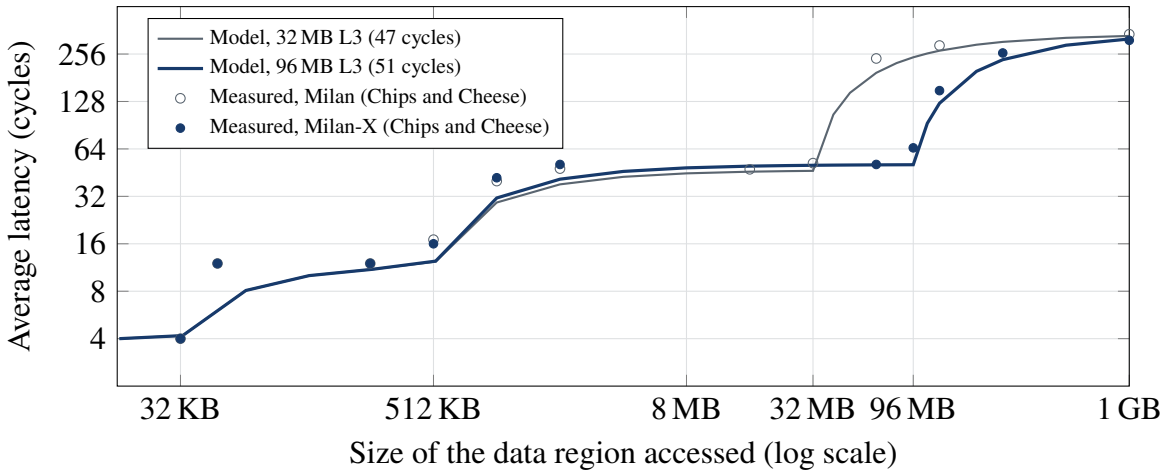


Figure 53: S0: the model's raw average latency (no overlap) for random accesses to a region of growing size (lines; mean of five seeds, spread smaller than the line width) against the latency sweep Chips and Cheese measured on Milan and Milan-X (points, read from figure 46) [31]. The level latencies are inputs, so their match is expected. The shape between them is not: the model's L3 stays flat up to its full capacity, whereas the measured curves rise earlier.

5600X provides data the model was not built from. It has one CCD with the same 32 MB Zen 3 L3, and its latency was measured at the machine's own 4.61 GHz with the kit described in appendix G, whose programs and raw results are on GitHub. Figure 54 sets the two side by side. The model's cache capacities and level latencies are inputs, so on each plateau its output is its input: the comparison there shows how close another Zen 3 chip's published latencies are to this one's, not how good the model is. The L1 and L2 latencies, 4 and 12 cycles, agree with the measured 4.0 and 12.1 cycles; the other two inputs differ by about 20% or less. The measured L3 plateau is 51–55 cycles against the assumed 47, close to the about 53 cycles reported for the Ryzen 7 5800X (table 15), and the latency at 1 GB is 98.6 ns against the model's 82 ns, a difference consistent with different memory and with the virtual machine (Windows Subsystem for Linux, WSL2) in which the measurements ran. The model's own behaviour, how hits and misses mix as a cache fills, is exercised only between the plateaus, and there the unfitted model differs from the measurement by up to 2.4 times (at 32 MB). Fitting the L3 and memory latencies to this machine, 54 cycles and 90 ns beyond the L3 (against the default 74 ns), is a calibration: it aligns the plateaus and the 1 GB value by construction (within 5% and 1%). It does not remove the differences near the capacity edges, which remain 1.2–2.1 times between 19 and 38 MB and 1.1–1.3 times between 45 and 108 MB. The comparison therefore shows two things: the published Zen 3 latencies the model uses are within about a fifth of this machine's, and close to each cache's capacity the model's latency is too low.

Where the shapes differ. Near each capacity edge the model and the hardware differ, on the 5600X as on Milan, in two ways (appendix G). Just above each capacity the measured latency is higher than the model's, partly because a latency sweep visits its lines in a fixed cycle, which the model's random accesses do not reproduce (table 38); and the measured L3 starts to slow down at about 16–20 MB, before it is full, for reasons these tests cannot separate. Both make the smaller cache slower in a latency sweep than in the model, so for a latency sweep the measured gain of a 96 MB over a 32 MB L3 is larger than the model's from about 20 to about 64 MB, and smaller close to 96 MB. How far this carries over to real programs depends on their order of access.

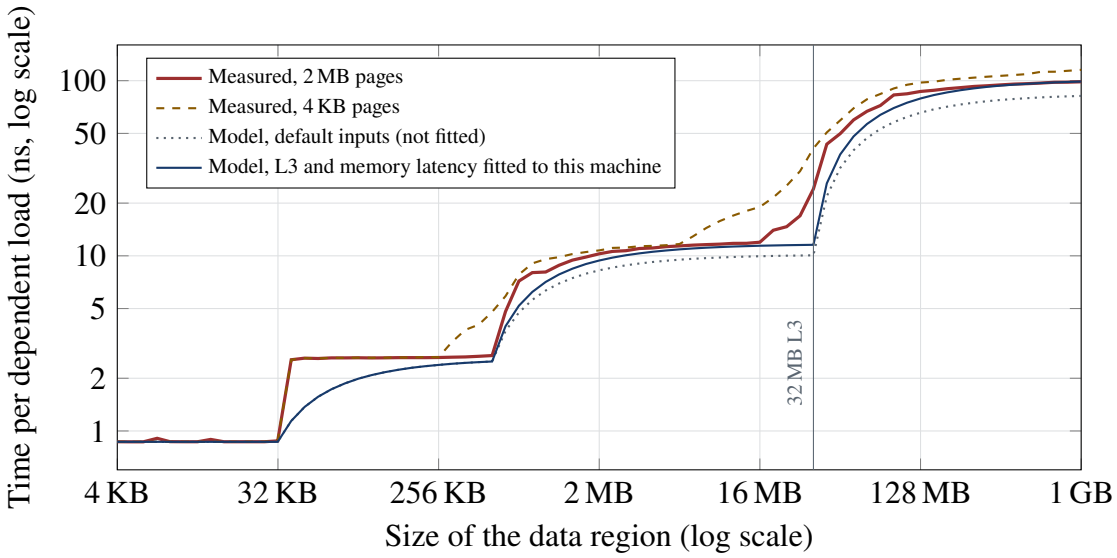


Figure 54: The group's Ryzen 5 5600X (one CCD, 32 MB L3, 4.61 GHz) against the model of section 28: pointer-chasing latency, median of three runs, under WSL2 (appendix G). With its default inputs (dotted), the model's plateaus are the published latencies of another Zen 3 chip, which it takes as inputs; L1 and L2 agree with this machine's, and L3 and memory are within about 20%. Its own behaviour shows only between the plateaus. Fitting those two latencies to this machine (54 cycles, and 90 ns beyond the L3; solid blue) is a calibration that aligns the plateaus and the 1 GB value by construction. Two differences remain after it. Just above each capacity the measured latency is higher, partly because a pointer chase visits its lines in a fixed cycle, which the model's random accesses do not reproduce (table 38); and the measured L3 begins to slow down at about 16–20 MB, before it is full. 4 KB pages add the TLB misses of table 49.

The overlap factor has to be fitted to measurements. Lam's libx264 run saved about 36 cycles per thousand instructions net while removing 0.40 misses and making 3.85 L3 accesses (section 17.1). Because the model also overlaps the four extra L3 cycles, the factor that reproduces this net saving solves an equation given in appendix E: it is 0.335. In the model a removed miss then saves about 103 cycles, fewer than the 130 apparent cycles of section 17.1, which charge the four extra cycles in full. Table 30 gives the same calculation for Cyberpunk 2077 and GHPC, which need 0.202 and 0.105. The experiments below use 0.335, and the sensitivity analyses use the other two values to cover the range Lam's workloads span. All three are fitted values and inherit the caveat of section 17.1: the savings they imply are apparent ones.

A plausibility check on the fitted range. The measurement itself is clear. With k independent pointer chains the 5600X's time per load falls almost exactly as $1/k$, to 0.071 of the single-miss latency with sixteen chains in memory and 0.077 in the L3, consistently across three runs (table 31): a Zen 3 core keeps at least sixteen misses in flight with little loss. What it says about the model is limited, because the overlap factor of Lam's programs cannot be measured on this hardware. The fitted factors correspond to about three misses in flight for libx264 (0.335, against 0.349 measured with three chains), about five for Cyberpunk 2077 (0.202) and about ten for GHPC (0.105, against 0.113 with ten chains). All three are physically possible on this core. As a test of the calibration the check is weak, since almost any fitted value would fall inside the measured range, and it does not show that overlap rather than prefetching explains the fitted values (section 17.1).

Table 30: S0: the overlap factor at which the model reproduces each of Lam's three workloads, solved so that the model's net saving per 1000 instructions, after the four extra L3 cycles are also overlapped, matches the measurement, and the gross cycles a removed miss then saves: the overlap factor times the 306.8 cycles by which a memory access at 4.2 GHz exceeds a stacked-L3 hit. A 64 MB random working set in the model reproduces these values within 1.5%. The per-miss values of table 24 charge the four extra cycles in full and are therefore larger.

Workload (Lam)	Net saving measured (cycles per 1000 instr.)	Overlap factor that reproduces it	Gross saving per removed miss (cycles)
GHPC	77	0.105	32
Cyberpunk 2077	94	0.202	62
libx264	36	0.335	103
no overlap	—	1	307

Table 31: Overlap of independent misses on the group's Ryzen 5 5600X: time per load when the core follows k independent pointer chains through memory (1 GB) or through the L3 (16 MB), and that time as a fraction of the single-chain latency (median of three runs; appendix G). The fraction is what the model's overlap factor stands for.

Chains k	Memory (ns/load)	Fraction	L3 (ns/load)	Fraction
1	104.9	1.000	11.88	1.000
2	53.5	0.510	6.65	0.560
3	36.6	0.349	4.47	0.376
4	27.8	0.265	3.37	0.284
6	18.8	0.179	2.29	0.192
8	14.8	0.141	1.74	0.147
10	11.9	0.113	1.41	0.119
16	7.5	0.071	0.91	0.077

28.3 S1: The working-set window

Figure 55 changes only the L3 size, 32 against 96 MB, and sweeps the working set. Two synthetic workloads bracket real programs: *random*, in which almost every access reaches the L3 or memory once the working set is well above the L2, and *hot/cold*, in which 90% of the accesses go to a tenth of the data. The three regimes of section 13 appear, as they must for workloads of this kind:

- **Working set up to 32 MB:** both caches hold the data. With random accesses the memory system of the larger cache is about 5% slower (speed-up 0.949–0.956), because of its four extra cycles; with hot/cold accesses it is 1–5% slower.
- **Between 32 and 96 MB:** the 32 MB hit rate falls while the 96 MB one stays at 100%. With random accesses the larger cache makes the memory system 1.71 times faster at 40 MB and 3.49 times faster at 96 MB; with hot/cold accesses, whose hot data already fits, 1.03 to 1.28 times.
- **Beyond 96 MB:** the gain falls again, to 1.95 times at 128 MB and 1.27 times at 256 MB with

random accesses, as both caches miss.

The shape resembles the one Domke *et al.* measured for MiniFE (figure 47): a rise once the data outgrows the smaller cache, a peak, and a fall towards 1. The heights are not comparable, 3.49× for the model's memory system against 3.4× for the whole of MiniFE, because the model's random workload sends almost every access to the L3 and MiniFE does not, and because the machines differ. These are speed-ups of the memory system in a synthetic workload, not of a program. S3 uses an access mix fitted to a real program to estimate an effect on time.

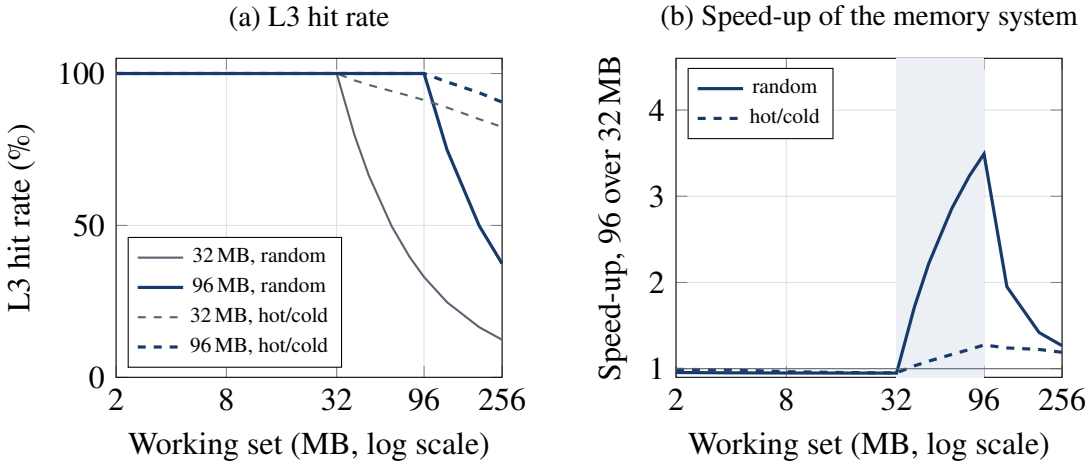


Figure 55: S1: the effect of tripling the L3 against the size of the working set, all else equal (one thread; overlap factor 0.335; mean of five seeds). (a) The L3 hit rate. (b) The model's memory-system time (exposed latency plus one cycle of other work per access) with 32 MB divided by that with 96 MB; values below 1 are the four extra cycles. The gain is confined to working sets between the two capacities (shaded) and fades beyond 96 MB, the three regimes of figure 29. Because almost every access in these synthetic workloads reaches the L3 or memory once the working set is well above the L2, the speed-ups are those of the memory system, not of a program.

28.4 S2: Stacked, planar or beside?

No product offers a planar 96 MB L3 or a cache die beside a Zen CCD, so here the model can only compare assumptions; its verdicts are only as good as the assumed latencies, which is why both are varied. Figure 56 sets five designs against the same random workload. The planar 96 MB L3 is assumed to take 59 cycles, as the longer wires of section 14 suggest. At that latency the stacked L3 has about 10% lower exposed latency than the planar one at every working set up to 96 MB (6.21 against 6.87 ns at 32 MB). For data that fits in 32 MB both pay for their capacity: compared with the 32 MB baseline (5.88 ns), the planar design is 17% slower and the stacked one 5.6% slower.

Table 32 asks how slow a planar 96 MB L3 could be and still beat the 32 MB baseline. For working sets in the window, the answer is "very slow": 107 cycles at 40 MB and 245 at 96 MB, because almost any hit beats a trip to memory. In the model, latency is therefore not the planar design's main disadvantage for programs in that window. Its disadvantages lie elsewhere: programs whose working sets fit in 32 MB pay every extra cycle and gain nothing, and the planar die costs area and yield, which the model cannot price (section 18.3). Stacking keeps the latency penalty small for programs that do not need the capacity, and it moves the area onto a die that

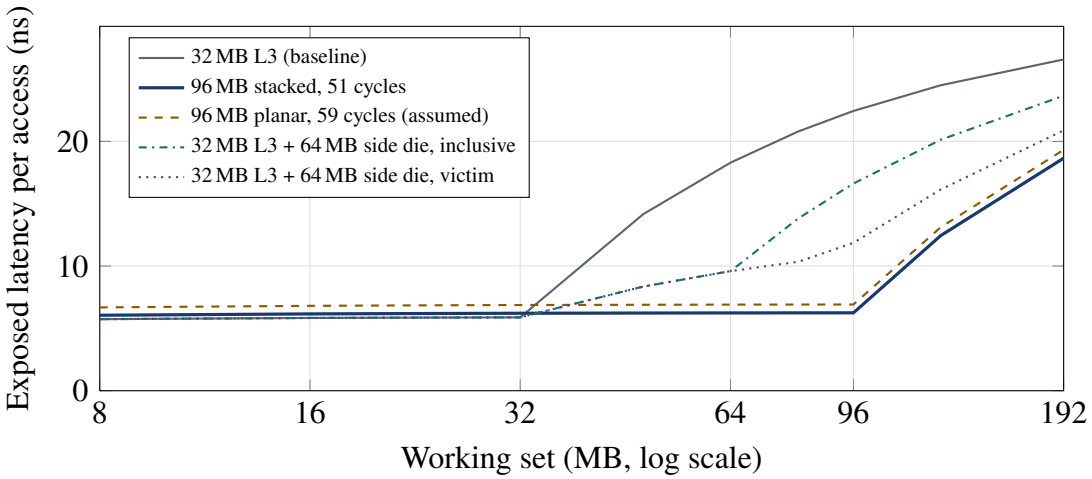


Figure 56: S2: ways to provide about 96 MB of last-level cache, compared on the same random-access workload (one thread at 4 GHz; average latency after the overlap factor of 0.335; mean of five seeds). The planar latency of 59 cycles is an assumption that table 32 varies. The side die is a 64 MB L4 at 22 ns, either filled on every miss (inclusive) or filled by L3 evictions (victim); table 33 varies its latency.

only some products carry.

Table 32: S2: the longest L3 latency at which a planar 96 MB L3 still gives a shorter exposed latency than the 32 MB, 47-cycle baseline (random accesses; overlap 0.335; found by bisection on the five-seed mean). The stacked design needs 51 cycles.

Working set (MB)	24	40	48	64	80	96
Break-even planar latency (cycles)	47	107	147	196	226	245

The cache die beside the CCD is modelled as a 64 MB L4 that is looked up after an L3 miss. Its default latency, 22 ns, gives a hit of about 34 ns in all, close to the more than 30 ns that Lam measured for Broadwell's embedded DRAM (eDRAM); this is an assumption, and a modern 2.5D die could be faster. Table 33 therefore varies it from 5 to 22 ns and compares two organisations. An *inclusive* side die is filled on every miss and holds copies of L3 data, so the L3 and the side die together hold only 64 MB of distinct data; a *victim* side die is filled only by L3 evictions, so together they hold 96 MB. The two are identical at 48 MB, where both hold the data. At 80 MB the victim die is much better (10.34 against 13.81 ns at 22 ns). For working sets beyond 32 MB, however, the side die does not match the stacked L3 even at 5 ns (6.46 against 6.23 ns at 48 MB), because in the model every side-die hit first pays a full L3 miss. Below 32 MB the side die is the faster of the two (5.88 against 6.21 ns at 32 MB), because it adds nothing to an L3 hit, whereas the stacked L3 adds four cycles. Whether a real side die would be looked up after the L3 or alongside it is a design choice the model does not explore.

28.5 S3: Cache against clock

To estimate an effect on time, S3 uses an access mix fitted to Lam's libx264 run (appendix E). Most accesses hit a 16 KB hot region in the L1, a small share go to a cold region whose size is swept, and a smaller share go to a far region that misses in both caches, so that the V-Cache CCD

Table 33: S2: exposed latency per access (ns) of a 64 MB side die beside a 32 MB L3, by its own latency and organisation, against the stacked 96 MB L3 (random accesses, one thread at 4 GHz, overlap 0.335). An inclusive side die is filled on every miss and holds copies of L3 data; a victim side die is filled by L3 evictions. In the model the side die is looked up after an L3 miss.

	48 MB working set		**80 MB working set**	
Side-die latency	**Inclusive**	**Victim**	**Inclusive**	**Victim**
5 ns	6.46	6.46	11.53	6.93
8 ns	6.79	6.79	11.93	7.53
12 ns	7.24	7.24	12.47	8.33
16 ns	7.68	7.68	13.01	9.14
22 ns	8.35	8.35	13.81	10.34
Stacked 96 MB L3	6.23	6.23	6.25	6.25

keeps about 1.04 misses per thousand instructions, as Lam's V-Cache CCD kept about 1.08. Three inputs of the mix are assumptions, not measurements. They are one memory access per three instructions, the textbook's typical figure [1, sec. 11.1.1]; one cycle for an L1 hit, standing for loads the pipeline hides; and 0.48 cycles of other work per access, chosen so that the plain CCD runs at about 730 cycles per thousand instructions, as libx264 did. The plain CCD runs at 5.15 GHz and the V-Cache CCD at a lower clock, and the cold region grows from 32 to 96 MB so that the larger cache removes more and more misses. Figure 57 plots the result against the number of misses removed, and table 34 reads off the break-even for each clock gap.

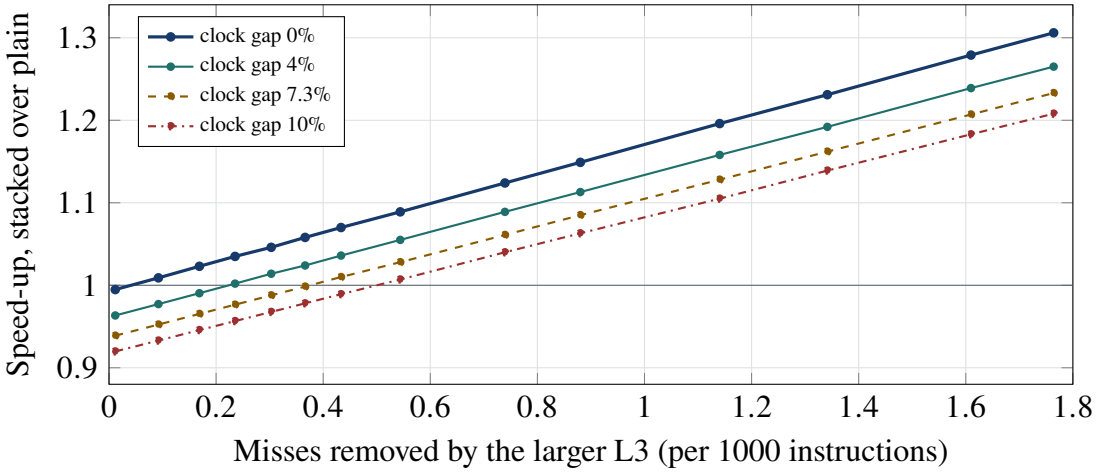


Figure 57: S3: the V-Cache CCD (96 MB, 51 cycles) at a lower clock against the plain CCD (32 MB, 47 cycles) at 5.15 GHz, as the working set grows and the larger cache removes more misses. The access mix is calibrated to Lam's libx264 run (about 3.8 L3 accesses per 1000 instructions, of which about 1.04 per 1000 instructions miss even in the larger cache); the overlap factor is 0.335. A 7.3% gap is the 4.8 against 5.15 GHz Lam measured on the 7950X3D. Each curve crosses 1 at the break-even of that clock gap; table 34 compares the crossings with the analytical model of section 27.2.

The simulation is consistent with the main result of section 27.2: the clock gap, not the extra latency, sets the break-even. Away from equal clocks the two agree closely, within 0.05 misses per thousand instructions at every gap from 2 to 10%; at Lam's 7.3% gap the V-Cache CCD needs 0.38 misses removed per thousand instructions in the simulation and 0.35 in the analytical

Table 34: S3: misses the larger L3 must remove per 1000 instructions for the V-Cache CCD to break even, read from figure 57 by linear interpolation, against the analytical model of section 27.2 at the same clocks (plain CCD at 5.15 GHz; the analytical equal-clock value is 0.12 at 4.2 GHz).

Clock gap	0%	2%	4%	6%	7.3%	10%
Simulation (overlap 0.335)	0.04	0.13	0.22	0.32	0.38	0.50
Analytical model, equation (13)	0.10	0.17	0.24	0.31	0.35	0.45

model. This agreement is expected rather than independent, since both are built on the same calibration and the same structure. At equal clocks the two disagree, 0.04 against 0.10 when both use 5.15 GHz (the analytical model gives 0.12 at the 4.2 GHz of section 27.2). Most of the difference, a factor of about 2.4, comes from one assumption: the simulation's overlap factor hides two thirds of the four extra L3 cycles as well as of each memory access, while the analytical model charges all four cycles. Neither has been measured.

Table 35 shows how far the break-even depends on the inputs. At equal clocks it hardly moves, 0.03 to 0.05. In the simulation the overlap factor scales both the saving of a removed miss and the cost of the four extra cycles, so it cancels, and only the memory latency moves the equal-clock break-even. At the realistic 7.3% gap it is far more sensitive. With the overlap factors fitted to Cyberpunk 2077 and GHPC it rises to 0.58 and 1.05, and with no overlap it falls to 0.17, so across the overlap factors fitted to Lam's workloads, with the libx264 mix held fixed, it changes about threefold. The break-even at a realistic gap is therefore known only to within that range, and it depends most on how much of a miss a core hides.

Table 35: Sensitivity of S3: the break-even (misses removed per 1000 instructions) when one input is changed. The overlap factors 0.202 and 0.105 are those that reproduce Lam's Cyberpunk 2077 and GHPC measurements (table 30).

Changed input	**Equal clocks**	**7.3% clock gap**
calibrated (overlap 0.335)	0.04	0.38
overlap 0.202 (Cyberpunk)	0.04	0.58
overlap 0.105 (GHPC)	0.04	1.05
overlap 1.0	0.04	0.17
memory 60 ns	0.05	0.46
memory 90 ns	0.03	0.31

As a check on the mix, the simulation was run at equal clocks of 4.2 GHz at the point where the V-Cache CCD removes 0.40 misses per thousand instructions, as Lam's libx264 run did. There the plain CCD makes 3.78 L3 accesses and 1.44 misses per thousand instructions and takes 729 cycles, and the V-Cache CCD gains 5.2%, against 5.1% from Lam's chart IPCs (Lam states 4.9%). Because the mix and the overlap factor are both fitted to this one measurement, the agreement shows that the construction is consistent; it is not a test of the model.

28.6 S4: The cost of the chiplet split

The model's default hop between dies is AMD's figure for one round trip between a CCD and the I/O die, under 9 ns (section 8). In the two-die configuration it is charged on every transfer between the CCDs and on every memory access, which crosses to the I/O die. Figure 58 compares this with Reguly's core-to-core measurement (table 11). Only the ratio can be fitted; the absolute values can be compared only roughly. Reguly's test times one message, whereas the model's round trip contains two messages and two write upgrades, and the model's preset is a planar Zen 3 CCD at 4 GHz rather than a Milan-X. With a 9 ns hop, the model's ping-pong round trip between CCDs takes 2.2 times as long as within one CCD (108.5 against 49 ns), against 4.5 times in the measurement (265 against 58.6 ns). One transfer between two different CCDs, a request and its reply, crosses the CCD–IOD link four times, that is, two of AMD's round trips or 18 ns; with an 18 ns hop the model gives 2.9 times. Matching the measured ratio needs a hop of about 37 ns. This value is fitted to a ratio; it is not the cost of a transfer. The model's exchanges are themselves too fast: halved as in section 8.2, its round trips give about 25 ns per message within a CCD against Reguly's 58.6 ns, and about 110 ns across CCDs with a 37 ns hop against about 265 ns, both about 2.4 times too fast. Matching Reguly's absolute extra cost would need a hop of about 97 ns. The measured extra cost is the 206 ns of section 8.2, from one cloud instance, most of which lies in the fabric and the coherence steps rather than in the link itself. The group's 5600X gives the one check of the model's coherence timing on data it was not built from: the model's same-CCD round trip is 49 ns at 4 GHz, against 98.5 ns measured under WSL2 (66–72 ns on the fastest pairs; appendix G). The model's sharing latencies should therefore be read as lower estimates.

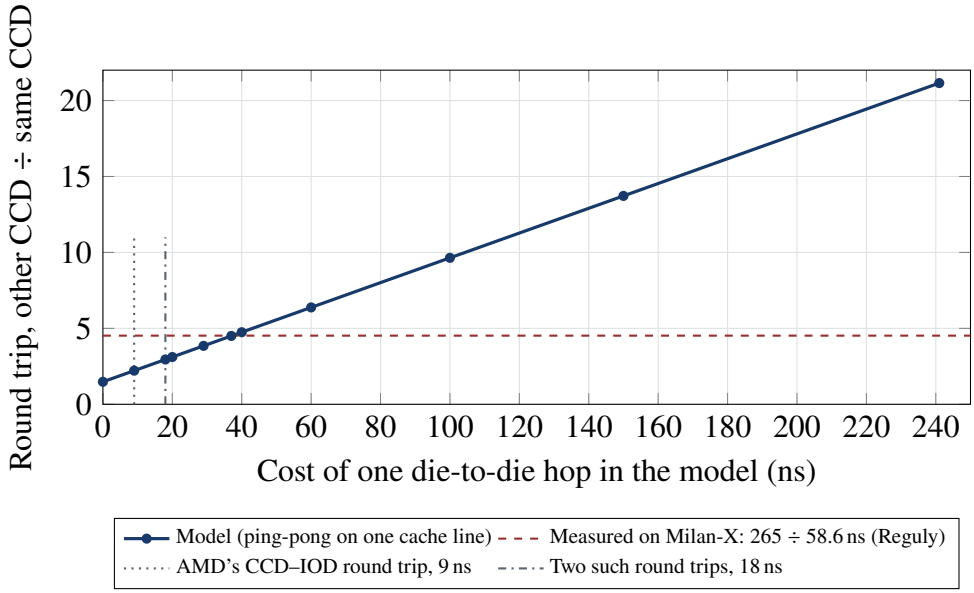


Figure 58: S4: the model's ratio of cross-CCD to same-CCD ping-pong time as the cost of a hop grows (planar Zen 3 CCDs at 4 GHz). Only the ratio is fitted to Reguly's measurement [26], because Reguly's test times one message while the model's round trip contains two messages and two write upgrades. One transfer between two different CCDs, a request and its reply, crosses the CCD–IOD link four times, two of AMD's 9 ns round trips. With a 9 ns hop the model gives 2.2× and with 18 ns 2.9×; matching the measured 4.5× needs a hop of about 37 ns. That value is fitted to the ratio only: the model's exchanges are about 2.4 times faster than the measured ones, so 37 ns is not the real cost of a transfer, which section 8.2 puts at about 206 ns on one cloud instance.

Table 36 then runs sixteen cores on two CCDs against a hypothetical single die with one 64 MB L3 at the same 47 cycles, which favours the single die, since a real 64 MB L3 would be slower (section 14). Two kinds of data are compared:

- **Shared data**, a 48 MB set read and written by all cores (5% writes). It costs the two-die design in two ways. First, 66 of every thousand accesses cross between the CCDs. Second, each CCD keeps its own copy of the lines it reads, so the 48 MB no longer fits in two 32 MB L3s: the two-die design makes 22 memory accesses per thousand, the single die none. Its exposed latency is 1.20 times the single die's with a 9 ns hop, 1.26 times with 18 ns, the two link round trips a transfer between CCDs needs, and 1.39 times with the fitted 37 ns. The 9 ns figure undercounts the link and is therefore a lower limit. The 18 and 37 ns figures also charge every memory access at that hop; with AMD's 9 ns for memory accesses they would be about 1.25 and 1.35 times, by the authors' estimate. Because the 37 ns hop matches only a ratio (figure 58), 1.39 times is not an upper limit either.
- **Private data**, 8 MB per thread, 128 MB in all, which overflows both designs. Nothing crosses between the CCDs, but every memory access pays the hop to the I/O die, and the two-die design is 1.08 times slower with a 9 ns hop. For memory accesses, AMD's 9 ns is the relevant figure, not the 37 ns fitted to sharing.

In the model, then, the split costs most for data shared across the boundary, where replication also wastes L3 capacity, and a few per cent for private data that misses to memory. This is the model's version of the argument for placing communicating threads on one CCD (section 8.2). It follows from how the hop is charged, so it shows the size of the effect for the assumed costs rather than proving that the costs are right.

Table 36: S4: sixteen cores on two CCDs (2 × 32 MB L3; the hop is charged on every transfer between CCDs and on every memory access) against a hypothetical single die with one 64 MB L3 at the same 47 cycles, for four hop costs (overlap 0.335; mean of five seeds). The shared data set is 48 MB, with 5% of accesses writes; the private data sets are 8 MB per thread, 128 MB in all, and overflow both organisations.

Data	Hop (ns)	Transfers between CCDs per 1000 accesses	Memory accesses per 1000 (two dies / one die)	Exposed latency, two dies (ns)	Two dies ÷ one die
shared	9	66	22 / 0	6.83	1.20×
shared	18	66	22 / 0	7.17	1.26×
shared	37	66	22 / 0	7.90	1.39×
shared	60	66	22 / 0	8.77	1.54×
private	9	0	500 / 500	19.65	1.08×
private	18	0	500 / 500	21.15	1.17×
private	37	0	500 / 500	24.34	1.34×
private	60	0	500 / 500	28.19	1.55×

28.7 S5: Zen 2 to Zen 3, one factor at a time

AMD's 19% IPC gain from Zen 2 to Zen 3 combines a unified L3 with a redesigned core (section 26). The model can separate the two L3 changes: the unification and the 7–8 cycles it

added (figure 59). For one thread with a working set of up to 16 MB, the unified L3's exposed latency is about 12% higher than that of the split Zen 2-style L3 (20% higher in raw latency), entirely because the split L3 has the lower latency: with equal latencies the two organisations give identical results. Between 16 and 32 MB the unified L3 is better by 1.7 to 3.0 times (exposed; 4.0 times in raw latency at 32 MB), because a thread on a Zen 2 CCX can use only its own 16 MB. With eight threads sharing one data set, the gain is smaller, 1.3 to 2.3 times, because a miss in one CCX's L3 can often be served from the other CCX's L3, at two L3 latencies plus the hop, instead of from memory. The unification is therefore worth a great deal for working sets between 16 and 32 MB and costs 7–8 cycles on every L3 access elsewhere; for a real program that cost scales with how often it reaches the L3. Its contribution to an average over many programs depends entirely on how many of them fall in the window, which is why the 19% figure cannot be credited to the L3.

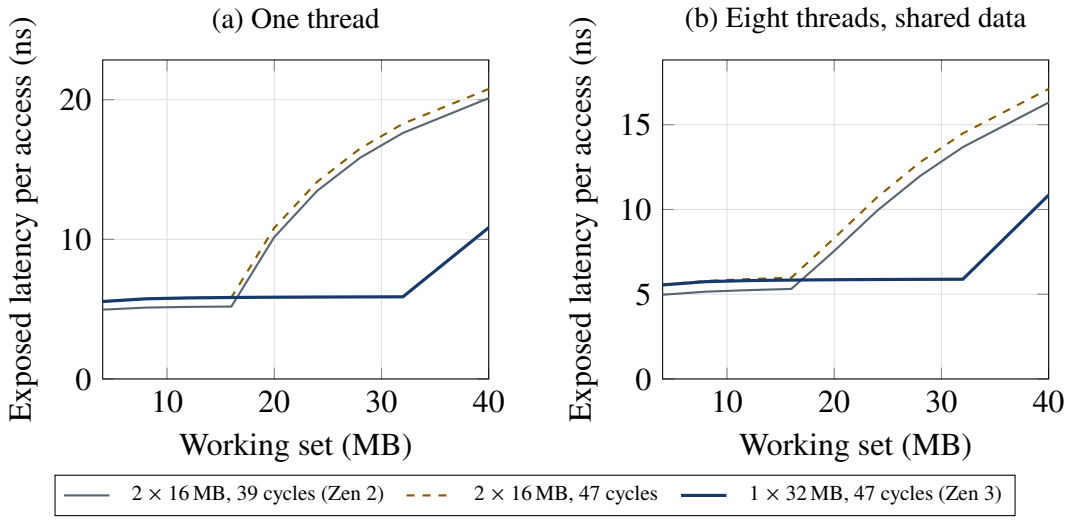


Figure 59: S5: the Zen 2 to Zen 3 change of L3 organisation, separated from the latency change (random accesses; average latency after the overlap factor of 0.335; mean of five seeds). Comparing the dashed line with the thick one isolates the unification itself, since both have a 47-cycle L3; comparing the thin solid line with the dashed one isolates the latency change.

28.8 S6: When bandwidth binds

Figure 60 shows up to eight cores streaming through private data with two DDR4-3200 channels, 51.2 GB/s. Here the overlap factor is set to 0.05, for a core whose prefetchers hide most latency. With 8 MB per thread, the 32 MB L3 holds the data of four threads; at five, the combined 40 MB no longer fits, and throughput falls from 1.08 to 0.68 billion lines per second, limited by memory latency, before levelling off at the DRAM limit of 0.8 billion from six threads. The 96 MB L3 holds all eight threads' data and keeps scaling to 2.1 billion, 2.7 times the DRAM limit. With 20 MB per thread, the 96 MB L3 holds the data of up to four threads; beyond that both caches overflow and meet the same limit. A larger L3 thus raises the bandwidth a chip can sustain only for data that fits in it. Beyond that, more cores add no throughput, which is the bandwidth wall of section 34. The result follows directly from the model's time limits, so it illustrates the argument rather than testing it. The 5600X shows the same wall in practice (appendix G): six threads streaming from memory reach 41 GB/s, 80% of the 51.2 GB/s peak and 6.8 GB/s per core, while the same threads reading data that fits in the L3 (8–16 MB in all) reach 280–296 GB/s. The

model's DRAM limit is the peak, so its absolute throughputs beyond the wall are about a quarter too high; the shape of the wall is the same.

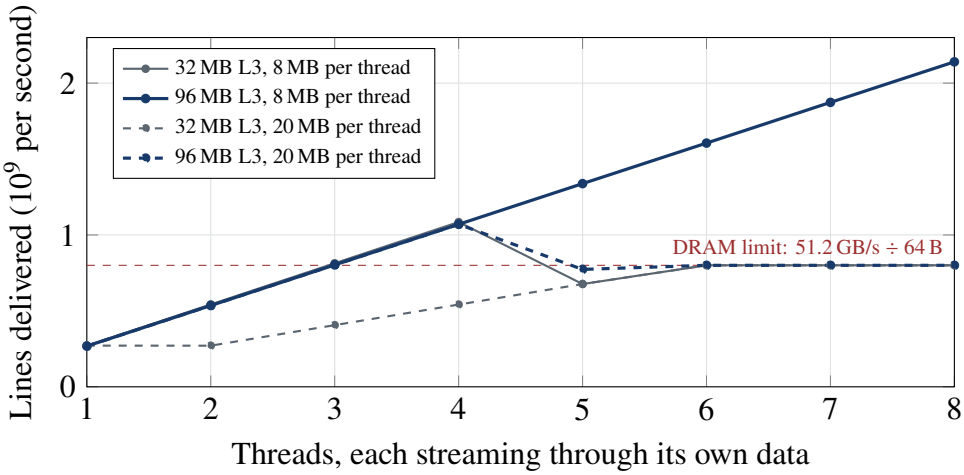


Figure 60: S6: up to eight cores streaming through private data (sequential reads; overlap 0.05, for a core whose prefetchers hide most latency; 51.2 GB/s of DRAM bandwidth; mean of five seeds). While the combined data fits in the L3, throughput grows with the number of threads; once it does not, every configuration meets the same DRAM limit. With 8 MB per thread, the 96 MB L3 holds all eight threads' data and keeps scaling to 2.1 billion lines per second, while the 32 MB L3 overflows at five threads and falls back to the DRAM limit.

28.9 Sensitivity to the assumed inputs

Table 37 repeats the S1 comparison with one input changed at a time, and table 35 did the same for S3. In S1 the pattern holds in every case: a loss (one minus the speed-up) of 2.4–9.6% when the data fits in 32 MB, a gain of 1.89 to 3.78 times at 64 MB, and a smaller gain beyond 96 MB. The size of the gain in the window depends on the overlap factor and on memory latency, from 34% below to 32% above the calibrated 2.86 times, so the model fixes the direction and the window but not the size of the effect. Table 38 varies the replacement policy. With random accesses no policy can change the result, since every resident line is equally likely to be used next, and the three policies agree to within 0.05%. With reuse or a cyclic order the policy matters: for hot/cold accesses the gain at 64 MB ranges from 1.15 to 1.19 times, and for cyclic sequential accesses from 4.27 to 4.75 times. In every case the larger cache still wins in the window and loses slightly when the data fits. A stacked L3 at 55 rather than 51 cycles almost doubles the loss when the data fits, from 5.1 to 9.6%, which shows why AMD's four-cycle figure matters (section 17).

28.10 What the model adds, and what it cannot show

With every other factor held fixed, the model reproduces, largely by construction, the patterns that the published evidence reached only through comparisons that change several things at once. The benefit of a larger L3 is confined to a window of working-set sizes (S1). The extra latency of the stacked cache is a small but real cost outside that window (S1, S2, sensitivity). The clock gap sets the break-even more than the extra latency does (S3). And the split costs most for data

Table 37: Sensitivity of S1: the memory-system speed-up of 96 over 32 MB (as in figure 55b, random accesses) when one assumed input is changed at a time. A 16 MB working set fits in both caches, 64 MB lies between them and 192 MB exceeds both.

Changed input	**16 MB**	**64 MB**	**192 MB**
none (calibrated defaults)	0.95	2.86	1.42
overlap 0.105 (GHPC)	0.98	1.89	1.30
overlap 0.202 (Cyberpunk)	0.96	2.40	1.38
overlap 1.0	0.92	3.78	1.47
memory 60 ns	0.95	2.50	1.38
memory 90 ns	0.95	3.27	1.45
stacked L3 55 cycles	0.90	2.72	1.39

Table 38: Replacement policy: the memory-system speed-up of 96 over 32 MB for three workloads under three policies (overlap 0.335; mean of five seeds). With random accesses every resident line is equally likely to be used next, so no policy can change the result; the policy matters for workloads with reuse or a cyclic order.

Workload	**Policy**	**16 MB**	**64 MB**	**192 MB**
random	LRU	0.95	2.86	1.42
	random	0.95	2.86	1.42
	tree pseudo-LRU	0.95	2.86	1.42
hot/cold	LRU	0.96	1.16	1.22
	random	0.96	1.19	1.22
	tree pseudo-LRU	0.96	1.15	1.16
sequential (cyclic)	LRU	0.95	4.75	0.99
	random	0.95	4.27	0.99
	tree pseudo-LRU	0.95	4.75	0.99

shared across dies (S4). Its value lies less in these reproductions than in the questions it lets the paper ask under stated assumptions. Within the model:

- a planar 96 MB L3 loses mainly on programs that do not need the capacity (S2);
- for working sets beyond 32 MB, a side die looked up after the L3 cannot match a stacked L3 even at 5 ns, and only a victim organisation makes full use of its capacity; below 32 MB the side die is the faster (S2);
- the break-even at a realistic clock gap varies about threefold across the overlap factors of Lam's workloads (S3);
- the two link round trips of one transfer between CCDs, 18 ns, explain only part of the measured cost of sharing between them (S4);
- the Zen 3 L3 unification helps only a window of working sets, so it cannot account for an average IPC gain on its own (S5).

The model has clear limits, which bound how its results may be used:

- It has no pipeline, prefetchers, TLB or clock behaviour, so it gives the time spent in the memory system, not IPC or frame rates. The S3 mix is fitted to one program and contains three assumed inputs.
- Its workloads are synthetic. Traces of real programs through Valgrind were tried but were too slow to finish (several hundred times slowdown), so none is reported.
- It maps each working set contiguously and accesses it at random, so it has no conflict misses below capacity and keeps some hits that a cyclic access pattern would lose. It is therefore optimistic near each capacity edge, as the 5600X measurement shows; for a latency sweep it understates the gain from about 20 to 64 MB and overstates it close to 96 MB (S0).
- Its L3 is filled on every miss; the sources used here do not state how Zen's L3 is filled, and real fill policies may differ. Its link-bandwidth limit is switched off in every experiment.
- Its key inputs are fitted or assumed: the overlap factor is fitted to measurements and the 37 ns hop to a measured ratio, and the latencies of the planar 96 MB L3 and of the side die are assumptions. Each is varied above where it matters.
- Its energy model has placeholder values and is not used in this paper.

Within these limits, the model's role is the one set out at the start of this section: a controlled illustration of the published evidence, not a substitute for it. What the 5600X measurements add is set out in S0 (section 28.2).

29 Summary of Part V

In answer to RQ4, the stacked L3 costs about four cycles of L3 latency in every generation measured, and in its first two generations also several percent of clock speed. In exchange it removes capacity misses for working sets between 32 and 96 MB per CCD. Where those misses dominate, the benefit is large: from about 10–20% higher IPC in miss-heavy games (a geometric mean of 11.4% over Lam's five quantified workloads at equal clock, section 24.3) to more than 1.5× in memory-bound computational fluid dynamics (OpenFOAM), and up to 3.4× for one HPC proxy at favourable problem sizes. Where they do not, the benefit is small, absent or slightly negative. The fourth generation repeats the pattern: Genoa-X gains up to 1.60× in OpenFOAM over a wider range of problem sizes, with almost no loss on small ones (section 24.6). The apparent value of each removed miss also varies, from about 50 to about 130 cycles in Lam's data, possibly because workloads with many misses overlap them, or because the counters include prefetches (table 24). Separating the cache effect from other factors requires fixed-clock comparisons or performance counters, which most vendor comparisons lack. The simulation of section 28 illustrates these patterns with every other factor held fixed, largely by construction: the working-set window, the small cost of the extra latency and the dominance of the clock gap at the break-even. It adds that the break-even at a realistic clock gap changes about threefold across the overlap factors fitted to Lam's workloads, and that the chiplet split costs most for data shared between dies. On the group's Ryzen 5 5600X the model's input latencies are nearly right, but close to each cache's capacity its latency is too low (section 28.2). The confidence of these conclusions is therefore high for the direction of the effect and for its latency cost, moderate for the size of the IPC gain, which rests on one controlled study, and lower still for the size of the effect in any particular workload.

Part VI

Comparative Synthesis, Limitations and Future Directions

Figure 61 maps the topics of this part.

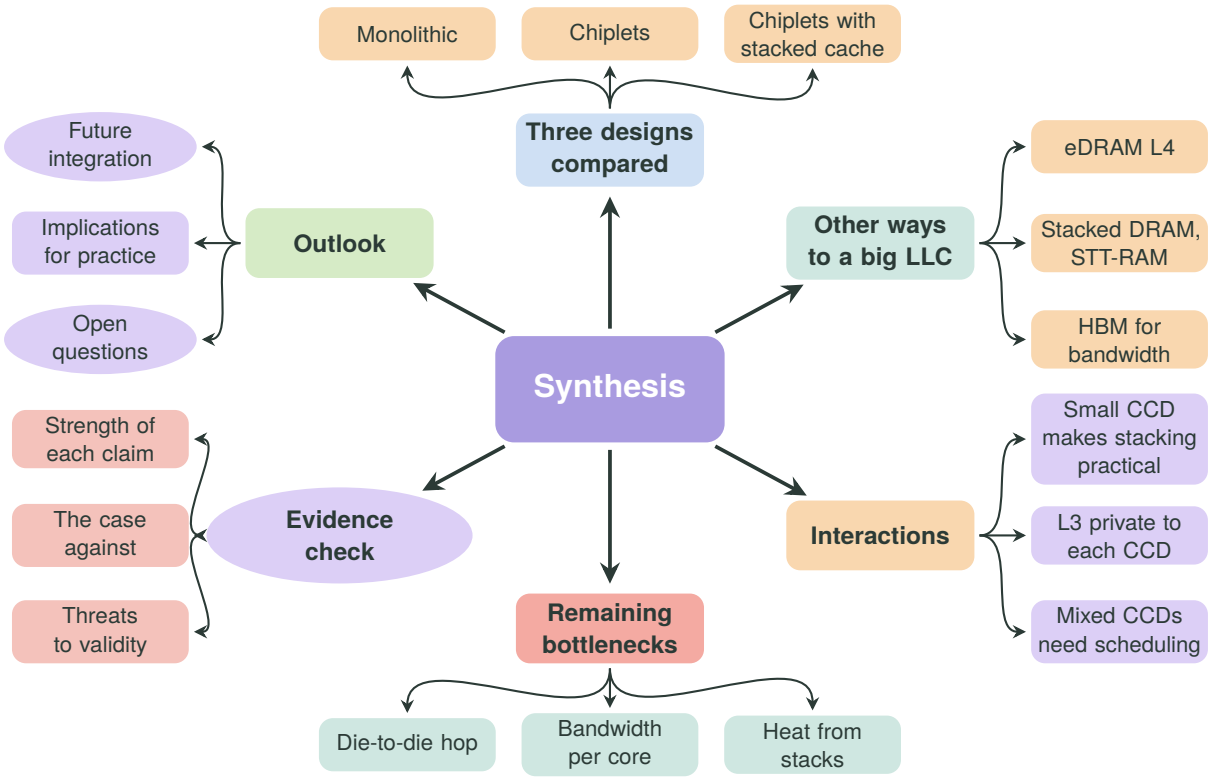


Figure 61: Overview of Part VI. The three bottlenecks at the bottom are the ones that remain after both techniques are applied (section 33); the evidence-check branch is where each conclusion is graded.

30 Monolithic, Chiplet-Based and Vertically Stacked Designs Compared

Table 39 compares the three organisations against the bottleneck–benefit–trade-off test of section 1.4, with the measured or modelled values from Parts III–V where they exist.

Table 39: Comparison of the three design approaches (model values illustrative; measured values from the cited sections).

	Monolithic die	**Chiplets (2D package)**	**Chiplets with stacked cache**
Main limit addressed	—	Die size, yield, uneven node scaling	Planar cost and wire length of a large LLC
Communication	All on-die	Die-to-die links through the package	Adds dense vertical links between cache and compute dies
Memory latency	Shortest on-die paths (no advantage visible in the Cezanne comparison, section 11)	Extra die-to-die hop to memory	Fewer trips to memory for cache-sensitive work
Cost and yield	Falls steeply with die area	Smaller dies, mixed nodes, tested before assembly	Extra die and bonding step; capacity only where sold
Thermal	One heat source per die	Heat spread over several dies	Stack alters heat path from cores
New trade-off	Scaling ceiling	Latency, link power, packaging	Heat, test complexity, workload dependence
64-core compute sets per wafer (model)	about 35	about 95	about 95, plus cache dies
L3 per compute die (Zen 3)	—	32 MB	96 MB
L3 hit latency (Milan / Milan-X)	—	47.5 cycles	51.1 cycles
Clock (Zen 4 desktop, same package)	—	reference	about 7% lower

31 Other Ways to Enlarge the Last-Level Cache

3D V-Cache is one of several ways to give a processor more last-level cache. Table 40 places it beside the alternatives for which the sources give numbers.

A side-by-side cache in practice. Intel's Broadwell processors offer a real example of the side-by-side cache die discussed in section 15. They placed a 128 MB embedded-DRAM die next to the processor die and used it as a fourth-level cache. Lam measured its latency at over 30 ns (34.18 ns in Lam's chart, against 10.63 ns for the 7950X3D's V-Cache CCD) and its bandwidth at about 50 GB/s, a quarter of Broadwell's own L3 bandwidth. Lam explains why: reaching the eDRAM involves another trip over the on-chip ring bus and then Intel's on-package I/O (OPIO) link, which on the related Haswell design is a 64-bit full-duplex interface at 6.4 GT/s, providing 51.2 GB/s in each direction [22]. Because the eDRAM was too slow to be part of the

Table 40: Ways to enlarge the last-level cache, and one way to raise memory bandwidth instead, with the figures the sources report (different studies, nodes and conditions; compare orders of magnitude, not exact values).

Approach	Reported figures	Main drawback	Source
Planar SRAM L3	32 MB per Zen 3 CCD; about 47 cycles	Die area, yield and wire length grow with capacity	[20], [31]
Stacked SRAM (3D V-Cache)	64 MB in 41 mm^2; +4 cycles; >2 TB/s (AMD peak)	Heat path, clocks, bonding cost	[3], [23]
On-package eDRAM L4 (Intel Broadwell, side by side)	128 MB; over 30 ns; about 50 GB/s	Too slow to join the L3; must be a separate L4	[22]
Stacked DRAM cache (simulated, 2006)	32 MB; average 13% fewer cycles per memory access; off-die traffic −66%; +0.08 °C	Higher latency than SRAM; large tag store	[13], [14]
On-package HBM as main memory (Intel Xeon Max 9480)	STREAM 1446 GB/s against 310 GB/s for two-socket Milan-X; cache-to-memory bandwidth ratio 3.8× against 14×	Raises bandwidth, not cache capacity; communication latency unchanged	[26]
STT-RAM (simulated, 16 MB)	5.0 against 11.4 mm^2 for SRAM; leakage 65 against 463 mW; write 11.9 against 3.3 ns	Slow, energy-hungry writes	[21]

L3, the L3 stayed in place to protect the cores from its latency, and Broadwell even used part of the L3's data array to hold the tags for the L4 [22]. Lam contrasts this with V-Cache: each nominally 4 MB slice of the Zen 4 L3 gets its own interface to an 8 MB extension on the cache die, complete with tag and LRU arrays, so the stacked SRAM can extend the L3 rather than add a level [22]. Lam also notes that Zen 4's own Infinity Fabric link between dies provides only about 64 GB/s of read bandwidth per CCD, little better than Broadwell's link, and concludes that package traces are not well suited to L3 bandwidths [22]. This is the comparison of table 16 with real numbers: the side-by-side cache had about three times the latency of the stacked one and, in Lam's multithreaded test, about a thirteenth of its bandwidth (53 against 702 GB/s), and it had to sit at a different level of the hierarchy [22]. The 702 GB/s is consistent with the 4.8 GHz the V-Cache CCD sustained at stock clocks (section 17).

Denser memory technologies. The two simulated alternatives trade speed for density in different ways. Kim *et al.*'s circuit-level estimates for 16 MB caches show STT-RAM needing less than half the area of SRAM (5.0 against 11.4 mm^2) and about a seventh of the leakage power (65 against 463 mW). STT-RAM has a similar read latency (6.9 against 6.6 ns) but a write latency more than three times longer (11.9 against 3.3 ns) and about six times the write energy (1.13 against 0.18 nJ) [21, Table 1]. That is why they place write-intensive blocks in a small SRAM region, an approach that gave a 26% average speedup over a baseline placement policy in their simulations [21]. Black *et al.*'s 32 MB stacked DRAM cache reduced the cycles per memory access of their benchmarks by 13% on average and up to 55%, cut off-die bandwidth and bus power by 66%, and raised the peak temperature by only 0.08 °C, much less than stacked SRAM, because DRAM dissipates less power [14]. Neither has appeared in a mainstream CPU as an L3 extension; V-Cache's advantage is that it changes the cache's location but not its technology, so

it adds capacity at nearly SRAM speed.

32 Interactions Between Chiplet Organisation and Cache Design

The two techniques studied in this paper work as layers of one design, and they interact in four ways.

The chiplet makes the stack practical. Stacking is applied to a small, self-contained compute die. Because the CCD is small, a defective stack loses only one CCD's worth of silicon. Because the same CCD is used across the product line, AMD could design V-Cache support into it "from Day 1" and then offer stacked and unstacked versions of the same die [3, slide 10]. And because the V-Cache CCD is built to the same height as a plain one, server parts with and without V-Cache share a package [3, slide 17].

Stacking moves the cost of the cache to where it is wanted. On a planar die every CCD would pay for a 96 MB L3 whether or not its workload needs it, which is the "product flexibility" barrier on AMD's own list [3, slide 6]. With stacking, the base CCD stays at 81 mm^2 and the 41 mm^2 cache die is added only to the products that benefit.

The chiplet boundary also bounds the cache. The L3 remains private to each CCD (section 5), so a thread gains only from the stacked cache on its own die, and data shared across CCDs still crosses the die-to-die link. A larger per-CCD L3 reduces trips to memory through the IOD, which partly offsets the chiplet's main latency cost (section 8), but it does nothing for traffic between CCDs.

Mixing stacked and plain CCDs creates a scheduling problem. Because stacking is per CCD, a product can combine a V-Cache CCD with a plain one, as the 16-core Ryzen 9 7950X3D does. The two CCDs then have different strengths: one has three times the L3, the other clocks about 7% higher, and neither is better for every program. At the time of Lam's testing, AMD's default policy placed ordinary applications on the higher-clocked CCD, so a user had to set processor affinity by hand to use the cache [22]. The chiplet organisation thus turns a hardware choice into an operating-system decision about where each thread should run.

33 Workload-Dependent Trade-Offs and Remaining Bottlenecks

Parts III–V suggest a simple map of where each technique helps. Throughput workloads with many independent threads favour more cores, which chiplets supply. Workloads whose working sets lie just beyond a CCD's planar L3 favour stacked cache. Latency-sensitive single-threaded code with a small working set favours clock frequency, which a V-Cache part may sacrifice (section 26).

Figure 62 turns this map into a sequence of questions for the stacked cache.

Part V quantified the cache–clock side of this map for the one processor that isolates it, the

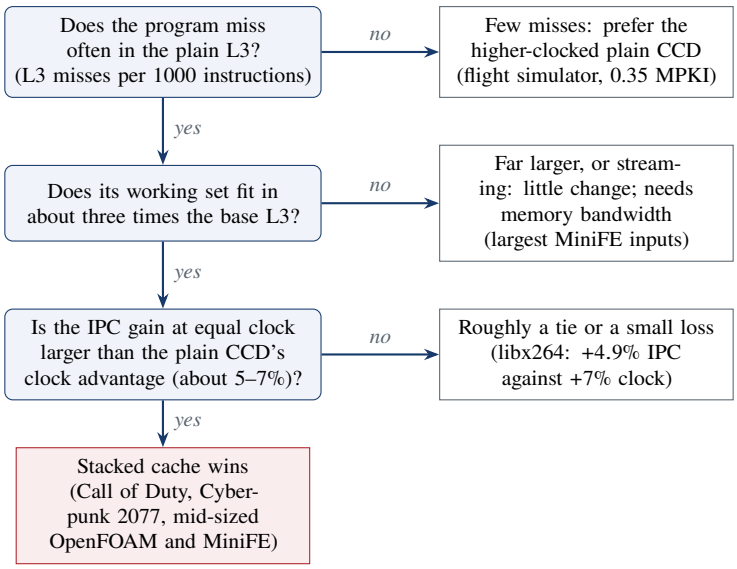


Figure 62: When does a stacked L3 pay off? A decision sequence summarising the evidence of Parts IV and V; the examples are measured cases from [16], [22], [34]. Cost, which also favours the plain CCD, is not shown.

Ryzen 9 7950X3D (section 27). In its analytical model, the 7% clock gap raises the number of misses the larger cache must remove to break even from about 0.1 to about 0.35 per thousand instructions, and the simulation, built on the same calibration and so not an independent check, gives 0.38. A program that gains nothing from the cache loses about 7% on the V-Cache CCD. Figure 62 and the placement policy of section 39 build on that model.

Three bottlenecks remain after both techniques are applied: memory bandwidth per core, which falls as CCDs are added behind a fixed IOD; the die-to-die hop for every L3 miss and for sharing between CCDs; and heat removal from 3D stacks. The alternative technologies of section 31 attack the capacity problem from another direction, at a cost in speed; 3D V-Cache instead keeps SRAM and changes only its placement, which is probably one reason why it could ship in a mainstream product with the latency cost held to four cycles. The figures in table 40 suggest that these alternatives differ chiefly in which workloads they serve, not in a single ranking: stacked SRAM gives the lowest-latency capacity, but only up to 96 MB per CCD in current products.

34 The Memory-Bandwidth Wall Behind Both Techniques

Of the three remaining bottlenecks, memory bandwidth per core is the one that both techniques make more pressing. Chiplets let a socket hold more cores, but the memory channels are fixed by the I/O die and the socket (section 8); a stacked cache raises the capacity per core but leaves the channels unchanged. Table 41 works out what each core receives on the server parts used in this paper.

The Zen 3 rows use AMD's EPYC 7003 description and the L3 sizes reported for the 7763 and 7773X [7], [11]; the Zen 4 rows use AMD's EPYC 9004 datasheet, which lists twelve DDR5-4800 channels and 460.8 GB/s per socket [27]. Between generations, AMD raised the bandwidth per core by half, from 3.2 to 4.8 GB/s, by adding channels and moving to DDR5, while raising the core count by half as well. Within a generation, V-Cache triples the L3 per

Table 41: Memory bandwidth and L3 capacity per core on EPYC processors with all cores busy (peak bandwidth from channel count and data rate; sources as cited).

Processor	Cores	Memory	Peak bandwidth	GB/s per core	L3 per core
EPYC 7763 (Milan)	64	8 × DDR4-3200	204.8 GB/s	3.2	4 MB
EPYC 7773X (Milan-X)	64	8 × DDR4-3200	204.8 GB/s	3.2	12 MB
EPYC 9654 (Genoa)	96	12 × DDR5-4800	460.8 GB/s	4.8	4 MB
EPYC 9684X (Genoa-X)	96	12 × DDR5-4800	460.8 GB/s	4.8	12 MB

core but adds no bandwidth. Stacked cache is therefore a way of needing less bandwidth, not of providing more: it helps when a program reuses data that now fits, and does nothing for data that is streamed once.

Measured bandwidth and the cache-to-memory ratio. Reguly's study of bandwidth-bound HPC codes measured this directly on a two-socket Milan-X system (EPYC 7V73X, an Azure variant with 60 usable cores per socket). The STREAM triad reached 310 GB/s, about 75% of the 2 × 204.8 GB/s peak [26]. The same measurement at small array sizes shows the bandwidth of the on-chip caches, and Reguly reports the ratio between cache and main-memory bandwidth: 14× on Milan-X, against 6× on an Intel Xeon Platinum 8360Y and 3.8× on an Intel Xeon CPU Max 9480 with on-package high-bandwidth memory (HBM) [26]. A high ratio means that data served from cache is much cheaper than data served from memory, so the payoff for reusing data is large.

Software decides how much of the cache is used. Reguly's tiling experiment shows this. Tiling, a cache-blocking optimisation, reorders a structured-mesh code's loops so that data is reused while it is still in cache. Applying it sped up the CloverLeaf 2D application by 4× on the Milan-X system, against 2.7× on the Xeon 8360Y and 1.84× on the HBM-equipped Xeon Max, and Reguly notes that the gains track the cache-to-memory bandwidth ratios of the three platforms [26]. This is consistent with the high cache-to-memory bandwidth ratio of the stacked-cache system rewarding code written to use it, but Reguly did not test a plain Milan, and the three platforms also differ in vendor, compiler, core count and vector width, so the share of the 4× due to V-Cache is unknown. The other direction is to raise memory bandwidth itself: on the Xeon Max, the STREAM triad reached 1446 GB/s, about 4.7× the Milan-X figure (Reguly gives 4.8× against the two other CPUs together), and bandwidth-bound codes ran 2 to 4.3× faster than on the other two CPUs [26]. Reguly also found that core-to-core communication latencies did not improve, so that some applications became limited by communication instead of bandwidth [26]. HBM therefore complements a large SRAM cache: it serves data that is streamed, and the cache serves data that is reused. Figure 63 sets the two measurements side by side.

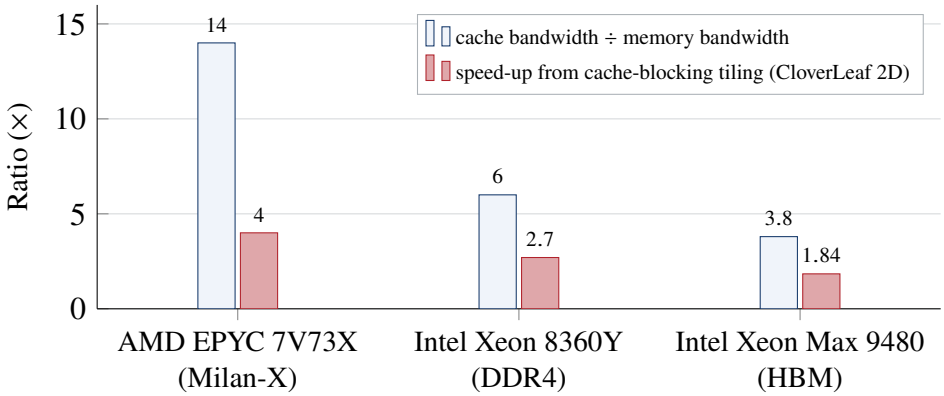


Figure 63: The platform with the largest gap between cache and memory bandwidth gains most from code that reuses data in cache. Measurements by Reguly [26]; the Xeon 8360Y ratio is given as 6× in the bandwidth section and 6.3× in the tiling section of the same paper.

35 Future Directions in Heterogeneous Integration and Advanced Packaging

AMD's Hot Chips 33 presentation sketches integration schemes that go beyond stacking cache on cores, including memory stacked on compute, cores stacked on cores or on uncore logic, and splitting or "folding" circuits across dies; AMD presents these as schemes that advanced packaging could enable but monolithic design could not [6]. These are vendor projections, not products.

AMD's more recent statements point the same way. Its 2023 EPYC white paper asks whether the same bonding technology could add "specialized processing units to the CPU die" rather than only cache [9]. A 2025 AMD packaging talk lists, among future directions, high-speed standardised die-to-die interfaces (UCIe, the Universal Chiplet Interconnect Express [12]), co-packaged optics and module sizes that grow beyond what a single wafer-scale process provides, with 3D interconnect pitch below 9 µm and 2.5D pitch below 40 µm [25, slides 12–13]. An independent review of chiplet research reaches a compatible conclusion from the other side. It finds substantial progress, especially in high-performance computing, but chiplet systems still far from delivering cost, performance, power and design agility together, and it calls for package, architecture and interconnect to be co-optimised by chip designers, package designers and design-tool vendors jointly [12].

These directions raise architectural questions that the evidence from the cache-on-top parts already hints at. Stacking logic on logic would concentrate far more power than stacking SRAM, which was itself thermally significant (section 18.2). Black *et al.* simulated exactly this: splitting a high-performance core's floorplan across two stacked dies removed enough wire to cut power by 15% and raise performance by 15%. The power density, however, rose by 1.3× and the peak temperature by 14 °C, or 26 °C if there were no power savings and the stacking doubled the power density; scaling voltage and frequency down to reach the original temperature still left an 8% performance gain with 34% less power [14]. The thermal cost is roughly three times that of their stacked SRAM cache (4.5 °C), which suggests why commercial stacking has started with cache. Standard interfaces would allow chiplets from different designers to be combined, but each added die-to-die hop adds latency of the kind measured in section 8. And as packages grow,

testing each die before assembly becomes more important for yield (appendix C).

Figure 64 maps these directions.

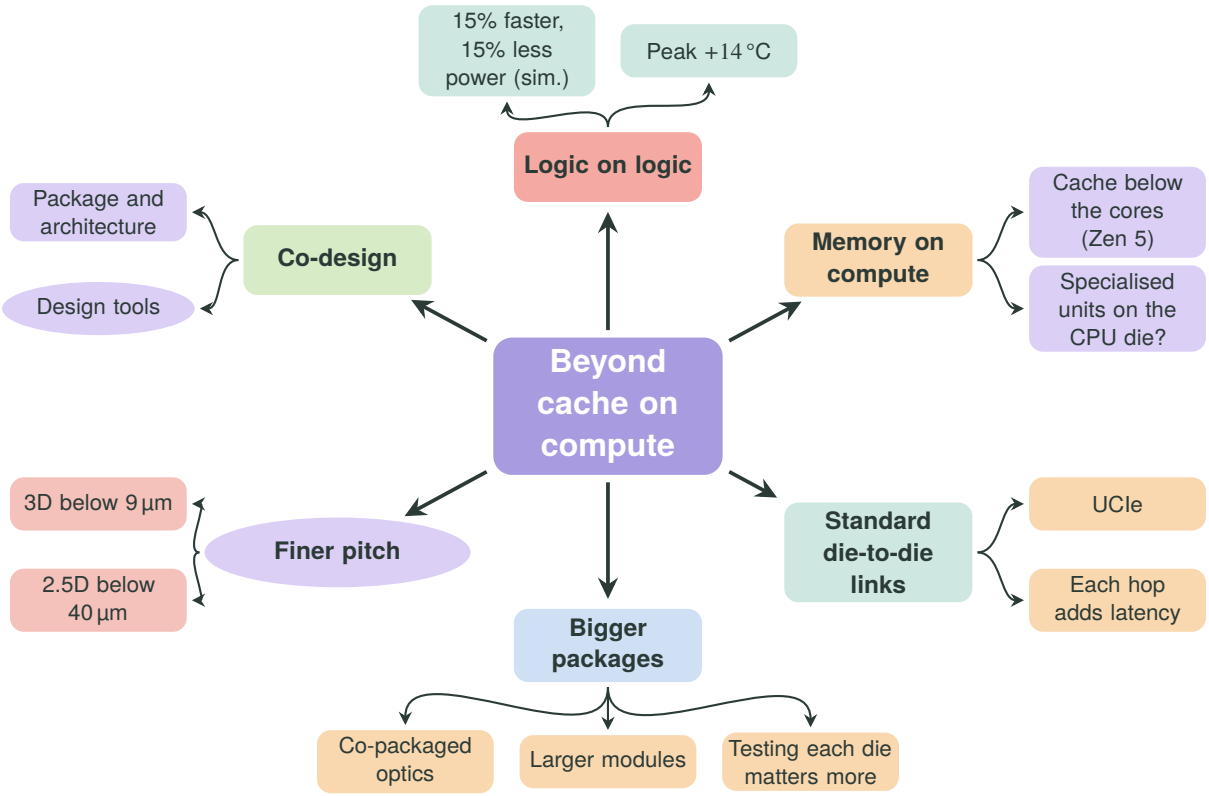


Figure 64: Overview of the future directions of section 35: AMD's stated directions [6], [9], [25], Black *et al.*'s simulated logic-on-logic stack [14] and the co-design conclusion of Liu *et al.*'s review [12]. These are projections and simulations, not products, except where a product is named.

36 How Strong Is the Evidence?

The conclusions below rest on evidence of uneven quality. Table 42 grades each main claim by the best evidence found for it, using the source grading of section 21: whether the evidence is independent of AMD, whether it controls the confounding factors of section 26, and how many workloads or products it covers.

The four cycles of added L3 latency are measured independently on three generations, though by one outlet (strong). The IPC gain on cache-sensitive work is controlled but shown on one processor only (moderate). The die-to-die hop is a mechanism described by AMD and consistent with, though too small to isolate in, the independent comparisons (moderate). The claims about the general size of the effects, how much a typical user gains and how much stacking costs, are weak, because the controlled measurements cover few workloads and the cost data are not public. The conclusions in section 40 are worded accordingly.

Table 42: Strength of the evidence for the paper's main claims. Claims about performance are graded strong when the evidence is independent, controlled and replicated, moderate when one of these is missing, and weak when it is vendor-only, uncontrolled or modelled with illustrative inputs. Two other kinds of claim are graded on their own terms: a fact stated in a datasheet is strong as a fact, and a mechanism is moderate when it rests on established theory, or on a vendor description with which independent evidence is consistent, without its size being measured.

Claim	Best evidence	Strength
Cost per good die rises faster than linearly with die area	Murphy and Dong–Xie yield models (section 6.2); AMD's cost comparison [2]	Moderate: established yield theory, independent of AMD, with illustrative inputs; no independent cost data, though AMD's chart agrees
Chiplets add a die-to-die hop to memory	AMD's description and NUMA latencies (figure 20 and table 10); the one-die against chiplet comparison shows no visible hop but cannot isolate its size (section 11)	Moderate: vendor-described, size not independently isolated
Sharing between CCDs costs several times sharing within one	One independent core-to-core test on a Milan-X cloud instance, 58.6 against about 265 ns (section 8.2)	Moderate: independent, but one system in a virtual machine
One compute die serves a wide product range	AMD's statements and datasheets, checked in table 14 (section 10.2)	Strong for the range (datasheet facts); weak for the per-product CCD configurations, which are inferred
Link power is a small share of the budget	An illustrative estimate from the predecessor link's 2 pJ/bit: about 2% of a Genoa processor's power budget, memory traffic only (section 10.1)	Weak: no measurement
The simulations (section 28)	A trace-driven model calibrated on the measurements above; reproduces the mechanisms, partly by construction; against the group's Ryzen 5 5600X, L1 and L2 inputs agree, L3 and memory within about 20%, and near the capacity edges its latency is up to 2.4 times too low (section 28.2)	Weak: model outputs
Stacked cache adds about four cycles of L3 latency	AMD, plus independent measurements on three generations, all from one outlet (Chips and Cheese; section 17)	Strong
Stacked cache raises IPC on cache-sensitive work	Fixed-clock per-CCD counters on one processor [22]; two HPC studies with unlocked clocks [16], [34]	Moderate: controlled on one processor, not replicated under equal clocks
Size of the average gain across real workloads	AMD averages; reviews at stock clocks (section 24)	Weak
The cache-on-top stack carried a thermal cost	Lower clocks and voltage limits of shipped parts; AMD's later inversion; a 2006 simulation (section 18.2)	Moderate, indirect
Peak V-Cache bandwidth above 2 TB/s	AMD figure; independent CCD-wide tests (Cozma, Lam) find no gain over the plain CCD, and Lam's figures imply about 0.6–0.7 TB/s (section 17)	Weak: vendor figure, not reproduced
Hybrid-bonded stacking is cost-effective	No public cost model; illustrative models only (section 18.3)	Weak

37 The Case Against, and How It Fares

Five objections can be raised against the thesis that chiplets and stacked cache together overcome the limits of monolithic scaling.

"Chiplets are slower than a single die." True for communication. Every L3 miss crosses to the IOD, and Rome's fastest memory access is slower than Naples's (94 against 90 ns), probably because of it (table 10). But AMD reports that the average memory latency of the socket fell by 19%, which it attributes to the central I/O die, decoupled clocks and the latency-optimised link together (section 8). A monolithic die is expected to remain faster for fine-grained sharing across the whole die, because transfers between CCDs cross the IOD (section 8), but the sources do not measure this: Reguly reports only adjacent cores on the Xeons, and EPYC's adjacent cores are faster still (table 11). For most other workloads, the penalty, paid by every memory access, is small next to the core count and yield it buys.

"The stacked cache helps only a narrow set of programs." Largely true. The evidence of Part V shows gains from slightly negative to more than 3× depending on the working set, and tables 28 and 50 show that a modest IPC gain can be cancelled by the clock penalty. This is why the paper calls stacked cache a *targeted* answer to the capacity limit rather than a general one. AMD's own product strategy agrees: it sells stacked and unstacked versions of the same CCD.

"The clock penalty makes the V-Cache part worse for most users." Partly true for the first two generations, in which the V-Cache CCD clocked several per cent lower (section 18.2). The refined model shows that a cache-insensitive program loses about as much as the clock gap (table 50). The inverted stack of the Zen 5 generation is AMD's answer, and whether it fully removes the penalty is one of the open questions below.

"The evidence is mostly AMD's." A fair criticism of the field, and the reason for table 42. Where independent measurements exist, they confirm AMD's latency figures and the direction of its performance claims, but the independent CCD-wide bandwidth tests do not reproduce AMD's aggregate peak (they find no gain over the plain CCD, section 17), and they find a wider spread of performance results than AMD's averages. The paper labels vendor figures as such and prefers independent results where they exist.

"The cost advantage is assumed, not shown." True for stacking. The yield argument for chiplets rests on robust mathematics (section 6.2), but no public cost model covers hybrid-bonded cache, and the models used here predate it (appendix C). The paper therefore makes no quantitative claim about the money cost of stacking; its silicon-area examples are illustrative, and it claims only that stacking places the cost of the extra capacity on the products that need it.

The objections narrow the thesis without refuting it: chiplets are a general and well-supported answer to the cost and size limits of large dies, while stacked cache is a workload-specific answer to the capacity limit whose benefit has to be argued for each class of program.

38 Threats to Validity

The limitations of this study are grouped below by the kind of error they could cause.

Construct validity: are the right things being measured? The paper uses IPC, L3 hit rate and

latency as measures of what the stacked cache changes. These are the quantities the mechanism acts on, but they are not end-user performance; a program can gain IPC and still run slower if its clock falls (section 27.2). Latency sweeps mix cache and TLB effects (section 24.2), and some cycle counts were derived from screenshots of a single run (section 22.1), so they are approximate.

Internal validity: are the effects correctly attributed? Most available comparisons change clock speed, core count or core design along with cache capacity (table 26). The paper grades every comparison against these factors and corrects for clock where the clocks are known (section 26.1), but the corrections assume either linear or fixed-time scaling of memory stalls, and the truth lies between. Only one independent source controls every listed factor [22].

External validity: do the results generalise? The best-controlled evidence covers a handful of workloads on one processor; the HPC studies cover a few codes on one server generation [16], [34]. All the processors studied are AMD's, so the conclusions describe one company's implementation of chiplets and stacking, not every possible one. Desktop and server results are combined in places, although their clocks, power limits and workloads differ.

Source reliability. Much of the detailed technical information comes from AMD itself; such claims are attributed to AMD and treated as engineering claims rather than independent results. Several independent sources are technical web publications that were not peer reviewed; they are used only for measurements whose method they state, and they are archived locally because web pages change after publication, but they are not refereed.

Modelling assumptions. The study works at the architectural level and does not model physical design, packaging processes or thermal behaviour in detail. The thermal argument rests on a 2006 simulation study, the measured clock and voltage limits of shipping parts, and AMD's description of its own redesign. The one temperature reported, a 9800X3D at 95 °C under load, does not isolate what the stack adds, and no measurement of that was found. The cost model uses 1964 and 2009 yield and cost formulations with illustrative parameters, and no public cost model for hybrid-bonded cache stacking was found. The AMAT example keeps three assumed miss rates, and the cache–clock model is calibrated on one program; both are labelled where they are used.

Limited own measurements. Apart from one set of measurements on the group's Ryzen 5 5600X, all measured numbers in this paper come from published sources or from arithmetic on them. The simulations of section 28 are model outputs, calibrated against those sources and labelled as such. Because the model is calibrated on the same measurements it is compared with, its agreement with them is partly by construction; it shows how the mechanisms work and which inputs matter, not how large the effects are in real programs (section 28.10). The 5600X measurements are the one comparison with data the model was not built from (section 28.2). They were taken in a virtual machine on a processor without stacked cache, so they say nothing directly about the stacked cache (appendix G).

39 Implications for Practice

The analysis leads to practical guidance for four groups of readers. Each point follows from evidence presented earlier and is no stronger than that evidence.

For those choosing hardware. A first screen for a stacked-cache part is the L3 miss rate of the intended workload on the plain CCD, not the workload's category (section 25): a low miss rate rules the cache out, and a high one still leaves open whether the working set fits in 96 MB. A workload with few L3 misses is better served by the higher clock of the plain CCD; one whose working set falls between the base and stacked capacities can gain substantially; one that streams far more data than either holds needs memory bandwidth instead (section 34). The miss rate can be measured with the processor's performance counters before buying.

For software developers. On a chiplet processor, threads that share data should run on one CCD, where they share an L3, and memory should be allocated close to the cores that use it; AMD's NPS and "LLC as NUMA" settings expose exactly this structure (section 8). On a stacked-cache part, code that reuses data within the larger cache benefits most, and Reguly's 4× gain from cache-blocking tiling on Milan-X shows how much a code can gain by being written for a high cache-to-memory bandwidth ratio, although that study did not separate the effect of V-Cache itself [26].

For operating-system designers. A processor with one V-Cache CCD and one plain CCD needs a placement policy that knows which threads are cache-sensitive. A default that favours the faster die leaves the cache unused unless the user intervenes [22]; equation (13) suggests that the miss rate per thousand instructions, which the hardware can count, is the right input for such a policy. Figure 65 sketches one.

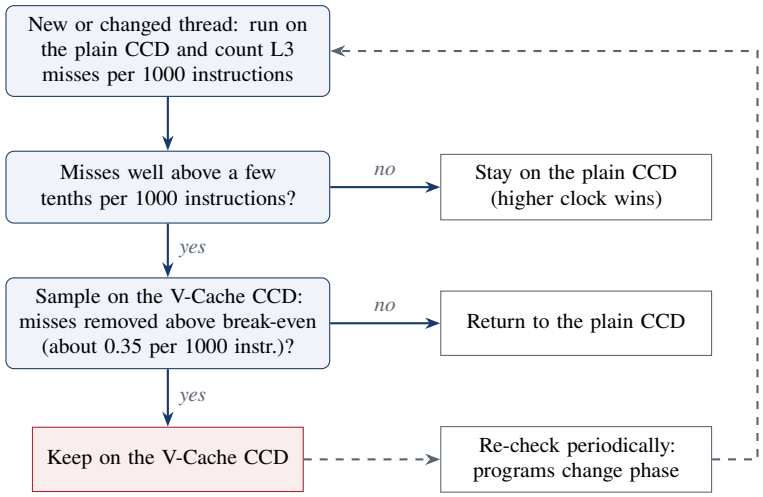


Figure 65: A possible placement policy for a processor with one V-Cache CCD and one plain CCD, built from the cache–clock model of section 27.2. It is a proposal, not AMD's policy: at the time of Lam's testing, the default placed ordinary applications on the higher-clocked CCD [22]. The thresholds come from the libx264 profile and the measured clock gap, and would differ for other programs and processors.

For those evaluating processors. A comparison that is meant to show the effect of cache should fix the clock speed, keep the core count and memory configuration constant, and report performance counters alongside run times (section 26). Reports should state the clocks reached during the test, so that others can apply the corrections of section 26.1.

Figure 66 gathers this guidance by audience.

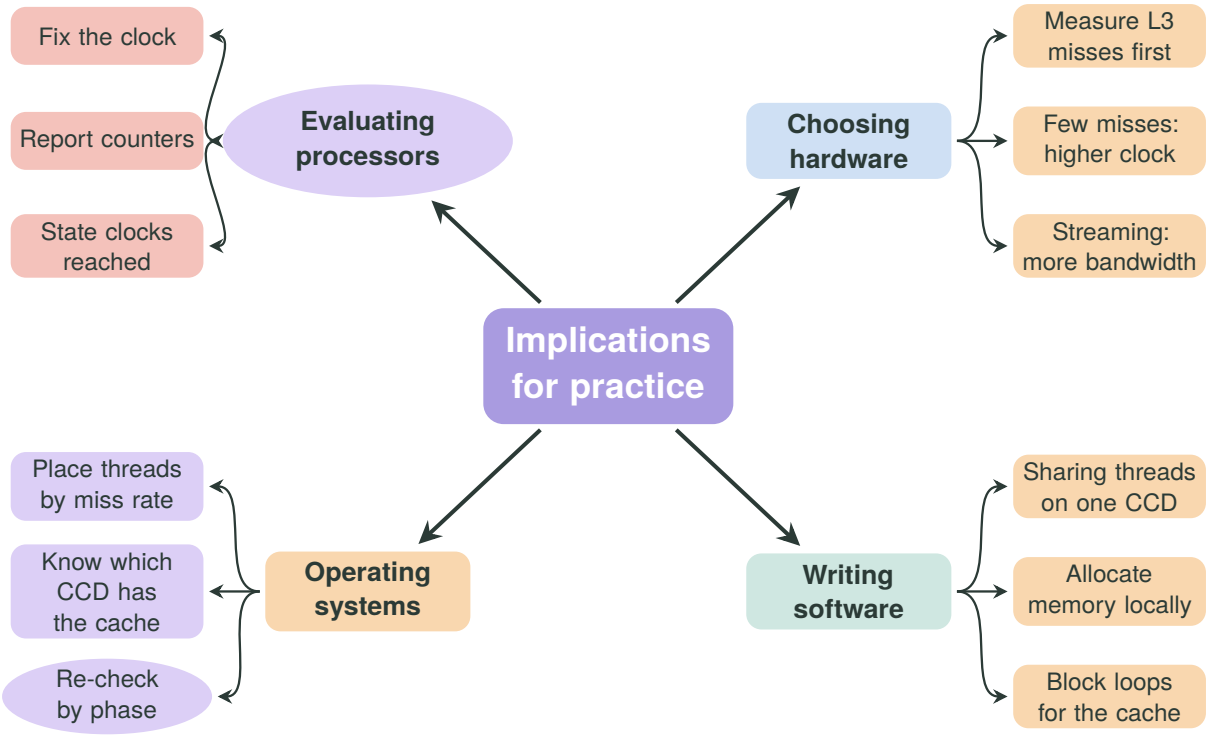


Figure 66: Overview of the guidance in section 39, one branch per group of readers.

40 Conclusions

RQ1: Why is a single large die an increasingly costly way to scale? Because the cost per good die rises faster than linearly with area, through both fewer dies per wafer and lower yield (section 6.2), the reticle limits the largest die (section 6), and analog, I/O and now SRAM circuits gain little from new process nodes (sections 6.3 and 13). Each of these makes the last square millimetres of a large die the most expensive.

RQ2: What does partitioning into compute and I/O dies change? AMD places the cores, their private L2 caches and the shared L3 on the CCDs, made on the newest node, and the memory controllers, I/O and the fabric that links the CCDs on the IOD, made on an older node. In Rome and Matisse it connects them through the organic substrate because, by AMD's account, that reach lets eight CCDs surround one IOD, keeping 2.5D and 3D connections for places where density matters more than reach (RQ2a, sections 7 and 8.4). The split makes high core counts practical, improves silicon yield substantially and moves I/O to a cheaper node, at the cost of a die-to-die hop on every L3 miss and on all communication between CCDs, per-link bandwidth limits and NUMA behaviour that software may need to manage (Part III). It also brings two gains beyond yield. AMD reports that Rome's average memory latency was 19% lower than the four-die Naples design's, which it attributes to the central I/O die, decoupled clocks and the optimised link (table 10). And within each generation one compute die serves a wide range of products, from 6-core desktop parts to 64-core Zen 3 servers, or, in the Zen 4 products of table 14, 16- to 96-core servers with 4 to 48 MB of L3 per core (section 10.2; the 112- and 128-core parts use a different, Zen 4c, die). The cost the split adds to sharing is larger than its cost to memory access: in the one measurement available, passing data between cores on different CCDs took about 4.5 times as long as within one (section 8.2).

On power the evidence is thin: the predecessor link's energy per bit puts the energy of moving memory data across the package at about 2% of a Genoa processor's power budget, leaving out sharing, I/O and idle power (section 10.1). The power side of RQ2 is therefore answered only for link energy, and otherwise qualitatively, beyond AMD's statement that decoupled uncore clocks and DVFS let power shift between the cores and the I/O die.

RQ3: Why does LLC capacity matter, and what does stacking change? A larger LLC removes capacity misses, but in the plane of the die it costs area, yield, wire length and power (section 14). Stacking adds 64 MB per CCD for about four cycles of latency, with dense, short connections and no growth in package footprint. Its price is a harder heat path, which AMD first managed with structural silicon, alongside lower clocks and voltage limits that the sources attribute at least partly to the stacked cache's voltage tolerance, and then addressed by inverting the stack (section 18.2). No measurement of a V-Cache part's energy use, or of the temperature the stack adds, was found: the one temperature reported is a 9800X3D at 95 °C under load, and the only power figures are rated defaults, 400 W for the EPYC 9684X against 360 W for the 9654, which also differ in clocks (table 20). The power side of RQ3 is therefore answered only indirectly.

RQ4: Where does the stacked cache help, and how can its effect be isolated? It helps workloads whose working sets fall between the base and stacked capacities, often substantially, and does little or slightly harms others. Its effect can be isolated only by fixed-clock comparisons or performance-counter measurements, and the best such evidence supports the mechanism, but on one processor and a handful of workloads (section 26, table 42). Even at equal clocks the larger cache pays off only once it removes a threshold number of misses, about 0.1 per thousand instructions for the libx264 example (0.12 at the 4.2 GHz of the calibration). The 7% clock gap of the Zen 4 Ryzen 9 7950X3D raises it to about 0.35 in that model and 0.38 in the simulation, which is built on the same calibration and so is not an independent check (sections 27.2 and 28.5). Both rest on an apparent saving of about 130 cycles per counted miss, which may reflect prefetching as well as overlap (section 17.1); across the range of that saving implied by Lam's workloads, the threshold at a realistic gap changes about threefold (section 28.9).

The two techniques therefore address different limits. Chiplet organisation is a general answer to the cost and size limits of monolithic scaling. Stacked cache is a targeted answer to the capacity limit of the last-level cache, whose value depends on the workload. AMD Zen shows that they work well together: the small, reusable compute die makes stacking practical, and stacking lets that die serve both cache-hungry and clock-hungry markets.

41 Open Research Questions

- When is 3D integration worth its extra complexity, compared with a side-by-side cache chiplet built with today's 2.5D packaging rather than Broadwell's older links?
- What limits the stacking of logic on logic, as opposed to SRAM on logic, from a thermal and power-delivery point of view?
- What is the stacked-cache benefit across a broad, representative workload set when clock speed and core count are held constant?
- How much of the clock penalty of the cache-on-top parts was thermal and how much

electrical, and how fully does the inverted stack remove it?

- What would a current, public cost model for hybrid-bonded cache stacking show?
- How should a large stacked SRAM cache and on-package high-bandwidth memory be combined, given that one serves reused data and the other streamed data (section 34)?
- How should operating systems place threads and data when L3 capacity differs between compute dies, and can a scheduler do it automatically from counted L3 misses per thousand instructions, close to the best static placement?
- How do the 5600X results of appendix G change under native Linux, and how does a V-Cache part such as the Ryzen 7 5800X3D compare on the same tests?

References

[1] S. R. Sarangi, *Basic Computer Architecture*, Version 3.09. Author's online edition, 2025.

[2] S. Naffziger, K. Lepak, M. Paraschou, and M. Subramony, "AMD chiplet architecture for high-performance server and desktop products," in *2020 IEEE International Solid-State Circuits Conference (ISSCC)*, IEEE, 2020, pp. 44–45. DOI: 10.1109/ISSCC19947.2020.9063103

[3] J. Wuu et al., "3D V-Cache: The implementation of a hybrid-bonded 64MB stacked cache for a 7nm x86-64 CPU," in *2022 IEEE International Solid-State Circuits Conference (ISSCC)*, IEEE, 2022, pp. 428–429. DOI: 10.1109/ISSCC42614.2022.9731565

[4] D. Suggs, D. Bouvier, M. Clark, K. Lepak, and M. Subramony, "AMD Zen 2," in *2019 IEEE Hot Chips 31 Symposium (HCS)*, IEEE, 2019.

[5] Advanced Micro Devices, Inc. "AMD Launches AMD Ryzen 5000 Series Desktop Processors: The Fastest Gaming CPUs in the World," AMD Investor Relations (press release). [Online]. Available: https://ir.amd.com/news-events/press-releases/detail/972/amd-launches-amd-ryzen-5000-series-desktop-processors-the-fastest-gaming-cpus-in-the-world

[6] R. Swaminathan, "Advanced Packaging: Enabling Moore's Law's Next Frontier Through Heterogeneous Integration," in *2021 IEEE Hot Chips 33 Symposium (HCS)*, Virtual: IEEE, 2021. [Online]. Available: https://hc33.hotchips.org/assets/program/tutorials/2021%20Hot%20Chips%20AMD%20Advanced%20Packaging%20Swaminathan%20Final%20%2020210820.pdf

[7] M. Larabel. "AMD EPYC 7773X "Milan-X" Benchmarks Show Very Strong HPC Performance Upgrade," Phoronix. [Online]. Available: https://www.phoronix.com/review/amd-epyc-7773x-linux

[8] A. Alam, T. Chen, and R. Swaminathan, "2nd Generation 3D V-Cache Enablement," presented at the IMAPS 20th Device Packaging Conference (Fountain Hills, AZ), Mar. 2024. [Online]. Available: https://imapsource.org/api/v1/articles/129075-2nd-generation-3d-v-cache-tm-enablement.pdf

[9] Advanced Micro Devices, Inc., "4th Gen AMD EPYC Processor Architecture," Advanced Micro Devices, Inc., White paper, second edition, Jun. 2023.

[10] Advanced Micro Devices, Inc. "The Gaming Legend Continues — AMD Introduces Next-Generation AMD Ryzen 7 9800X3D Processor," AMD Investor Relations (press release). [Online]. Available: https://ir.amd.com/news-events/press-releases/detail/1225/the-gaming-legend-continues-amd-introduces-next-generation-amd-ryzen-7-9800x3d-processor

[11] Advanced Micro Devices, Inc., "AMD EPYC 7003 processor microarchitecture overview," Advanced Micro Devices, Inc., Publication 57075, 2022. [Online]. Available: https://www.amd.com/content/dam/amd/en/documents/epyc-technical-docs/white-papers/overview-amd-epyc7003-series-processors-microarchitecture.pdf

[12] Y. Liu, X. Li, and S. Yin, "Review of chiplet-based design: System architecture and interconnection," *Science China Information Sciences*, vol. 67, no. 10, p. 200 401, Oct. 2024. DOI: 10.1007/s11432-023-3926-8

[13] S. Mittal and J. S. Vetter, "A Survey Of Techniques for Architecting DRAM Caches," *IEEE Transactions on Parallel and Distributed Systems*, vol. 27, no. 6, pp. 1852–1863, Jun. 1, 2016. DOI: 10.1109/TPDS.2015.2461155

[14] B. Black et al., "Die Stacking (3D) Microarchitecture," in *2006 39th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO'06)*, Orlando, FL: IEEE, Dec. 2006, pp. 469–479. DOI: 10.1109/MICRO.2006.18

[15] X. Dong and Y. Xie, "System-level cost analysis and design exploration for three-dimensional integrated circuits (3D ICs)," in *2009 Asia and South Pacific Design Automation Conference*, Yokohama, Japan: IEEE, Jan. 2009, pp. 234–241. DOI: 10.1109/ASPDAC.2009.4796486

[16] J. Domke et al., "At the Locus of Performance: Quantifying the Effects of Copious 3D-Stacked Cache on HPC Workloads," *ACM Transactions on Architecture and Code Optimization*, vol. 20, no. 4, pp. 1–26, Dec. 31, 2023. DOI: 10.1145/3629520

[17] B. Justice. "AMD Ryzen 7 5800X vs Ryzen 7 3700X Performance Review," The FPS Review. [Online]. Available: https://www.thefpsreview.com/2021/10/06/amd-ryzen-7-5800x-vs-ryzen-7-3700x-performance-review/

[18] Advanced Micro Devices, Inc., "Using SEV with AMD EPYC Processors," Advanced Micro Devices, Inc., User guide 58207, Rev. 1.0, Mar. 2023. [Online]. Available: https://www.amd.com/content/dam/amd/en/documents/developer/58207-using-sev-with-amd-epyc-processors.pdf

[19] B. Wang, "Design and Implementation of Cache Memory with Dual Unit Tile/Line Accessibility," *Mathematical Problems in Engineering*, vol. 2019, no. 1, Jan. 2019, Art. no. 9601961. DOI: 10.1155/2019/9601961

[20] 7-cpu.com. "AMD Zen3," 7-cpu.com. [Online]. Available: https://www.7-cpu.com/cpu/Zen3.html

[21] B. Kim, Y. Kim, P. Nair, and S. Hong, "Exploiting Data Compression for Adaptive Block Placement in Hybrid Caches," *Electronics*, vol. 11, no. 2, Jan. 12, 2022, Art. no. 240. DOI: 10.3390/electronics11020240

[22] C. Lam. "AMD's 7950X3D: Zen 4 Gets VCache," Chips and Cheese. [Online]. Available: https://chipsandcheese.com/p/amds-7950x3d-zen-4-gets-vcache

[23] G. Cozma. "Deep Diving Zen 3 V-Cache," Chips and Cheese. [Online]. Available: https://chipsandcheese.com/p/deep-diving-zen-3-v-cache

[24] B. Murphy, "Cost-size optima of monolithic integrated circuits," *Proceedings of the IEEE*, vol. 52, no. 12, pp. 1537–1545, 1964. DOI: 10.1109/PROC.1964.3442

[25] W. Lambert, "Heterogenous Integration in High Volume: Now and Looking Forward," presented at the IMAPS 2025 Device Packaging Conference (Phoenix, AZ), 2025. [Online]. Available: https://imapsource.org/article/147811.pdf

[26] I. Z. Reguly, "Comparative evaluation of bandwidth-bound applications on the Intel Xeon CPU MAX Series," in *Proceedings of the SC23 Workshops: Performance Modeling, Benchmarking and Simulation of High Performance Computer Systems (PMBS '23)*, Denver, CO, Nov. 2023. arXiv: 2309.09084. [Online]. Available: https://arxiv.org/abs/2309.09084

[27] Advanced Micro Devices, Inc., "AMD EPYC 9004 Series Processors," Advanced Micro Devices, Inc., Data sheet, 2023. [Online]. Available: https://www.amd.com/content/dam/amd/en/documents/epyc-business-docs/datasheets/amd-epyc-9004-series-processors-datasheet.pdf

[28] Advanced Micro Devices, Inc., "AMD Ryzen 3000 and 5000 Series Processors," Advanced Micro Devices, Inc., Datasheet, 2022. [Online]. Available: https://www.amd.com/content/dam/amd/en/documents/products/processors/ryzen/5000/amd-ryzen-5000-datasheet.pdf

[29] S. Walton. "AMD Ryzen 7 5700G APU Review: Radeon GPU Inside," TechSpot. [Online]. Available: https://www.techspot.com/review/2293-amd-ryzen-5700g/

[30] devtechprofile. "For whom Cezanne could be disappointing," CapFrameX Blog. [Online]. Available: https://www.capframex.com/tests/For%20whom%20Cezanne%20could%20be%20disappointing

[31] G. Cozma. "AMD's V-Cache Tested: The Latency Teaser," Chips and Cheese. [Online]. Available: https://chipsandcheese.com/p/amds-v-cache-tested-the-latency-teaser

[32] G. Cozma. "AMD's 9800X3D: 2nd Generation V-Cache," Chips and Cheese. [Online]. Available: https://chipsandcheese.com/p/amds-9800x3d-2nd-generation-v-cache

[33] Advanced Micro Devices, Inc. "AMD Ryzen 7 9800X3D Desktop Processor," AMD (product specifications). [Online]. Available: https://www.amd.com/en/products/processors/desktops/ryzen/9000-series/amd-ryzen-7-9800x3d.html

[34] M. Lawenda, Ł. Szustak, L. Környei, F. C. C. Galeazzo, and P. Bratek. "Evaluating the impact of the L3 cache size of AMD EPYC CPUs on the performance of CFD applications." arXiv: 2505.17934, pre-published.

[35] M. Larabel. "AMD Ryzen 7 5800X vs. Ryzen 7 5800X3D On Linux 6.0 Benchmarks," Phoronix. [Online]. Available: https://www.phoronix.com/review/amd-5800x3d-linux6

[36] S. Walton. "AMD Ryzen 7 5800X Review," TechSpot. [Online]. Available: https://www.techspot.com/review/2134-amd-ryzen-5800x/

A Plagiarism Report

The similarity of this paper has been checked with Turnitin on 12 November 2026 (submission ID trn:oid:::1:3709247204, 151 pages), with the bibliography and quoted text excluded. Turnitin displayed an overall similarity of 2% (table 43); no single source accounted for 1% or more.

Table 43: Turnitin similarity report, as displayed by the tool.

Item	**Matches**	**Similarity**
Overall similarity		2%
Not cited or quoted	142	2%
Missing quotations	1	0%
Missing citation	0	0%
Cited and quoted	0	0%
Internet sources		2%
Publications		1%
Submitted works (student papers)		0%
Largest single source (67 sources in all)		<1%

B Reading Indices

The reading indices of the main text (Parts I to VI) have been calculated with the `textstat` library (version 0.7.13) on the US school grade-level scale, except Flesch Reading Ease, which is on a 0–100 scale where a higher score means easier text (table 44).

Table 44: Reading indices of the main text (Parts I–VI).

Reading index	**Score**
Gunning Fog	11.95
Flesch–Kincaid Grade Level	9.43
SMOG	11.60
Coleman–Liau	9.33
Automated Readability Index	10.42
Average grade level	10.55
Median grade level	10.42
Flesch Reading Ease (0–100)	61.81

C Yield and Cost-per-Good-Die Model

This appendix derives the illustrative cost model used in section 6.2. It is a textbook-style model for reasoning about trends, not a model of any real AMD product or foundry process. The yield expressions follow Murphy [24] and Dong and Xie [15]; the numerical examples are this paper's own.

C.1 From Poisson to Murphy yield

If yield-killing defects fall independently and uniformly with a *fixed* density D per unit area, the number of defects on a die of area A is Poisson-distributed with mean AD. A die is good only if it has no defects, so its yield is

$$Y_{\text{Poisson}}(A, D) = e^{-AD}.$$

In practice the defect density is not fixed: it varies from chip to chip and from slice to slice. Murphy therefore averaged the Poisson yield over a normalised distribution $f(D)$ of defect densities, $Y = \int_0^\infty e^{-DA} f(D)\, dD$, and solved it for three simplified distributions [24, Eqs. 7–11]. If D is spread uniformly (a "rectangular" distribution) between 0 and $2D_0$, so that its mean is D_0, then

$$Y_{\text{Murphy}}(A) = \frac{1}{2D_0} \int_0^{2D_0} e^{-AD}\, dD = \frac{1 - e^{-2AD_0}}{2AD_0}.$$

This is Murphy's Eq. 11, and multiplying it by the fraction of wafers that survive processing, Y_{wafer}, gives the expression Dong and Xie use (their Eq. 15), which is equation (8).

Murphy also considered a narrower, "triangular" distribution, which gives

$$Y_{\text{triangular}}(A) = \left(\frac{1 - e^{-AD_0}}{AD_0} \right)^2.$$

Murphy regarded the zero-width (Poisson) and rectangular cases as extremes that are "almost certainly inappropriate" and used the triangular form for most of the paper's results, while noting that the three forms differ little at high yields [24, p. 1539]. This paper keeps the rectangular form because it is the one used in the 3D-IC cost model of Dong and Xie. For the 1 and 2 cm^2 dies of the example the choice changes the result by less than one percentage point (82.4 against 82.1%, 68.8 against 67.9%). For the 592 mm^2 monolithic die used in Parts I and III the rectangular and triangular forms give 38.3% and 34.4%, a gap of about four points that slightly favours the monolithic design. Because the averaged expression is never smaller than e^{-AD_0} for the same mean density, Murphy's model is less pessimistic than the Poisson model for large dies.

With $D_0 = 0.2\,\text{cm}^{-2}$ and $Y_{\text{wafer}} = 1$: for $A = 1\,\text{cm}^2$, $Y = (1 - e^{-0.4})/0.4 \approx 0.824$; for $A = 2\,\text{cm}^2$, $Y = (1 - e^{-0.8})/0.8 \approx 0.688$. For comparison, the triangular model gives 0.821 and 0.679, and the Poisson model 0.819 and 0.670.

C.2 Cost per good die

Let C_w be the cost of a processed wafer and $N(A)$ the number of whole dies of area A that fit on it. Ignoring edge losses, $N(A) \approx W/A$ for usable wafer area W. Dong and Xie use a more exact

dies-per-wafer expression that also counts the partial dies lost at the wafer edge [15]. The cost per good die is

$$C_{\text{good}}(A) = \frac{C_w}{N(A)\,Y(A)} \approx \frac{C_w}{W} \cdot \frac{A}{Y(A)}.$$

Doubling the area therefore multiplies the cost per good die by approximately $2\,Y(A)/Y(2A)$. In the example above (from 1 to 2 cm^2 with $D_0 = 0.2\,\text{cm}^{-2}$), the factor is $2 \times 0.824/0.688 \approx 2.40$: the larger die costs about 2.4 times as much, not twice as much.

C.3 Splitting a design into chiplets

Suppose a design of area A is split into k chiplets of area A/k, each with an additional interface overhead δ. If each chiplet is tested before assembly, so that only known-good dies are packaged, the silicon cost of the assembled part is approximately

$$k \cdot C_{\text{good}}\left(\tfrac{A}{k} + \delta\right) \approx \frac{C_w}{W} \cdot \frac{A + k\delta}{Y\left(\tfrac{A}{k} + \delta\right)}.$$

Smaller chiplets yield better, so the denominator grows with k, while the overhead term $k\delta$ in the numerator also grows. As an illustration with $D_0 = 0.2\,\text{cm}^{-2}$, a 4 cm^2 design split into four chiplets of 1 cm^2 plus a 0.05 cm^2 interface each costs about $(4.2/0.817)/(4/0.499) \approx 0.64$ of the monolithic silicon, a saving of roughly 36%. The saving grows for larger designs and higher defect densities. Dong and Xie reach the same qualitative conclusion for die stacking: it becomes cheaper than a single planar die only above a size threshold, and testing each die before stacking (die-to-wafer bonding) avoids multiplying together the yields of all dies in the stack [15]. The model omits packaging, assembly yield, test cost and the use of different process nodes for different chiplets, all of which section 9 discusses qualitatively.

C.4 Silicon across a product line

This calculation supports the summary in section 18.3. Suppose a fraction p of the parts a company sells needs 96 MB of L3 and the rest needs 32 MB, and compare three ways of building the line, using the silicon per good part from table 17 and a bonding yield of 95%. The bonding yield is an assumed value; at 99% each stacked part would cost about 140.7 mm^2 instead of 146.6 mm^2. The three ways are:

- **(A) One planar die for everything.** Every part uses the 122 mm^2 die with 96 MB, so every part costs 154 mm^2 of silicon.
- **(B) Two planar dies.** A 32 MB die (94.8 mm^2 per good part) and a 96 MB die (154 mm^2), each designed, verified and made separately.
- **(C) One CCD with an optional cache die.** Every part uses the 81 mm^2 CCD (94.8 mm^2 per good die). The fraction p is bonded to a cache die, and a failed bond scraps the CCD as well, so each stacked part costs $(94.8 + 44.5)/0.95 \approx 146.6\,\text{mm}^2$, as in equation (11).

Table 45 separates the two parts of AMD's argument. Against strategy (A), stacking saves a third of the silicon when few parts need the large cache, because the parts that do not need it no longer pay for it. Against strategy (B), the saving in silicon is small, 0.7–3%, so stacking's

Table 45: Average silicon per good part across a product line in which a fraction p of parts needs 96 MB (from table 17; bonding yield 95%, assumed; thinning, bonding and test costs not included).

Strategy	$p = 0.1$	$p = 0.25$	$p = 0.5$
(A) One planar 96 MB die	154 mm^2	154 mm^2	154 mm^2
(B) Two planar dies	101 mm^2	110 mm^2	124 mm^2
(C) One CCD, optional cache die	100 mm^2	108 mm^2	121 mm^2

real advantage over two planar dies is not silicon but everything outside this table. Strategy (B) needs a second die to be designed, verified and given its own masks, and it fixes the product mix when wafers are started, because a wafer of 96 MB dies cannot later become 32 MB parts without wasting the extra area. Strategy (C) needs one CCD design, and the decision to add a cache die is taken at assembly, so the mix can follow demand. The table also shows the limit of the argument. Each V-Cache part costs $139.3/Y_{\text{bonding}}$ mm^2 under (C), against 154 mm^2 under (A) and (B). Below a bonding yield of about $139.3/154 \approx 90\%$, strategy (C) therefore loses its advantage over (A) for V-Cache parts, and at the same point its small silicon lead over (B).

C.5 Good dies per wafer

This calculation supports the summary in section 9. Yield is only half of the cost story; the other half is how many dies fit on a wafer. Dong and Xie estimate the number of dies per wafer as

$$N_{\text{die}} = \frac{\pi\,(\phi_{\text{wafer}}/2)^2}{A_{\text{die}}} - \frac{\pi\,\phi_{\text{wafer}}}{\sqrt{2\,A_{\text{die}}}}, \tag{14}$$

where ϕ_{wafer} is the wafer diameter and A_{die} the die area [15, Eq. 14]. The first term divides the wafer area by the die area; the second subtracts the partial dies lost around the circular edge, a loss that grows with die size. For a 300 mm wafer, table 46 applies equation (14) and the yield model of equation (8).

Table 46: Dies and good dies per 300 mm wafer ($D_0 = 0.2\,\text{cm}^{-2}$, $Y_{\text{wafer}} = 1$; illustrative).

Die	Dies per wafer	Modelled yield	Good dies	64-core sets per wafer
74 mm^2 CCD	$955.2 - 77.5 \approx 878$	86.6%	760	$760/8 = 95$
592 mm^2 monolithic	$119.4 - 27.4 \approx 92$	38.3%	35	35

A wafer of CCDs supplies the compute silicon for about 95 64-core processors, against 35 for the monolithic design, a factor of 2.7. With $D_0 = 0.1$ the factor is about 1.9, and with $D_0 = 0.3$ about 3.5. The edge loss alone is small for the CCD (8% of the gross count) but noticeable for the large die (23%). This calculation still ignores the I/O die, packaging and test, and assumes that a 592 mm^2 die could be made at all; at the full 64-core size, Naffziger *et al.* note that the monolithic design would exceed the maximum reticle size [2].

D Organisation of the Stacked L3 Arrays

This appendix collects the arithmetic on the organisation of the 64 MB stacked L3 die (L3D) summarised in section 17.2.

How the stacked arrays are organised. The L3D contains 512 data macros of 128 KB each and 1088 tag and LRU macros of 6 KB each [3, slide 25]. The first figure checks: $512 \times 128\,\text{KB} = 64\,\text{MB}$. The arrays use a high-density SRAM bit cell and two supply rails, one for the bit cells and one for the peripheral circuits, with "extensive power reduction features" [3, slide 25]. The L3 is protected by DECTED (double-error-correcting, triple-error-detecting) error-correcting code (ECC) [3, slide 19].

The cost of tags. The tag and LRU macros hold $1088 \times 6\,\text{KB} = 6528\,\text{KB}$, about 10% of the 64 MB of data. With the 64-byte lines used throughout Zen 3 [11], the L3D holds $64\,\text{MB}/64\,\text{B} = 1\,048\,576$ lines, so the tag, state and replacement storage averages $6528 \times 1024 \times 8/1\,048\,576 \approx 51$ bits per line. The address split of section 4.5 shows where those bits go. A 64 MB, 16-way cache with 64-byte lines has $2^{26}/(2^6 \times 16) = 65\,536 = 2^{16}$ sets, so 6 address bits select the byte and 16 select the set. With the 48-bit physical addresses AMD documents for Zen 3 EPYC processors [18, p. 10], each tag would need $48 - 16 - 6 = 26$ bits, leaving roughly 25 bits per line for coherence state, replacement information and error-correction bits. The overhead is significant: a cache die is not all data storage.

An open detail. The combined 96 MB L3 is 16-way set associative [3, slide 20]. With 64-byte lines that is $96\,\text{MB}/(64\,\text{B} \times 16) = 98\,304 = 3 \times 2^{15}$ sets, which is not a power of two, so the simple indexing of section 4.5, in which a fixed group of address bits selects the set, cannot cover it directly. Because each die's portion of the L3 has its own data, tag and LRU arrays [3, slide 20], the base and stacked portions may be indexed separately, but AMD's materials do not say how addresses are divided between them. Lam, examining the Zen 4 version, describes each nominally 4 MB L3 slice as having its own interface to an 8 MB extension on the cache die, with its own tags and LRU state, and suggests that the controller *probably* chooses between the base and stacked portions using a subset of the address bits [22]. That arrangement would split each slice's lines between the two dies in a 1:2 ratio and would explain the non-power-of-two total, but it remains an informed inference rather than an AMD statement.

E Simulation Set-up and Reproduction

This appendix records what the model of section 28 contains and how to repeat every result. The simulator, the experiment scripts and the raw results are in the paper's GitHub repository (`github.com/TheInvincible2111/ELL305-Term-Paper`): the simulator in `tools/cachesim`, the scripts in `tools/cachesim/study`, and one CSV file per experiment in `research/measurements/cachesim`. A script generates the figures and tables of section 28 from those CSV files. The only values typed by hand are the measured Milan and Milan-X points in figure 53, read from figure 46, and Reguly's measured ratio in figure 58.

What the model contains

The model is trace-driven and functional: it follows every memory access through the caches but does not model a pipeline.

- **Caches.** Each core has a private L1 data cache and a private L2 that contains everything in its L1. Cores are divided evenly into L3 groups (a CCX or a CCD), each sharing one L3. The L3 is filled on every miss; the sources used here do not state how Zen's L3 is filled, and real fill policies may differ. An optional L4, the side die of S2, is shared by all groups. It is either filled on every miss, so that it holds copies of L3 data (*inclusive*), or filled only by L3 evictions and emptied when a line moves back up (*victim*). It is looked up after an L3 miss. Replacement is least-recently-used unless stated otherwise.
- **Placement.** A working set occupies consecutive line numbers, so it spreads evenly over the sets of each cache and has no conflict misses below capacity. Real programs with 4 KB pages are scattered over physical memory, which causes some conflict misses before a cache is full; the 5600X measurement shows that other effects also make a real L3 slow down before it is full (section 28.2).
- **Coherence.** A directory records which cores hold each line (section 5). A read of a line that another core has modified is served core to core. A write invalidates all other copies. A write that hits in the writer's own caches while other cores hold the line is charged one extra L3 latency, plus the hop if any of them is on another die. A write that misses pays for fetching the line, as a read does, plus a further hop if a holder is on another die. Copies held only in another group's L3, by no core, are invalidated at no cost.
- **Latency.** An access is charged the latency of the level that served it. An L1, L2 or L3 hit costs that level's latency. Another group's L3 or core costs two L3 latencies plus the hop. An L4 hit costs the L3 latency plus the L4 latency. DRAM costs the L3 latency plus a fixed time in nanoseconds, plus the hop in the two-die configuration of S4, where every memory access crosses to the I/O die.
- **Overlap.** The part of any latency beyond an L2 hit is multiplied by an *overlap factor* ω, the fraction an out-of-order core cannot hide; it is the complement of the hidden fraction h of section 4.8, $\omega = 1 - h$, so 1 means no overlap and smaller values mean more overlap. The model reports both the raw average latency, which a latency sweep measures, and the *exposed* average latency after the overlap factor, which the experiments compare.
- **Time.** The estimated time is the largest of three limits: the slowest core's latency-bound time (exposed latency plus a fixed number of cycles of other work per access), L3 traffic

over L3 bandwidth, and DRAM traffic over DRAM bandwidth. The model can also limit die-to-die link bandwidth, but that limit is switched off in every experiment here.

- **Warm-up.** Every run except the ping-pong test warms the caches before measuring and resets the statistics afterwards, so compulsory misses do not distort small working sets. The ping-pong test has no separate warm-up; it ignores its first 40 accesses.

Table 47: Default model parameters and their sources.

Parameter	Value and source
L1D, L2	32 KB 8-way, 4 cycles; 512 KB 8-way, 12 cycles; 64-byte lines [20]
L3, one CCD (Zen 3)	32 MB 16-way, 47 cycles [20]
L3, two CCXs (Zen 2 style)	2 × 16 MB, 39 cycles: 47 cycles minus the 7–8 cycles that unifying the L3 added [23] (derived)
L3 with cache die	96 MB, 51 cycles, four more than the planar L3 [3]
Planar 96 MB L3	59 cycles: an assumption, varied in table 32
Side die (L4)	64 MB, 22 ns, so that a hit totals about 34 ns, in line with the more than 30 ns Lam measured for Broadwell's eDRAM [22]; an assumption, varied from 5 to 22 ns in table 33
DRAM	L3 latency plus 74 ns [20]; 51.2 GB/s, two DDR4-3200 channels
Hop between dies	9 ns, AMD's CCD–IOD round trip [2]; 37 ns when fitted to Reguly's measured ratio (figure 58); also 18 and 60 ns in table 36
L3 bandwidth per core	32 bytes per cycle [3, slide 20]
Overlap factor	0.335, fitted to Lam's libx264 measurement (table 30); 0.05 in S6
Other work per access	1 cycle; 0.48 in S3 (assumption, see below)
Clock	4 GHz unless stated
Seeds; measured accesses per point	5 seeds; 10^6 accesses after warm-up (2.5×10^5 per thread in S6)

Calibration of the overlap factor

Lam's libx264 measurement gives, per thousand instructions, 3.85 L3 accesses, 0.40 misses removed by the V-Cache and a net saving of about 36 cycles (section 17.1). In the model, a removed miss saves $\omega\,(L3 + t_{\text{mem}} - L3_{\text{stacked}})$ cycles and each L3 access costs 4ω more, where ω is the overlap factor, because the model overlaps the four extra cycles too. Requiring $0.40 \times 306.8\,\omega - 3.85 \times 4\,\omega = 36$ at 4.2 GHz gives $\omega = 0.335$; the same equation gives 0.202 for Cyberpunk 2077 and 0.105 for GHPC. These are fitted values. They inherit the caveats of the 130-cycle estimate (section 17.1).

Workloads

Random accesses lines uniformly at random within the working set. A latency sweep instead follows a cyclic chain through its working set: within a cache's capacity the two give the same latency, but above it the random workload keeps some hits that a cyclic chase loses (section 28.2 and table 38). *Hot/cold* sends a fixed share of accesses to a small hot region, and the rest to the

whole working set or, optionally, to a separate far region beyond it. *Sequential* streams through private data. *Ping-pong* has two cores write one line in turn.

The S3 mix is built to match Lam's libx264 run:

- 98.889% of accesses go to a 16 KB hot region that stays in the L1;
- 0.799% go to a cold region whose size is swept;
- 0.312% go to a 1 GB far region that misses in both caches, so that the V-Cache CCD keeps about 1.04 misses per thousand instructions where Lam measured 1.08;
- one access in three instructions is a memory access, the textbook's typical figure [1, sec. 11.1.1] (an assumption here);
- L1 hits are charged one cycle, standing for loads the pipeline hides (an assumption);
- each access adds 0.48 cycles of other work (an assumption), chosen so that the plain CCD runs at about 730 cycles per thousand instructions at Lam's operating point, against the 735 measured.

At that operating point the mix gives about 3.8 L3 accesses and 1.44 misses per thousand instructions on the plain CCD. Because the mix and the overlap factor are both fitted to the same measurement, its agreement with Lam's gain at equal clocks (5.2% against 5.1% from the chart IPCs; section 28.5) is a check of the construction, not an independent test.

How to repeat the study

After cloning the GitHub repository, on Linux or WSL with a C++ compiler and Python 3:

```
cd tools/cachesim
g++ -O2 -std=c++17 -o cachesim src/cachesim.cpp
./cachesim --selftest
node test/crosscheck.js ./cachesim # C++ and JavaScript give identical results
python3 test/check_golden.py ./cachesim # without Node.js: against stored
    JavaScript results
python3 study/sims.py --bin ./cachesim --out results
python3 study/make_figures.py results figures
python3 study/compare_5600x.py --bin ./cachesim \
    --results ../../research/measurements/5600X/results-20261108-0724 \
    --csv model-comparison.csv --figures figures # App. G comparison
```

The full study takes about seven minutes on a 16-thread desktop and, with the same seeds, reproduces the stored CSV files exactly. The same model also runs in the browser, in `tools/cachesim/gui/index.html` on GitHub, where its parameters can be changed with sliders and experiments like S1, S2 and S4–S6 rerun.

F Further Calculations

This appendix holds three calculations whose results are summarised in the main text.

F.1 A directory protocol, step by step

This calculation supports section 5.

Table 48 repeats the five accesses of table 6 with a directory instead of a bus. The textbook specifies that caches send their requests to the directory, that the directory fetches a block from a cache that holds it before forwarding it to the requester, and that it sends invalidations only to the caches on the sharer list [1, sec. 12.4.6]. It does not fix the exact messages, so the table counts them under three assumptions made for this example: each request goes to the directory; the directory sends one message to each cache that must act, and each such cache replies once; and the directory then answers the requester. The count is an illustration of the textbook's scheme, not a description of AMD's protocol.

Table 48: The accesses of table 6 under a directory protocol (D is the directory; counting assumptions in the text).

	Access	Directory entry after	Messages	Count
1	A reads x	sharers {A}	A→D request; D→A data from memory	2
2	B reads x	sharers {A, B}	B→D; D→A forward; A→D data; D→B data	4
3	A writes x	owner A	A→D; D→B invalidate; B→D acknowledge; D→A permission	4
4	B reads x	sharers {A, B}	B→D; D→A forward; A→D data (written back); D→B data	4
5	B writes x	owner B	B→D; D→A invalidate; A→D acknowledge; D→B permission	4
			Total	**18**

The comparison with snooping shows what a directory buys and what it costs. On a bus, each of the five steps is one broadcast that every other cache must examine: with N caches, $5(N-1)$ deliveries, or 35 for eight caches and 315 for 64. The directory sends 18 messages whatever the number of caches, because only A and B ever hold the block. The price is indirection. A bus transaction is a single step, whereas each of steps 2 to 5 passes through a third party and takes two sequential round trips: requester to directory, directory to owner or sharer, and back again. The textbook also notes that the sharer list is in general a superset of the caches that actually hold the block [1, sec. 12.4.6]; an extra entry costs an unnecessary message, not a wrong result.

F.2 TLB reach against cache capacity

This calculation supports section 4.7. The memory a TLB can translate without a miss, its *reach*, is its number of entries times the page size. 7-cpu.com reports a 64-entry first-level and a

2048-entry second-level data TLB for Zen 3 [20], and Chips and Cheese also gives the 64-entry first level [23]. Table 49 compares the resulting reach with the cache capacities.

Table 49: Data-TLB reach of Zen 3 (entries from [20]) against its cache capacities.

	4 KB pages	**2 MB pages**
First-level DTLB, 64 entries	256 KB	128 MB
Second-level DTLB, 2048 entries	8 MB	4 GB
For comparison: L2 512 KB; L3 32 MB per CCD; L3 with V-Cache 96 MB		

The table multiplies entries by page size. 7-cpu.com's own page gives "about 800 KB" for the first-level TLB with 4 KB pages, more than 64×4 KB; it does not say why [20]. The group's own measurement on a Ryzen 5 5600X favours the 256 KB figure (appendix G). With 4 KB pages, the latency of a pointer chase starts to rise just above 256 KB, whereas with 2 MB pages it stays at the L2 latency up to the 512 KB capacity of the L2. From about 7 MB the 4 KB curve rises again, near the reach of the second-level TLB, and at 1 GB address translation adds about 17 ns to every access under WSL2, whose nested translation probably inflates the figure. With ordinary 4 KB pages, the second-level TLB covers 8 MB, a quarter of a 32 MB L3 and a twelfth of a 96 MB stacked L3. A working set that fills the stacked cache spans $96\,\text{MB}/4\,\text{KB} = 24\,576$ pages, twelve times as many as the TLB can hold, so a program that touches it at random can hit in the L3 and still miss in both TLBs. With 2 MB pages the first-level TLB alone covers 128 MB, more than either L3. This is why Chips and Cheese repeated its latency tests with 2 MB pages, which removes TLB penalties for test sizes below 128 MB [23]. With 2 MB pages, 7-cpu.com measured a penalty of 7 cycles for a first-level TLB miss and 24 cycles for a second-level miss [20]; the second figure is about half of a 47-cycle L3 hit.

F.3 Calibration of the cache–clock model

This calculation calibrates equation (13) of section 27.2. The libx264 example of section 17.1 supplies every input. On the plain CCD, $A = 3.85$, $M = 1.48$ and an IPC of 1.36 give 735 cycles per thousand instructions; the apparent stall of about 130 cycles per removed miss at 4.2 GHz gives $s \approx 130/4.2 \approx 31$ ns and $C \approx 735 - 1.48 \times 130 \approx 543$ cycles. The clocks are those Lam measured under an all-core load on an L3-sized data set, 4.8 GHz for the V-Cache CCD and 5.15 GHz for the plain one, a ratio of 1.073 [22]; Lam did not report the clocks during the libx264 run itself, so these are an assumption. At equal clocks and $\Delta M = 0.40$ the model gives a 5.2% gain, against 5.1% from the chart IPCs and the 4.9% Lam states.

Table 50: Net speed-up of the V-Cache CCD over the plain CCD for the libx264 profile, from equation (13) (calibration and clocks as in the text).

Misses removed per 1000 instr.	**Equal clocks (both 4.2 GHz)**	**4.8 against 5.15 GHz, memory time fixed**	**4.8 against 5.15 GHz, linear rule**
0	−2.1%	−6.7%	−8.7%
0.2	+1.5%	−3.0%	−5.4%
0.4 (libx264)	+5.2%	+1.0%	−1.9%
0.6	+9.3%	+5.3%	+1.9%
1.0	+18.5%	+15.3%	+10.4%
Break-even	0.12	0.35	0.50

G Measurements on the Group's Ryzen 5 5600X

This appendix records the one set of hardware measurements made for this paper. The group ran them on its own Ryzen 5 5600X, a single-CCD Zen 3 chiplet processor without stacked cache, so that the model of section 28 could be compared with a machine it was not built from. The results are used in section 28.2 (latency and overlap of misses), section 4.7 (TLB reach), section 8.2 (core-to-core hand-over) and section 28.8 (memory bandwidth). The raw results, the run notes and an analysis are on GitHub (`github.com/TheInvincible2111/ELL305-Term-Paper`), in `research/measurements/5600X`, and the measurement programs in `tools/measurements/5600x-kit`.

Machine and conditions

Table 51 records the machine and the conditions of the run.

Table 51: Conditions of the 5600X measurements (8 November 2026), as recorded in the run notes and by the kit's system report.

Item	Value
Processor	AMD Ryzen 5 5600X: one CCD, six cores, twelve threads, 32 MB L3, 512 KB L2 per core
Board, cooling	MSI B550 Gaming Edge WiFi; DeepCool AS500 air cooler
Firmware settings	Precision Boost Overdrive (PBO) enabled; no Curve Optimizer and no other overclock
Memory	32 GB of DDR4-3200, CL16-20-20-40, command rate 1T; Infinity Fabric clock 1600 MHz, 1:1 with the memory clock
Core clock	4.614 GHz, estimated by the kit from a chain of dependent additions before and after the run; Task Manager showed 4.6 GHz
Software	Windows with the High performance power plan; tests run in Ubuntu under the Windows Subsystem for Linux (WSL2); transparent huge pages on request (`madvise`); no other programs deliberately running

Method

The kit runs four C programs and the model, and takes about ten minutes.

- **Latency sweep.** A pointer chase walks a randomly ordered cyclic chain of 64-byte lines through a buffer of increasing size; each load depends on the previous one, so the time per step is the latency of the level that holds the buffer. It runs three times with 4 KB pages and three times with 2 MB pages. The paper uses the median of the three runs.
- **Overlap of misses.** The same chase follows k independent chains at once, $k = 1$ to 16, through a 1 GB buffer (memory) and a 16 MB buffer (L3), with 2 MB pages.
- **Bandwidth.** A read kernel and a triad kernel run over buffers from 16 KB to 256 MB with 1, 3, 6 and 12 threads.

- **Core-to-core.** Two threads pinned to two logical CPUs pass a cache line back and forth by writing a shared counter in turn; every pair of the twelve logical CPUs is measured, and each pair's value is the fastest of three trials.
- **Model.** The model of section 28 is run at the measured clock with its default inputs, and again with its L3 and memory latencies set to this machine's values (figure 54).

Results

Table 52: Latency at each level on the 5600X (pointer chase, median of three runs, 2 MB pages unless stated; cycles at 4.614 GHz) against the model's default inputs.

Level	Measured	Model's default input
L1 (up to 32 KB)	0.87 ns, 4.0 cycles	4 cycles
L2 (64–512 KB)	2.61–2.69 ns, 12.0–12.4 cycles	12 cycles
L3 (4–16 MB)	11.1–12.0 ns, 51–55 cycles	47 cycles
Memory (1 GB)	98.6 ns; 115.3 ns with 4 KB pages	L3 + 74 ns, 84 ns at this clock (the model gives 82 ns at 1 GB, where a few accesses still hit)

Table 52 gives the latency of each level. The L1 and L2 agree with the model's inputs, which were measured on another Zen 3 chip. The L3 plateau of 51–55 cycles agrees with the about 53 cycles reported for the Ryzen 7 5800X (table 15) and is 4–8 cycles above the 47 cycles the model takes from a Ryzen 5 5600G. Memory latency is about 14 ns above the model's input (17 ns above its output at 1 GB), which was measured on another machine with other memory. The overlap measurements are in table 31. Their single-chain value at 1 GB, 104.9 ns, is about 6% above the sweep's 98.6 ns. The two are separate runs of the same program in different modes, and the fractions in that table use 104.9 ns as their base. The time per step grows only from 105 to 120 ns between one and sixteen chains, so the core overlaps at least sixteen memory accesses with little loss.

Bandwidth behaved as expected. One thread read 108 GB/s from the L3, about 23 bytes per cycle against AMD's figure of 32 bytes per cycle per core [3, slide 20], and 30 GB/s from memory. Six threads read 280–296 GB/s from the L3 (8–16 MB in all) and 41 GB/s from memory, 80% of the 51.2 GB/s peak of two DDR4-3200 channels, or 6.8 GB/s per core.

The core-to-core round trip was 98.5 ns (median over the 132 ordered pairs, each the fastest of three trials), about 49 ns for each hand-over of the line. The model's same-CCD round trip, 49 ns at 4 GHz, is about half this (section 28.6).

Where the model and the 5600X differ

Near each capacity edge the model and the hardware differ, on the 5600X as on Milan, in two ways. First, just above each capacity the measured latency is higher than the model's: at

64 MB the 5600X measures 66.9 ns against 56.6 ns for the refitted model, and the same happens just above the 32 KB L1, where pages and the virtual machine play no part. A likely partial explanation is the order of access. A latency sweep visits its lines in a fixed cycle, which least-recently-used replacement handles badly once the data no longer fits, whereas the model's random accesses keep hitting in a share of the cache equal to its capacity divided by the working set. The model shows the same effect when it is given a cyclic workload (table 38): the 32 MB L3 then misses on every access at 64 MB, and the gain of the larger cache rises from 2.86 to 4.75 times. The explanation is only partial: at 64 MB the 5600X measures 66.9 ns, between the random model and an all-miss chase (about 99 ns), so about a third of its accesses still hit. Second, the measured L3 starts to slow down at about 16–20 MB, before it is full; at 32 MB its latency is 2.4 times the model's with default inputs and 2.1 times the refitted model's. The model maps each working set to consecutive lines, so it has no conflict misses below capacity. On Milan, part of the early rise comes from 4 KB pages scattered over physical memory and from TLB misses (table 49). On the 5600X the rise remains with 2 MB pages, so other causes contribute as well: replacement that is not exact LRU, other software's use of the L3, and the virtual machine's address translation. These tests cannot separate them, and the early rise is also the least repeatable part of the measurement: at 32 MB the three runs gave 24.1, 23.9 and 73.8 ns.

Both differences make the smaller cache slower in a latency sweep than in the model. For a latency sweep, therefore, the measured gain of a 96 MB over a 32 MB L3 is larger than the model's from about 20 to about 64 MB: at 64 MB, Milan's measured latency is about 240 cycles against the 32 MB model's 195, while Milan-X stays near its 51-cycle plateau. Close to the larger cache's own capacity the model overstates the gain: at 96 MB, Milan-X measures about 65 cycles against the 96 MB model's 51. How far this carries over to real programs depends on their order of access, which need resemble neither a random nor a cyclic pattern, and the early L3 slowdown may partly reflect the virtual machine.

Reliability and limits

- **Repeatability.** The clock estimate was the same before and after the run. Most points of the three runs of each sweep agree within about 1–3%. One of the three 2 MB-page runs was slower from about 19 MB upwards, by a factor of up to 3 at 32 MB and by 3–7% from about 150 MB to 1 GB, probably because the system granted it fewer huge pages; the median removes it. The 4 KB runs differ by up to about 30% near the TLB and cache limits.
- **Virtual machine.** WSL2 is a virtual machine. Its second level of address translation can add to the L3 and memory latencies, and pinning a thread to a logical CPU inside it does not fix the physical core. This probably explains why no pair of logical CPUs in the core-to-core test shows the short time expected for two threads on one core, and why pairs in which the responding thread ran on logical CPU 0 were faster (66–72 ns). The core-to-core values are therefore used only as a same-CCD magnitude.
- **Unusable values.** With twelve threads, bandwidth for buffers below about 32 MB was erratic (0.1 to 480 GB/s), and with any thread count the smallest buffers are limited by the overhead of starting each parallel pass. These values are not used.
- **Scope.** The processor has no stacked cache and one CCD, so the measurements do not test the stacked cache or the cost of crossing between CCDs. Repeating them under native Linux and on a V-Cache part is an open question (section 41).

H Evidence Ledger

This appendix records, for each independent source on the stacked cache used in Part V, the systems it compares, the conditions it states and what it measures (section 24.1).

Table 53: Evidence ledger: the independent sources on the stacked cache, with the conditions each one states.

Source; type	Systems compared	Clock and memory	Metric	Workloads; notes
Lam 2023 [22]; technical article	the two CCDs of one Ryzen 9 7950X3D	both CCDs fixed at 4.2 GHz, boost off; same memory; GPU clock fixed	IPC and L3 hit rate from performance counters	four games, libx264, 7-Zip; one Call of Duty scenario not exactly repeatable (multiplayer)
Domke *et al.* 2023 [16]; peer-reviewed	dual-socket EPYC 7773X vs 7763, 64 cores per socket	clocks not fixed (base 2.20 vs 2.45 GHz); same 280 W TDP and 16-channel DDR4	runtime-based performance	MiniFE across problem sizes 100^3 to 400^3
Lawenda *et al.* 2025 [34]; preprint	seven dual-socket platforms, Rome to Genoa-X and Bergamo	turbo on, SMT off; clocks measured as ranges	FVOPS (cells solved per second); L3 miss rate from AMD µProf	OpenFOAM motorBike (eight meshes) and an urban air-pollution case; each test repeated five times
Cozma 2022a [31]; technical article	Milan and Milan-X cloud instances	not fixed; Milan-X observed boosting higher in this test	load latency (cycles, ns)	pointer-chasing latency sweep
Cozma 2022b [23]; technical article	Milan-X vs Milan, one CCD each, cloud instances	clocks estimated (Milan-X about 5% lower)	runtime; latency	gem5, libx264, 7-Zip; latency with 4 KB and 2 MB pages
Cozma 2024 [32]; technical article	Ryzen 7 9800X3D (sample supplied by AMD)	stock	L3 latency; clock behaviour; SPEC CPU 2017	microbenchmarks; observed clocks; SPEC subtests
Larabel 2022 [35]; benchmark article	Ryzen 7 5800X3D vs 5800X	stock clocks; memory not restated per test	each benchmark's own metric	420 Linux benchmarks; 350 with a significant difference
Reguly 2023 [26]; paper	EPYC 7V73X (Milan-X) against two Intel Xeons	cloud virtual machine; SMT off	bandwidth; core-to-core latency	BabelStream; message-passing test; bandwidth-bound applications

Index

2.5D integration, 53
3D V-Cache, 19
    cache-on-top stack, 67

AMAT, 31

break-even (cache against clock), 100

cache coherence, 38
cache miss, 32
CCD (core complex die), 46
CCX (core complex), 46
chiplet, 19, 46

directory protocol, 39
dynamic power, 28

hybrid bonding, 68

I/O die (IOD), 46
inclusion, 35
Infinity Fabric, 48

leakage power, 28

memory hierarchy, 29
memory-level parallelism, 36
micro-bump, 68
Moore's law, 17
Murphy yield model, 43

NUMA, 47

performance equation, 27
plagiarism report, 134

readability indices, 135
reticle limit, 42

set-associative cache, 32
snooping protocol, 38
SRAM, 30

thermal resistance, 29
TLB, 35
    reach, 35

working set, 32
write-back cache, 33

yield, 43, 136